# MÁY VECTOR HỖ TRỢ (SVM/SVR)

*Định hướng: Tìm siêu phẳng có lề cực đại cho phân loại, và ống ε cho hồi quy*

## 1. Từ Perceptron và Logistic Regression đến bài toán lề cực đại

Qua hai bài trước, ta đã xây dựng hai mô hình phân loại tuyến tính. **Perceptron** tính điểm số $s = \mathbf{xw}$ rồi dùng hàm dấu để trả về một nhãn cứng, và được huấn luyện bằng quy tắc cập nhật mỗi khi gặp mẫu sai. **Logistic Regression** giữ nguyên điểm số tuyến tính nhưng đưa nó qua hàm sigmoid để thu được xác suất, và được huấn luyện bằng Gradient Descent trên hàm mất mát cross-entropy. Hai mô hình khác nhau về đầu ra và hàm mất mát, nhưng có chung một bộ khung: đường biên là một siêu phẳng, và đại lượng margin $m = y\,s$ cho biết mẫu được phân loại đúng hay sai.

Tuy nhiên, cả hai bài đều kết thúc bằng cùng một câu hỏi còn bỏ ngỏ. Khi dữ liệu phân tách tuyến tính được, có **vô số** đường biên phân loại đúng toàn bộ dữ liệu huấn luyện, và cả hai mô hình đều không có cơ chế nào để chọn ra một đường biên **tốt hơn** các đường còn lại. Bài này trả lời câu hỏi đó bằng một ý tưởng hình học rất tự nhiên: chọn đường biên **cách xa hai lớp nhất có thể**. Ý tưởng này là nền tảng của **máy vector hỗ trợ (Support Vector Machine, SVM)**. Trong mục 1, ta trả lời ba câu hỏi:

1. Perceptron và Logistic Regression còn để lại điều gì chưa giải quyết trong việc chọn đường biên?
2. Margin và khoảng cách từ một mẫu đến đường biên được viết như thế nào khi tách hệ số chệch khỏi vector trọng số?
3. Vì sao đường biên có lề lớn là tốt, và bài toán lề cực đại được phát biểu ra sao?

### Quy ước ký hiệu

Ta giữ nguyên quy ước của hai bài trước: chữ thường in nghiêng là đại lượng vô hướng, **chữ thường in đậm là vector**, **chữ hoa in đậm là ma trận**. Vector đặc trưng là **vector hàng**, vector tham số là **vector cột**.

Bài này có **một thay đổi về ký hiệu** so với hai bài trước: hệ số chệch *b* được **tách riêng** khỏi vector trọng số **w**, thay vì gộp vào **w** bằng cách thêm số 1 vào đầu vector đặc trưng. Lý do sẽ rõ ở mục 1.2.

| Đối tượng | Ký hiệu | Kích thước | Ý nghĩa |
|---|---|---|---|
| Số mẫu, số đặc trưng | $n,\ d$ | vô hướng | $n$ quan sát, mỗi quan sát có $d$ đặc trưng |

| Đối tượng | Ký hiệu | Kích thước | Ý nghĩa |
|---|---|---|---|
| Vector đặc trưng (hàng) | $\mathbf{x}_i$ | $1 \times d$ | Đặc trưng của mẫu thứ $i$, **không còn** phần tử 1 ở đầu |
| Ma trận đặc trưng | $\mathbf{X}$ | $n \times d$ | Hàng thứ $i$ của $\mathbf{X}$ là $\mathbf{x}_i$ |
| Vector trọng số (cột) | $\mathbf{w}$ | $d \times 1$ | Gồm $w_1, \dots, w_d$; là vector pháp tuyến của đường biên |
| Hệ số chệch | $b$ | vô hướng | Quyết định vị trí của đường biên |
| Vector nhãn thực (cột) | $\mathbf{y}$ | $n \times 1$ | Nhãn thực $y_i \in \{-1, +1\}$ |
| Vector toàn số 1 | $\mathbf{1}$ | $n \times 1$ | Dùng để cộng $b$ cho mọi mẫu |
| Điểm số | $s_i$, $\mathbf{s}$ | vô hướng, $n \times 1$ | $s_i = \mathbf{x}_i\mathbf{w} + b$ và $\mathbf{s} = \mathbf{X}\mathbf{w} + b\mathbf{1}$ |
| Margin | $m_i$, $\mathbf{m}$ | vô hướng, $n \times 1$ | $m_i = y_i s_i$ và $\mathbf{m} = \mathbf{y} \odot \mathbf{s}$ |
| Nhãn dự đoán | $\hat{y}_i$ | vô hướng | $\hat{y}_i = \text{sign}(s_i) \in \{-1, +1\}$ |

Hai cách viết cũ và mới cho **cùng một giá trị**. Gọi $\tilde{\mathbf{x}}_i = [1 \quad \mathbf{x}_i]$ và $\tilde{\mathbf{w}} = [b \quad \mathbf{w}^T]^T$ là vector đặc trưng và vector tham số theo cách viết cũ (có dấu ngã để phân biệt). Khi đó

$$\tilde{\mathbf{x}}_i\tilde{\mathbf{w}} = b + \mathbf{x}_i\mathbf{w}.$$

**Về nhãn.** Perceptron dùng nhãn $\{-1, +1\}$ còn Logistic Regression dùng nhãn $\{0,1\}$. SVM quay lại cách mã hóa $\{-1, +1\}$ của Perceptron, vì nó cho phép gói gọn việc đúng hay sai vào một phép nhân $y_i s_i$. Hai cách mã hóa chuyển qua lại bằng $y^{\pm} = 2y - 1$, như đã dùng ở bài Logistic Regression.

**Dữ liệu xuyên suốt.** Ta tiếp tục dùng bài toán dự đoán sinh viên đậu/rớt với $n = 6$ sinh viên và $d = 2$ đặc trưng ($x_1$: số giờ tự học mỗi tuần, $x_2$: số buổi vắng mặt). Với ký hiệu mới, ma trận đặc trưng **không còn cột số** 1:

$$\mathbf{X} = \begin{bmatrix} 2 & 4 \\ 3 & 3 \\ 4 & 5 \\ 6 & 1 \\ 8 & 2 \\ 9 & 0 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} -1 \\ -1 \\ -1 \\ +1 \\ +1 \\ +1 \end{bmatrix}.$$

## 1.1. Hạn chế của hai mô hình trước: có vô số đường biên đúng, chọn đường nào?

### Nhìn lại: hai mô hình đã cho ta những gì?

Ta đặt hai mô hình cạnh nhau, với điểm số $s = \mathbf{x}\mathbf{w} + b$ và margin $m = y\,s$:

| | Perceptron | Logistic Regression |
|---|---|---|
| Đầu ra | Nhãn cứng $\hat{y} = \text{sign}(s)$ | Xác suất $p = \sigma(s)$, rồi suy ra nhãn |
| Hàm mất mát theo margin | $\max(0, -m)$ | $\ln(1 + e^{-m})$ |
| Thuật toán huấn luyện | Cập nhật từng mẫu sai | Gradient Descent |
| Khi dữ liệu phân tách được | Dừng ngay khi hết mẫu sai | Mất mát giảm dần về 0 nhưng không bao giờ đạt, $\mathbf{w}$ tăng không giới hạn |

Cả hai mô hình có hai điểm chung quan trọng. Thứ nhất, ranh giới quyết định là siêu phẳng $\mathbf{x}\mathbf{w} + b = 0$. Thứ hai, hàm mất mát của cả hai đều là hàm của **margin** $m_i = y_i(\mathbf{x}_i\mathbf{w} + b)$. Hai điểm chung này sẽ được giữ nguyên trong SVM. Điều khác biệt nằm ở **tiêu chí chọn** $\mathbf{w}$ và $b$.

## Khi dữ liệu phân tách được, nghiệm đúng không duy nhất

Nhắc lại, dữ liệu **phân tách tuyến tính được** khi tồn tại $\mathbf{w}$ và $b$ sao cho

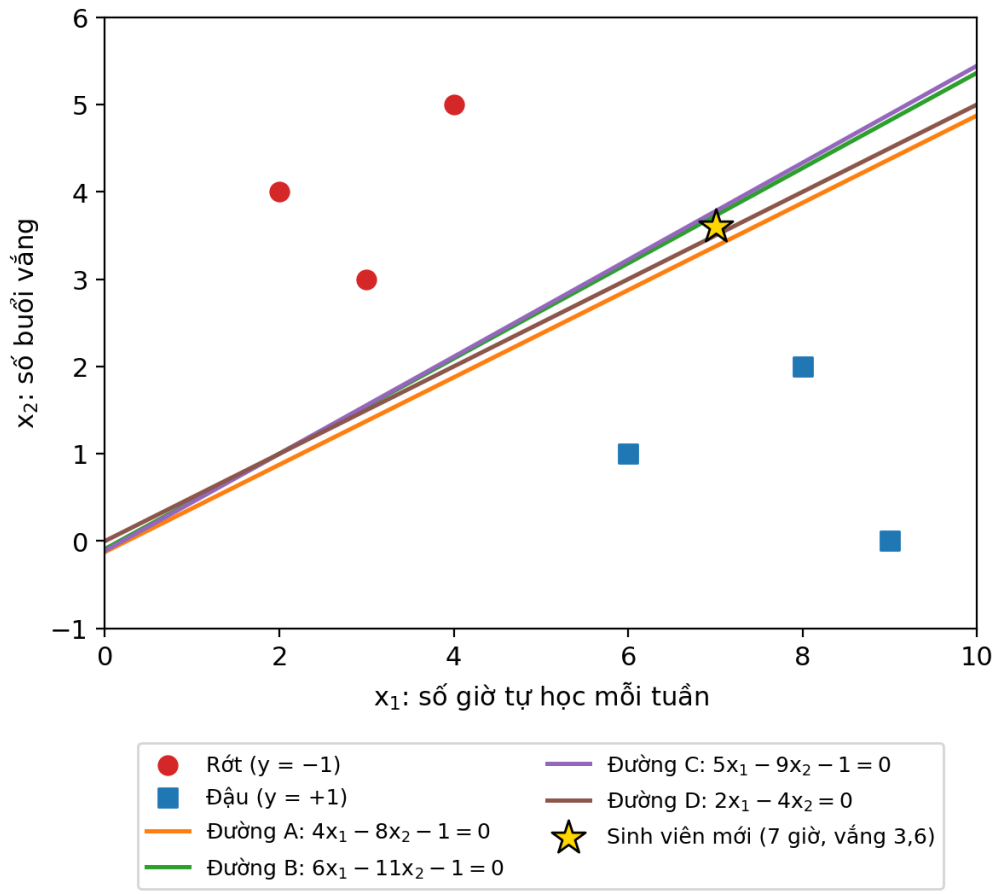
$$y_i(\mathbf{x}_i\mathbf{w} + b) > 0 \quad \text{với mọi } i = 1, \dots, n.$$

Dữ liệu 6 sinh viên thỏa điều kiện này. Khi đó **mọi** $(\mathbf{w}, b)$ thỏa bất đẳng thức trên đều phân loại đúng cả 6 mẫu, và có vô số bộ như vậy. Hai bài trước đã cho ta thấy hậu quả cụ thể.

**Với Perceptron.** Chạy thuật toán với tất cả $6! = 720$ thứ tự duyệt mẫu, ta thu được tới 20 bộ tham số cuối cùng khác nhau, và bộ nào cũng phân loại đúng 6 sinh viên huấn luyện. Bốn trong số đó, viết lại theo ký hiệu mới, là:

| Đường | Thứ tự duyệt | $b$ | $\mathbf{w}^T$ | Điểm số của sinh viên mới $\mathbf{x} = [7 \quad 3{,}6]$ | Dự đoán |
|---|---|---|---|---|---|
| A | 1,2,3,4,5,6 | $-1$ | $[4 \quad -8]$ | $-1{,}8$ | Rớt |
| B | 6,5,4,3,2,1 | $-1$ | $[6 \quad -11]$ | $1{,}4$ | Đậu |
| C | 1,4,2,5,3,6 | $-1$ | $[5 \quad -9]$ | $1{,}6$ | Đậu |
| D | 3,1,2,4,5,6 | $0$ | $[2 \quad -4]$ | $-0{,}4$ | Rớt |

Bốn đường biên này (Hình 1) đều đúng trên dữ liệu huấn luyện nhưng **đưa ra hai kết luận trái ngược nhau** cho cùng một sinh viên mới. Perceptron chỉ đảm bảo dừng ở một nghiệm "đúng hết", còn nghiệm cụ thể nào thì phụ thuộc vào thứ tự duyệt và giá trị khởi tạo, những yếu tố không liên quan gì đến chất lượng của đường biên.




*Hình 1. Bốn đường biên A, B, C, D do Perceptron tìm được với bốn thứ tự duyệt mẫu khác nhau. Cả bốn đều phân loại đúng 6 sinh viên huấn luyện, nhưng sinh viên mới (ngôi sao) nằm ở phía "đậu" của đường B, C và phía "rớt" của đường A, D.*

**Với Logistic Regression.** Tình huống không khá hơn. Trên dữ liệu phân tách hoàn hảo, hàm mất mát có thể nhỏ tùy ý bằng cách nhân $\mathbf{w}$ với một hệ số lớn, nên không tồn tại cực tiểu hữu hạn. Khi chạy Gradient Descent, độ dài $\|\mathbf{w}\|$ tăng từ 1,62 sau 10 bước lên 8,56 sau $10^6$ bước mà vẫn chưa dừng. Kết quả cuối cùng phụ thuộc vào **điều kiện dừng** mà ta đặt ra. Nếu thêm điều chuẩn hóa L2, ta thu được nghiệm duy nhất, nhưng nghiệm đó lại phụ thuộc vào hệ số $\lambda$, và $\lambda$ được chọn theo tiêu chí xác suất chứ không theo một tiêu chí hình học rõ ràng.

Logistic Regression còn có một đặc điểm đáng chú ý: mất mát $\ln(1 + e^{-m})$ luôn **dương** với mọi $m$, nên ngay cả mẫu nằm rất xa đường biên, về phía đúng, vẫn đóng góp một lượng nhỏ vào gradient. Ta sẽ thấy ở mục 3 rằng SVM có cách xử lý khác: chỉ những mẫu **sát đường biên** mới thực sự quan trọng.

## Câu hỏi trung tâm

Từ những quan sát trên, ta rút ra câu hỏi cần trả lời:

> **Trong vô số đường biên phân loại đúng dữ liệu huấn luyện, đường biên nào là tốt nhất, và "tốt nhất" được định nghĩa như thế nào?**

Một câu trả lời hợp lý phải thỏa hai yêu cầu: (i) dựa trên một tiêu chí **hình học rõ ràng**, và (ii) cho **cùng một đường biên** bất kể thứ tự duyệt mẫu hay giá trị khởi tạo. Bảng sau tổng hợp những gì ta cần:

| Hạn chế của hai mô hình trước | Điều ta mong muốn |
|---|---|
| Nghiệm phụ thuộc thứ tự duyệt, khởi tạo (Perceptron) | Một tiêu chí cho ra **nghiệm duy nhất** |
| $\mathbf{w}$ tăng không giới hạn trên dữ liệu phân tách được (Logistic) | Có một **cực tiểu hữu hạn** ngay cả khi dữ liệu phân tách hoàn hảo |
| Dừng ngay khi hết mẫu sai, đường biên có thể sát mẫu | Đường biên **cách đều và xa** các mẫu của hai lớp |
| Mọi mẫu đều ảnh hưởng đến nghiệm | Chỉ các mẫu **quan trọng** (sát đường biên) quyết định nghiệm |

Cả hai bài trước cũng đã gợi ý hướng đi này. Bài Perceptron kết luận rằng ý tưởng tìm đường biên có margin lớn nhất dẫn đến Support Vector Machine, còn bài Logistic Regression chỉ ra rằng Gradient Descent trên dữ liệu phân tách được dần tiến tới hướng của đường biên có margin lớn nhất. Hai mô hình khác nhau cùng chỉ về một đích. Các mục 1.2 và 1.3 làm rõ đích đến đó.

## 1.2. Mô tả bài toán nhị phân với nhãn $\{-1, +1\}$, margin và khoảng cách có dấu

### Mô tả bài toán

Cho bộ dữ liệu huấn luyện gồm $n$ quan sát

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}, \qquad y_i \in \{-1, +1\}.$$

Mô hình tuyến tính tính điểm số và dự đoán nhãn bằng dấu của điểm số:

$$s = \mathbf{x}\mathbf{w} + b, \qquad \hat{y} = \text{sign}(s) = \begin{cases} +1 & \text{nếu } s \geq 0 \\ -1 & \text{nếu } s < 0. \end{cases}$$

Đường biên quyết định là **siêu phẳng**

$$\mathcal{H} = \{\mathbf{x}\colon \mathbf{x}\mathbf{w} + b = 0\}.$$

Vector $\mathbf{w}$ vuông góc với siêu phẳng này. Thật vậy, lấy hai điểm bất kỳ $\mathbf{x}_a, \mathbf{x}_b$ nằm trên $\mathcal{H}$, ta có $\mathbf{x}_a\mathbf{w} + b = 0$ và $\mathbf{x}_b\mathbf{w} + b = 0$. Trừ hai đẳng thức:

$$(\mathbf{x}_a - \mathbf{x}_b)\,\mathbf{w} = 0.$$

Vector $\mathbf{x}_a - \mathbf{x}_b$ nằm dọc theo siêu phẳng và có tích vô hướng bằng 0 với $\mathbf{w}$, nên $\mathbf{w}$ là **vector pháp tuyến** của $\mathcal{H}$, chỉ về phía điểm số tăng.

### Margin hàm: đúng hay sai, và đúng đến mức nào

Như ở bài Perceptron, **margin** của mẫu thứ $i$ là tích giữa nhãn thực và điểm số:

$$m_i = y_i(\mathbf{x}_i\mathbf{w} + b), \qquad \mathbf{m} = \mathbf{y} \odot (\mathbf{X}\mathbf{w} + b\mathbf{1}).$$

Nếu $m_i > 0$ thì mẫu được phân loại đúng, nếu $m_i < 0$ thì mẫu bị phân loại sai. Dữ liệu phân tách được khi tồn tại $(\mathbf{w}, b)$ làm **mọi** $m_i > 0$.

Margin này còn được gọi là **margin hàm** (functional margin), để phân biệt với khoảng cách hình học ở phần tiếp theo. Nó có một nhược điểm sẽ được nêu ở cuối mục này.

## Khoảng cách từ một điểm đến siêu phẳng

Để đo mẫu cách đường biên **bao xa** theo nghĩa hình học, ta tính khoảng cách từ điểm $\mathbf{x}$ đến siêu phẳng $\mathcal{H}$. Gọi $\mathbf{x}_p$ là hình chiếu vuông góc của $\mathbf{x}$ lên $\mathcal{H}$. Vì $\mathbf{x} - \mathbf{x}_p$ vuông góc với $\mathcal{H}$ nên nó song song với pháp tuyến $\mathbf{w}^T$. Ta kiểm tra rằng

$$\mathbf{x}_p = \mathbf{x} - \frac{\mathbf{x}\mathbf{w} + b}{\| \mathbf{w} \|^2} \mathbf{w}^T$$

thuộc $\mathcal{H}$. Thật vậy, dùng $\mathbf{w}^T\mathbf{w} = \| \mathbf{w} \|^2$:

$$\mathbf{x}_p\mathbf{w} + b = \mathbf{x}\mathbf{w} - \frac{\mathbf{x}\mathbf{w} + b}{\| \mathbf{w} \|^2} \| \mathbf{w} \|^2 + b = 0.$$

Khoảng cách từ $\mathbf{x}$ đến $\mathcal{H}$ chính là độ dài $\| \mathbf{x} - \mathbf{x}_p \|$:

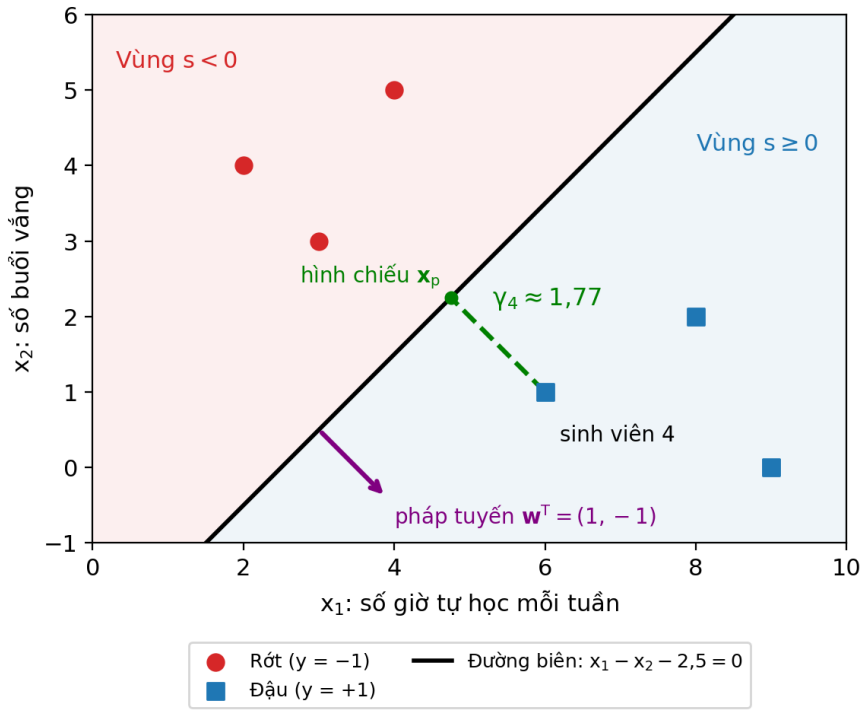
$$d(\mathbf{x}) = \frac{|\mathbf{x}\mathbf{w} + b|}{\| \mathbf{w} \|^2} \| \mathbf{w} \| = \frac{|\mathbf{x}\mathbf{w} + b|}{\| \mathbf{w} \|}.$$

Kết quả này khớp với công thức đã dùng ở bài Perceptron. Có một điểm cần lưu ý: **độ dài** $\| \mathbf{w} \|$ ở đây chỉ tính trên các trọng số $w_1, \dots, w_d$ và **không chứa** $b$. Đây chính là lý do ta tách $b$ khỏi $\mathbf{w}$: nếu gộp $b$ vào $\tilde{\mathbf{w}}$, độ dài $\| \tilde{\mathbf{w}} \|$ sẽ chứa cả $b$ và không còn là mẫu số của khoảng cách hình học. Hệ số chệch chỉ tịnh tiến đường biên song song với chính nó, còn độ rộng "khoảng trống" giữa hai lớp chỉ do $\mathbf{w}$ quyết định.

Thêm nhãn vào, ta được **khoảng cách có dấu** của mẫu thứ $i$:

$$\gamma_i = \frac{y_i(\mathbf{x}_i\mathbf{w} + b)}{\| \mathbf{w} \|} = \frac{m_i}{\| \mathbf{w} \|}.$$

Đại lượng $\gamma_i$ **dương** nếu mẫu nằm đúng phía của đường biên và **âm** nếu nằm sai phía, còn $|\gamma_i|$ là khoảng cách thật đến đường biên. Nó còn được gọi là **margin hình học** (geometric margin) của mẫu.




*Hình 2. Khoảng cách từ sinh viên 4 đến đường biên $x_1 - x_2 - 2{,}5 = 0$. Vector pháp tuyến $\boldsymbol{w}^T = (1, -1)$ chỉ về phía điểm số tăng, và khoảng cách bằng độ dài đoạn vuông góc từ mẫu đến hình chiếu $\boldsymbol{x}_p$ của nó.*

**Ví dụ với dữ liệu sinh viên.** Lấy lại bộ tham số quen thuộc từ hai bài trước, $\mathbf{w} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$ và $b = -2{,}5$, nên $\| \mathbf{w} \| = \sqrt{2} \approx 1{,}414$. Điểm số của 6 sinh viên tính đồng thời bằng một phép nhân ma trận:

$$\mathbf{s} = \mathbf{X}\mathbf{w} + b\mathbf{1} = \begin{bmatrix} 2 & 4 \\ 3 & 3 \\ 4 & 5 \\ 6 & 1 \\ 8 & 2 \\ 9 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} - 2{,}5 \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -4{,}5 \\ -2{,}5 \\ -3{,}5 \\ 2{,}5 \\ 3{,}5 \\ 6{,}5 \end{bmatrix}.$$

Nhân từng phần tử với nhãn, ta được margin $\mathbf{m}$ và khoảng cách có dấu $\gamma_i = m_i/\sqrt{2}$:

| Sinh viên | $y_i$ | $s_i$ | $m_i = y_i s_i$ | $\gamma_i = m_i/\sqrt{2}$ |
|---|---|---|---|---|
| 1 | $-1$ | $-4{,}5$ | $4{,}5$ | $3{,}18$ |
| 2 | $-1$ | $-2{,}5$ | $2{,}5$ | $1{,}77$ |
| 3 | $-1$ | $-3{,}5$ | $3{,}5$ | $2{,}47$ |
| 4 | $+1$ | $2{,}5$ | $2{,}5$ | $1{,}77$ |
| 5 | $+1$ | $3{,}5$ | $3{,}5$ | $2{,}47$ |
| 6 | $+1$ | $6{,}5$ | $6{,}5$ | $4{,}60$ |

Sinh viên 4 cách đường biên $2{,}5/\sqrt{2} \approx 1{,}77$, đúng như tính ở bài Perceptron. Mọi $\gamma_i$ đều dương nên bộ tham số này phân loại đúng cả 6 sinh viên.

### Margin hàm không đo được "độ xa": vai trò của thang đo

Margin hàm $m_i$ có một nhược điểm: nó **phụ thuộc vào cách chọn thang đo** của $(\mathbf{w}, b)$. Với mọi $c > 0$, bộ tham số $(c\mathbf{w}, cb)$ cho **cùng một đường biên** và **cùng nhãn dự đoán**, nhưng

$$m_i(c\mathbf{w}, cb) = c\, m_i(\mathbf{w}, b), \qquad \gamma_i(c\mathbf{w}, cb) = \frac{c\, m_i}{c \parallel \mathbf{w} \parallel} = \gamma_i(\mathbf{w}, b).$$

Chẳng hạn, nhân đôi $(\mathbf{w}, b)$ ở ví dụ trên làm mọi margin hàm tăng gấp đôi (từ 4,5; 2,5; ... lên 9; 5; ...), trong khi đường biên không đổi và mọi khoảng cách $\gamma_i$ vẫn giữ nguyên. Hệ quả:

1. Không thể dùng margin hàm để so sánh hai đường biên, vì ta có thể làm nó lớn tùy ý chỉ bằng cách nhân tham số lên.
2. Đây cũng chính là lý do Logistic Regression trên dữ liệu phân tách được có $\parallel \mathbf{w} \parallel$ tăng mãi: nhân $\mathbf{w}$ lên làm mọi margin tăng, nên mất mát $\ln(1 + e^{-m})$ giảm, dù đường biên không hề tốt hơn.
3. Đại lượng **không phụ thuộc thang đo** và có ý nghĩa hình học thật sự là khoảng cách có dấu $\gamma_i$. Ở mục 2, ta sẽ **cố định thang đo** bằng cách chuẩn hóa margin hàm nhỏ nhất bằng 1.

### Lề của một bộ phân loại

Từ khoảng cách có dấu của từng mẫu, ta định nghĩa **lề (margin)** của bộ phân loại $(\mathbf{w}, b)$ trên toàn bộ dữ liệu là khoảng cách có dấu **nhỏ nhất**:

$$\gamma(\mathbf{w}, b) = \min_{i=1,\dots,n} \gamma_i = \min_{i=1,\dots,n} \frac{y_i(\mathbf{x}_i\mathbf{w} + b)}{\parallel \mathbf{w} \parallel}.$$

Nói bằng lời, $\gamma$ là khoảng cách từ đường biên đến **mẫu gần nhất**. Nếu $\gamma > 0$ thì mọi mẫu đều nằm đúng phía, và quanh đường biên có một **dải trống** rộng $2\gamma$ (mỗi bên $\gamma$) không chứa mẫu nào. Với bộ tham số ở trên, $\gamma = \min_i \gamma_i \approx 1{,}77$, đạt tại sinh viên 2 và sinh viên 4.

## 1.3. Ý tưởng lề cực đại và vì sao lề lớn là tốt

### Ý tưởng lề cực đại

Giờ ta đã có một thước đo hình học, không phụ thuộc thang đo, cho chất lượng của một đường biên: **lề** $\gamma(\mathbf{w}, b)$. Câu trả lời cho câu hỏi trung tâm ở mục 1.1 trở nên tự nhiên:

> **Trong tất cả các đường biên phân loại đúng dữ liệu, chọn đường biên có lề lớn nhất.**

Viết thành bài toán tối ưu, đây là **bài toán lề cực đại (maximum margin)**:

$$\max_{\mathbf{w},\,b}\ \gamma(\mathbf{w},b) = \max_{\mathbf{w},\,b}\ \min_{i=1,\dots,n}\ \frac{y_i(\mathbf{x}_i\mathbf{w}+b)}{\|\,\mathbf{w}\,\|}.$$

Về mặt hình học, ta tìm "con đường" rộng nhất có thể đặt giữa hai lớp, và đường biên là đường tim của con đường đó. Về mặt cấu trúc, dạng maxmin cho thấy đường biên tối ưu hoàn toàn do các mẫu **gần nó nhất** quyết định, còn các mẫu ở xa không có vai trò. Những mẫu nằm đúng trên mép của con đường được gọi là **vector hỗ trợ (support vector)**, và đây là nguồn gốc của tên gọi SVM. Mục 2 sẽ biến bài toán maxmin này thành một bài toán tối ưu có dạng quen thuộc bằng cách cố định thang đo, và mục 4 sẽ giải thích vì sao chỉ các vector hỗ trợ mới quyết định nghiệm.

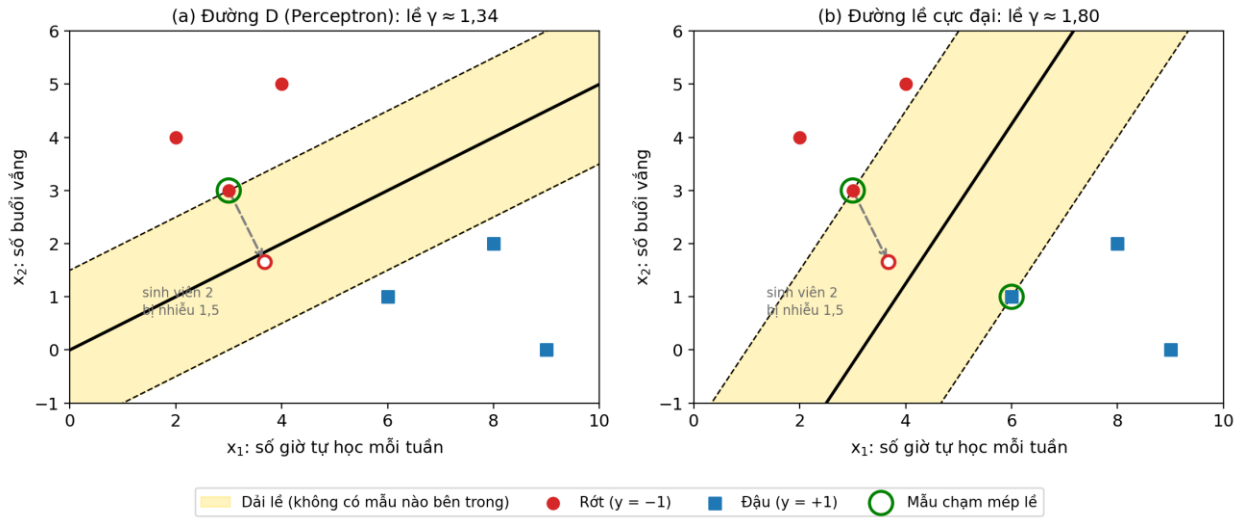
**Ví dụ với dữ liệu sinh viên.** Bộ phân loại lề cực đại của 6 sinh viên (được tính chi tiết ở mục 2.5) là

$$\mathbf{w} = \begin{bmatrix} \dfrac{6}{13} \\ -\dfrac{4}{13} \end{bmatrix}, \qquad b = -\frac{19}{13},$$

tương ứng đường biên $3x_1 - 2x_2 - 9{,}5 = 0$. Hai mẫu gần đường biên nhất là sinh viên 2 và sinh viên 4, với $\gamma_2 = \gamma_4 = \sqrt{13}/2 \approx 1{,}80$, nên lề là $\gamma \approx 1{,}80$ và độ rộng dải trống là $\sqrt{13} \approx 3{,}61$. Ta so sánh với bốn đường biên của Perceptron ở mục 1.1:

| Đường | $b$ | $\mathbf{w}^T$ | Lề $\gamma$ | Dự đoán sinh viên mới $[7 \quad 3{,}6]$ |
|---|---|---|---|---|
| A | $-1$ | $[4 \quad -8]$ | 1,45 | Rớt |
| B | $-1$ | $[6 \quad -11]$ | 1,28 | Đậu |
| C | $-1$ | $[5 \quad -9]$ | 1,26 | Đậu |
| D | $0$ | $[2 \quad -4]$ | 1,34 | Rớt |
| **Lề cực đại** | $-\dfrac{19}{13}$ | $\left[\dfrac{6}{13} \quad -\dfrac{4}{13}\right]$ | $\mathbf{1,80}$ | Đậu (cách đường biên $\approx 1{,}19$) |

Cả bốn đường của Perceptron đều có lề nhỏ hơn rõ rệt so với đường lề cực đại. Hình 3 minh họa dải lề của đường D và của đường lề cực đại trên cùng một bộ dữ liệu.




*Hình 3. Cùng một bộ dữ liệu, hai đường biên đúng nhưng có lề khác nhau. (a) Đường D của Perceptron có lề $\gamma \approx 1{,}34$ và chỉ một mẫu chạm mép lề. (b) Đường lề cực đại có lề $\gamma \approx 1{,}80$, hai mẫu chạm mép lề (vector hỗ trợ). Vòng tròn rỗng là sinh viên 2 sau khi bị dịch chuyển một đoạn dài 1,5: đường D phân loại sai mẫu này, còn đường lề cực đại vẫn phân loại đúng.*

**Một lưu ý về hai cách đo độ dài.** Bài Logistic Regression nói rằng Gradient Descent tiến dần tới hướng $[-1 \quad 2 \quad -4]$, là hướng có margin lớn nhất ở bài Perceptron. Hướng đó thực ra là hướng lề cực đại khi hệ số chệch **được tính vào độ dài** của vector tham số, tức là khi đo $\| \tilde{\mathbf{w}} \|$. SVM chuẩn đo độ dài chỉ bằng $\| \mathbf{w} \|$, **không tính** $b$, vì đó mới là khoảng cách hình học thật sự như đã chứng minh ở mục 1.2. Vì thế hai nghiệm khác nhau đôi chút (đường $2x_1 - 4x_2 - 1 = 0$ có lề $\approx 1{,}57$, nhỏ hơn lề $\approx 1{,}80$ của đường lề cực đại). Ý tưởng chung thì giống nhau: đẩy đường biên ra xa các mẫu nhất có thể.

## Lý do 1: lề lớn nghĩa là bài toán "dễ" hơn cho Perceptron

Ở bài Perceptron, định lý hội tụ của Novikoff cho ta một mối liên hệ định lượng giữa lề và độ khó của việc học. Với ký hiệu cũ (có gộp hệ số chệch), đặt $R = \max_i \| \tilde{\mathbf{x}}_i \|$ và giả sử tồn tại vector $\tilde{\mathbf{w}}^*$ có $\| \tilde{\mathbf{w}}^* \| = 1$ và $\gamma > 0$ sao cho $y_i \tilde{\mathbf{x}}_i \tilde{\mathbf{w}}^* \geq \gamma$ với mọi $i$. Khi đó số lần cập nhật $k$ của thuật toán Perceptron thỏa

$$k \leq \frac{R^2}{\gamma^2}.$$

Đại lượng $R$ là tính chất của dữ liệu (kích thước của đám mây điểm), còn $\gamma$ là lề của bộ phân loại $\tilde{\mathbf{w}}^*$. Cận trên **không phụ thuộc số mẫu** $n$ hay số chiều $d$, chỉ phụ thuộc tỷ số $R/\gamma$: lề càng lớn so với kích thước dữ liệu thì thuật toán càng ít phải cập nhật. Mô phỏng ở bài Perceptron minh họa rõ điều này, khi hai lớp cách nhau một khoảng trống $g$:

| Khoảng trống $g$ | 0,25 | 0,5 | 1 | 2 | 4 | 8 |
|---|---|---|---|---|---|---|
| Số lần cập nhật trung bình | 9,7 | 5,6 | 3,4 | 2,0 | 1,5 | 1,0 |

Vì cận đúng với **mọi** bộ phân loại tách được, cận tốt nhất ta có thể khẳng định là cận ứng với bộ phân loại có $\gamma$ **lớn nhất**. Nói cách khác, lề cực đại là thước đo "độ dễ" nội tại của bài toán: dữ liệu có lề tối đa càng lớn thì càng dễ phân loại.

Cần nói rõ giới hạn của lập luận này. Định lý nói về thuật toán Perceptron, không phải về chất lượng dự đoán trên dữ liệu mới. Ngoài ra, khi tách hệ số chệch khỏi $\mathbf{w}$, cận có dạng tương tự với $\gamma$ là khoảng cách hình học, sai khác một hằng số nhỏ khi đường biên nằm trong phạm vi của dữ liệu. Ý nghĩa không đổi: số lần cập nhật tỷ lệ với $(R/\gamma)^2$.

## Lý do 2: lề lớn thì ổn định trước nhiễu

Dữ liệu thực tế luôn có sai số: đặc trưng đo không chính xác, hoặc mẫu mới hơi khác mẫu huấn luyện. Một đường biên tốt phải **không đổi nhãn khi mẫu bị dịch chuyển nhẹ**. Lề cho ta một bảo đảm định lượng cho điều này.

**Mệnh đề.** Nếu mẫu $(\mathbf{x}, y)$ có khoảng cách có dấu $\gamma_{\mathbf{x}} > 0$ và mẫu bị nhiễu thành $\mathbf{x}' = \mathbf{x} + \mathbf{e}$ với $\| \mathbf{e} \| < \gamma_{\mathbf{x}}$ thì mẫu $\mathbf{x}'$ vẫn được phân loại đúng.

*Chứng minh.* Theo bất đẳng thức Cauchy–Schwarz, $|\mathbf{ew}| \le \| \mathbf{e} \| \, \| \mathbf{w} \|$, nên

$$y(\mathbf{x}'\mathbf{w} + b) = y(\mathbf{x}\mathbf{w} + b) + y\,\mathbf{ew} \;\ge\; y(\mathbf{x}\mathbf{w} + b) - \| \mathbf{e} \| \, \| \mathbf{w} \|.$$

Chia hai vế cho $\| \mathbf{w} \|$ ta được khoảng cách có dấu của mẫu bị nhiễu $\gamma_{\mathbf{x}'} \ge \gamma_{\mathbf{x}} - \| \mathbf{e} \| > 0$. ■

Áp dụng cho mọi mẫu huấn luyện: nếu bộ phân loại có lề $\gamma$ thì **mọi** dịch chuyển có độ dài nhỏ hơn $\gamma$ đều không làm đổi nhãn của bất kỳ mẫu nào. Lề càng lớn, "vùng an toàn" càng rộng.

**Ví dụ.** Dịch chuyển sinh viên 2, $\mathbf{x}_2 = [3 \quad 3]$ (thực tế rớt), một đoạn có độ dài 1,5 theo hướng tiến về phía đường biên D, tức là cộng $\mathbf{e} = [0{,}67 \quad -1{,}34]$ để được $\mathbf{x}_2{}' \approx [3{,}67 \quad 1{,}66]$ (Hình 3). Đường D có lề $1{,}34 < 1{,}5$ nên không còn được bảo đảm, và quả thực điểm số mới là $\mathbf{x}_2{}'\mathbf{w}_D = 2 \cdot 3{,}67 - 4 \cdot 1{,}66 \approx 0{,}7 > 0$, tức là mẫu bị dự đoán thành **đậu**. Đường lề cực đại có lề $1{,}80 > 1{,}5$ nên bảo đảm vẫn đúng, và điểm số mới là $\frac{6}{13} \cdot 3{,}67 - \frac{4}{13} \cdot 1{,}66 - \frac{19}{13} \approx -0{,}28 < 0$, tức là mẫu vẫn được dự đoán **rớt**, cách đường biên khoảng $0{,}50$.

## Lý do 3: một gợi ý từ lý thuyết học thống kê

Hai lý do trên đều có thể kiểm chứng trực tiếp bằng phép tính. Lý do thứ ba mang tính lý thuyết hơn và ta chỉ nêu kết quả mà không chứng minh: trong lý thuyết học thống kê, sai số trên dữ liệu **chưa thấy** của một bộ phân loại tuyến tính có thể được chặn trên bằng một đại lượng phụ thuộc vào tỷ số $R/\gamma$ giữa kích thước dữ liệu và lề, chứ không phụ thuộc trực tiếp vào số chiều $d$. Hiểu một cách trực giác, lề lớn nghĩa là bộ phân loại "đơn giản" hơn theo nghĩa nó khó bị ép vào những cấu hình quá khớp với nhiễu, nên khả năng tổng quát hóa tốt hơn. Đây là một lý do quan trọng khiến SVM hoạt động tốt kể cả khi số đặc trưng lớn.

## Khi nào bài toán lề cực đại không áp dụng được?

Bài toán $\max\gamma$ chỉ có ý nghĩa khi tồn tại ít nhất một đường biên có $\gamma > 0$, tức là dữ liệu **phân tách tuyến tính được**. Đây gọi là trường hợp **lề cứng (hard margin)**, và là nội dung của mục 2.

Thực tế có hai tình huống phá vỡ giả thiết này, và mỗi tình huống có một lời giải đã được nhắc đến ở các bài trước:

1. **Dữ liệu chồng lấn hoặc có nhiễu**, nên không có đường biên nào đúng hết. Lời giải là cho phép một số mẫu **lấn vào lề hoặc bị sai** với một mức phạt, gọi là **lề mềm (soft margin)**, ở mục 3.
2. **Đường biên cần cong**, như bài toán XOR hay dữ liệu hình vòng tròn. Lời giải là ánh xạ dữ liệu lên không gian đặc trưng cao hơn mà không cần tính tường minh, gọi là **thủ thuật kernel**, ở mục 5.

### Bức tranh tổng thể

Ba mô hình trả lời cùng một bài toán phân loại tuyến tính nhưng bằng ba tiêu chí khác nhau:

| | Perceptron | Logistic Regression | SVM |
|---|---|---|---|
| Câu hỏi chính | Có phân loại đúng không? | Xác suất đúng của từng mẫu là bao nhiêu? | Đường biên nào **cách xa hai lớp nhất**? |
| Hàm mất mát theo margin | $\max(0, -m)$ | $\ln(1 + e^{-m})$ | $\max(0,\ 1 - m)$ (mất mát hinge, mục 3) |
| Nghiệm khi dữ liệu phân tách được | Không duy nhất | Không có nghiệm hữu hạn nếu không điều chuẩn | **Duy nhất** |
| Mẫu nào quyết định nghiệm | Các mẫu từng bị sai | Mọi mẫu | Các **vector hỗ trợ** |

Cột SVM được điền sẵn theo những gì ta sẽ chứng minh ở các mục sau. Điều đáng chú ý là hàm mất mát hinge $\max(0,1 - m)$ khác Perceptron ở đúng một chỗ: nó vẫn phạt cả những mẫu **đúng nhưng chưa đủ xa** đường biên (có $0 \le m < 1$), nhờ đó đẩy đường biên ra xa các mẫu. Đó là cơ chế cụ thể thực hiện ý tưởng lề cực đại.

## Tóm tắt mục 1

1. Perceptron và Logistic Regression đều tìm một siêu phẳng $\mathbf{x}\mathbf{w} + b = 0$, nhưng khi dữ liệu phân tách được thì có **vô số** nghiệm đúng, và cả hai mô hình đều không có tiêu chí hình học để chọn ra một nghiệm. Perceptron cho nghiệm phụ thuộc thứ tự duyệt và khởi tạo; Logistic Regression có $\| \mathbf{w} \|$ tăng không giới hạn nếu không điều chuẩn.
2. Với nhãn $\{-1, +1\}$, **margin hàm** $m_i = y_i(\mathbf{x}_i\mathbf{w} + b)$ cho biết mẫu đúng hay sai, nhưng phụ thuộc thang đo: nhân $(\mathbf{w}, b)$ với $c > 0$ làm $m_i$ tăng $c$ lần mà đường biên không đổi.
3. **Khoảng cách có dấu** $\gamma_i = m_i / \| \mathbf{w} \|$ không phụ thuộc thang đo và là khoảng cách hình học thật sự. Ta tách $b$ khỏi $\mathbf{w}$ vì $\| \mathbf{w} \|$ trong mẫu số không được chứa $b$.

4. **Lề** của bộ phân loại là $\gamma(\mathbf{w}, b) = \min_i \gamma_i$, tức khoảng cách đến mẫu gần nhất. **Bài toán lề cực đại** là $\max_{\mathbf{w},b} \gamma(\mathbf{w}, b)$.

5. Lề lớn là tốt vì: (i) cận $k \le R^2/\gamma^2$ của Perceptron cho thấy lề lớn là bài toán dễ hơn, (ii) mọi dịch chuyển nhỏ hơn $\gamma$ không làm đổi nhãn, (iii) lý thuyết học thống kê gắn khả năng tổng quát hóa với tỷ số $R/\gamma$.

6. Bài toán lề cực đại cần dữ liệu phân tách được (lề cứng). Hai trường hợp còn lại được giải bằng lề mềm (mục 3) và kernel (mục 5).

## Câu hỏi kiểm tra nhanh

1. Với $\mathbf{w} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$, $b = -2{,}5$ và một sinh viên thực tế đậu có $\mathbf{x} = [5 \quad 2]$, hãy tính điểm số $s$, margin $m$ và khoảng cách có dấu $\gamma$.

2. Nhân bộ tham số ở câu 1 với $c = 10$. Margin hàm và khoảng cách có dấu của sinh viên đó thay đổi thế nào? Vì sao điều này cho thấy margin hàm không phù hợp để so sánh hai đường biên?

3. Với đường D ($b = 0$, $\mathbf{w} = \begin{bmatrix} 2 \\ -4 \end{bmatrix}$) và sinh viên 2 ($\mathbf{x}_2 = [3 \quad 3]$, $y_2 = -1$), hãy tính khoảng cách có dấu $\gamma_2$.

4. Một bộ phân loại có lề $\gamma = 1{,}8$. Một mẫu huấn luyện có thể bị dịch chuyển tối đa bao xa mà chắc chắn không đổi nhãn? Điều ngược lại có đúng không, tức là dịch chuyển lớn hơn 1,8 thì chắc chắn đổi nhãn?

5. Nếu lề $\gamma$ của một bộ phân loại giảm đi một nửa (với $R$ không đổi), cận $R^2/\gamma^2$ của Perceptron thay đổi thế nào?

### Gợi ý đáp án

1. $s = 1 \cdot 5 + (-1) \cdot 2 - 2{,}5 = 0{,}5$, margin $m = y\,s = (+1)(0{,}5) = 0{,}5$, và $\gamma = m/\|\,\mathbf{w}\,\| = 0{,}5/\sqrt{2} \approx 0{,}354$. Sinh viên được phân loại đúng nhưng nằm rất sát đường biên.

2. Margin hàm tăng 10 lần, từ 0,5 lên 5. Khoảng cách có dấu không đổi vì $\|\,10\mathbf{w}\,\| = 10\sqrt{2}$ nên $\gamma = 5/(10\sqrt{2}) \approx 0{,}354$. Vì ta có thể làm margin hàm lớn tùy ý mà đường biên không thay đổi, nó không phản ánh chất lượng của đường biên. Chỉ $\gamma$ mới so sánh được.

3. $s_2 = 2 \cdot 3 - 4 \cdot 3 + 0 = -6$, $m_2 = y_2 s_2 = (-1)(-6) = 6$, $\|\,\mathbf{w}\,\| = \sqrt{4 + 16} = \sqrt{20}$, nên $\gamma_2 = 6/\sqrt{20} \approx 1{,}34$. Đây cũng là lề của đường D, vì sinh viên 2 là mẫu gần nhất.

4. Dịch chuyển bất kỳ có độ dài **nhỏ hơn** 1,8 đều không làm đổi nhãn. Điều ngược lại **không** đúng: mệnh đề chỉ là điều kiện đủ. Dịch chuyển lớn hơn 1,8 **có thể** làm đổi nhãn (nếu đi đúng hướng vuông góc với đường biên) nhưng không chắc chắn, chẳng hạn khi dịch chuyển dọc theo đường biên.

5. Cận tăng gấp 4 lần, vì $\gamma$ nằm ở mẫu số dưới dạng bình phương: $R^2/(\gamma/2)^2 = 4R^2/\gamma^2$.

# 2. Mô hình SVM tuyến tính lề cứng

Ở mục 1, ta đã đi đến một ý tưởng rõ ràng: trong vô số đường biên phân loại đúng dữ liệu, hãy chọn đường biên có **lề lớn nhất**, tức là đường biên cách xa mẫu gần nhất của cả hai lớp nhất có thể. Ý tưởng được viết thành bài toán

$$\max_{\mathbf{w},\, b} \min_{i=1,\dots,n} \frac{y_i(\mathbf{x}_i\mathbf{w} + b)}{\| \mathbf{w} \|}.$$

Bài toán này đúng về mặt ý tưởng nhưng khó dùng: nó có cấu trúc maxmin, và hàm mục tiêu chứa $\| \mathbf{w} \|$ ở mẫu số. Mục này biến nó thành một bài toán tối ưu **gọn và có nghiệm duy nhất**, đó chính là mô hình **SVM tuyến tính lề cứng (hard-margin SVM)**. Từ "cứng" nghĩa là mô hình không cho phép **bất kỳ** mẫu huấn luyện nào nằm sai phía hoặc lấn vào dải lề. Điều kiện này chỉ thỏa được khi dữ liệu phân tách tuyến tính được, và ta sẽ nới lỏng nó ở mục 3. Trong mục 2, ta trả lời bốn câu hỏi:

1. Làm sao "chốt" thang đo của $(\mathbf{w}, b)$ để biểu diễn hai biên lề và độ rộng lề một cách gọn gàng?
2. Bài toán maxmin trở thành bài toán tối ưu nào, và nó có những tính chất gì?
3. Vector hỗ trợ là gì, và vì sao chỉ chúng mới quyết định đường biên?
4. Bài toán được viết bằng vector và ma trận ra sao, và giải tay trên dữ liệu 6 sinh viên như thế nào?

Ta giữ nguyên quy ước ký hiệu ở mục 1: chữ thường in đậm là vector, chữ hoa in đậm là ma trận, vector đặc trưng $\mathbf{x}_i$ là vector hàng, $\mathbf{w}$ là vector cột, hệ số chệch $b$ tách riêng, nhãn $y_i \in \{-1, +1\}$. Các ký hiệu mới được giới thiệu khi cần dùng. Số thứ tự hình tiếp tục từ mục 1.

## 2.1. Siêu phẳng, hai biên lề $\mathbf{xw} + \boldsymbol{b} = \pm\mathbf{1}$ và độ rộng lề $\mathbf{2}/\| \mathbf{w} \|$

### Thang đo của $(\mathbf{w}, b)$ là tự do, ta có thể "chốt" nó

Ở mục 1.2, ta đã thấy rằng với mọi $c > 0$, bộ tham số $(c\mathbf{w}, cb)$ cho **cùng một đường biên** và **cùng nhãn dự đoán**, còn khoảng cách có dấu $\gamma_i = m_i/\| \mathbf{w} \|$ không đổi. Nói cách khác, mỗi đường biên tương ứng với cả một "họ" tham số tỷ lệ với nhau. Sự tự do này đã gây rắc rối cho Logistic Regression (cứ nhân $\mathbf{w}$ lên là mất mát giảm), nhưng với SVM ta tận dụng nó: ta được phép **chọn một đại diện** cho mỗi họ.

Giả sử $(\mathbf{w}, b)$ phân loại đúng mọi mẫu, tức $m_i = y_i(\mathbf{x}_i\mathbf{w} + b) > 0$ với mọi $i$. Gọi

$$\mu = \min_{i=1,\dots,n} m_i > 0$$

là margin hàm nhỏ nhất. Thay $(\mathbf{w}, b)$ bằng $(\mathbf{w}/\mu,\ b/\mu)$, ta được một bộ tham số **cùng đường biên** nhưng có margin hàm nhỏ nhất bằng đúng 1. Phép chuẩn hóa này luôn thực hiện được trên dữ liệu phân tách được, và ta gọi dạng thu được là **dạng chuẩn tắc (canonical form)**:

$$\min_{i=1,\dots,n} y_i(\mathbf{x}_i\mathbf{w} + b) = 1.$$

Con số 1 không có gì đặc biệt: ta có thể chọn 2 hay 10 và mọi kết luận hình học vẫn như cũ. Chọn 1 chỉ vì công thức gọn nhất. Điều quan trọng là sau khi chốt thang đo, $\| \mathbf{w} \|$ **không còn tự do** và trở thành đại lượng mang ý nghĩa hình học, như ta thấy ngay sau đây.

## Hai biên lề và dải lề

Với dạng chuẩn tắc, mọi mẫu thỏa $y_i(\mathbf{x}_i\mathbf{w} + b) \geq 1$, tức là

$$\mathbf{x}_i\mathbf{w} + b \;\geq\; +1 \;\text{ nếu } y_i = +1, \qquad \mathbf{x}_i\mathbf{w} + b \;\leq\; -1 \;\text{ nếu } y_i = -1.$$

Ta gọi hai siêu phẳng

$$\mathcal{H}_+ = \{\mathbf{x}:\ \mathbf{x}\mathbf{w} + b = +1\}, \qquad \mathcal{H}_- = \{\mathbf{x}:\ \mathbf{x}\mathbf{w} + b = -1\}$$

là **hai biên lề**. Chúng song song với đường biên quyết định $\mathcal{H}_0$: $\mathbf{x}\mathbf{w} + b = 0$ (cùng pháp tuyến $\mathbf{w}$) và nằm hai bên của nó. Vùng nằm giữa hai biên lề, tức $-1 < \mathbf{x}\mathbf{w} + b < 1$, gọi là **dải lề**. Vì mọi mẫu huấn luyện có $|s_i| \geq 1$ nên **không có mẫu huấn luyện nào nằm bên trong dải lề**. Mẫu có $m_i = 1$ nằm đúng trên một biên lề, và đó là những mẫu gần đường biên nhất.

## Độ rộng lề

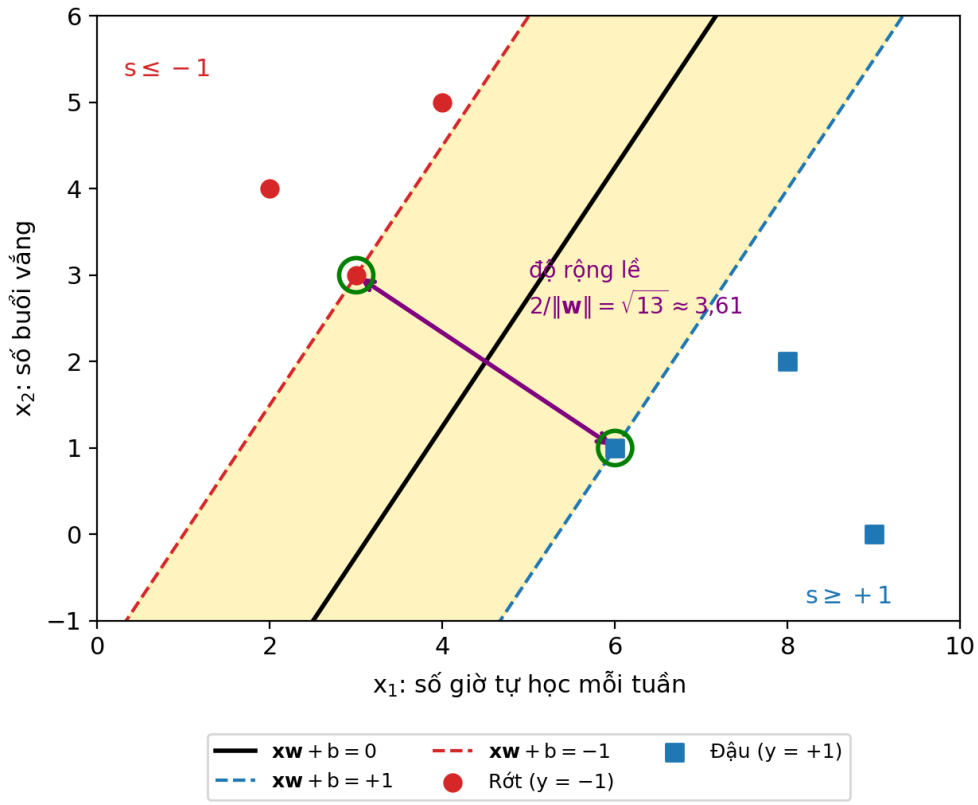
Lấy một điểm $\mathbf{x}$ nằm trên $\mathcal{H}_+$, tức $\mathbf{x}\mathbf{w} + b = 1$. Theo công thức khoảng cách ở mục 1.2, khoảng cách từ $\mathbf{x}$ đến đường biên $\mathcal{H}_0$ là

$$d(\mathbf{x}) = \frac{|\mathbf{x}\mathbf{w} + b|}{\| \mathbf{w} \|} = \frac{1}{\| \mathbf{w} \|}.$$

Tương tự cho điểm trên $\mathcal{H}_-$. Vậy mỗi biên lề cách đường biên đúng $1/\| \mathbf{w} \|$, và **độ rộng của dải lề** (khoảng cách giữa hai biên lề) là

$$\text{độ rộng lề} = \frac{2}{\| \mathbf{w} \|}, \qquad \gamma = \frac{1}{\| \mathbf{w} \|}.$$

Ở đây $\gamma$ là lề đã định nghĩa ở mục 1.2. Trong dạng chuẩn tắc, $\gamma = \min_i m_i / \| \mathbf{w} \| = 1/\| \mathbf{w} \|$.




*Hình 4. Đường biên quyết định* $\boldsymbol{xw} + b = 0$*, hai biên lề* $\boldsymbol{xw} + b = \pm 1$ *và dải lề (vùng vàng) của bộ phân loại lề cực đại trên dữ liệu 6 sinh viên. Độ rộng dải lề là* $2/\| \boldsymbol{w} \| = \sqrt{13} \approx 3{,}61$*. Hai mẫu được khoanh tròn nằm đúng trên hai biên lề.*

### Vì sao ‖ w ‖ nhỏ nghĩa là lề rộng?

Kết quả $\gamma = 1/\| \mathbf{w} \|$ có một cách hiểu trực giác. Điểm số $s = \mathbf{xw} + b$ thay đổi theo tốc độ $\| \mathbf{w} \|$ khi ta rời xa đường biên theo hướng pháp tuyến: đi thêm một đoạn $d$ thì $s$ thay đổi $\| \mathbf{w} \| \ d$. Muốn $s$ đạt mức $\pm 1$ (đến mép lề), ta phải đi một đoạn $d = 1/\| \mathbf{w} \|$. Nếu $\| \mathbf{w} \|$ **nhỏ**, điểm số tăng chậm, ta phải đi **xa** mới đến mép lề, nên lề rộng. Nếu $\| \mathbf{w} \|$ **lớn**, điểm số tăng nhanh, mép lề sát đường biên, nên lề hẹp.

Đây là hình ảnh đối xứng với Logistic Regression ở bài trước. Ở đó, $\| \mathbf{w} \|$ lớn làm sigmoid dốc và mô hình "tự tin quá mức", và mất mát cứ muốn đẩy $\| \mathbf{w} \|$ tăng mãi. SVM làm điều ngược lại: cố định margin hàm nhỏ nhất bằng 1 rồi **ép** $\| \mathbf{w} \|$ **xuống thấp nhất có thể**, và chính việc đó làm lề rộng nhất.

## 2.2. Bài toán tối ưu gốc: $\min \frac{1}{2} \| \mathbf{w} \|^2$ với ràng buộc $y_i(\mathbf{x}_i \mathbf{w} + b) \geq 1$

### Từ bài toán maxmin đến bài toán tối thiểu hóa ‖ w ‖

Ta quay lại bài toán lề cực đại

$$\max_{\mathbf{w},b} \min_i \frac{m_i}{\|\mathbf{w}\|}, \qquad m_i = y_i(\mathbf{x}_i\mathbf{w} + b).$$

Mỗi bộ tham số khả thi (phân loại đúng mọi mẫu) đều có thể chuẩn hóa về dạng chuẩn tắc $\min_i m_i = 1$ mà không đổi đường biên và không đổi giá trị mục tiêu. Với dạng chuẩn tắc, giá trị mục tiêu chỉ còn là $1/\|\mathbf{w}\|$. Do đó bài toán tương đương với

$$\max_{\mathbf{w},b} \frac{1}{\|\mathbf{w}\|} \quad \text{với ràng buộc} \quad \min_i m_i = 1.$$

Hai bước đơn giản hóa tiếp theo:

1. **Đổi** $\max 1/\|\mathbf{w}\|$ **thành** $\min \|\mathbf{w}\|$, và vì $\|\mathbf{w}\| \geq 0$ nên có thể thay bằng $\min \frac{1}{2}\|\mathbf{w}\|^2$. Hệ số $\frac{1}{2}$ chỉ để đạo hàm sau này gọn hơn ($\nabla_{\mathbf{w}} \frac{1}{2}\|\mathbf{w}\|^2 = \mathbf{w}^T$), không ảnh hưởng nghiệm. Hàm bình phương này **trơn và lồi chặt**, rất thuận lợi cho tối ưu.
2. **Nới ràng buộc** $\min_i m_i = 1$ **thành** $m_i \geq 1$ **với mọi** $i$**.** Việc nới này không làm đổi nghiệm. Thật vậy, giả sử nghiệm tối ưu có $\min_i m_i = \mu > 1$. Khi đó $(\mathbf{w}/\mu, b/\mu)$ vẫn thỏa $m_i \geq 1$ và có $\|\mathbf{w}/\mu\| < \|\mathbf{w}\|$, mâu thuẫn với tính tối ưu. Vậy tại nghiệm tối ưu, ràng buộc $m_i \geq 1$ luôn có ít nhất một mẫu đạt dấu bằng, tức $\min_i m_i = 1$ tự động được thỏa.

## Bài toán tối ưu gốc của SVM lề cứng

Kết hợp lại, ta thu được **bài toán gốc (primal problem)** của SVM tuyến tính lề cứng:

$$\boxed{\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{với ràng buộc} \quad y_i(\mathbf{x}_i\mathbf{w} + b) \geq 1, \;\; i = 1, \ldots, n.}$$

Khai triển hàm mục tiêu theo từng thành phần, $\frac{1}{2}\|\mathbf{w}\|^2 = \frac{1}{2}(w_1^2 + w_2^2 + \cdots + w_d^2)$. Có ba điểm đáng chú ý khi đọc bài toán này:

1. **Hàm mục tiêu không chứa** $b$**.** Hệ số chệch chỉ xuất hiện trong ràng buộc: nó tịnh tiến đường biên để các mẫu nằm đúng phía, nhưng không làm lề rộng hay hẹp. Điều này khớp với việc ta đã tách $b$ khỏi $\mathbf{w}$ ở mục 1.2, và tương tự việc Logistic Regression điều chuẩn hóa L2 không phạt $w_0$.
2. **Có** $n$ **ràng buộc**, mỗi mẫu một ràng buộc $m_i \geq 1$. Mỗi ràng buộc là một bất đẳng thức **tuyến tính** theo $(\mathbf{w}, b)$.
3. **Ràng buộc có vế phải là 1, không phải 0.** So với điều kiện phân tách được của Perceptron $m_i > 0$, ràng buộc $m_i \geq 1$ đòi hỏi mọi mẫu không chỉ đúng phía mà còn cách đường biên ít nhất $1/\|\mathbf{w}\|$. Số 1 là thang đo ta đã chốt ở mục 2.1.

## Tính chất của bài toán

**Đây là bài toán quy hoạch toàn phương lồi.** Hàm mục tiêu là hàm bậc hai lồi theo $\mathbf{w}$, còn tập các $(\mathbf{w}, b)$ thỏa $n$ ràng buộc tuyến tính là một tập lồi (giao của các nửa không gian). Hệ quả là mọi cực tiểu địa phương đều là cực tiểu toàn cục, và bài toán giải được hiệu quả bằng các bộ giải quy hoạch toàn phương (QP).

**Vector w tối ưu là duy nhất.** Giả sử có hai nghiệm tối ưu $(\mathbf{w}_1, b_1)$ và $(\mathbf{w}_2, b_2)$ với cùng giá trị mục tiêu, tức $\| \mathbf{w}_1 \| = \| \mathbf{w}_2 \|$. Điểm giữa $(\bar{\mathbf{w}}, \bar{b}) = \frac{1}{2}(\mathbf{w}_1 + \mathbf{w}_2,\ b_1 + b_2)$ cũng thỏa mọi ràng buộc (vì tập khả thi lồi). Theo đẳng thức hình bình hành,

$$\| \bar{\mathbf{w}} \|^2 = \frac{1}{2} \| \mathbf{w}_1 \|^2 + \frac{1}{2} \| \mathbf{w}_2 \|^2 - \frac{1}{4} \| \mathbf{w}_1 - \mathbf{w}_2 \|^2.$$

Nếu $\mathbf{w}_1 \neq \mathbf{w}_2$ thì $\| \bar{\mathbf{w}} \| < \| \mathbf{w}_1 \|$, mâu thuẫn với tính tối ưu. Vậy $\mathbf{w}_1 = \mathbf{w}_2$. Ở mục 2.3, ta sẽ thấy $b$ cũng được xác định duy nhất. Đây là điểm khác căn bản với Perceptron (nhiều nghiệm) và với Logistic Regression không điều chuẩn (không có nghiệm hữu hạn): **SVM lề cứng cho đúng một đường biên**, không phụ thuộc thứ tự duyệt mẫu hay giá trị khởi tạo.

**Bài toán có nghiệm khi và chỉ khi dữ liệu phân tách tuyến tính được.** Nếu tồn tại $(\mathbf{w}, b)$ phân loại đúng mọi mẫu, ta chuẩn hóa nó về $m_i \geq 1$ như ở mục 2.1, nên tập khả thi khác rỗng. Ngược lại, nếu dữ liệu không phân tách được thì không có $(\mathbf{w}, b)$ nào thỏa $m_i > 0$ với mọi $i$, huống chi $m_i \geq 1$, và bài toán vô nghiệm. Ví dụ, thêm một sinh viên thứ 7 có $\mathbf{x}_7 = [7 \quad 1]$ nhưng bị rớt ($y_7 = -1$). Điểm này là tổ hợp lồi của ba sinh viên đậu:

$$[7 \quad 1] = 0{,}6[6 \quad 1] + 0{,}2[8 \quad 2] + 0{,}2[9 \quad 0].$$

Vì điểm số $s$ là hàm affine nên $s_7 = 0{,}6\, s_4 + 0{,}2\, s_5 + 0{,}2\, s_6$. Nếu ba sinh viên đậu đều có $s \geq 1$ thì $s_7 \geq 1 > 0$, tức sinh viên 7 bị dự đoán đậu, trái với $y_7 = -1$. Vậy không có đường thẳng nào phân loại đúng cả 7 sinh viên, và bài toán lề cứng vô nghiệm. Tình huống này là động lực của lề mềm ở mục 3.

## Đặt cạnh Perceptron và Logistic Regression

Cả ba mô hình đều dùng điểm số tuyến tính $s = \mathbf{x}\mathbf{w} + b$ và margin $m = y\, s$. Chúng khác nhau ở cách dùng margin:

| | Perceptron | Logistic Regression (có L2) | SVM lề cứng |
|---|---|---|---|
| Mục tiêu | $\sum_i \max(0, -m_i)$ | $\sum_i \ln(1 + e^{-m_i}) + \frac{\lambda}{2} \| \mathbf{w} \|^2$ | $\frac{1}{2} \| \mathbf{w} \|^2$ |
| Ràng buộc | Không | Không | $m_i \geq 1$ với mọi $i$ |
| Vai trò của $\| \mathbf{w} \|$ | Không kiểm soát | Bị phạt nhẹ | **Là mục tiêu cần tối thiểu** |

| | Perceptron | Logistic Regression (có L2) | SVM lề cứng |
|---|---|---|---|
| Nghiệm | Không duy nhất | Duy nhất khi $\lambda > 0$ | Duy nhất khi dữ liệu phân tách được |

Có thể nhìn SVM lề cứng như trường hợp cực đoan của hai bài trước: thay vì đánh đổi giữa "mất mát dữ liệu" và "độ lớn của $\mathbf{w}$" bằng một hệ số mềm, SVM lề cứng **đòi hỏi mất mát dữ liệu bằng** 0 (mọi $m_i \geq 1$, tức mất mát hinge $\max(0, 1 - m_i) = 0$) rồi mới tối thiểu hóa $\| \mathbf{w} \|$. Hàm hinge sẽ được dùng đầy đủ ở mục 3 khi ta cho phép mất mát khác 0.

## 2.3. Vector hỗ trợ và ý nghĩa hình học

### Định nghĩa

Gọi $(\mathbf{w}^*, b^*)$ là nghiệm của bài toán gốc. Mỗi ràng buộc $m_i \geq 1$ có thể ở một trong hai trạng thái: **chặt** ($m_i = 1$, mẫu nằm đúng trên biên lề) hoặc **lỏng** ($m_i > 1$, mẫu nằm ngoài dải lề, còn "dư" một khoảng). Ta gọi **vector hỗ trợ (support vector)** là các mẫu có ràng buộc chặt:

$$\mathcal{S} = \{ i : \ y_i(\mathbf{x}_i \mathbf{w}^* + b^*) = 1 \}.$$

Hiểu theo hình học, đó là những mẫu nằm đúng trên $\mathcal{H}_+$ hoặc $\mathcal{H}_-$, gần đường biên nhất, và chúng "chống đỡ" dải lề giống như các cột chống đỡ một cây cầu. Tên gọi "vector hỗ trợ" xuất phát từ đây (mỗi mẫu là một vector trong không gian đặc trưng).

### Hai lớp đều có vector hỗ trợ

**Mệnh đề.** Tại nghiệm tối ưu, mỗi lớp có ít nhất một vector hỗ trợ.

*Chứng minh.* Giả sử không có mẫu dương nào có $m_i = 1$, tức mọi mẫu dương có $m_i > 1$. Giảm $b$ đi một lượng $\varepsilon > 0$ đủ nhỏ. Với mẫu dương, $m_i$ giảm đi $\varepsilon$ nên vẫn $\geq 1$ nếu $\varepsilon$ đủ nhỏ. Với mẫu âm, $m_i = -(\mathbf{x}_i \mathbf{w} + b)$ tăng thêm $\varepsilon$, nên các mẫu âm đang có $m_i = 1$ nay có $m_i = 1 + \varepsilon > 1$. Khi đó **mọi** $m_i > 1$. Đặt $\mu = \min_i m_i > 1$ và thay $(\mathbf{w}, b)$ bằng $(\mathbf{w}/\mu, b/\mu)$: bộ mới vẫn khả thi nhưng có $\| \mathbf{w} \|$ nhỏ hơn, mâu thuẫn với tính tối ưu. Lập luận cho lớp âm hoàn toàn tương tự. ■

Hệ quả hình học: cả $\mathcal{H}_+$ lẫn $\mathcal{H}_-$ đều "chạm" ít nhất một mẫu, nên đường biên nằm đúng giữa hai lớp.

### Tính $b$ từ vector hỗ trợ

Với mọi $k \in \mathcal{S}$ ta có $y_k(\mathbf{x}_k \mathbf{w}^* + b^*) = 1$. Nhân hai vế với $y_k$ và dùng $y_k^2 = 1$:

$$b^* = y_k - \mathbf{x}_k \mathbf{w}^*, \qquad k \in \mathcal{S}.$$

Vì $\mathbf{w}^*$ duy nhất và $\mathcal{S}$ khác rỗng, công thức này xác định $b^*$ duy nhất. Trong thực hành, người ta thường lấy trung bình công thức trên qua mọi $k \in \mathcal{S}$ để giảm sai số làm tròn. Chú ý rằng mọi $k \in \mathcal{S}$ phải cho cùng một giá trị $b^*$, nên đây cũng là một phép kiểm tra nghiệm.

## Chỉ vector hỗ trợ quyết định đường biên

Đây là tính chất quan trọng nhất của SVM, và cũng là điểm đối lập rõ nhất với Logistic Regression (mọi mẫu đều đóng góp vào gradient).

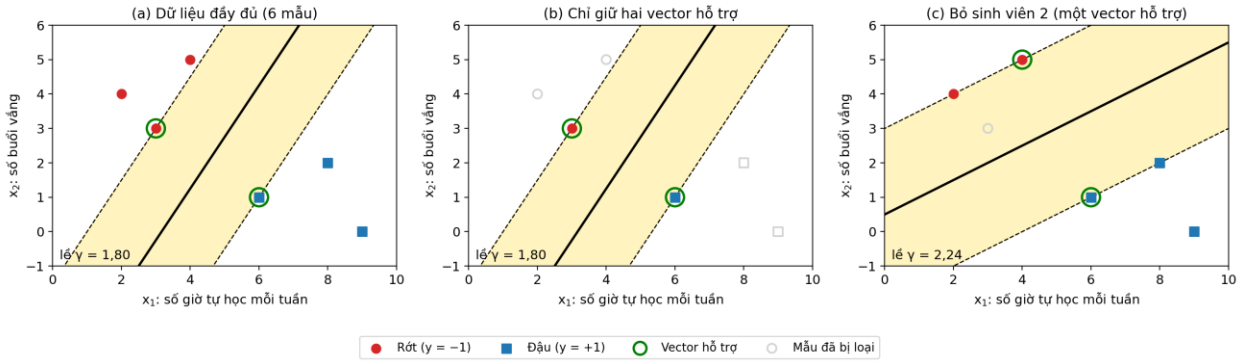
**Mệnh đề.** Nếu bỏ khỏi dữ liệu một mẫu $j \notin \mathcal{S}$ (ràng buộc lỏng, $m_j > 1$), vector $\mathbf{w}^*$ tối ưu không đổi.

*Chứng minh.* Gọi $(\mathbf{w}', b')$ là nghiệm tối ưu của bài toán đã bỏ mẫu $j$. Vì $(\mathbf{w}^*, b^*)$ cũng khả thi cho bài toán rút gọn nên $\| \mathbf{w}' \| \leq \| \mathbf{w}^* \|$. Xét điểm nằm giữa $(\mathbf{w}_t, b_t) = (1-t)(\mathbf{w}^*, b^*) + t(\mathbf{w}', b')$ với $t \in (0,1)$. Nó thỏa mọi ràng buộc $i \neq j$ (tập khả thi lồi), và thỏa cả ràng buộc $j$ khi $t$ đủ nhỏ, vì tại $t = 0$ ta có $m_j > 1$ (còn "dư") và $m_j$ liên tục theo $t$. Như vậy $(\mathbf{w}_t, b_t)$ khả thi cho bài toán ban đầu. Nếu $\mathbf{w}' \neq \mathbf{w}^*$ thì do tính lồi chặt của $\|\cdot\|^2$,

$$\| \mathbf{w}_t \|^2 < (1-t) \| \mathbf{w}^* \|^2 + t \| \mathbf{w}' \|^2 \leq \| \mathbf{w}^* \|^2,$$

mâu thuẫn với tính tối ưu của $\mathbf{w}^*$ trong bài toán ban đầu. Vậy $\mathbf{w}' = \mathbf{w}^*$. ■

Áp dụng liên tiếp, ta có thể bỏ **tất cả** mẫu không phải vector hỗ trợ mà nghiệm không đổi: chỉ cần chạy SVM trên riêng tập vector hỗ trợ là thu được cùng một đường biên. Cũng bằng lập luận tương tự, **di chuyển** một mẫu không phải vector hỗ trợ sang vị trí khác (miễn là vẫn nằm ngoài dải lề, đúng phía) không làm thay đổi đường biên. Ngược lại, bỏ hoặc di chuyển một vector hỗ trợ **có thể** làm đường biên thay đổi.




*Hình 5. (a) Nghiệm SVM lề cứng trên cả 6 mẫu, lề $\gamma \approx 1{,}80$, vector hỗ trợ là sinh viên 2 và 4. (b) Chỉ giữ lại hai vector hỗ trợ (các mẫu khác bị loại, vẽ mờ): đường biên giữ nguyên. (c) Bỏ vector hỗ trợ sinh viên 2: đường biên xoay và lề đổi thành $\gamma \approx 2{,}24$ với vector hỗ trợ mới là sinh viên 3 và 4.*

Bảng sau tóm tắt ba thí nghiệm trong Hình 5, cùng kết quả khi bỏ riêng sinh viên 6 (không phải vector hỗ trợ):

| Dữ liệu | $\mathbf{w}^T$ | $b$ | Lề $\gamma$ | Vector hỗ trợ |
|---|---|---|---|---|
| Cả 6 sinh viên | $\left[\frac{6}{13} \quad -\frac{4}{13}\right]$ | $-\frac{19}{13}$ | 1,80 | 2, 4 |
| Bỏ sinh viên 6 | $\left[\frac{6}{13} \quad -\frac{4}{13}\right]$ | $-\frac{19}{13}$ | 1,80 | 2, 4 |

| Dữ liệu | $\mathbf{w}^T$ | $b$ | Lề $\gamma$ | Vector hỗ trợ |
|---|---|---|---|---|
| Chỉ giữ sinh viên 2 và 4 | $\left[\frac{6}{13} \quad -\frac{4}{13}\right]$ | $-\frac{19}{13}$ | 1,80 | 2, 4 |
| Bỏ sinh viên 2 | $[0{,}2 \quad -0{,}4]$ | 0,2 | 2,24 | 3, 4 |

Đối chiếu với Perceptron ở mục 1.1: Perceptron cập nhật dựa trên các mẫu **từng bị sai**, nên nghiệm phụ thuộc cả lịch sử duyệt mẫu. SVM lề cứng chỉ phụ thuộc vào các mẫu **sát đường biên**, và vì thế nghiệm ổn định hơn và không phụ thuộc thứ tự. Tính chất "nghiệm chỉ dựa trên một số ít mẫu" gọi là **tính thưa (sparsity)**, và sẽ được làm rõ bằng dạng đối ngẫu ở mục 4.

## 2.4. Biểu diễn vector và ma trận của bài toán

### Ràng buộc cho toàn bộ $n$ mẫu

Với mẫu thứ $i$, ràng buộc là $y_i(\mathbf{x}_i\mathbf{w} + b) \geq 1$. Gom $n$ ràng buộc lại bằng margin dạng vector đã dùng ở mục 1.2:

$$\mathbf{m} = \mathbf{y} \odot (\mathbf{X}\mathbf{w} + b\,\mathbf{1}) \;\geq\; \mathbf{1},$$

trong đó $\geq$ giữa hai vector hiểu là **từng phần tử một**, và $\mathbf{1}$ là vector cột toàn số 1 kích thước $n \times 1$. Để đưa bài toán về dạng một phép nhân ma trận duy nhất, ta đặt

$$\mathbf{Z} = \mathrm{diag}(\mathbf{y})\,\mathbf{X},$$

tức là ma trận $n \times d$ có hàng thứ $i$ bằng $y_i\mathbf{x}_i$ (nhân từng hàng của $\mathbf{X}$ với nhãn tương ứng). Vì $\mathbf{y} \odot \mathbf{1} = \mathbf{y}$ nên

$$\mathbf{y} \odot (\mathbf{X}\mathbf{w} + b\,\mathbf{1}) = \mathbf{Z}\mathbf{w} + b\,\mathbf{y}.$$

Ghép $\mathbf{w}$ và $b$ thành một vector tham số duy nhất $\mathbf{u}$ gồm $d + 1$ phần tử, và ghép $\mathbf{Z}$ với $\mathbf{y}$ thành ma trận $\mathbf{G}$:

$$\mathbf{u} = \begin{bmatrix}\mathbf{w}\\ b\end{bmatrix}, \qquad \mathbf{G} = [\mathbf{Z} \quad \mathbf{y}], \qquad \mathbf{G}\mathbf{u} = \mathbf{Z}\mathbf{w} + b\,\mathbf{y}.$$

Hàm mục tiêu cũng viết được theo $\mathbf{u}$. Đặt ma trận đường chéo khối

$$\mathbf{P} = \begin{bmatrix}\mathbf{I}_d & \mathbf{0}\\ \mathbf{0}^T & 0\end{bmatrix},$$

với $\mathbf{I}_d$ là ma trận đơn vị cấp $d$, thì $\mathbf{u}^T\mathbf{P}\,\mathbf{u} = \|\,\mathbf{w}\,\|^2$. Phần tử 0 ở góc dưới bên phải của $\mathbf{P}$ chính là việc **không phạt** $b$, giống ma trận $\mathbf{D}_0$ ở bài Logistic Regression (khi đó $b$ đứng đầu nên số 0 nằm ở góc trên bên trái).

### Bài toán gốc ở dạng ma trận

Tổng hợp lại, bài toán gốc của SVM tuyến tính lề cứng viết gọn là

$$\min_{\mathbf{u}\in\mathbb{R}^{d+1}} \frac{1}{2}\,\mathbf{u}^T\mathbf{P}\,\mathbf{u} \qquad \text{với ràng buộc} \qquad \mathbf{G}\,\mathbf{u} \;\geq\; \mathbf{1}.$$

Đây đúng là **dạng chuẩn của một bài toán quy hoạch toàn phương** (hàm mục tiêu bậc hai $\frac{1}{2}\mathbf{u}^T\mathbf{P}\mathbf{u}$, ràng buộc tuyến tính $\mathbf{G}\mathbf{u} \geq \mathbf{1}$), nên có thể đưa trực tiếp cho một bộ giải QP. Chú ý rằng $\mathbf{P}$ chỉ **nửa xác định dương** (do số 0 ứng với $b$), nên hàm mục tiêu lồi chặt theo $\mathbf{w}$ chứ không lồi chặt theo cả $\mathbf{u}$. Điều này khớp với kết quả ở mục 2.2: $\mathbf{w}$ duy nhất, còn $b$ được xác định nhờ vector hỗ trợ.

Bảng kích thước các đại lượng:

| Đại lượng | Biểu thức | Kích thước |
|---|---|---|
| Ma trận đặc trưng | $\mathbf{X}$ | $n \times d$ |
| Vector nhãn | $\mathbf{y}$ | $n \times 1$ |
| Ma trận nhãn-đặc trưng | $\mathbf{Z} = \mathrm{diag}(\mathbf{y})\mathbf{X}$ | $n \times d$ |
| Vector tham số gộp | $\mathbf{u} = \begin{bmatrix}\mathbf{w}\\ b\end{bmatrix}$ | $(d+1) \times 1$ |
| Ma trận ràng buộc | $\mathbf{G} = [\mathbf{Z} \quad \mathbf{y}]$ | $n \times (d+1)$ |
| Ma trận mục tiêu | $\mathbf{P} = \begin{bmatrix}\mathbf{I}_d & \mathbf{0}\\ \mathbf{0}^T & 0\end{bmatrix}$ | $(d+1) \times (d+1)$ |
| Vector margin | $\mathbf{m} = \mathbf{G}\mathbf{u}$ | $n \times 1$ |

Vì $\mathbf{m} = \mathbf{G}\mathbf{u}$, nên sau khi huấn luyện, **vector hỗ trợ chính là các hàng của G mà tại đó** $(\mathbf{G}\mathbf{u})_i = 1$, và mọi hàng khác có $(\mathbf{G}\mathbf{u})_i > 1$.

## Dự đoán

Với mẫu mới $\mathbf{x}$, nhãn dự đoán là $\hat{y} = \mathrm{sign}(\mathbf{x}\mathbf{w}^* + b^*)$. Với nhiều mẫu mới xếp thành ma trận $\mathbf{X}_{\text{mới}}$, điểm số là $\mathbf{X}_{\text{mới}}\mathbf{w}^* + b^*\mathbf{1}$. Cần nhớ rằng ràng buộc $|s| \geq 1$ chỉ áp dụng cho **mẫu huấn luyện**. Mẫu mới hoàn toàn có thể rơi vào bên trong dải lề, và khi đó dấu của $s$ vẫn quyết định nhãn.

## Ví dụ: ma trận của dữ liệu 6 sinh viên

Với $\mathbf{X}$ và $\mathbf{y}$ ở mục 1, ta có

$$\mathbf{Z} = \mathrm{diag}(\mathbf{y})\mathbf{X} = \begin{bmatrix}-2 & -4\\ -3 & -3\\ -4 & -5\\ 6 & 1\\ 8 & 2\\ 9 & 0\end{bmatrix}, \qquad \mathbf{G} = \begin{bmatrix}-2 & -4 & -1\\ -3 & -3 & -1\\ -4 & -5 & -1\\ 6 & 1 & 1\\ 8 & 2 & 1\\ 9 & 0 & 1\end{bmatrix}.$$

Với $\mathbf{u} = [w_1 \quad w_2 \quad b]^T$, bất đẳng thức $\mathbf{G}\mathbf{u} \geq \mathbf{1}$ tương đương sáu ràng buộc cụ thể sau, mỗi dòng ứng với một sinh viên:

$$\begin{aligned} -2w_1 - 4w_2 - b &\geq 1, & 6w_1 + w_2 + b &\geq 1,\\ -3w_1 - 3w_2 - b &\geq 1, & 8w_1 + 2w_2 + b &\geq 1,\\ -4w_1 - 5w_2 - b &\geq 1, & 9w_1 + b &\geq 1. \end{aligned}$$

Bài toán là tìm $(w_1, w_2, b)$ làm $\frac{1}{2}(w_1^2 + w_2^2)$ nhỏ nhất trong khi thỏa cả sáu bất đẳng thức này.

## 2.5. Ví dụ tính tay trên 6 sinh viên (chỉ có hai vector hỗ trợ)

Bài toán có ba biến và sáu ràng buộc, đủ nhỏ để giải tay. Ta chưa có công cụ tổng quát (dạng đối ngẫu ở mục 4), nên sẽ dùng một chiến lược dựa trên tính chất ở mục 2.3: **đoán tập vector hỗ trợ, giải bài toán với các ràng buộc đó ở dạng đẳng thức, rồi kiểm tra các ràng buộc còn lại.**

### Bước 1: chọn ứng viên vector hỗ trợ

Vector hỗ trợ là những mẫu gần đường biên nhất, và đường biên nằm giữa hai lớp. Vì vậy ứng viên tự nhiên là **cặp mẫu khác lớp gần nhau nhất**. Bảng sau cho bình phương khoảng cách giữa mỗi sinh viên rớt (hàng) và mỗi sinh viên đậu (cột):

| | Sinh viên 4 (6,1) | Sinh viên 5 (8,2) | Sinh viên 6 (9,0) |
|---|---|---|---|
| Sinh viên 1 (2,4) | 25 | 40 | 65 |
| Sinh viên 2 (3,3) | **13** | 26 | 45 |
| Sinh viên 3 (4,5) | 20 | 25 | 50 |

Cặp gần nhất là **sinh viên 2 và sinh viên 4**, với khoảng cách $\sqrt{13} \approx 3{,}61$. Ta giả thiết hai sinh viên này là vector hỗ trợ, tức ràng buộc 2 và 4 là đẳng thức.

### Bước 2: giải bài toán với hai ràng buộc đẳng thức

Với $y_2 = -1$ và $y_4 = +1$, hai đẳng thức là

$$\begin{aligned} 3w_1 + 3w_2 + b &= -1, \\ 6w_1 + w_2 + b &= +1. \end{aligned}$$

Trừ vế theo vế để khử $b$:

$$(6w_1 + w_2) - (3w_1 + 3w_2) = 2 \;\Rightarrow\; 3w_1 - 2w_2 = 2, \quad \text{tức} \quad [3 \quad -2]\mathbf{w} = 2.$$

Ta cần tối thiểu $\| \mathbf{w} \|$ trong số các $\mathbf{w}$ thỏa phương trình này. Theo bất đẳng thức Cauchy–Schwarz,

$$2 = [3 \quad -2]\mathbf{w} \le \|[3 \quad -2]\| \; \| \mathbf{w} \| = \sqrt{13} \; \| \mathbf{w} \| \Rightarrow \| \mathbf{w} \| \ge \frac{2}{\sqrt{13}}.$$

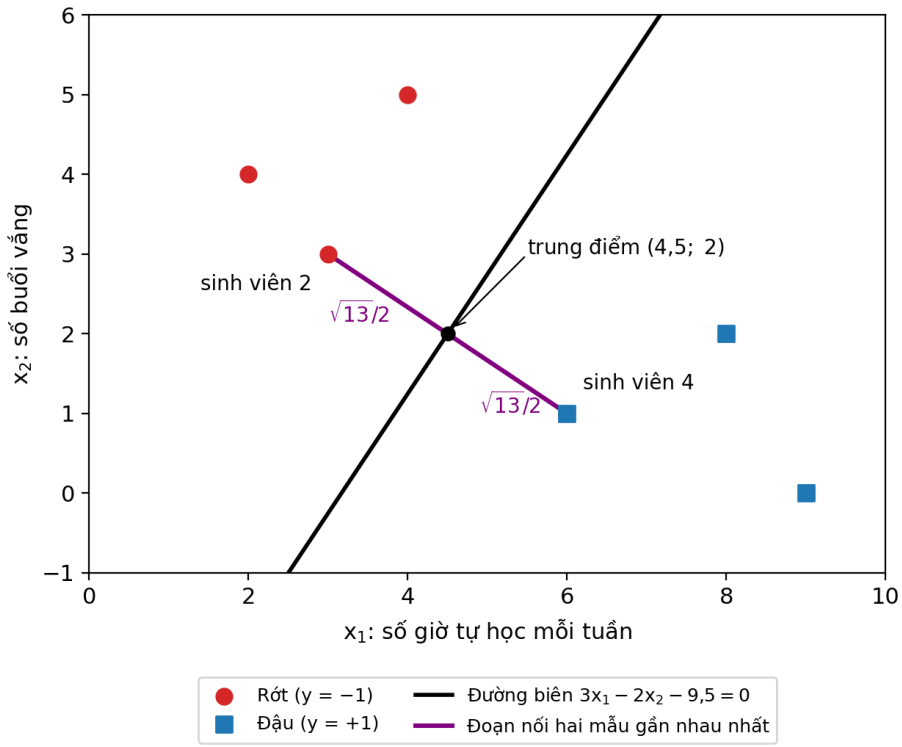
Dấu bằng xảy ra khi $\mathbf{w}$ cùng hướng với $[3 \quad -2]^T$, tức $\mathbf{w} = t\begin{bmatrix} 3 \\ -2 \end{bmatrix}$ với $t > 0$. Thay vào $3w_1 - 2w_2 = 2$ được $13t = 2$, nên $t = \frac{2}{13}$ và

$$\mathbf{w}^* = \begin{bmatrix} \frac{6}{13} \\ -\frac{4}{13} \end{bmatrix}, \qquad \| \mathbf{w}^* \|^2 = \frac{36 + 16}{169} = \frac{4}{13}.$$

Hệ số chệch lấy từ ràng buộc của vector hỗ trợ 4, $b = y_4 - \mathbf{x}_4\mathbf{w}^*$:

$$b^* = 1 - \left(6 \cdot \frac{6}{13} + 1 \cdot \left(-\frac{4}{13}\right)\right) = 1 - \frac{32}{13} = -\frac{19}{13}.$$

Kết quả này có một ý nghĩa hình học đẹp: vector pháp tuyến $\mathbf{w}^*$ cùng phương với đoạn thẳng nối hai vector hỗ trợ, $\mathbf{x}_4 - \mathbf{x}_2 = [3 \quad -2]$. Nghĩa là đường biên là **đường trung trực** của đoạn nối sinh viên 2 và 4, đi qua trung điểm $[4{,}5 \quad 2]$ (Hình 6). Kiểm tra: $3 \cdot 4{,}5 - 2 \cdot 2 - 9{,}5 = 0$.




*Hình 6. Đường biên lề cực đại $3x_1 - 2x_2 - 9{,}5 = 0$ là đường trung trực của đoạn nối hai sinh viên gần nhau nhất (sinh viên 2 và 4). Mỗi vector hỗ trợ cách đường biên $\sqrt{13}/2 \approx 1{,}80$.*

## Bước 3: kiểm tra bốn ràng buộc còn lại

Ta cần chắc rằng các sinh viên 1, 3, 5, 6 không vi phạm $m_i \geq 1$. Tính điểm số và margin bằng một phép nhân ma trận:

$$\mathbf{s} = \mathbf{X}\mathbf{w}^* + b^*\mathbf{1} = \frac{1}{13}\begin{bmatrix} 12-16-19 \\ 18-12-19 \\ 24-20-19 \\ 36-4-19 \\ 48-8-19 \\ 54-0-19 \end{bmatrix} = \frac{1}{13}\begin{bmatrix} -23 \\ -13 \\ -15 \\ 13 \\ 21 \\ 35 \end{bmatrix}, \qquad \mathbf{m} = \mathbf{y} \odot \mathbf{s} = \frac{1}{13}\begin{bmatrix} 23 \\ 13 \\ 15 \\ 13 \\ 21 \\ 35 \end{bmatrix}.$$

| Sinh viên | $y_i$ | $s_i$ | $m_i$ | Trạng thái |
|---|---|---|---|---|
| 1 | $-1$ | $-\frac{23}{13} \approx -1{,}77$ | 1,77 | Lỏng ($m_i > 1$) |
| 2 | $-1$ | $-1$ | 1 | **Chặt, vector hỗ trợ** |
| 3 | $-1$ | $-\frac{15}{13} \approx -1{,}15$ | 1,15 | Lỏng |
| 4 | $+1$ | $+1$ | 1 | **Chặt, vector hỗ trợ** |
| 5 | $+1$ | $\frac{21}{13} \approx 1{,}62$ | 1,62 | Lỏng |
| 6 | $+1$ | $\frac{35}{13} \approx 2{,}69$ | 2,69 | Lỏng |

Cả sáu ràng buộc đều thỏa, và đúng hai ràng buộc chặt (sinh viên 2 và 4) như giả thiết.

**Vì sao đây chắc chắn là nghiệm tối ưu của bài toán đầy đủ?** Bài toán ở Bước 2 chỉ có **hai** trong sáu ràng buộc, nên tập khả thi của nó **lớn hơn** tập khả thi của bài toán đầy đủ. Do đó giá trị tối thiểu của nó không thể lớn hơn giá trị tối thiểu của bài toán đầy đủ. Vì nghiệm tìm được ở Bước 2 lại thỏa cả bốn ràng buộc còn lại, nó cũng khả thi cho bài toán đầy đủ, nên đạt đúng giá trị tối thiểu đó. Vậy $(\mathbf{w}^*, b^*)$ là nghiệm tối ưu, và vì $\mathbf{w}$ duy nhất nên đây là nghiệm duy nhất.

Cũng có thể kiểm tra bằng một lập luận hình học độc lập. Với mọi bộ phân loại tách được và mọi cặp mẫu $\mathbf{p}$ (đậu), $\mathbf{q}$ (rớt), ta có $\| \mathbf{p} - \mathbf{q} \| \geq \frac{|(\mathbf{p}-\mathbf{q})\mathbf{w}|}{\|\mathbf{w}\|} = \frac{m_p + m_q}{\|\mathbf{w}\|} \geq 2\gamma$. Vậy lề không thể vượt quá một nửa khoảng cách giữa cặp gần nhất, tức $\gamma \leq \frac{\sqrt{13}}{2}$. Nghiệm của ta đạt đúng $\gamma = \sqrt{13}/2$ nên không thể cải thiện thêm.

## Bước 4: đọc kết quả

Từ $\| \mathbf{w}^* \| = 2/\sqrt{13}$, ta có lề $\gamma = \frac{1}{\|\mathbf{w}^*\|} = \frac{\sqrt{13}}{2} \approx 1{,}80$ và độ rộng dải lề $\frac{2}{\|\mathbf{w}^*\|} = \sqrt{13} \approx 3{,}61$. Ba đường của bộ phân loại là

$$\begin{aligned} \mathcal{H}_0&: \quad 6/13\, x_1 - 4/13\, x_2 - 19/13 = 0 \;\Leftrightarrow\; 3x_1 - 2x_2 - 9{,}5 = 0, \\ \mathcal{H}_+&: \quad 6/13\, x_1 - 4/13\, x_2 - 19/13 = +1 \;\Leftrightarrow\; 3x_1 - 2x_2 - 16 = 0, \\ \mathcal{H}_-&: \quad 6/13\, x_1 - 4/13\, x_2 - 19/13 = -1 \;\Leftrightarrow\; 3x_1 - 2x_2 - 3 = 0. \end{aligned}$$

Dễ kiểm tra sinh viên 4 $(6{,}1)$ thỏa phương trình của $\mathcal{H}_+$ $(18 - 2 - 16 = 0)$ và sinh viên 2 $(3{,}3)$ thỏa phương trình của $\mathcal{H}_-$ $(9 - 6 - 3 = 0)$.

**Dự đoán sinh viên mới.** Với sinh viên học 7 giờ, vắng 3,6 buổi ($\mathbf{x} = [7 \quad 3{,}6]$), mà bốn đường của Perceptron ở mục 1.1 đưa ra hai kết luận trái ngược:

$$s = 7 \cdot \frac{6}{13} - 3{,}6 \cdot \frac{4}{13} - \frac{19}{13} = \frac{42 - 14{,}4 - 19}{13} = \frac{8{,}6}{13} \approx 0{,}66 > 0 \;\Rightarrow\; \hat{y} = +1 \text{ (đậu)}.$$

Sinh viên này cách đường biên $0{,}66/\| \mathbf{w}^* \| \approx 1{,}19$, và có $|s| < 1$ nên nằm **bên trong dải lề**. Điều đó hoàn toàn hợp lệ: ràng buộc $|s| \geq 1$ chỉ áp dụng cho mẫu huấn luyện. Điểm quan trọng là SVM luôn cho **một** câu trả lời xác định, không phụ thuộc thứ tự duyệt mẫu như Perceptron.

### Kiểm tra bằng thư viện

Ta kiểm tra bằng scikit-learn. SVM lề cứng tương ứng với tham số $C$ rất lớn (ý nghĩa của $C$ được giải thích ở mục 3):

```
import numpy as np
from sklearn.svm import SVC

X = np.array([[2, 4], [3, 3], [4, 5], [6, 1], [8, 2], [9, 0]], dtype=float)
y = np.array([-1, -1, -1, 1, 1, 1])

svm = SVC(kernel="linear", C=1e8).fit(X, y)
print("w =", svm.coef_[0])             # [ 0.46153846 -0.30769231]  (= 6/13, -4/13)
print("b =", svm.intercept_[0])        # -1.46153846                (= -19/13)
print("vector hỗ trợ (sinh viên):", svm.support_ + 1)   # [2 4]
print("lề =", 1 / np.linalg.norm(svm.coef_[0]))          # 1.8027756   (= sqrt(13)/2)
```

Kết quả trùng khớp với phép tính tay.

### So sánh với Perceptron

Ta đặt đường lề cực đại cạnh các đường của Perceptron ở mục 1.3:

| Mô hình | Lề $\gamma$ | Số nghiệm có thể có |
|---|---|---|
| Perceptron (bốn đường A, B, C, D) | 1,26 đến 1,45 | Ít nhất 20 nghiệm khác nhau, tùy thứ tự duyệt |
| SVM lề cứng | $\mathbf{1,80}$ | **Đúng một** nghiệm |

SVM không chỉ cho lề lớn hơn mà còn **lớn nhất có thể**, theo đúng bài toán ta đã đặt ra ở mục 1.

## Giới hạn của lề cứng và hướng tiếp theo

Mô hình lề cứng đẹp về mặt toán học nhưng có hai hạn chế đã thấy ngay trong mục này:

1. **Chỉ áp dụng được khi dữ liệu phân tách tuyến tính được.** Chỉ cần thêm một sinh viên "ngoại lệ" như sinh viên 7 ở mục 2.2 thì bài toán vô nghiệm.
2. **Rất nhạy với các mẫu sát biên hoặc ngoại lai.** Vì đường biên do vector hỗ trợ quyết định, một mẫu nhiễu nằm gần phía bên kia có thể kéo đường biên đi rất xa, và làm lề thu hẹp đáng kể. Hình 5(c) cho thấy chỉ bỏ một vector hỗ trợ đã làm đường biên xoay mạnh.

Mục 3 giải quyết cả hai bằng cách cho phép một số mẫu **lấn vào dải lề hoặc bị phân loại sai** với một mức phạt, tạo ra mô hình **lề mềm** và hàm mất mát hinge. Mục 4 quay lại giải thích bằng dạng đối ngẫu vì sao nghiệm chỉ phụ thuộc vào các vector hỗ trợ.

## Tóm tắt mục 2

1. Vì $(\mathbf{w}, b)$ và $(c\mathbf{w}, cb)$ cho cùng đường biên, ta **chốt thang đo** bằng dạng chuẩn tắc $\min_i y_i(\mathbf{x}_i\mathbf{w} + b) = 1$. Khi đó hai biên lề là $\mathbf{xw} + b = \pm 1$, lề là $\gamma = 1/\| \mathbf{w} \|$ và độ rộng dải lề là $2/\| \mathbf{w} \|$.

2. Bài toán maxmin $m_i/\| \mathbf{w} \|$ tương đương với **bài toán gốc** $\min \frac{1}{2} \| \mathbf{w} \|^2$ với ràng buộc $y_i(\mathbf{x}_i\mathbf{w} + b) \geq 1$. Đây là bài toán quy hoạch toàn phương lồi, có nghiệm khi và chỉ khi dữ liệu phân tách được, và cho **w duy nhất**.

3. **Vector hỗ trợ** là các mẫu nằm đúng trên hai biên lề ($m_i = 1$). Mỗi lớp có ít nhất một vector hỗ trợ, $b^* = y_k - \mathbf{x}_k\mathbf{w}^*$ với mọi $k$ là vector hỗ trợ, và **bỏ các mẫu không phải vector hỗ trợ không làm đổi nghiệm**.

4. Ở dạng ma trận, bài toán là $\min \frac{1}{2}\mathbf{u}^T\mathbf{P}\mathbf{u}$ với $\mathbf{Gu} \geq \mathbf{1}$, trong đó $\mathbf{u} = \begin{bmatrix}\mathbf{w} \\ b\end{bmatrix}$, $\mathbf{G} = [\text{diag}(\mathbf{y})\mathbf{X} \quad \mathbf{y}]$.

5. Trên dữ liệu 6 sinh viên, nghiệm là $\mathbf{w}^* = \begin{bmatrix}\frac{6}{13} & -\frac{4}{13}\end{bmatrix}^T$, $b^* = -\frac{19}{13}$, lề $\frac{\sqrt{13}}{2} \approx 1{,}80$, và chỉ có **hai vector hỗ trợ** (sinh viên 2 và 4).

6. Lề cứng không dùng được khi dữ liệu chồng lấn hoặc có ngoại lai. Đó là động lực của lề mềm ở mục 3.

## Câu hỏi kiểm tra nhanh

1. Với nghiệm ở mục 2.5, hãy tính margin $m_3$ của sinh viên 3 (học 4 giờ, vắng 5 buổi, rớt). Sinh viên 3 có phải vector hỗ trợ không?

2. Nếu ta chốt thang đo bằng điều kiện $\min_i m_i = 2$ thay vì 1, thì $\mathbf{w}$, $b$ của đường biên ở mục 2.5 thay đổi thế nào? Độ rộng dải lề giữa hai biên $\mathbf{xw} + b = \pm 2$ có đổi không?

3. Cho dữ liệu một chiều gồm hai mẫu: $x_1 = 1$ với $y_1 = -1$ và $x_2 = 3$ với $y_2 = +1$. Hãy viết ma trận $\mathbf{G}$, rồi giải bài toán SVM lề cứng để tìm $w$, $b$, đường biên và độ rộng lề.

4. Nếu bỏ sinh viên 5 (học 8 giờ, vắng 2 buổi) khỏi dữ liệu 6 sinh viên, đường biên SVM có thay đổi không? Giải thích bằng định nghĩa vector hỗ trợ.

5. Giải thích vì sao thêm sinh viên thứ 7 với $\mathbf{x}_7 = [7 \quad 1]$, $y_7 = -1$ làm bài toán SVM lề cứng vô nghiệm, trong khi Perceptron không dừng và Logistic Regression vẫn huấn luyện được (dù không cho nghiệm phân loại đúng hết).

## Gợi ý đáp án

1. $s_3 = \frac{1}{13}(24 - 20 - 19) = -\frac{15}{13}$, nên $m_3 = y_3 s_3 = \frac{15}{13} \approx 1{,}15 > 1$. Ràng buộc lỏng nên sinh viên 3 **không** phải vector hỗ trợ.

2. Khi chốt thang đo ở mức 2, ta nhân đôi tham số: $\mathbf{w}' = 2\mathbf{w}^* = \left[\frac{12}{13} \quad -\frac{8}{13}\right]^T$, $b' = 2b^* = -\frac{38}{13}$. Đường biên $\mathbf{x}\mathbf{w}' + b' = 0$ không đổi. Khoảng cách giữa $\mathbf{x}\mathbf{w}' + b' = \pm 2$ là $\frac{2 \cdot 2}{\|\mathbf{w}\prime\|} = \frac{4}{2\|\mathbf{w}^*\|} = \frac{2}{\|\mathbf{w}^*\|} = \sqrt{13}$, **không đổi**. Thang đo chỉ là quy ước, hình học không thay đổi.

3. $\mathbf{Z} = \text{diag}(\mathbf{y})\mathbf{X} = \begin{bmatrix} -1 \\ 3 \end{bmatrix}$ và $\mathbf{G} = \begin{bmatrix} -1 & -1 \\ 3 & 1 \end{bmatrix}$. Hai ràng buộc là $-w - b \geq 1$ và $3w + b \geq 1$. Cả hai mẫu là vector hỗ trợ nên dấu bằng xảy ra: $w + b = -1$ và $3w + b = 1$, suy ra $w = 1$, $b = -2$. Đường biên là $x - 2 = 0$, tức $x = 2$ (trung điểm của 1 và 3). Độ rộng lề là $\frac{2}{|w|} = 2$, đúng bằng khoảng cách giữa hai mẫu.

4. **Không đổi.** Sinh viên 5 có $m_5 = \frac{21}{13} > 1$ nên không phải vector hỗ trợ, và theo mệnh đề ở mục 2.3, bỏ một mẫu như vậy không làm đổi $\mathbf{w}^*$ (và do đó cũng không đổi $b^*$ tính từ vector hỗ trợ).

5. Sinh viên 7 là tổ hợp lồi của ba sinh viên đậu $(0{,}6;\ 0{,}2;\ 0{,}2)$. Vì điểm số là hàm affine nên $s_7$ là cùng tổ hợp đó của $s_4, s_5, s_6$; nếu ba sinh viên đậu có $s \geq 1$ thì $s_7 \geq 1$, mâu thuẫn với yêu cầu $s_7 \leq -1$ của một mẫu rớt. Do đó không có $(\mathbf{w}, b)$ nào thỏa cả bảy ràng buộc và tập khả thi rỗng. Perceptron không dừng vì luôn còn mẫu sai. Logistic Regression vẫn tối thiểu hóa được cross-entropy (hàm này luôn xác định) và cho một nghiệm hữu hạn vì dữ liệu không còn phân tách hoàn hảo, nhưng sẽ vẫn phân loại sai ít nhất một mẫu.

# 3. SVM lề mềm và huấn luyện bằng hàm mất mát hinge

Ở mục 2, ta đã xây dựng SVM lề cứng: tìm đường biên có lề lớn nhất với ràng buộc **mọi** mẫu huấn luyện phải thỏa $y_i(\mathbf{x}_i\mathbf{w} + b) \geq 1$. Mô hình này đẹp và cho nghiệm duy nhất, nhưng nó có một điều kiện sống còn: dữ liệu phải phân tách tuyến tính được. Chỉ cần một sinh viên "ngoại lệ" như sinh viên 7 ở mục 2.2 là bài toán vô nghiệm. Dữ liệu thực tế hầu như luôn có nhiễu, nhãn sai hoặc hai lớp chồng lấn, nên ta cần một mô hình **chịu được vi phạm**.

Mục này nới lỏng ràng buộc của lề cứng bằng cách cho phép một số mẫu **lấn vào dải lề hoặc nằm sai phía**, với một mức phạt tỷ lệ với mức độ vi phạm. Kết quả là **SVM lề mềm (soft-margin SVM)**. Điều đáng chú ý là bài toán lề mềm, sau khi khử các biến phụ, trở thành **cực tiểu hóa một hàm mất mát không ràng buộc** gồm hàm mất mát **hinge** cộng với điều chuẩn L2. Đó chính là cùng khuôn khổ "hàm mất mát cộng điều chuẩn, huấn luyện bằng gradient" mà ta đã dùng cho Perceptron và Logistic Regression, chỉ khác ở hàm mất mát. Trong mục 3, ta trả lời các câu hỏi:

1. Vì sao lề cứng không đủ, và lề mềm cho phép vi phạm như thế nào?
2. Hàm mất mát hinge là gì, và nó khác mất mát của Perceptron và Logistic Regression ở điểm nào?
3. Gradient (đúng hơn là dưới đạo hàm) của hàm mục tiêu là gì, và quy tắc cập nhật đối chiếu với Perceptron ra sao?
4. Tham số $C$, tốc độ học và việc chuẩn hóa đặc trưng ảnh hưởng thế nào đến huấn luyện?
5. Thuật toán chạy từng bước ra sao, và cài đặt từ đầu bằng Python như thế nào?

**Dữ liệu xuyên suốt.** Ta thêm vào bộ 6 sinh viên một sinh viên thứ 7 như ở mục 2.2: $\mathbf{x}_7 = [7 \quad 1]$ nhưng bị rớt ($y_7 = -1$). Sinh viên này nằm lẫn giữa các sinh viên đậu nên dữ liệu **không còn phân tách tuyến tính được**. Với $n = 7$ và $d = 2$:

$$\mathbf{X} = \begin{bmatrix} 2 & 4 \\ 3 & 3 \\ 4 & 5 \\ 6 & 1 \\ 8 & 2 \\ 9 & 0 \\ 7 & 1 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} -1 \\ -1 \\ -1 \\ +1 \\ +1 \\ +1 \\ -1 \end{bmatrix}.$$

Ký hiệu giữ nguyên như các mục trước: $s_i = \mathbf{x}_i\mathbf{w} + b$, margin $m_i = y_i s_i$, $\mathbf{m} = \mathbf{y} \odot (\mathbf{X}\mathbf{w} + b\mathbf{1})$. Số thứ tự hình tiếp tục từ mục 2.

## 3.1. Vì sao cần lề mềm: dữ liệu chồng lấn và nhiễu

Lề cứng bộc lộ hai điểm yếu, cả hai đã thấy ở mục 2:

**Thứ nhất, nó không tồn tại khi dữ liệu không phân tách được.** Với 7 sinh viên, sinh viên 7 là tổ hợp lồi của ba sinh viên đậu nên không có đường thẳng nào phân loại đúng cả bảy. Tập khả thi của bài toán lề cứng rỗng. Hai nguyên nhân phổ biến của tình huống này là:

1. **Chồng lấn tự nhiên giữa hai lớp**: hai nhóm sinh viên thật sự có những người học ít vẫn đậu, học nhiều vẫn rớt, vì còn các yếu tố không nằm trong đặc trưng.
2. **Nhiễu nhãn hoặc nhiễu đo đạc**: một sinh viên bị ghi nhầm nhãn, hoặc số giờ học khai báo sai.

**Thứ hai, ngay cả khi dữ liệu phân tách được, lề cứng rất nhạy với mẫu sát biên hoặc ngoại lai.** Vì đường biên chỉ do vài vector hỗ trợ quyết định, một mẫu lạ nằm gần phía bên kia có thể kéo đường biên đi xa và làm lề thu hẹp mạnh. Ta đã thấy ở Hình 5(c) rằng chỉ cần bỏ một vector hỗ trợ, đường biên đã xoay đáng kể. Lề cứng bắt đường biên "chiều theo" từng mẫu, kể cả mẫu nhiễu.

Điều ta muốn là một sự **đánh đổi có kiểm soát** giữa hai mục tiêu thường mâu thuẫn nhau:

| Mục tiêu | Ý nghĩa | Nếu chỉ theo đuổi mục tiêu này |
|---|---|---|
| Lề rộng | $\|\| \mathbf{w} \|\|$ nhỏ, đường biên đơn giản và ổn định | Nhiều mẫu lấn lề hoặc sai |
| Ít vi phạm | Mọi mẫu nằm đúng phía và ngoài lề | Lề hẹp, quá khớp với nhiễu |

Lề mềm cho phép ta chọn điểm cân bằng giữa hai mục tiêu này bằng một tham số duy nhất.

## 3.2. Biến lỏng $\xi_i$ và tham số $C$

### Nới lỏng ràng buộc bằng biến lỏng

Ràng buộc của lề cứng là $y_i(\mathbf{x}_i\mathbf{w} + b) \geq 1$. Ta cho mỗi mẫu một "khoảng nới" không âm $\xi_i \geq 0$, gọi là **biến lỏng (slack variable)**:

$$y_i(\mathbf{x}_i\mathbf{w} + b) \;\geq\; 1 - \xi_i, \qquad \xi_i \geq 0, \qquad i = 1, \dots, n.$$

Nếu $\xi_i = 0$ ta quay lại ràng buộc cứng. Nếu $\xi_i > 0$, mẫu $i$ được phép có margin nhỏ hơn 1, và $\xi_i$ chính là mức vi phạm. Có ba trường hợp, tùy margin $m_i$ của mẫu:

| Vị trí của mẫu | Margin | Biến lỏng nhỏ nhất | Ý nghĩa |
|---|---|---|---|
| Ngoài dải lề, đúng phía | $m_i \geq 1$ | $\xi_i = 0$ | Không vi phạm |
| Trong dải lề, vẫn đúng phía | $0 \leq m_i < 1$ | $0 < \xi_i \leq 1$ | Đúng nhưng chưa đủ xa đường biên |
| Sai phía | $m_i < 0$ | $\xi_i > 1$ | Bị phân loại sai |

Về mặt hình học, mẫu vi phạm cách biên lề của lớp mình một đoạn $\xi_i / \|\| \mathbf{w} \|\|$. Một hệ quả hữu ích: mọi mẫu bị phân loại sai có $\xi_i > 1$, nên

$$\#\{i:\ m_i < 0\} \ \le\ \sum_{i=1}^{n} \xi_i.$$

Tổng các biến lỏng là **cận trên của số mẫu bị phân loại sai** trên tập huấn luyện, và cũng là thước đo tổng mức vi phạm.

## Bài toán gốc của SVM lề mềm

Ta vẫn muốn lề rộng (tối thiểu $\frac{1}{2} \| \mathbf{w} \|^2$), nhưng bây giờ phải trả giá cho mỗi vi phạm. Với hệ số phạt $C > 0$, bài toán gốc của SVM lề mềm là

$$\boxed{\min_{\mathbf{w}, b, \boldsymbol{\xi}} \frac{1}{2} \| \mathbf{w} \|^2 + C \sum_{i=1}^{n} \xi_i \quad \text{với} \quad y_i(\mathbf{x}_i \mathbf{w} + b) \ge 1 - \xi_i,\ \ \xi_i \ge 0.}$$

Ở dạng ma trận, gọi $\boldsymbol{\xi}$ là vector cột $n \times 1$ gồm các biến lỏng, và dùng $\mathbf{u}$, $\mathbf{P}$, $\mathbf{G}$ như ở mục 2.4:

$$\min_{\mathbf{u}, \boldsymbol{\xi}} \frac{1}{2} \mathbf{u}^T \mathbf{P}\, \mathbf{u} + C\, \mathbf{1}^T \boldsymbol{\xi} \qquad \text{với} \qquad \mathbf{G}\, \mathbf{u} \ \ge\ \mathbf{1} - \boldsymbol{\xi}, \quad \boldsymbol{\xi} \ge \mathbf{0}.$$

Đây vẫn là bài toán quy hoạch toàn phương lồi, nay có $d + 1 + n$ biến. Khác với lề cứng, **tập khả thi luôn khác rỗng**: với bất kỳ $(\mathbf{w}, b)$ nào ta chọn $\xi_i = \max(0, 1 - m_i)$ là thỏa. Vì vậy bài toán lề mềm luôn có nghiệm, kể cả khi dữ liệu chồng lấn.

## Tham số $C$: cán cân giữa lề rộng và ít vi phạm

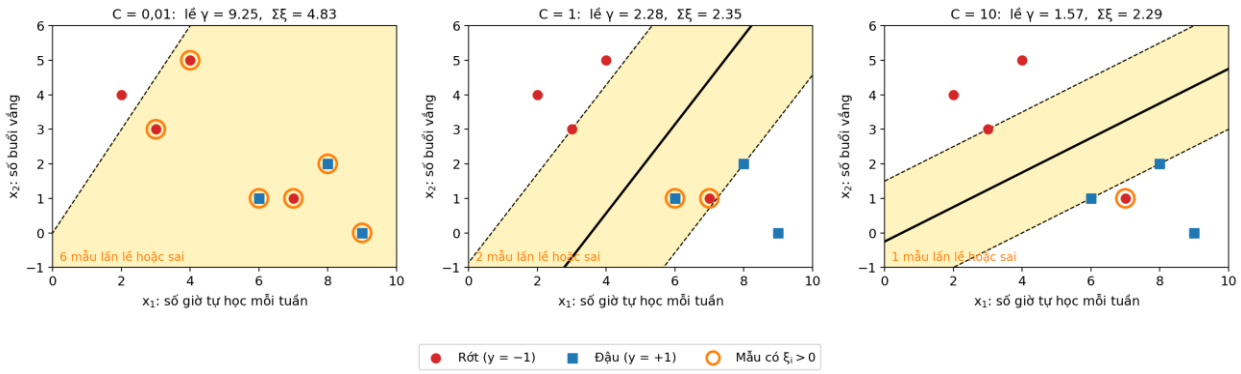
Tham số $C$ cân bằng hai số hạng của hàm mục tiêu:

1. $\frac{1}{2} \| \mathbf{w} \|^2$ ứng với **lề rộng** (nhớ rằng lề là $1/\| \mathbf{w} \|$).
2. $C \sum_i \xi_i$ ứng với **tổng vi phạm**.

Hai trường hợp giới hạn giúp hiểu $C$:

3. $C$ **rất lớn**: mỗi đơn vị vi phạm bị phạt rất nặng, mô hình cố gắng để $\sum \xi_i$ nhỏ nhất có thể, nên tiến gần SVM lề cứng. Khi dữ liệu phân tách được và $C$ đủ lớn, nghiệm trùng với nghiệm lề cứng ở mục 2.
4. $C$ **rất nhỏ**: vi phạm gần như "miễn phí", mô hình ưu tiên $\| \mathbf{w} \|$ nhỏ, tức lề rất rộng, và chấp nhận nhiều mẫu lấn lề hoặc sai.

Hình 8 minh họa ba giá trị $C$ trên dữ liệu 7 sinh viên. Phần 3.5 sẽ phân tích kỹ hơn.




*Hình 8. SVM lề mềm trên 7 sinh viên với $C = 0{,}01$, $C = 1$ và $C = 10$. $C$ nhỏ cho lề rất rộng và nhiều mẫu vi phạm; $C$ lớn cho lề hẹp hơn và ít mẫu vi phạm hơn. Vòng cam đánh dấu các mẫu có $\xi_i > 0$ (lấn lề hoặc sai phía). Sinh viên 7 luôn bị phân loại sai với $C \geq 0{,}1$ vì dữ liệu không phân tách được.*

## 3.3. Hàm mất mát hinge: dạng không ràng buộc và so sánh với mất mát Perceptron, Logistic theo margin

### Khử biến lỏng

Bài toán lề mềm có ràng buộc, nhưng các ràng buộc có cấu trúc rất đơn giản. Với $(\mathbf{w}, b)$ cố định, mỗi $\xi_i$ chỉ cần thỏa $\xi_i \geq 1 - m_i$ và $\xi_i \geq 0$, đồng thời hàm mục tiêu **tăng** theo $\xi_i$ (vì $C > 0$). Do đó giá trị tối ưu của $\xi_i$ là giá trị nhỏ nhất có thể:

$$\xi_i = \max(0,\ 1 - m_i).$$

Biểu thức này chính là **hàm mất mát hinge** của mẫu $i$:

$$\ell_{\text{hinge}}(m) = \max(0,\ 1 - m), \qquad m = y(\mathbf{x}\mathbf{w} + b).$$

Thay $\xi_i$ vào hàm mục tiêu, ta thu được bài toán **không ràng buộc** theo $(\mathbf{w}, b)$:

$$\boxed{J(\mathbf{w}, b) = \frac{1}{2} \| \mathbf{w} \|^2 + C \sum_{i=1}^{n} \max\big(0,\ 1 - y_i(\mathbf{x}_i\mathbf{w} + b)\big).}$$

Ở dạng vector, với $\mathbf{m} = \mathbf{y} \odot (\mathbf{X}\mathbf{w} + b\mathbf{1})$ và hàm max lấy theo từng phần tử,

$$J(\mathbf{w}, b) = \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\ \mathbf{1}^T \max(\mathbf{0},\ \mathbf{1} - \mathbf{m}).$$

Chia $J$ cho $nC$, ta được dạng tương đương thường dùng trong các thuật toán SGD:

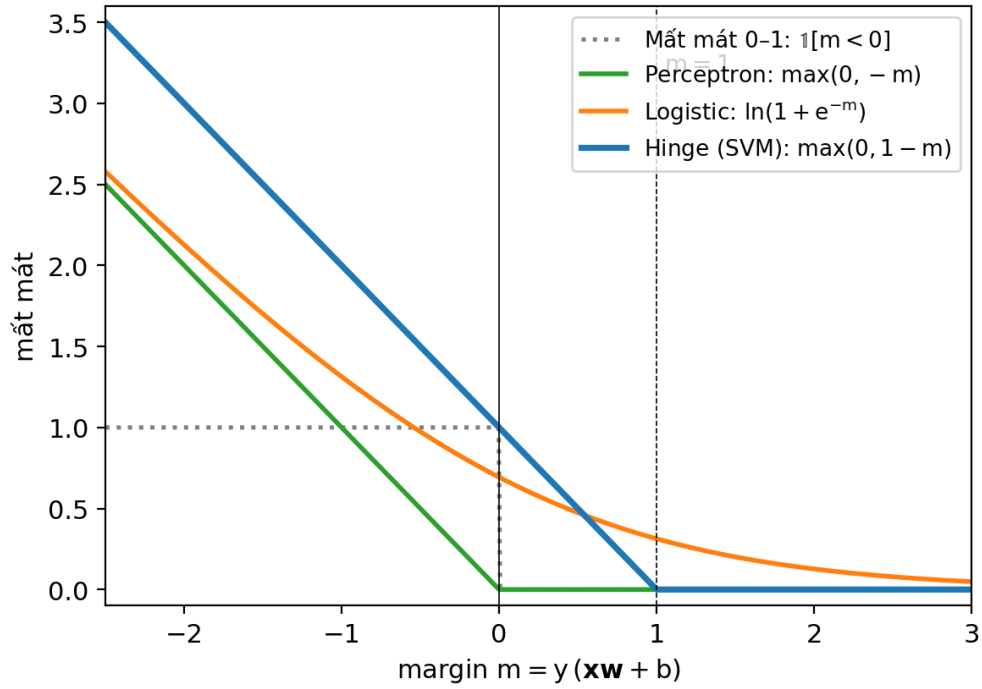
$$\frac{J}{nC} = \frac{\lambda}{2} \| \mathbf{w} \|^2 + \frac{1}{n} \sum_{i=1}^{n} \max(0,\ 1 - m_i), \qquad \lambda = \frac{1}{nC}.$$

Dạng này giống hệt hàm mục tiêu của Logistic Regression có điều chuẩn L2 ở bài trước, "trung bình mất mát cộng $\frac{\lambda}{2} \| \mathbf{w} \|^2$", chỉ khác ở **hàm mất mát**. Hệ số $C$ lớn tương ứng $\lambda$ nhỏ, tức điều chuẩn yếu.

## So sánh ba hàm mất mát theo margin

Cả ba mô hình đều có mất mát là hàm của margin $m = y\,s$. Bảng sau cho giá trị tại vài mức margin tiêu biểu:

| Margin $m$ | Tình huống | Perceptron $\max(0, -m)$ | Hinge $\max(0, 1-m)$ | Logistic $\ln(1+e^{-m})$ |
|---|---|---|---|---|
| $-2$ | Sai nặng | 2 | 3 | 2,127 |
| $-1$ | Sai | 1 | 2 | 1,313 |
| 0 | Đúng trên đường biên | 0 | 1 | 0,693 |
| 0,5 | Đúng, nằm trong dải lề | 0 | 0,5 | 0,474 |
| 1 | Đúng trên biên lề | 0 | 0 | 0,313 |
| 2 | Đúng, xa đường biên | 0 | 0 | 0,127 |




*Hình 7. Ba hàm mất mát theo margin $m$, so với mất mát 0–1 (đếm số mẫu sai). Hinge là hàm lồi, bằng $0$ khi $m \geq 1$ và tăng tuyến tính khi $m < 1$. Hinge và Logistic đều là cận trên của mất mát 0–1 (với Logistic dùng logarit cơ số $e$ thì cận này đúng sau khi nhân với $1/\ln 2$), còn mất mát Perceptron thì không.*

Từ bảng và Hình 7, ta rút ra những khác biệt cốt lõi:

1. **Perceptron và hinge chỉ khác nhau ở ngưỡng.** Mất mát Perceptron $\max(0, -m)$ bằng 0 ngay khi mẫu được phân loại đúng ($m \geq 0$), nên nó "thỏa mãn" khi mẫu chỉ vừa đúng phía. Hinge $\max(0, 1-m)$ dời ngưỡng từ 0 lên 1: mẫu đúng nhưng nằm trong dải lề ($0 \leq m < 1$) **vẫn bị phạt**. Chính bước dời ngưỡng này, kết hợp với $\frac{1}{2} \| \mathbf{w} \|^2$, tạo ra hiệu ứng đẩy đường biên ra xa mẫu, tức là tạo ra **lề**.

2. **Hinge bằng đúng 0 với mọi mẫu có $m \geq 1$.** Mẫu đúng và nằm ngoài dải lề không đóng góp gì vào mất mát, cũng không đóng góp vào gradient. Đây là nguồn gốc của **tính thưa**: chỉ các mẫu có $m_i \leq 1$ (vector hỗ trợ và mẫu vi phạm) mới ảnh hưởng nghiệm. Logistic Regression thì ngược lại: $\ln(1 + e^{-m}) > 0$ với mọi $m$, nên mọi mẫu, kể cả mẫu rất xa, đều đóng góp một chút.

3. **Hinge tăng tuyến tính khi $m \to -\infty$**, giống Perceptron và Logistic (Logistic cũng tiệm cận tuyến tính). Một mẫu bị sai rất nặng không bị phạt bình phương như trong MSE, nên ảnh hưởng của ngoại lai có giới hạn.

4. **Hinge không trơn tại $m = 1$** (có "góc nhọn"), trong khi Logistic trơn mọi nơi. Vì vậy ta không có gradient thông thường tại $m = 1$ mà phải dùng dưới đạo hàm, như ở mục 3.4. Ngược lại, hinge không cho xác suất như Logistic (xem mục 7.2).

5. **Cả ba đều là hàm lồi theo $(\mathbf{w}, b)$** (hàm lồi của một hàm affine), nên $J$ là hàm lồi, và lồi chặt theo $\mathbf{w}$ nhờ số hạng $\frac{1}{2} \| \mathbf{w} \|^2$. Mọi điểm dừng của thuật toán dưới đạo hàm ở mức tối ưu đều là tối ưu toàn cục.

**Lề cứng là trường hợp giới hạn.** Khi $C \to \infty$, để $J$ hữu hạn ta buộc $\sum_i \max(0, 1 - m_i) = 0$, tức mọi $m_i \geq 1$, và phần còn lại là cực tiểu hóa $\frac{1}{2} \| \mathbf{w} \|^2$. Đó chính là bài toán lề cứng ở mục 2. Như vậy hai mục liên kết với nhau một cách liền mạch: lề cứng yêu cầu hinge bằng 0 tuyệt đối, còn lề mềm cho phép hinge dương với một mức giá $C$.

## 3.4. Gradient (dưới đạo hàm) và quy tắc cập nhật, đối chiếu với quy tắc Perceptron

### Dưới đạo hàm của hinge

Hàm $\max(0, 1-m)$ không khả vi tại $m = 1$, nhưng tại điểm đó ta vẫn chọn được một **dưới đạo hàm (subgradient)**, tức một "hướng dốc hợp lệ". Với mẫu $i$, hàm mất mát $h_i = \max(0, 1 - m_i)$ có dưới đạo hàm theo $\mathbf{w}$ và $b$ như sau:

$$\partial_{\mathbf{w}} h_i = \begin{cases} -y_i \mathbf{x}_i^T & \text{nếu } m_i < 1, \\ \mathbf{0} & \text{nếu } m_i \geq 1, \end{cases} \qquad \partial_b h_i = \begin{cases} -y_i & \text{nếu } m_i < 1, \\ 0 & \text{nếu } m_i \geq 1. \end{cases}$$

(Tại $m_i = 1$ mọi giá trị giữa $-y_i \mathbf{x}_i^T$ và $\mathbf{0}$ đều là dưới đạo hàm hợp lệ, ta chọn $\mathbf{0}$.) Đặt **vector chỉ báo a** cỡ $n \times 1$ với

$$a_i = \mathbb{1}[m_i < 1] = \begin{cases} 1 & \text{nếu mẫu } i \text{ vi phạm lề (lấn lề hoặc sai),} \\ 0 & \text{nếu mẫu } i \text{ nằm ngoài dải lề.} \end{cases}$$

Gộp qua $n$ mẫu, dưới đạo hàm của $J(\mathbf{w}, b) = \frac{1}{2} \| \mathbf{w} \|^2 + C \sum_i h_i$ là

$$\nabla_{\mathbf{w}} J = \mathbf{w} - C \sum_{i=1}^{n} a_i \, y_i \, \mathbf{x}_i^T = \mathbf{w} - C \, \mathbf{X}^T (\mathbf{y} \odot \mathbf{a}), \qquad \frac{\partial J}{\partial b} = -C \sum_{i=1}^{n} a_i \, y_i = -C \, \mathbf{y}^T \mathbf{a}.$$

Chú ý rằng chỉ các mẫu có $a_i = 1$ góp mặt trong tổng. Các mẫu ngoài dải lề không ảnh hưởng gì đến gradient.

## Quy tắc cập nhật

Với tốc độ học $\eta > 0$, một bước của **Gradient Descent dưới đạo hàm** là

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \big( \mathbf{w} - C \, \mathbf{X}^T (\mathbf{y} \odot \mathbf{a}) \big) = (1 - \eta) \, \mathbf{w} + \eta C \, \mathbf{X}^T (\mathbf{y} \odot \mathbf{a}), \qquad b \leftarrow b + \eta C \, \mathbf{y}^T \mathbf{a}.$$

Cập nhật của $\mathbf{w}$ gồm hai thành phần có ý nghĩa rõ ràng:

1. $(1 - \eta)\mathbf{w}$: **co** $\mathbf{w}$ về $\mathbf{0}$ ở mỗi bước (còn gọi là weight decay). Đây là tác dụng của $\frac{1}{2} \| \mathbf{w} \|^2$, giữ $\| \mathbf{w} \|$ nhỏ, tức giữ lề rộng.
2. $\eta C \, \mathbf{X}^T (\mathbf{y} \odot \mathbf{a}) = \eta C \sum_{i: m_i < 1} y_i \, \mathbf{x}_i^T$: **đẩy** $\mathbf{w}$ về phía $y_i \mathbf{x}_i^T$ của mỗi mẫu vi phạm, để nâng margin của chúng lên.

## Đối chiếu với Perceptron và Logistic Regression

Nhớ lại quy tắc cập nhật của Perceptron trên một mẫu bị sai: $\mathbf{w} \leftarrow \mathbf{w} + \eta \, y_i \mathbf{x}_i^T$. Hình thức "cộng $y_i \mathbf{x}_i^T$ khi mẫu có vấn đề" giống hệt với thành phần đẩy ở trên. Logistic Regression cũng cộng $y_i \mathbf{x}_i^T$ nhưng nhân với một trọng số trơn. Thật vậy, đạo hàm của $\ln(1 + e^{-m})$ theo $m$ là $-\sigma(-m)$, nên gradient của Logistic là $-\sum_i \sigma(-m_i) \, y_i \mathbf{x}_i^T$ với $\sigma(-m) = 1/(1 + e^m)$. Ba mô hình có cùng một dạng cập nhật

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \sum_{i=1}^{n} \varphi(m_i) \, y_i \mathbf{x}_i^T \quad \big( - \text{co } \mathbf{w} \text{ nếu có điều chuẩn} \big),$$

chỉ khác nhau ở **trọng số** $\varphi(m_i)$ của mẫu $i$:

| | Perceptron | Logistic Regression | SVM lề mềm (hinge) |
|---|---|---|---|
| Trọng số $\varphi(m)$ | $\mathbb{1}[m < 0]$ | $\sigma(-m) = \dfrac{1}{1 + e^m}$ | $\mathbb{1}[m < 1]$ |

| | Perceptron | Logistic Regression | SVM lề mềm (hinge) |
|---|---|---|---|
| Mẫu nào được dùng | Chỉ mẫu **bị sai** | **Mọi** mẫu (mẫu càng sai, trọng số càng gần 1) | Mẫu **sai hoặc lấn lề** |
| Dạng trọng số | Bậc thang tại 0 | Trơn, giảm dần | Bậc thang tại 1 |
| Điều chuẩn / co **w** | Không | Tùy chọn (L2) | Có sẵn $(1-\eta)\mathbf{w}$ |

Bảng này tóm tắt toàn bộ ba bài học: SVM là Perceptron với **ngưỡng dời từ** 0 **lên** 1 và **có co w**, hoặc là Logistic Regression với **trọng số bậc thang thay cho sigmoid**.

### Lưu ý về tính chất của thuật toán

Vì $J$ không khả vi tại $m_i = 1$ nên Gradient Descent dưới đạo hàm **không đảm bảo giá trị** $J$ **giảm ở mỗi bước**. Nếu dùng tốc độ học hằng số, $J$ dao động quanh giá trị tối ưu. Để hội tụ, ta dùng tốc độ học **giảm dần**. Do $J$ là hàm lồi chặt với hệ số 1 (nhờ $\frac{1}{2} \| \mathbf{w} \|^2$), lựa chọn $\eta_t = 1/t$ cho đảm bảo hội tụ, như sẽ thấy ở mục 3.6.

## 3.5. Vai trò của $C$, learning rate và chuẩn hóa đặc trưng

### Vai trò của $C$

Ta giải bài toán lề mềm trên 7 sinh viên với các giá trị $C$ khác nhau (nghiệm chính xác, tính bằng bộ giải QP của scikit-learn):

| $C$ | $\mathbf{w}^T$ | $b$ | Lề $\gamma = 1/\| \mathbf{w} \|$ | $\sum \xi_i$ | Mẫu có $\xi_i > 0$ | Mẫu sai ($m_i < 0$) | Sinh viên mới (7; 3,6) |
|---|---|---|---|---|---|---|---|
| 0,01 | [0,09 −0,06] | −1,00 | 9,25 | 4,83 | 6 | 3 | Rớt |
| 0,1 | [0,36 −0,18] | −1,54 | 2,49 | 2,38 | 3 | 1 | Đậu |
| 1 | [0,35 −0,27] | −1,23 | 2,28 | 2,35 | 2 | 1 | Đậu |
| 10 | [0,29 −0,57] | −0,14 | 1,57 | 2,29 | 1 | 1 | Rớt |

Từ bảng ta rút ra các quan sát sau.

1. $C$ **tăng thì lề** $\gamma$ **giảm và tổng vi phạm** $\sum \xi_i$ **giảm.** Đây đúng là cán cân giữa hai số hạng của $J$: $C$ nhỏ ưu tiên lề rộng ($\gamma = 9{,}25$, một dải lề rộng đến mức bao trùm hầu hết dữ liệu và 3 mẫu bị sai), $C$ lớn ưu tiên ít vi phạm (chỉ còn 1 mẫu có $\xi_i > 0$).
2. **Dữ liệu không phân tách nên** $C \to \infty$ **không đưa về lề cứng** (lề cứng vô nghiệm). Với $C \geq 10$ nghiệm không đổi nữa: mô hình đã tối thiểu hóa $\sum \xi_i$ đến mức thấp nhất có thể, và sinh viên 7 vẫn là mẫu sai duy nhất.

3. **Dự đoán cho sinh viên mới phụ thuộc vào $C$** (rớt, đậu, đậu, rớt). Vì vậy $C$ là siêu tham số cần chọn cẩn thận, bằng kiểm định chéo ở mục 7.5, chứ không phải một hằng số tùy ý.

4. **$C$ nhỏ tương ứng điều chuẩn mạnh** theo quan hệ $\lambda = 1/(nC)$ ở mục 3.3, và đây cũng là cách đọc quen thuộc từ Logistic Regression: điều chuẩn mạnh làm $\mathbf{w}$ nhỏ và mô hình đơn giản, có nguy cơ "dưới khớp"; điều chuẩn yếu làm mô hình bám sát dữ liệu huấn luyện, có nguy cơ quá khớp.

## Vai trò của tốc độ học

Số hạng $\frac{1}{2} \| \mathbf{w} \|^2$ làm $J$ lồi chặt với hệ số 1, nên với Gradient Descent dưới đạo hàm trên $J$, lịch giảm bước $\eta_t = 1/t$ là lựa chọn tự nhiên, và với dạng chuẩn hóa $J/(nC)$ ở mục 3.3 là $\eta_t = 1/(\lambda t)$. Một số lưu ý:

1. **Bước hằng số** làm $J$ dao động quanh cực tiểu với biên độ tỷ lệ với bước và với $nC$, và không hội tụ đúng. Ngược lại, bước giảm dần $1/t$ làm biên độ dao động giảm dần.

2. **Bước đầu rất lớn là bình thường.** Với $\eta_1 = 1$, hệ số co $(1 - \eta_1) = 0$ xóa hoàn toàn $\mathbf{w}$ cũ, và $\mathbf{w}$ mới chỉ gồm thành phần đẩy. Các bước sau co lại dần nhờ $(1 - \eta_t) = 1 - 1/t$. Ta sẽ thấy hiện tượng này trong ví dụ tính tay ở mục 3.6: $J$ không giảm đều mà dao động rồi mới lắng xuống.

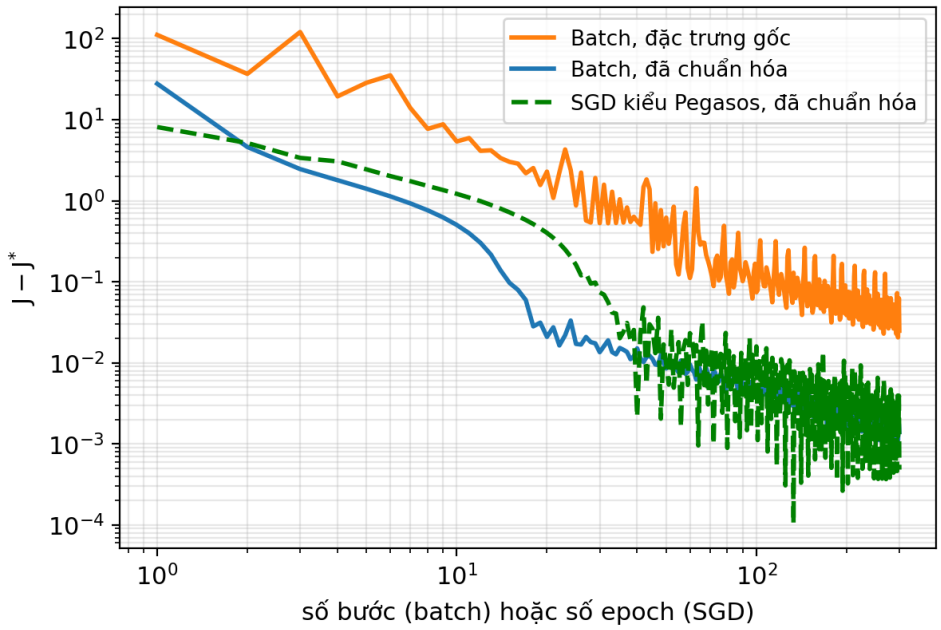
3. **Số $b$ không được điều chuẩn** nên tính chất hội tụ theo $b$ chậm hơn theo $\mathbf{w}$. Trong thực hành, chuẩn hóa đặc trưng giúp cải thiện điều này.

## Chuẩn hóa đặc trưng

Với SVM, chuẩn hóa đặc trưng gần như là **bắt buộc**, vì hai lý do:

1. **Lề và điều chuẩn đo bằng khoảng cách Euclid.** $\| \mathbf{w} \|$ gom mọi đặc trưng vào một thước đo chung, nên đặc trưng có thang đo lớn sẽ chi phối. Ví dụ, nếu đổi $x_1$ từ giờ sang phút (nhân 60), hệ số tương ứng của $x_1$ co lại 60 lần và bị điều chuẩn "nhẹ" hơn rất nhiều. Trên dữ liệu 7 sinh viên với $C = 1$, việc đổi đơn vị này làm hệ số quy về thang đo gốc đổi từ $[0{,}346 \quad -0{,}269]$ thành $[0{,}360 \quad -0{,}200]$, nghĩa là **đường biên đổi** chỉ vì đổi đơn vị.

2. **Điều kiện hóa của Gradient Descent.** Khi các đặc trưng có thang đo rất khác nhau, mặt mục tiêu "dẹt" theo một số hướng và Gradient Descent hội tụ chậm. Trên dữ liệu 7 sinh viên, để $J$ gần $J^*$ trong phạm vi 1%, Gradient Descent dưới đạo hàm cần khoảng 266 bước với đặc trưng gốc nhưng chỉ khoảng 20 bước với đặc trưng đã chuẩn hóa. Hình 9 cho thấy khoảng cách $J - J^*$ theo số bước.




*Hình 9. Khoảng cách $J - J^*$ đến giá trị tối ưu theo số bước (hoặc số epoch) khi huấn luyện trên 7 sinh viên với $C = 1$. Chuẩn hóa đặc trưng giúp batch hội tụ nhanh và ổn định hơn nhiều. SGD kiểu Pegasos dao động nhẹ quanh cực tiểu vì mỗi bước chỉ dùng một mẫu.*

Công thức chuẩn hóa chuẩn: với $\boldsymbol{\mu}$ và $\boldsymbol{\sigma}$ là vector hàng $1 \times d$ gồm trung bình và độ lệch chuẩn từng cột **của tập huấn luyện**,

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}, \qquad \mathbf{X}_{\text{std}} = (\mathbf{X} - \mathbf{1}\boldsymbol{\mu}) \oslash (\mathbf{1}\boldsymbol{\sigma}),$$

với $\oslash$ là phép chia từng phần tử. Dữ liệu kiểm tra phải được chuẩn hóa bằng **cùng** $\boldsymbol{\mu}, \boldsymbol{\sigma}$ của tập huấn luyện. Sau khi huấn luyện trên $\mathbf{X}_{\text{std}}$ được $(\mathbf{w}_{\text{std}}, b_{\text{std}})$, có thể quy ngược về thang đo gốc bằng

$$\mathbf{w}_{\text{gốc}} = \mathbf{w}_{\text{std}} \oslash \boldsymbol{\sigma}^T, \qquad b_{\text{gốc}} = b_{\text{std}} - \boldsymbol{\mu}\,\mathbf{w}_{\text{gốc}}.$$

Cần nhớ rằng nghiệm tối ưu **trên dữ liệu đã chuẩn hóa** và nghiệm tối ưu **trên dữ liệu gốc** là hai nghiệm khác nhau (vì điều chuẩn $\frac{1}{2} \| \mathbf{w} \|^2$ tác động lên hai thang đo khác nhau), nên $C$ cũng cần chọn lại khi đổi cách chuẩn hóa.

## 3.6. Thuật toán qua từng bước (batch và SGD kiểu Pegasos) và ví dụ tính tay một vòng

### Thuật toán batch (Gradient Descent dưới đạo hàm)

**Đầu vào:** $\mathbf{X}$ $(n \times d)$, $\mathbf{y}$, hệ số phạt $C$, số bước $T$.

1. Chuẩn hóa đặc trưng (mục 3.5).

2. Khởi tạo $\mathbf{w} = \mathbf{0}$, $b = 0$.
3. Với $t = 1,2,\dots,T$:
   a. Tính margin $\mathbf{m} = \mathbf{y} \odot (\mathbf{X}\mathbf{w} + b\mathbf{1})$ và vector chỉ báo $a_i = \mathbb{1}[m_i < 1]$.
   b. Tính dưới đạo hàm $\mathbf{g}_\mathbf{w} = \mathbf{w} - C\,\mathbf{X}^T(\mathbf{y} \odot \mathbf{a})$ và $g_b = -C\,\mathbf{y}^T\mathbf{a}$.
   c. Cập nhật với tốc độ học $\eta_t = 1/t$: $\mathbf{w} \leftarrow \mathbf{w} - \eta_t \mathbf{g}_\mathbf{w}$, $b \leftarrow b - \eta_t g_b$.
4. Trả về $(\mathbf{w}, b)$. Dự đoán $\hat{y} = \text{sign}(\mathbf{x}\mathbf{w} + b)$.

## Thuật toán SGD kiểu Pegasos

Mỗi bước của batch phải duyệt toàn bộ $n$ mẫu. Khi $n$ lớn, ta dùng **SGD**: mỗi bước chỉ lấy ngẫu nhiên một mẫu. Thuật toán **Pegasos** (Primal Estimated sub-GrAdient SOlver for SVM) áp dụng SGD cho dạng chuẩn hóa $f(\mathbf{w}, b) = \frac{\lambda}{2} \parallel \mathbf{w} \parallel^2 + \frac{1}{n}\sum_i h_i$ với $\lambda = 1/(nC)$. Với mẫu $i$ được chọn ngẫu nhiên ở bước $t$, dưới đạo hàm của $\frac{\lambda}{2} \parallel \mathbf{w} \parallel^2 + h_i$ là $\lambda\mathbf{w} - a_i y_i \mathbf{x}_i^T$ (theo $\mathbf{w}$) và $-a_i y_i$ (theo $b$), với $a_i = \mathbb{1}[m_i < 1]$. Chọn tốc độ học $\eta_t = \frac{1}{\lambda t}$ thì $\eta_t \lambda = \frac{1}{t}$ và cập nhật trở thành

$$\mathbf{w} \leftarrow \left(1 - \frac{1}{t}\right)\mathbf{w} + \eta_t\, a_i\, y_i\, \mathbf{x}_i^T, \qquad b \leftarrow b + \eta_t\, a_i\, y_i.$$

Điều này đúng với tinh thần của batch: **co** $\mathbf{w}$, rồi **đẩy** về phía mẫu vi phạm. Khác biệt là mỗi bước chỉ nhìn một mẫu, nên rẻ hơn $n$ lần nhưng "nhiễu" hơn. Các bản Pegasos đầy đủ còn có bước chiếu $\mathbf{w}$ vào hình cầu bán kính $1/\sqrt{\lambda}$, và ta bỏ qua chi tiết này.

## Ví dụ tính tay một vòng: batch

Dùng 7 sinh viên (đặc trưng gốc, để tính tay dễ), $C = 0{,}2$, khởi tạo $\mathbf{w} = \begin{bmatrix}0\\0\end{bmatrix}$, $b = 0$, tốc độ học $\eta_t = 1/t$.

**Bước $t = 1$.** Với $\mathbf{w} = \mathbf{0}$, $b = 0$: mọi $m_i = 0 < 1$ nên $\mathbf{a} = \mathbf{1}$ (cả bảy mẫu đều vi phạm). Ta cần $\mathbf{X}^T\mathbf{y}$ và $\mathbf{y}^T\mathbf{a} = \sum_i y_i$:

$$\mathbf{X}^T(\mathbf{y} \odot \mathbf{a}) = \mathbf{X}^T\mathbf{y} = \begin{bmatrix}-2-3-4+6+8+9-7\\-4-3-5+1+2+0-1\end{bmatrix} = \begin{bmatrix}7\\-10\end{bmatrix}, \qquad \mathbf{y}^T\mathbf{a} = -3+3-1 = -1.$$

Dưới đạo hàm và cập nhật với $\eta_1 = 1$:

$$\mathbf{g}_\mathbf{w} = \mathbf{0} - 0{,}2\begin{bmatrix}7\\-10\end{bmatrix} = \begin{bmatrix}-1{,}4\\2\end{bmatrix}, \quad g_b = -0{,}2 \cdot (-1) = 0{,}2 \;\Rightarrow\; \mathbf{w} = \begin{bmatrix}1{,}4\\-2\end{bmatrix}, \quad b = -0{,}2.$$

Giá trị hàm mục tiêu trước bước này là $J = 0 + 0{,}2 \cdot 7 \cdot 1 = 1{,}4$.

**Bước $t = 2$.** Tính điểm số $s_i = 1{,}4x_{i1} - 2x_{i2} - 0{,}2$ rồi margin $m_i = y_i s_i$:

| Sinh viên | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $s_i$ | $-5{,}4$ | $-2{,}0$ | $-4{,}6$ | 6,2 | 7,0 | 12,4 | 7,6 |

| Sinh viên | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $m_i = y_i s_i$ | 5,4 | 2,0 | 4,6 | 6,2 | 7,0 | 12,4 | $-7{,}6$ |
| $a_i = \mathbb{1}[m_i < 1]$ | 0 | 0 | 0 | 0 | 0 | 0 | 1 |

Chỉ sinh viên 7 (bị sai nặng) vi phạm. Khi đó $\mathbf{X}^T(\mathbf{y} \odot \mathbf{a}) = y_7 \mathbf{x}_7^T = -\begin{bmatrix}7\\1\end{bmatrix}$, $\mathbf{y}^T\mathbf{a} = -1$, nên với $\eta_2 = \frac{1}{2}$:

$$\mathbf{g}_\mathbf{w} = \begin{bmatrix}1{,}4\\-2\end{bmatrix} - 0{,}2\begin{bmatrix}-7\\-1\end{bmatrix} = \begin{bmatrix}2{,}8\\-1{,}8\end{bmatrix}, \quad g_b = 0{,}2 \Rightarrow \mathbf{w} = \begin{bmatrix}1{,}4 - 1{,}4\\-2 + 0{,}9\end{bmatrix} = \begin{bmatrix}0\\-1{,}1\end{bmatrix}, \quad b = -0{,}2 - 0{,}1 = -0{,}3.$$

Giá trị $J$ trước bước này là $\frac{1}{2}(1, 4^2 + 2^2) + 0{,}2 \cdot (1 + 7{,}6) = 2{,}98 + 1{,}72 = 4{,}70$, **lớn hơn** 1,4. Đây là hiện tượng thường gặp khi dùng dưới đạo hàm với bước đầu lớn. Tiếp tục theo cách tương tự, các bước đầu cho:

| Bước $t$ | Trước cập nhật: $J(\mathbf{w}, b)$ | Số mẫu vi phạm | Sau cập nhật: $\mathbf{w}^T$ | Sau cập nhật: $b$ |
|---|---|---|---|---|
| 1 | 1,400 | 7 | $[1{,}400 \quad -2{,}000]$ | $-0{,}200$ |
| 2 | 4,700 | 1 | $[0 \quad -1{,}100]$ | $-0{,}300$ |
| 3 | 2,045 | 3 | $[1{,}533 \quad -0{,}533]$ | $-0{,}100$ |
| 4 | 5,558 | 4 | $[0{,}350 \quad -1{,}050]$ | $-0{,}300$ |
| 5 | 1,203 | 3 | $[0{,}560 \quad -0{,}760]$ | $-0{,}260$ |

Giá trị tối ưu của bài toán với $C = 0{,}2$ là $J^* \approx 0{,}555$ tại $\mathbf{w}^* = [0{,}364 \quad -0{,}182]^T$, $b^* \approx -1{,}545$. Năm bước đầu **chưa hội tụ** và $J$ dao động mạnh, nhưng biên độ dao động giảm dần vì $\eta_t = 1/t$ giảm và hệ số co $(1 - \eta_t)$ ngày càng nhẹ. Chạy tiếp hàng trăm bước (mục 3.7) thì $\mathbf{w}$ tiến sát nghiệm tối ưu.

## Ví dụ tính tay: SGD kiểu Pegasos

Cùng dữ liệu, $C = 0{,}2$ nên $\lambda = \frac{1}{nC} = \frac{1}{1{,}4} \approx 0{,}714$ và $\eta_t = \frac{1}{\lambda t} = \frac{1{,}4}{t}$. Khởi tạo $\mathbf{w} = \mathbf{0}$, $b = 0$. Giả sử thứ tự mẫu ngẫu nhiên là sinh viên 4, 7, 2:

| Bước $t$ | Mẫu | $\eta_t$ | $m_i$ trước cập nhật | $a_i$ | $\mathbf{w}$ sau | $b$ sau |
|---|---|---|---|---|---|---|
| 1 | SV 4, $y = +1$, $\mathbf{x} = (6,1)$ | 1,4 | 0 | 1 | $0 \cdot \mathbf{0} + 1{,}4\,(6,1) = (8{,}4;\ 1{,}4)$ | 1,4 |
| 2 | SV 7, $y = -1$, $\mathbf{x} = (7,1)$ | 0,7 | $-61{,}6$ | 1 | $0{,}5\,(8{,}4;\ 1{,}4) - 0{,}7\,(7,1) = (-0{,}7;\ 0)$ | 0,7 |
| 3 | SV 2, $y = -1$, $\mathbf{x} = (3,3)$ | 0,467 | 1,4 | 0 | $\frac{2}{3}(-0{,}7;\ 0) = (-0{,}467;\ 0)$ | 0,7 |

Với $t = 1$ hệ số co $1 - 1/t = 0$ nên $\mathbf{w}$ cũ bị xóa. Ở $t = 3$, mẫu sinh viên 2 có $m_2 = 1{,}4 \geq 1$ (không vi phạm), nên chỉ có bước co chứ không có bước đẩy. Đây là điểm khác biệt trọng yếu

với batch: một bước SGD chỉ phản ánh một mẫu, nên trajectory nhiễu hơn, nhưng $\mathbb{E}$ theo thứ tự ngẫu nhiên của dưới đạo hàm một mẫu là dưới đạo hàm của $f$, nên SGD vẫn hội tụ về nghiệm tối ưu khi $\eta_t = 1/(\lambda t)$.

## 3.7. Mã hóa SVM tuyến tính từ đầu bằng Python

Ta cài đặt thuật toán ở mục 3.6 bằng NumPy, gói trong lớp *LinearSVM* hỗ trợ cả hai phương pháp *batch* và *sgd*. Công thức trong mã bám sát công thức trong bài: *a* là vector chỉ báo $\mathbf{a}$, *X.T @ (y * a)* là $\mathbf{X}^T(\mathbf{y} \odot \mathbf{a})$.

```
import numpy as np


class LinearSVM:
    """SVM tuyến tính lề mềm, huấn luyện bằng hàm mất mát hinge.

    J(w, b) = 1/2 ||w||^2 + C * sum_i max(0, 1 - y_i (x_i w + b))
    Dạng chuẩn hóa chia cho n*C:  f = (lam/2)||w||^2 + mean(hinge),  lam = 1/
(n*C)
    """

    def __init__(self, C=1.0, n_iter=500, method="batch", seed=0):
        self.C, self.n_iter, self.method, self.seed = C, n_iter, method, seed

    def objective(self, X, y):
        m = y * (X @ self.w + self.b)
        return 0.5 * self.w @ self.w + self.C * np.maximum(0, 1 - m).sum()

    def fit(self, X, y):
        n, d = X.shape
        lam = 1.0 / (n * self.C)
        self.w, self.b = np.zeros(d), 0.0
        self.history = []
        rng = np.random.default_rng(self.seed)
        t = 0
        for epoch in range(self.n_iter):
            if self.method == "batch":
                order = [None]                          # một bước dùng toàn bộ
dữ liệu
            else:
                order = rng.permutation(n)              # SGD: duyệt từng mẫu
            for i in order:
                t += 1
                eta = 1.0 / (lam * t)                   # bước giảm dần 1/(lam t
)
                if i is None:
                    a = (y * (X @ self.w + self.b) < 1).astype(float)
                    g_w = lam * self.w - X.T @ (y * a) / n
                    g_b = -np.sum(y * a) / n
                else:
```

```
                    a = float(y[i] * (X[i] @ self.w + self.b) < 1)
                    g_w = lam * self.w - a * y[i] * X[i]
                    g_b = -a * y[i]
                self.w = self.w - eta * g_w
                self.b = self.b - eta * g_b
            self.history.append(self.objective(X, y))
        return self

    def decision_function(self, X):
        return X @ self.w + self.b

    def predict(self, X):
        return np.where(self.decision_function(X) >= 0, 1, -1)
```

Lưu mã trên vào tệp *svm_scratch.py* để các đoạn mã sau nhập được lớp *LinearSVM*.

Giải thích các dòng chính:

1. *lam = 1/(n*C)* là $\lambda = 1/(nC)$, và *eta = 1/(lam*t)* là $\eta_t = 1/(\lambda t)$. Với dạng chuẩn hóa $f = J/(nC)$, bước batch này **tương đương** bước $\eta_t = 1/t$ trên $J$ ở mục 3.6.
2. Ở nhánh *batch*, *a = (m < 1)* là vector chỉ báo, *g_w = lam*w - X.T @ (y*a)/n* là dưới đạo hàm của $f$ theo $\mathbf{w}$ và *g_b = -sum(y*a)/n* là theo $b$. Nhân $nC$ vào hai vế ta được đúng $\mathbf{w} - C\mathbf{X}^T(\mathbf{y} \odot \mathbf{a})$ và $-C\mathbf{y}^T\mathbf{a}$ ở mục 3.4.
3. Ở nhánh *sgd*, mỗi bước dùng một mẫu $i$ và cập nhật tương ứng công thức Pegasos. Vì *history* ghi $J$ cuối mỗi epoch (một lượt qua $n$ mẫu), số *n_iter* ở chế độ *sgd* là **số epoch**.

## Chạy trên dữ liệu 7 sinh viên

```
import numpy as np
from svm_scratch import LinearSVM
from sklearn.svm import SVC, LinearSVC

# Dữ liệu 7 sinh viên (sinh viên 7 là mẫu gây chồng lấn)
X = np.array([[2, 4], [3, 3], [4, 5], [6, 1], [8, 2], [9, 0], [7, 1]], dtype=
float)
y = np.array([-1, -1, -1, 1, 1, 1, -1], dtype=float)

# Chuẩn hóa bằng trung bình và độ lệch chuẩn của tập huấn luyện
mu, sigma = X.mean(axis=0), X.std(axis=0)
Z = (X - mu) / sigma

batch = LinearSVM(C=1.0, n_iter=1000, method="batch").fit(Z, y)
sgd   = LinearSVM(C=1.0, n_iter=1000, method="sgd").fit(Z, y)
ref   = SVC(kernel="linear", C=1.0, tol=1e-12).fit(Z, y)
```

```
print("batch :", batch.w.round(4), round(batch.b, 4), round(batch.history[-1]
, 4))
print("sgd   :", sgd.w.round(4), round(sgd.b, 4), round(sgd.history[-1], 4))
print("sklearn:", ref.coef_[0].round(4), ref.intercept_[0].round(4))
print("nhãn dự đoán:", batch.predict(Z).astype(int))

# Đưa tham số về thang đo gốc: w_goc = w_std / sigma ; b_goc = b_std - mu . w
_goc
w_goc = batch.w / sigma
b_goc = batch.b - mu @ w_goc
print("w gốc =", w_goc.round(4), " b gốc =", round(b_goc, 4))
x_new = np.array([7, 3.6])
print("điểm số sinh viên mới =", round(x_new @ w_goc + b_goc, 3))
```

Kết quả:

```
batch : [ 0.8663 -0.3746] 0.0732 2.8011
sgd   : [ 0.8673 -0.3736] 0.0741 2.8016
sklearn: [ 0.8668 -0.3741] 0.0735
nhãn dự đoán: [-1 -1 -1  1  1  1  1]
w gốc = [ 0.3549 -0.2249]  b gốc = -1.3899
điểm số sinh viên mới = 0.285
```

Cả hai phương pháp tự cài đặt đều cho nghiệm gần như trùng với bộ giải của scikit-learn (sai khác ở chữ số thập phân thứ ba), và giá trị mục tiêu $J \approx 2{,}801$ gần sát giá trị tối ưu $J^{*} \approx 2{,}8008$. Nhãn dự đoán đúng 6 trong 7 sinh viên: sinh viên 7 vẫn bị phân loại sai, như đã dự đoán vì dữ liệu không phân tách được. Sinh viên mới $(7;\ 3{,}6)$ có điểm số $0{,}285 > 0$ nên được dự đoán là **đậu**.

## Chạy trên dữ liệu lớn hơn và so sánh với scikit-learn

Để kiểm tra trên bài toán chồng lấn lớn hơn, ta sinh 400 điểm từ hai đám mây Gaussian chồng lấn, chia 70% huấn luyện và 30% kiểm tra, chuẩn hóa theo tập huấn luyện:

```
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split

Xb, yb = make_blobs(n_samples=400, centers=[(0, 0), (2.2, 2.2)],
                    cluster_std=1.3, random_state=3)
yb = 2.0 * yb - 1                                   # nhãn {0,1} -> {-1,+1}
Xtr, Xte, ytr, yte = train_test_split(Xb, yb, test_size=0.3, random_state=0)
mu, sd = Xtr.mean(axis=0), Xtr.std(axis=0)
Ztr, Zte = (Xtr - mu) / sd, (Xte - mu) / sd

for method, n_iter in [("batch", 500), ("sgd", 100)]:
    m = LinearSVM(C=1.0, n_iter=n_iter, method=method).fit(Ztr, ytr)
    print(method, (m.predict(Ztr) == ytr).mean(), (m.predict(Zte) == yte).mea
n())

ref = SVC(kernel="linear", C=1.0).fit(Ztr, ytr)
```

```
lin = LinearSVC(C=1.0, loss="hinge", max_iter=100000, tol=1e-8).fit(Ztr, ytr)
print("SVC      ", ref.score(Ztr, ytr), ref.score(Zte, yte))
print("LinearSVC", lin.score(Ztr, ytr), lin.score(Zte, yte))
```

| Mô hình | $\mathbf{w}^T$ | $b$ | Chính xác huấn luyện | Chính xác kiểm tra | $J$ |
|---|---|---|---|---|---|
| *LinearSVM* (batch, 500 bước) | $[1{,}461 \quad 1{,}444]$ | $-0{,}200$ | 0,879 | 0,867 | 77,01 |
| *LinearSVM* (SGD, 100 epoch) | $[1{,}472 \quad 1{,}412]$ | $-0{,}228$ | 0,879 | 0,867 | 77,08 |
| *SVC(kernel="linear")* | $[1{,}459 \quad 1{,}439]$ | $-0{,}198$ | 0,879 | 0,867 | 77,01 |
| *LinearSVC(loss="hinge")* | $[1{,}460 \quad 1{,}442]$ | $-0{,}198$ | 0,879 | 0,867 | không in |

Cả bốn mô hình cho cùng độ chính xác và nghiệm gần như trùng nhau. Điều này xác nhận rằng huấn luyện bằng **hinge cộng điều chuẩn L2** trực tiếp trên bài toán không ràng buộc cho kết quả tương đương bộ giải QP dựa trên dạng ràng buộc. Hai ưu điểm của cách tự cài đặt là mã ngắn gọn (khoảng 40 dòng) và dễ mở rộng cho dữ liệu lớn bằng SGD. Nhược điểm là cần chuẩn hóa đặc trưng và chọn lịch giảm bước cẩn thận, còn bộ giải QP (dạng đối ngẫu, mục 4) cho nghiệm chính xác hơn trên dữ liệu nhỏ.

## Tóm tắt mục 3

1. **Lề cứng** không tồn tại khi dữ liệu chồng lấn và rất nhạy với ngoại lai. **Lề mềm** thêm biến lỏng $\xi_i \geq 0$: $y_i(\mathbf{x}_i\mathbf{w} + b) \geq 1 - \xi_i$, và tối thiểu hóa $\frac{1}{2} \| \mathbf{w} \|^2 + C\sum\xi_i$. Tập khả thi luôn khác rỗng, và $\sum\xi_i$ chặn trên số mẫu sai.

2. $C$ cân bằng **lề rộng** ($\| \mathbf{w} \|$ nhỏ) và **ít vi phạm** ($\sum\xi_i$ nhỏ). $C \to \infty$ tiến về lề cứng, $C$ nhỏ cho lề rất rộng. Quan hệ với Logistic Regression: $\lambda = 1/(nC)$.

3. Khử $\xi_i = \max(0, 1 - m_i)$ cho **bài toán không ràng buộc** $J(\mathbf{w}, b) = \frac{1}{2} \| \mathbf{w} \|^2 + C\sum\max(0, 1 - m_i)$. **Hinge** khác mất mát Perceptron ở chỗ dời ngưỡng từ $m = 0$ lên $m = 1$, và khác Logistic ở chỗ bằng 0 chính xác khi $m \geq 1$ (tính thưa).

4. Dưới đạo hàm: $\nabla_{\mathbf{w}} J = \mathbf{w} - C\mathbf{X}^T(\mathbf{y} \odot \mathbf{a})$, $\partial_b J = -C\mathbf{y}^T\mathbf{a}$, với $a_i = \mathbb{1}[m_i < 1]$. Ba mô hình có cùng dạng cập nhật $\mathbf{w} \leftarrow \mathbf{w} + \eta \sum_i \varphi(m_i) y_i \mathbf{x}_i^T$, chỉ khác trọng số $\varphi$: $\mathbb{1}[m < 0]$ (Perceptron), $\sigma(-m)$ (Logistic), $\mathbb{1}[m < 1]$ (SVM), và SVM có thêm bước co $\mathbf{w}$.

5. Thực hành: dùng tốc độ học giảm dần $\eta_t = 1/t$ trên $J$ (hay $1/(\lambda t)$ trên dạng chuẩn hóa), **chuẩn hóa đặc trưng**, và chọn $C$ bằng kiểm định chéo. Batch dùng toàn bộ dữ liệu mỗi bước, SGD kiểu Pegasos dùng một mẫu mỗi bước với $\eta_t\lambda = 1/t$.

6. Cài đặt từ đầu bằng NumPy cho nghiệm trùng với `SVC` và `LinearSVC` của scikit-learn.

## Câu hỏi kiểm tra nhanh

1. Với $\mathbf{w} = \begin{bmatrix} 0{,}4 \\ -0{,}2 \end{bmatrix}$, $b = -1{,}5$, tính margin $m_i$, biến lỏng $\xi_i = \max(0, 1 - m_i)$ của sinh viên 7 ($\mathbf{x}_7 = [7 \quad 1]$, $y_7 = -1$) và của sinh viên 4 ($\mathbf{x}_4 = [6 \quad 1]$, $y_4 = +1$). Mẫu nào bị phân loại sai, mẫu nào chỉ lấn lề?

2. Giải thích vì sao mất mát hinge bằng 0 với mọi $m \geq 1$ lại dẫn đến việc nghiệm chỉ phụ thuộc vào một số ít mẫu, trong khi Logistic Regression thì không.

3. Với $\mathbf{w} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$, $b = 0$, $C = 1$ và một mẫu duy nhất $\mathbf{x} = [1 \quad 2]$, $y = +1$, hãy tính $m$, vector chỉ báo $a$, và dưới đạo hàm của hàm mất mát hinge của mẫu này theo $\mathbf{w}$ và $b$.

4. Vì sao tăng $C$ làm lề $\gamma$ giảm? Chuyện gì xảy ra với số mẫu có $\xi_i > 0$?

5. Nếu không chuẩn hóa đặc trưng, đổi đơn vị của một đặc trưng từ "giờ" sang "phút" ảnh hưởng thế nào đến nghiệm SVM? Có ảnh hưởng tương tự đến Perceptron không?

### Gợi ý đáp án

1. Sinh viên 7: $s_7 = 0{,}4 \cdot 7 - 0{,}2 \cdot 1 - 1{,}5 = 1{,}1$, $m_7 = y_7 s_7 = -1{,}1$, nên $\xi_7 = \max(0, 1 + 1{,}1) = 2{,}1 > 1$: **bị phân loại sai**. Sinh viên 4: $s_4 = 2{,}4 - 0{,}2 - 1{,}5 = 0{,}7$, $m_4 = 0{,}7$, nên $\xi_4 = 0{,}3 \in (0, 1]$: **đúng phía nhưng lấn vào dải lề**.

2. Mẫu có $m_i \geq 1$ cho mất mát 0 và dưới đạo hàm $\mathbf{0}$ (vì $a_i = 0$), nên không góp mặt trong $\nabla J$ và không làm nghiệm đổi nếu bị loại bỏ. Chỉ mẫu có $m_i < 1$ mới đẩy $\mathbf{w}$. Với Logistic, đạo hàm $-\sigma(-m) y \mathbf{x}^T$ luôn khác 0 (dù rất nhỏ khi $m$ lớn) nên mọi mẫu đều ảnh hưởng.

3. $s = 1 \cdot 1 - 1 \cdot 2 + 0 = -1$, $m = y\, s = -1 < 1$ nên $a = 1$ (mẫu bị sai). Dưới đạo hàm của hinge là $\partial_{\mathbf{w}} h = -y \mathbf{x}^T = -[1 \quad 2]^T$ và $\partial_b h = -y = -1$. Thêm số hạng điều chuẩn, $\nabla_{\mathbf{w}} J = \mathbf{w} - C\, a\, y\, \mathbf{x}^T = \begin{bmatrix} 1 \\ -1 \end{bmatrix} - \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 0 \\ -3 \end{bmatrix}$ và $\partial_b J = -C\, a\, y = -1$.

4. Tăng $C$ làm vi phạm bị phạt nặng hơn nên mô hình chấp nhận $\| \mathbf{w} \|$ lớn hơn để giảm $\sum \xi_i$. Vì $\gamma = 1 / \| \mathbf{w} \|$ nên lề giảm. Số mẫu có $\xi_i > 0$ nói chung **giảm** (ở bảng mục 3.5: từ 6 xuống 1 khi $C$ từ 0,01 lên 10).

5. Với SVM, đổi đơn vị làm thay đổi cách $\frac{1}{2} \| \mathbf{w} \|^2$ phạt từng hệ số, nên **đường biên thay đổi** (đã thấy ở mục 3.5). Với Perceptron, việc nhân một đặc trưng với hằng

số dương chỉ co giãn trục tọa độ mà không có điều chuẩn can thiệp, nên tính phân tách được của dữ liệu và số lần cập nhật hữu hạn vẫn đúng, nhưng hình dạng đường biên tìm được vẫn có thể phụ thuộc thang đo qua cận $R^2/\gamma^2$ và thứ tự cập nhật.

# 4. Phân tích cơ chế học của SVM: đối ngẫu và vector hỗ trợ

Ở mục 2 và 3, ta đã làm việc với SVM trong **không gian gốc** của $(\mathbf{w}, b)$: mục 2 phát biểu bài toán lề cứng, mục 3 nới nó thành lề mềm và huấn luyện bằng Gradient Descent trên hàm mất mát hinge. Hai mục đó để lại ba điều chưa được giải thích đến nơi:

1. Ở mục 2, ta đã chứng minh rằng nghiệm lề cứng chỉ do các **vector hỗ trợ** quyết định, nhưng chưa nói được lực nào của từng mẫu tạo nên $\mathbf{w}$. Thuật toán ở mục 3 thì lại duyệt **mọi** mẫu ở mỗi bước, vậy mẫu nào thật sự có ảnh hưởng, và ảnh hưởng đó lớn đến đâu?
2. Hàm mục tiêu $J(\mathbf{w}, b) = \frac{1}{2} \| \mathbf{w} \|^2 + C \sum_i \max(0, 1 - m_i)$ không khả vi, vậy nghiệm của nó có tồn tại và duy nhất hay không, **kể cả khi dữ liệu phân tách hoàn hảo** (trường hợp khiến Logistic Regression không điều chuẩn có $\| \mathbf{w} \|$ tăng mãi).
3. Ở mục 1.3, ta thấy Perceptron có thể viết $\mathbf{w}$ như một tổ hợp của các mẫu huấn luyện. Cách viết đó có áp dụng được cho SVM không, và nó đem lại điều gì?

Mục này trả lời các câu hỏi đó bằng cách **mổ xẻ cơ chế học** của SVM: từ điều kiện tối ưu của bài toán hinge, ta suy ra **dạng đối ngẫu**, trong đó mỗi mẫu có một hệ số $\alpha_i$ cho biết nó "tác động" vào đường biên mạnh đến mức nào. Từ đó tính thưa của nghiệm và khái niệm vector hỗ trợ hiện ra một cách tự nhiên. Các câu hỏi cụ thể:

1. Điều gì xảy ra với đường biên khi một mẫu bị phân loại sai hoặc lấn vào lề?
2. Vì sao nghiệm luôn tồn tại và $\mathbf{w}$ luôn duy nhất, ngay cả khi dữ liệu phân tách hoàn hảo?
3. Dạng đối ngẫu và các điều kiện KKT là gì, và vì sao $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i^T$?
4. Ba loại mẫu ứng với $\alpha_i = 0$, $0 < \alpha_i < C$, $\alpha_i = C$ khác nhau thế nào?
5. Dạng đối ngẫu của SVM liên hệ ra sao với Perceptron dạng đối ngẫu, và nó được giải như thế nào?

**Dữ liệu xuyên suốt.** Ta dùng lại hai bộ dữ liệu đã quen: (i) 7 sinh viên của mục 3 với $C = 1$, nghiệm ở mục 3.5 là $\mathbf{w} = \left[\frac{9}{26} \quad -\frac{7}{26}\right]^T$, $b = -\frac{16}{13}$; (ii) 6 sinh viên của mục 2 với lề cứng, nghiệm $\mathbf{w} = \left[\frac{6}{13} \quad -\frac{4}{13}\right]^T$, $b = -\frac{19}{13}$.

**Ký hiệu bổ sung.** Ngoài các ký hiệu đã dùng ($\mathbf{X}$, $\mathbf{y}$, $\mathbf{m} = \mathbf{y} \odot (\mathbf{X}\mathbf{w} + b\mathbf{1})$, $\mathbf{Z} = \mathrm{diag}(\mathbf{y})\mathbf{X}$), mục này dùng thêm:

| Đối tượng | Ký hiệu | Kích thước | Ý nghĩa |
|---|---|---|---|
| Hệ số đối ngẫu | $\boldsymbol{\alpha}$, $\alpha_i$ | $n \times 1$ | Hệ số (nhân tử Lagrange) gắn với ràng buộc của mẫu $i$ |
| Ma trận Gram | $\mathbf{K} = \mathbf{X}\mathbf{X}^T$ | $n \times n$ | $K_{ij} = \mathbf{x}_i \mathbf{x}_j^T$, tích vô hướng giữa các mẫu |
| Ma trận đối ngẫu | $\mathbf{Q} = \operatorname{diag}(\mathbf{y})\, \mathbf{K}\, \operatorname{diag}(\mathbf{y}) = \mathbf{Z}\mathbf{Z}^T$ | $n \times n$ | $Q_{ij} = y_i y_j\, \mathbf{x}_i \mathbf{x}_j^T$ |
| Vector $\mathbf{0}$, $\mathbf{1}$ | $\mathbf{0}$, $\mathbf{1}$ | $n \times 1$ | Vector toàn số 0, toàn số 1 |

## 4.1. Điều gì xảy ra khi một mẫu bị phân loại sai hoặc lấn vào lề?

### Điều kiện tối ưu: $\mathbf{w}$ là hợp lực của các mẫu

Ta quay lại hàm mục tiêu dạng không ràng buộc ở mục 3.3:

$$J(\mathbf{w}, b) = \frac{1}{2} \| \mathbf{w} \|^2 + C \sum_{i=1}^{n} \max(0,\ 1 - m_i).$$

Ở mục 3.4, ta tìm dưới đạo hàm của $J$ và dùng nó để cập nhật. Một điều kiện cần của cực tiểu là $\mathbf{0}$ thuộc tập dưới đạo hàm. Với mẫu $i$, dưới đạo hàm của hinge theo $\mathbf{w}$ là $-\theta_i y_i \mathbf{x}_i^T$, trong đó hệ số $\theta_i$ được xác định bởi vị trí của mẫu:

$$\theta_i = \begin{cases} 1 & \text{nếu } m_i < 1 \text{ (lấn lề hoặc sai)}, \\ 0 & \text{nếu } m_i > 1 \text{ (ngoài dải lề)}, \\ \text{một số bất kỳ trong } [0,1] & \text{nếu } m_i = 1 \text{ (đúng trên biên lề)}. \end{cases}$$

Điều kiện $\mathbf{0} \in \partial J$ viết thành hai phương trình. Đặt $\alpha_i = C\theta_i$, tức $\alpha_i \in [0, C]$, ta có

$$\mathbf{w} = \sum_{i=1}^{n} \alpha_i\, y_i\, \mathbf{x}_i^T = \mathbf{X}^T(\boldsymbol{\alpha} \odot \mathbf{y}), \qquad \sum_{i=1}^{n} \alpha_i\, y_i = \mathbf{y}^T \boldsymbol{\alpha} = 0,$$

với

$$\alpha_i = \begin{cases} C & \text{nếu } m_i < 1, \\ 0 & \text{nếu } m_i > 1, \\ \in [0, C] & \text{nếu } m_i = 1. \end{cases}$$

Hai phương trình này có ý nghĩa trực tiếp, và chúng chính là **điểm dừng** của quy tắc cập nhật ở mục 3.4. Thật vậy, quy tắc $\mathbf{w} \leftarrow (1 - \eta)\mathbf{w} + \eta C \sum_{i:\, m_i < 1} y_i\, \mathbf{x}_i^T$ ngừng thay đổi $\mathbf{w}$ khi $\mathbf{w} = C \sum_{i:\, m_i < 1} y_i\, \mathbf{x}_i^T$, đúng bằng công thức trên khi không có mẫu nào nằm chính xác trên biên lề.

## Đọc theo "lực": mỗi mẫu tác động lên đường biên với cường độ $\alpha_i$

Công thức $\mathbf{w} = \sum_i \alpha_i\, y_i \mathbf{x}_i^T$ cho phép hình dung đường biên như một vật chịu **lực** từ các mẫu. Mẫu $i$ tác động theo hướng $y_i \mathbf{x}_i^T$ với cường độ $\alpha_i$, và $\mathbf{w}$ là hợp lực. Có ba kiểu mẫu:

1. **Mẫu nằm ngoài dải lề, đúng phía ($m_i > 1$): lực bằng 0.** Mẫu này đã "an toàn", đường biên không cần để ý đến nó.
2. **Mẫu lấn vào dải lề hoặc nằm sai phía ($m_i < 1$): lực đạt mức tối đa $C$.** Điểm đáng chú ý là lực này **bị chặn tại** $C$, bất kể mẫu sai nặng đến đâu. Một ngoại lai nằm rất xa vẫn chỉ kéo đường biên bằng $C\, y_i \mathbf{x}_i^T$, không hơn. Đây là cơ chế làm lề mềm bền vững trước ngoại lai.
3. **Mẫu nằm đúng trên biên lề ($m_i = 1$): lực trung gian** $\alpha_i \in [0, C]$, được điều chỉnh vừa đủ để hệ ở trạng thái cân bằng.

Phương trình thứ hai $\mathbf{y}^T\boldsymbol{\alpha} = 0$ chính là **điều kiện cân bằng lực**: tổng cường độ lực của lớp dương phải bằng tổng cường độ lực của lớp âm, $\sum_{y_i=+1} \alpha_i = \sum_{y_i=-1} \alpha_i$. Nếu một lớp "đẩy mạnh hơn", đường biên sẽ tịnh tiến sang phía lớp kia, tức là $b$ thay đổi, cho đến khi hai bên cân bằng. Như vậy $b$ không bị phạt trong hàm mục tiêu nhưng vẫn được xác định bởi điều kiện cân bằng này.

## Ví dụ: 7 sinh viên với $C = 1$

Nghiệm ở mục 3.5 cho margin $m_i$ của 7 sinh viên như bảng dưới. Từ quy tắc ba trường hợp, ta suy ra $\alpha_i$ cho các mẫu có $m_i \neq 1$, và cho các mẫu $m_i = 1$ (sinh viên 2 và 5), ta giải $\alpha_2, \alpha_5$ từ hai phương trình cân bằng:

$$\sum_i \alpha_i\, y_i = \alpha_4 + \alpha_5 - \alpha_2 - \alpha_7 = 1 + \alpha_5 - \alpha_2 - 1 = 0 \;\Rightarrow\; \alpha_5 = \alpha_2 = \alpha,$$

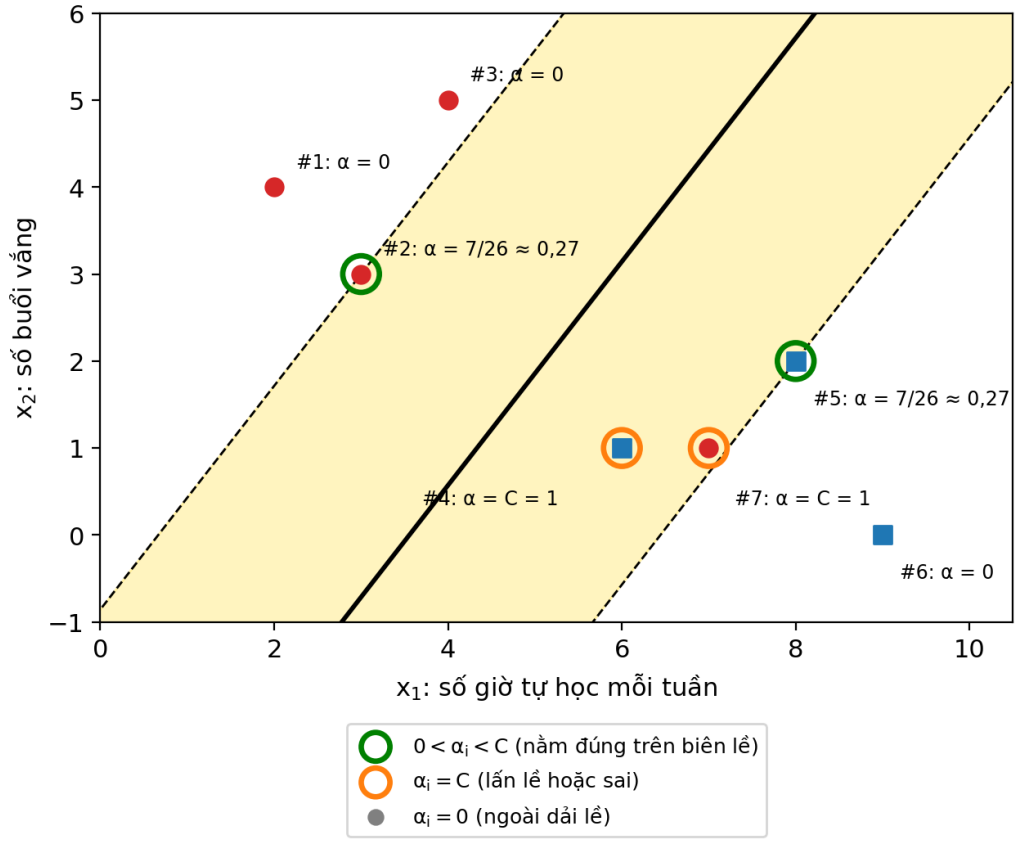
$$\mathbf{w} = \alpha_4 \mathbf{x}_4^T + \alpha\, \mathbf{x}_5^T - \alpha\, \mathbf{x}_2^T - \alpha_7 \mathbf{x}_7^T = \begin{bmatrix} -1 + 5\alpha \\ -\alpha \end{bmatrix} = \begin{bmatrix} \frac{9}{26} \\ -\frac{7}{26} \end{bmatrix} \;\Rightarrow\; \alpha = \frac{7}{26}.$$

Kết quả được tổng hợp ở bảng:

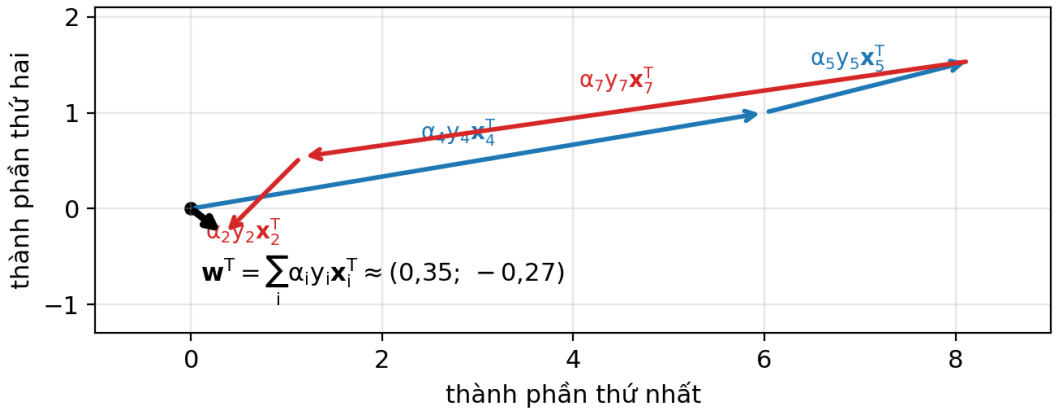
| Sinh viên | $y_i$ | $m_i$ | Vị trí | $\alpha_i$ | Lực $\alpha_i y_i \mathbf{x}_i^T$ |
|---|---|---|---|---|---|
| 1 | $-1$ | 1,62 | Ngoài lề | 0 | $[0 \quad 0]$ |
| 2 | $-1$ | 1,00 | Trên biên lề | $\frac{7}{26} \approx 0{,}27$ | $[-0{,}81 \quad -0{,}81]$ |
| 3 | $-1$ | 1,19 | Ngoài lề | 0 | $[0 \quad 0]$ |
| 4 | $+1$ | 0,58 | **Lấn lề** | $C = 1$ | $[6 \quad 1]$ |
| 5 | $+1$ | 1,00 | Trên biên lề | $\frac{7}{26} \approx 0{,}27$ | $[2{,}15 \quad 0{,}54]$ |

| Sinh viên | $y_i$ | $m_i$ | Vị trí | $\alpha_i$ | Lực $\alpha_i y_i \mathbf{x}_i^T$ |
|---|---|---|---|---|---|
| 6 | $+1$ | 1,88 | Ngoài lề | 0 | $[0 \quad 0]$ |
| 7 | $-1$ | $-0{,}92$ | **Sai phía** | $C = 1$ | $[-7 \quad -1]$ |

Cộng bốn lực khác không cho đúng $\mathbf{w}^T = [0{,}346 \quad -0{,}269]$ (Hình 11). Cũng kiểm tra được cân bằng lực: lớp dương có $1 + \frac{7}{26}$, lớp âm có $\frac{7}{26} + 1$. Chỉ **bốn trong bảy** sinh viên có lực khác 0, và hai sinh viên bị chặn ở $\alpha = C$ (sinh viên 4 lấn lề, sinh viên 7 bị sai) cũng chỉ tác động tối đa bằng $C$.




*Hình 10. Ba loại mẫu của SVM lề mềm trên 7 sinh viên với $C = 1$. Mẫu ngoài dải lề có $\alpha_i = 0$. Mẫu nằm đúng trên biên lề có $0 < \alpha_i < C$ (vòng xanh lá). Mẫu lấn lề hoặc sai phía có $\alpha_i = C$ (vòng cam).*




*Hình 11.* $\mathbf{w}$ *là tổng của các "lực"* $\alpha_i y_i \boldsymbol{x}_i^T$*. Đi lần lượt theo bốn vector khác không (nối đầu vào đuôi) từ gốc tọa độ, điểm cuối chính là* $\boldsymbol{w}^T \approx (0{,}35; -0{,}27)$ *(mũi tên đen).*

### Khác biệt với Perceptron và Logistic Regression

Mỗi mô hình cho các mẫu tác động khác nhau lên $\mathbf{w}$:

1. **Perceptron**: mỗi lần sai, mẫu cộng thêm $y_i \mathbf{x}_i^T$ vào $\mathbf{w}$. Một mẫu bị sai nhiều lần cộng dồn nhiều lần, **không có trần**.
2. **Logistic Regression**: mọi mẫu tác động với cường độ $\sigma(-m_i) \in (0,1)$, giảm dần nhưng không bao giờ bằng $0$.
3. **SVM**: cường độ là $\alpha_i \in [0, C]$, **bằng đúng** $0$ với mẫu ngoài lề và **bị chặn ở** $C$ với mẫu vi phạm.

## 4.2. Hàm mất mát lồi chặt và nghiệm duy nhất, ngay cả khi dữ liệu phân tách hoàn hảo

### Nghiệm tồn tại

Hàm $J(\mathbf{w}, b)$ là tổng của $\frac{1}{2} \| \mathbf{w} \|^2$ và các hàm hinge, đều liên tục và không âm. Để chứng minh $J$ có cực tiểu, ta chỉ cần chỉ ra rằng $J \to \infty$ khi $\| (\mathbf{w}, b) \| \to \infty$:

1. Nếu $\| \mathbf{w} \| \to \infty$ thì $J \geq \frac{1}{2} \| \mathbf{w} \|^2 \to \infty$.
2. Nếu $\mathbf{w}$ bị chặn nhưng $|b| \to \infty$: khi $b \to +\infty$ thì mọi mẫu **âm** có $m_i = -(\mathbf{x}_i \mathbf{w} + b) \to -\infty$ nên hinge $\to \infty$. Khi $b \to -\infty$ thì mọi mẫu **dương** có $m_i \to -\infty$. Cả hai trường hợp đều cần dữ liệu có đủ hai lớp.

Do đó $J$ luôn có cực tiểu toàn cục, bất kể dữ liệu có phân tách được hay không.

## $\mathbf{w}$ tối ưu là duy nhất

Hàm $\frac{1}{2} \| \mathbf{w} \|^2$ lồi chặt theo $\mathbf{w}$, còn $H(\mathbf{w}, b) = C \sum_i \max(0, 1 - m_i)$ lồi (hàm lồi của hàm affine). Giả sử có hai điểm cực tiểu $(\mathbf{w}_1, b_1) \neq (\mathbf{w}_2, b_2)$ với $\mathbf{w}_1 \neq \mathbf{w}_2$, cùng giá trị $J^*$. Gọi $\left(\bar{\mathbf{w}}, \bar{b}\right)$ là trung điểm. Theo đẳng thức hình bình hành và tính lồi của $H$,

$$J\left(\bar{\mathbf{w}}, \bar{b}\right) \leq \frac{1}{2} J(\mathbf{w}_1, b_1) + \frac{1}{2} J(\mathbf{w}_2, b_2) - \frac{1}{8} \| \mathbf{w}_1 - \mathbf{w}_2 \|^2 < J^*,$$

mâu thuẫn với tính tối ưu. Vậy $\mathbf{w}^*$ **duy nhất**. Hệ số chệch $b^*$ duy nhất khi có ít nhất một mẫu nằm đúng trên biên lề (trường hợp $0 < \alpha_i < C$ ở mục 4.4, vì khi đó $b^* = y_i - \mathbf{x}_i \mathbf{w}^*$). Trong trường hợp suy biến hiếm gặp không có mẫu nào trên biên lề, $b^*$ có thể nằm trong một đoạn, nhưng đường biên khi đó chỉ khác nhau về vị trí tịnh tiến, còn hướng $\mathbf{w}^*$ và lề vẫn như nhau.

## Ngay cả khi dữ liệu phân tách hoàn hảo

Đây là điểm đối lập rõ nhất với hai bài trước:

| Mô hình | Khi dữ liệu phân tách hoàn hảo |
|---|---|
| Perceptron | Có **vô số** nghiệm, tùy thứ tự duyệt và khởi tạo |
| Logistic Regression (không điều chuẩn) | **Không có** cực tiểu: $J \to 0$ khi $\| \mathbf{w} \| \to \infty$ nhưng không bao giờ đạt |
| Logistic Regression + L2 | Duy nhất, phụ thuộc $\lambda$ |
| SVM lề cứng | Duy nhất (mục 2.2) |
| SVM lề mềm | Duy nhất theo $\mathbf{w}$, với mọi $C > 0$ |

Điều khiển SVM không "chạy mãi" như Logistic Regression là **cách đặt mục tiêu**. Logistic Regression thưởng cho margin lớn một cách vô hạn, nên có thể kéo $\mathbf{w}$ ra xa vô cùng. SVM thì **cố định margin ở mức** 1 (hinge bằng 0 khi $m \geq 1$, không thưởng thêm) và phạt $\| \mathbf{w} \|$, nên điểm dừng nằm ở vị trí hữu hạn.

## Khi nào lề mềm trùng với lề cứng?

Khi dữ liệu phân tách được, câu hỏi tự nhiên là: $C$ lớn đến mức nào thì nghiệm lề mềm **đúng bằng** nghiệm lề cứng? Ta dự đoán câu trả lời bằng một thí nghiệm trên 6 sinh viên (phân tách được), rồi chứng minh ở mục 4.3.

| $C$ | $\mathbf{w}^T$ | $b$ | Các $\alpha_i > 0$ | Kết luận |
|---|---|---|---|---|
| 0,05 | $[0{,}326 \quad -0{,}232]$ | $-1{,}144$ | $\alpha_2 = 0{,}05,\ \alpha_3 = 0{,}044,\ \alpha_4 = 0{,}05,\ \alpha_5 = 0{,}044$ | Khác lề cứng |
| 0,1 | $[0{,}332 \quad -0{,}224]$ | $-1{,}208$ | $\alpha_2 = 0{,}1,\ \alpha_3 = 0{,}008,\ \alpha_4 = 0{,}1,\ \alpha_5 = 0{,}008$ | Khác lề cứng |
| 0,15 | $[0{,}450 \quad -0{,}300]$ | $-1{,}425$ | $\alpha_2 = \alpha_4 = 0{,}15$ | Khác lề cứng |

| $C$ | $\mathbf{w}^T$ | $b$ | Các $\alpha_i > 0$ | Kết luận |
|---|---|---|---|---|
| 0,16 | $[0{,}4615 \quad -0{,}3077]$ | $-1{,}4615$ | $\alpha_2 = \alpha_4 = \frac{2}{13} \approx 0{,}154$ | **Trùng lề cứng** |
| 1; 100 | $[0{,}4615 \quad -0{,}3077]$ | $-1{,}4615$ | $\alpha_2 = \alpha_4 = \frac{2}{13}$ | **Trùng lề cứng** |

Ngưỡng nằm ở $C^* = \frac{2}{13} \approx 0{,}154$, đúng bằng giá trị $\alpha_2 = \alpha_4$ của nghiệm lề cứng. Với $C < C^*$, các hệ số đối ngẫu bị "cắt" ở $C$ nên đường biên khác đi. Với $C \geq C^*$, ràng buộc $\alpha_i \leq C$ không còn tác dụng nên nghiệm lề mềm **trùng khít** với lề cứng. Như vậy "$C$ đủ lớn" ở mục 3.2 có một nghĩa cụ thể: $C \geq \max_i \alpha_i^{\text{cứng}}$.

## 4.3. Dạng đối ngẫu, điều kiện KKT và công thức $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i^T$

Ở 4.1, ta đã "đoán" công thức của $\mathbf{w}$ từ điều kiện tối ưu của bài toán hinge. Phần này thiết lập nó một cách chặt chẽ từ bài toán gốc **có ràng buộc** ở mục 3.2, bằng phương pháp nhân tử Lagrange.

### Hàm Lagrange

Bài toán gốc của SVM lề mềm:

$$\min_{\mathbf{w}, b, \boldsymbol{\xi}} \frac{1}{2} \| \mathbf{w} \|^2 + C \sum_{i=1}^{n} \xi_i \quad \text{với} \quad y_i(\mathbf{x}_i \mathbf{w} + b) - 1 + \xi_i \geq 0, \;\; \xi_i \geq 0.$$

Gắn nhân tử $\alpha_i \geq 0$ cho ràng buộc $y_i(\mathbf{x}_i \mathbf{w} + b) - 1 + \xi_i \geq 0$, và nhân tử $\mu_i \geq 0$ cho ràng buộc $\xi_i \geq 0$. Hàm Lagrange là

$$L(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) = \frac{1}{2} \| \mathbf{w} \|^2 + C \sum_i \xi_i - \sum_i \alpha_i \left[ y_i(\mathbf{x}_i \mathbf{w} + b) - 1 + \xi_i \right] - \sum_i \mu_i \xi_i.$$

### Điều kiện dừng theo các biến gốc

Đạo hàm riêng theo $\mathbf{w}$, $b$, $\xi_i$ bằng 0:

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w}^T - \sum_i \alpha_i y_i \mathbf{x}_i = \mathbf{0} \;\Rightarrow\; \boxed{\mathbf{w} = \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i^T = \mathbf{X}^T(\boldsymbol{\alpha} \odot \mathbf{y})}$$

$$\frac{\partial L}{\partial b} = -\sum_i \alpha_i y_i = 0 \;\Rightarrow\; \mathbf{y}^T \boldsymbol{\alpha} = 0, \qquad \frac{\partial L}{\partial \xi_i} = C - \alpha_i - \mu_i = 0 \;\Rightarrow\; \alpha_i = C - \mu_i \leq C.$$

Vì $\mu_i \geq 0$ nên $0 \leq \alpha_i \leq C$. Đây chính xác là các phương trình ta thu được ở 4.1 (với $\alpha_i = C\theta_i$), nay được suy ra chặt chẽ từ bài toán gốc.

## Thay ngược để thu bài toán đối ngẫu

Thay ba điều kiện trên vào $L$. Các số hạng chứa $\xi_i$ gom lại thành $\sum_i (C - \alpha_i - \mu_i)\,\xi_i = 0$, số hạng chứa $b$ là $-b\sum_i \alpha_i\, y_i = 0$. Phần còn lại:

$$L = \frac{1}{2} \parallel \mathbf{w} \parallel^2 - \sum_i \alpha_i\, y_i \mathbf{x}_i \mathbf{w} + \sum_i \alpha_i = \frac{1}{2} \parallel \mathbf{w} \parallel^2 - \parallel \mathbf{w} \parallel^2 + \sum_i \alpha_i = \sum_i \alpha_i - \frac{1}{2} \parallel \mathbf{w} \parallel^2,$$

với $\sum_i \alpha_i\, y_i \mathbf{x}_i \mathbf{w} = \mathbf{w}^T \mathbf{w}$. Thay $\mathbf{w} = \mathbf{X}^T(\boldsymbol{\alpha} \odot \mathbf{y}) = \mathbf{Z}^T \boldsymbol{\alpha}$ (vì $\mathbf{Z} = \mathrm{diag}(\mathbf{y})\mathbf{X}$) ta được $\parallel \mathbf{w} \parallel^2 = \boldsymbol{\alpha}^T \mathbf{Z}\mathbf{Z}^T \boldsymbol{\alpha} = \boldsymbol{\alpha}^T \mathbf{Q} \boldsymbol{\alpha}$. Vậy ta thu được **bài toán đối ngẫu (dual problem)**:

$$\max_{\boldsymbol{\alpha}} \; \mathbf{1}^T \boldsymbol{\alpha} - \frac{1}{2} \boldsymbol{\alpha}^T \mathbf{Q}\, \boldsymbol{\alpha} \quad \text{với} \quad \mathbf{y}^T \boldsymbol{\alpha} = 0, \;\; \mathbf{0} \le \boldsymbol{\alpha} \le C\, \mathbf{1}.$$

Khai triển theo từng phần tử, hàm mục tiêu là

$$\sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i\, \alpha_j y_i y_j\, \mathbf{x}_i \mathbf{x}_j^T.$$

Với **lề cứng**, bài toán đối ngẫu giống hệt nhưng **bỏ cận trên** $\alpha_i \le C$, chỉ còn $\boldsymbol{\alpha} \ge \mathbf{0}$ và $\mathbf{y}^T \boldsymbol{\alpha} = 0$.

Có ba nhận xét quan trọng về dạng đối ngẫu:

1. **Số biến là $n$** (một hệ số cho mỗi mẫu), thay vì $d + 1$ (hay $d + 1 + n$ với biến lỏng) như bài toán gốc. Ràng buộc đơn giản: một đẳng thức và các cận trên dưới (gọi là ràng buộc "hộp").
2. **Dữ liệu chỉ xuất hiện qua các tích vô hướng $\mathbf{x}_i \mathbf{x}_j^T$**, nằm trong ma trận Gram $\mathbf{K}$. Đây là chìa khóa của thủ thuật kernel ở mục 5.
3. **Đối ngẫu mạnh**: vì bài toán gốc là quy hoạch toàn phương lồi với ràng buộc tuyến tính, giá trị tối ưu của bài toán đối ngẫu **bằng** giá trị tối ưu của bài toán gốc.

## Điều kiện KKT

Các điều kiện KKT (Karush–Kuhn–Tucker) là điều kiện cần và đủ cho nghiệm tối ưu của bài toán lồi này. Gồm bốn nhóm:

| Nhóm | Nội dung |
|---|---|
| Dừng | $\mathbf{w} = \sum_i \alpha_i\, y_i \mathbf{x}_i^T, \;\; \sum_i \alpha_i\, y_i = 0, \;\; \alpha_i + \mu_i = C$ |
| Khả thi gốc | $y_i(\mathbf{x}_i \mathbf{w} + b) \ge 1 - \xi_i, \;\; \xi_i \ge 0$ |
| Khả thi đối ngẫu | $\alpha_i \ge 0, \;\; \mu_i \ge 0$ |
| Bù trừ (complementary slackness) | $\alpha_i [y_i(\mathbf{x}_i \mathbf{w} + b) - 1 + \xi_i] = 0, \;\; \mu_i \xi_i = 0$ |

Nhóm bù trừ là nhóm quyết định. Nó nói rằng: hoặc nhân tử bằng 0, hoặc ràng buộc tương ứng "chặt". Vì $\mu_i = C - \alpha_i$, điều kiện $\mu_i \xi_i = 0$ trở thành $(C - \alpha_i)\,\xi_i = 0$. Xét ba trường hợp của $\alpha_i$:

1. $\alpha_i = 0$**:** khi đó $C - \alpha_i = C > 0$ nên $\xi_i = 0$, và ràng buộc khả thi cho $m_i \geq 1$. Vậy $m_i \geq 1$.
2. $0 < \alpha_i < C$**:** vì $\alpha_i > 0$ nên ràng buộc chặt: $m_i = 1 - \xi_i$. Vì $C - \alpha_i > 0$ nên $\xi_i = 0$. Vậy $m_i = 1$.
3. $\alpha_i = C$**:** ràng buộc chặt, $m_i = 1 - \xi_i$, còn $\xi_i \geq 0$ tùy ý. Vậy $m_i \leq 1$.

Chiều ngược lại cũng đúng: $m_i > 1$ buộc $\alpha_i = 0$, còn $m_i < 1$ buộc $\alpha_i = C$ (vì $\xi_i = 1 - m_i > 0$ nên $C - \alpha_i = 0$). Kết quả này **khớp hoàn toàn** với ba trường hợp suy từ hinge ở 4.1.

## Tính $b$ và dự đoán

Với mọi $k$ có $0 < \alpha_k < C$ (mẫu nằm đúng trên biên lề), $m_k = 1$ cho

$$b = y_k - \mathbf{x}_k \mathbf{w} = y_k - \sum_{i=1}^{n} \alpha_i \, y_i \, \mathbf{x}_k \mathbf{x}_i^T.$$

Trong thực hành, lấy trung bình trên mọi mẫu như vậy để giảm sai số số học. Hàm quyết định cho mẫu mới $\mathbf{x}$ là

$$f(\mathbf{x}) = \mathbf{x}\mathbf{w} + b = \sum_{i=1}^{n} \alpha_i \, y_i \, \mathbf{x}\mathbf{x}_i^T + b, \qquad \hat{y} = \operatorname{sign}\big(f(\mathbf{x})\big).$$

Margin của mọi mẫu huấn luyện cũng viết được qua $\mathbf{Q}$. Vì $m_i = y_i\big(\sum_j \alpha_j \, y_j \mathbf{x}_i \mathbf{x}_j^T + b\big)$, ta có

$$\mathbf{m} = \mathbf{Q}\,\boldsymbol{\alpha} + b\,\mathbf{y}.$$

## Ví dụ 1: lề cứng trên 6 sinh viên, giải tay trong không gian đối ngẫu

Nghiệm lề cứng chỉ có $\alpha_2, \alpha_4 > 0$ (mục 2.5). Giả sử ta không biết điều đó mà chỉ giả thiết hai mẫu này là vector hỗ trợ. Từ $\mathbf{y}^T\boldsymbol{\alpha} = -\alpha_2 + \alpha_4 = 0$ suy ra $\alpha_2 = \alpha_4 = \alpha$. Các tích vô hướng cần dùng: $\mathbf{x}_2\mathbf{x}_2^T = 18$, $\mathbf{x}_4\mathbf{x}_4^T = 37$, $\mathbf{x}_2\mathbf{x}_4^T = 21$. Ma trận $\mathbf{Q}$ thu gọn trên hai mẫu này là

$$\mathbf{Q}_{\{2,4\}} = \begin{bmatrix} y_2 y_2 \cdot 18 & y_2 y_4 \cdot 21 \\ y_4 y_2 \cdot 21 & y_4 y_4 \cdot 37 \end{bmatrix} = \begin{bmatrix} 18 & -21 \\ -21 & 37 \end{bmatrix},$$

nên $\boldsymbol{\alpha}^T\mathbf{Q}\boldsymbol{\alpha} = \alpha^2(18 + 37 - 2 \cdot 21) = 13\alpha^2$. Hàm mục tiêu đối ngẫu trở thành một hàm một biến:

$$D(\alpha) = 2\alpha - \frac{1}{2} \cdot 13\alpha^2, \qquad D'(\alpha) = 2 - 13\alpha = 0 \;\Rightarrow\; \alpha = \frac{2}{13}.$$

Từ đó $\mathbf{w} = \alpha(\mathbf{x}_4 - \mathbf{x}_2)^T = \frac{2}{13}[3 \quad -2]^T = \left[\frac{6}{13} \quad -\frac{4}{13}\right]^T$, trùng với kết quả mục 2.5, và $b = y_4 - \mathbf{x}_4\mathbf{w} = -\frac{19}{13}$. Giá trị đối ngẫu là $D = \frac{4}{13} - \frac{2}{13} = \frac{2}{13}$, **bằng đúng** giá trị gốc $\frac{1}{2} \| \mathbf{w} \|^2 = \frac{1}{2} \cdot \frac{4}{13} = \frac{2}{13}$. Bốn mẫu còn lại có $\alpha_i = 0$ và $m_i > 1$ (đã kiểm tra ở mục 2.5), nên mọi điều kiện KKT đều thỏa và đây là nghiệm tối ưu.

Ví dụ này cũng chứng minh ngưỡng ở mục 4.2. Nghiệm lề cứng thỏa các điều kiện KKT của bài toán lề mềm nếu $\alpha_i \le C$ với mọi $i$ (khi đó $\xi_i = 0$ và $0 \le \alpha_i \le C$). Vì $\max_i \alpha_i = \frac{2}{13}$ nên bất kỳ $C \ge \frac{2}{13}$ đều cho nghiệm lề mềm trùng nghiệm lề cứng.

## Ví dụ 2: lề mềm trên 7 sinh viên, kiểm tra đối ngẫu mạnh

Với 7 sinh viên, ma trận Gram $\mathbf{K} = \mathbf{X}\mathbf{X}^T$ và ma trận đối ngẫu $\mathbf{Q} = \mathrm{diag}(\mathbf{y})\mathbf{K}\mathrm{diag}(\mathbf{y})$ là

$$\mathbf{K} = \begin{bmatrix} 20 & 18 & 28 & 16 & 24 & 18 & 18 \\ 18 & 18 & 27 & 21 & 30 & 27 & 24 \\ 28 & 27 & 41 & 29 & 42 & 36 & 33 \\ 16 & 21 & 29 & 37 & 50 & 54 & 43 \\ 24 & 30 & 42 & 50 & 68 & 72 & 58 \\ 18 & 27 & 36 & 54 & 72 & 81 & 63 \\ 18 & 24 & 33 & 43 & 58 & 63 & 50 \end{bmatrix}, \qquad \mathbf{Q}$$

$$= \begin{bmatrix} 20 & 18 & 28 & -16 & -24 & -18 & 18 \\ 18 & 18 & 27 & -21 & -30 & -27 & 24 \\ 28 & 27 & 41 & -29 & -42 & -36 & 33 \\ -16 & -21 & -29 & 37 & 50 & 54 & -43 \\ -24 & -30 & -42 & 50 & 68 & 72 & -58 \\ -18 & -27 & -36 & 54 & 72 & 81 & -63 \\ 18 & 24 & 33 & -43 & -58 & -63 & 50 \end{bmatrix}.$$

Ma trận $\mathbf{Q}$ có cùng giá trị với $\mathbf{K}$ nhưng đổi dấu ở các ô nối hai mẫu khác lớp. Với $\boldsymbol{\alpha} = \begin{bmatrix} 0 & \frac{7}{26} & 0 & 1 & \frac{7}{26} & 0 & 1 \end{bmatrix}^T$:

$$\boldsymbol{\alpha}^T \mathbf{Q} \boldsymbol{\alpha} = \| \mathbf{w} \|^2 = \frac{5}{26}, \qquad \mathbf{1}^T \boldsymbol{\alpha} = \frac{33}{13} \Rightarrow D = \frac{33}{13} - \frac{1}{2} \cdot \frac{5}{26} = \frac{127}{52} \approx 2{,}4423.$$

Giá trị gốc là $J = \frac{1}{2} \| \mathbf{w} \|^2 + C \sum_i \xi_i = \frac{5}{52} + \left( \frac{11}{26} + \frac{25}{13} \right) = \frac{5}{52} + \frac{61}{26} = \frac{127}{52}$, trong đó $\xi_4 = 1 - \frac{15}{26} = \frac{11}{26}$ và $\xi_7 = 1 + \frac{12}{13} = \frac{25}{13}$. Hai giá trị **bằng nhau**, đúng như đối ngẫu mạnh.

## Kiểm tra bằng thư viện

Trong scikit-learn, thuộc tính *dual_coef_* lưu $\alpha_i y_i$ cho các vector hỗ trợ, *support_* lưu chỉ số của chúng:

```
import numpy as np
from sklearn.svm import SVC

X = np.array([[2, 4], [3, 3], [4, 5], [6, 1], [8, 2], [9, 0], [7, 1]], dtype=
float)
y = np.array([-1, -1, -1, 1, 1, 1, -1], dtype=float)

svm = SVC(kernel="linear", C=1.0, tol=1e-12).fit(X, y)
alpha = np.zeros(len(y))
alpha[svm.support_] = np.abs(svm.dual_coef_[0])          # alpha_i = |alpha_i
```

```
y_i|

Q = np.outer(y, y) * (X @ X.T)                          # Q_ij = y_i y_j x_i
x_j^T
print("alpha      :", alpha.round(4))                    # [0. 0.2692 0. 1. 0.
2692 0. 1.]
print("y^T alpha  :", round(alpha @ y, 6))                 # 0.0
print("w          :", (X.T @ (alpha * y)).round(4))       # [ 0.3462 -0.2692]
print("dual value :", round(alpha.sum() - 0.5 * alpha @ Q @ alpha, 4))   # 2.
4423
print("m = Q a + b y:", (Q @ alpha + svm.intercept_[0] * y).round(4))
# [ 1.6154  1.      1.1923  0.5769  1.      1.8846 -0.9231]
```

Kết quả trùng khớp với phép tính tay: $\alpha$, $\mathbf{w}$, giá trị đối ngẫu $\frac{127}{52}$ và vector margin $\mathbf{m}$.

## 4.4. Tính thưa của nghiệm: ba loại mẫu ($\alpha_i = 0$, $0 < \alpha_i < C$, $\alpha_i = C$)

### Ba loại mẫu

Từ các điều kiện bù trừ, mỗi mẫu rơi vào đúng một trong ba loại, tùy giá trị $\alpha_i$:

| Loại | $\alpha_i$ | Margin | $\xi_i$ | Tên gọi | Vai trò |
|---|---|---|---|---|---|
| 1 | $0$ | $m_i \geq 1$ | $0$ | **Không phải vector hỗ trợ** | Không ảnh hưởng đến $\mathbf{w}$ và $b$ |
| 2 | $0 < \alpha_i < C$ | $m_i = 1$ | $0$ | **Vector hỗ trợ tự do** (free) | Nằm đúng trên biên lề; dùng để tính $b$ |
| 3 | $\alpha_i = C$ | $m_i \leq 1$ | $\xi_i \geq 0$ | **Vector hỗ trợ bị chặn** (bounded) | Lấn lề hoặc sai phía; lực bị chặn ở $C$ |

Khái niệm **vector hỗ trợ** nay được tổng quát hóa: một mẫu là vector hỗ trợ khi $\alpha_i > 0$, tức thuộc loại 2 hoặc 3. Với lề cứng (không có cận trên $C$), chỉ có loại 1 và loại 2, tức vector hỗ trợ là các mẫu có $m_i = 1$ đúng như định nghĩa ở mục 2.3. Với lề mềm, định nghĩa mở rộng để bao gồm cả các mẫu lấn lề và sai phía.

Ví dụ trên 7 sinh viên với $C = 1$ (Hình 10): sinh viên 1,3,6 thuộc loại 1; sinh viên 2,5 thuộc loại 2; sinh viên 4,7 thuộc loại 3. Như vậy 4 trong 7 mẫu là vector hỗ trợ.

### Bỏ mẫu loại 1 không đổi nghiệm

Tính chất chứng minh ở mục 2.3 cho lề cứng nay có chứng minh ngắn hơn nhờ KKT, và đúng cho cả lề mềm.

**Mệnh đề.** Nếu bỏ khỏi dữ liệu các mẫu có $\alpha_i = 0$, nghiệm $(\mathbf{w}, b)$ không đổi.

*Chứng minh.* Gọi $\mathcal{A} = \{i : \alpha_i > 0\}$ là tập vector hỗ trợ. Xét bài toán chỉ gồm các mẫu thuộc $\mathcal{A}$ và giữ nguyên $\alpha_i$ cho chúng, đặt $\alpha_i = 0$ cho phần còn lại (đã bị bỏ). Công thức $\mathbf{w} = \sum_{i \in \mathcal{A}} \alpha_i y_i \mathbf{x}_i^T$ và $\sum_{i \in \mathcal{A}} \alpha_i y_i = 0$ vẫn đúng vì các số hạng bị bỏ đều bằng 0. Điều kiện bù trừ của mọi $i \in \mathcal{A}$ không thay đổi (chúng chỉ phụ thuộc $\mathbf{w}, b$ và $\alpha_i$). Như vậy $(\mathbf{w}, b, \boldsymbol{\alpha}_{\mathcal{A}})$ thỏa mọi điều kiện KKT

của bài toán rút gọn. Vì KKT đủ cho tối ưu trong bài toán lồi, nó là nghiệm của bài toán rút gọn, và do **w** duy nhất nên đó chính là nghiệm cũ. ■

Trên thực nghiệm (mục 2.3, Hình 5), chỉ cần giữ lại hai vector hỗ trợ của 6 sinh viên là thu được nguyên đường biên.

## Hệ quả của tính thưa

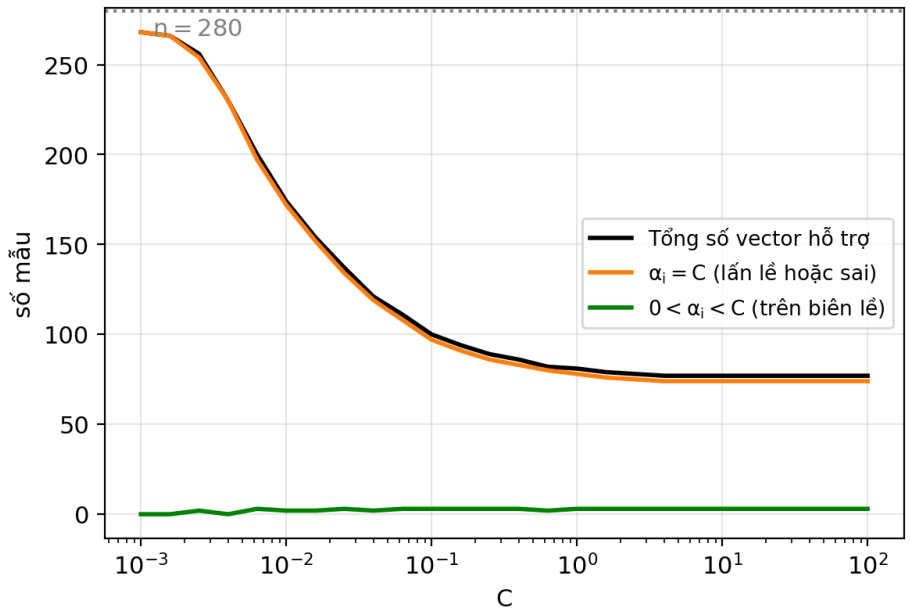
1. **Dự đoán nhanh.** Hàm quyết định $f(\mathbf{x}) = \sum_{i \in \mathcal{A}} \alpha_i \, y_i \mathbf{x}\mathbf{x}_i^T + b$ chỉ cần các vector hỗ trợ. Với mô hình tuyến tính ta có thể tính trước **w**, nhưng với kernel (mục 5) thì chi phí dự đoán tỷ lệ với số vector hỗ trợ.

2. **Cận trên cho sai số tổng quát hóa.** Xét sai số kiểu *leave-one-out* (bỏ lần lượt từng mẫu, huấn luyện trên phần còn lại, rồi dự đoán mẫu bị bỏ). Một mẫu loại 1 có $m_i \geq 1 > 0$ nên được phân loại đúng, và nếu bỏ nó đi thì nghiệm không đổi (mệnh đề trên), nên nó vẫn được dự đoán đúng. Do đó chỉ các vector hỗ trợ mới có thể gây ra lỗi, và

$$\#\{\text{lỗi leave-one-out}\} \;\leq\; \#\{\text{vector hỗ trợ}\}.$$

Số vector hỗ trợ **ít** là dấu hiệu tốt cho khả năng tổng quát hóa.

- **Ảnh hưởng của $C$ đến số vector hỗ trợ.** $C$ nhỏ làm các hệ số $\alpha_i$ bị chặn ở mức thấp nên **nhiều** mẫu có $\alpha_i = C$ (lề rộng, nhiều mẫu lấn lề). $C$ lớn làm số vector hỗ trợ giảm. Hình 12 minh họa trên một bộ dữ liệu chồng lấn gồm 280 mẫu huấn luyện: khi $C = 10^{-3}$ có 268 mẫu là vector hỗ trợ, còn khi $C \geq 10$ chỉ còn khoảng 77 mẫu. Vì hai lớp chồng lấn nên phần lớn vector hỗ trợ ở đây thuộc loại bị chặn; chỉ có khoảng 3 mẫu nằm đúng trên biên lề (loại 2).




*Hình 12. Số vector hỗ trợ theo $C$ trên bộ dữ liệu hai đám mây chồng lấn (280 mẫu huấn luyện, đã chuẩn hóa). $C$ nhỏ thì hầu hết mẫu đều là vector hỗ trợ bị chặn ($\alpha_i = C$); khi $C$ tăng, tổng số vector hỗ trợ giảm và ổn định.*

## 4.5. Mối liên hệ với Perceptron dạng đối ngẫu và cách giải bài toán (SMO ở mức trực giác)

### Perceptron cũng có dạng đối ngẫu

Giả sử huấn luyện Perceptron từ $\mathbf{w} = \mathbf{0}$, $b = 0$ với tốc độ học $\eta$. Mỗi lần mẫu $i$ bị sai, ta cập nhật $\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i^T$ và $b \leftarrow b + \eta y_i$. Gọi $n_i$ là số lần mẫu $i$ gây ra cập nhật, đặt $\alpha_i = \eta n_i \geq 0$. Khi kết thúc,

$$\mathbf{w} = \sum_{i=1}^{n} \alpha_i \, y_i \mathbf{x}_i^T, \qquad b = \sum_{i=1}^{n} \alpha_i \, y_i, \qquad f(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i \, y_i (\mathbf{x}\mathbf{x}_i^T + 1).$$

Thuật toán **Perceptron dạng đối ngẫu** bỏ hẳn $\mathbf{w}$ và chỉ duy trì $\boldsymbol{\alpha}$: nếu $y_i f(\mathbf{x}_i) \leq 0$ thì tăng $\alpha_i \leftarrow \alpha_i + \eta$. Dữ liệu cũng chỉ xuất hiện qua các tích vô hướng $\mathbf{x}\mathbf{x}_i^T$.

**Ví dụ.** Chạy Perceptron với $\eta = 1$ trên 6 sinh viên theo thứ tự 1,2, … ,6 (đường A ở mục 1.1), số lần cập nhật là $\mathbf{n} = [1 \quad 2 \quad 0 \quad 2 \quad 0 \quad 0]^T$. Kiểm tra:

$$\mathbf{w} = -1 \cdot \begin{bmatrix} 2 \\ 4 \end{bmatrix} - 2 \begin{bmatrix} 3 \\ 3 \end{bmatrix} + 2 \begin{bmatrix} 6 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ -8 \end{bmatrix}, \qquad b = -1 - 2 + 2 = -1,$$

đúng bằng nghiệm $(b, \mathbf{w}) = \big(-1, \, (4, -8)\big)$ ở bảng mục 1.1. Với thứ tự ngược 6,5, … ,1 ta được $\mathbf{n} = [0 \quad 1 \quad 2 \quad 0 \quad 1 \quad 1]^T$ và đường B.

## SVM đối ngẫu: cùng dạng nghiệm, khác cách tìm $\boldsymbol{\alpha}$

Cả hai mô hình đều viết $\mathbf{w}$ như tổ hợp tuyến tính của các mẫu huấn luyện, $\mathbf{w} = \sum_i \alpha_i \, y_i \mathbf{x}_i^T$. Khác biệt nằm ở **bản chất và cách tìm** $\alpha_i$:

| | Perceptron dạng đối ngẫu | SVM dạng đối ngẫu |
|---|---|---|
| Ý nghĩa $\alpha_i$ | Số lần mẫu $i$ gây cập nhật (nhân $\eta$) | Nhân tử Lagrange, "cường độ lực" của mẫu $i$ |
| Giá trị | Số nguyên (nhân $\eta$), **không có trần** | Số thực trong $[0, C]$ |
| Hệ số chệch | $b = \sum_i \alpha_i \, y_i$, **không bị ràng buộc** | $\sum_i \alpha_i \, y_i = 0$, **cân bằng lực** |
| Cách tìm | Duyệt tuần tự, tăng khi sai (trực tuyến) | Giải bài toán tối ưu $\max \mathbf{1}^T\boldsymbol{\alpha} - \frac{1}{2}\boldsymbol{\alpha}^T\mathbf{Q}\boldsymbol{\alpha}$ |
| Mẫu có $\alpha_i > 0$ | Mẫu từng bị sai (phụ thuộc thứ tự) | Vector hỗ trợ (xác định bởi hình học, không phụ thuộc thứ tự) |
| Nghiệm | Không duy nhất | $\mathbf{w}$ duy nhất, lề lớn nhất |

Trên 6 sinh viên, nghiệm Perceptron theo thứ tự $1, \ldots, 6$ dùng ba sinh viên 1,2,4 với $\boldsymbol{\alpha} = (1,2,0,2,0,0)$, còn nghiệm SVM lề cứng chỉ dùng sinh viên 2 và 4 với $\boldsymbol{\alpha} = \left(0, \frac{2}{13}, 0, \frac{2}{13}, 0, 0\right)$. Hai bộ hệ số có cùng "hình dạng" (một tổ hợp của vài mẫu) nhưng khác hẳn về ý nghĩa: $\boldsymbol{\alpha}$ của Perceptron là **vết lịch sử** của các lần sai, còn $\boldsymbol{\alpha}$ của SVM là **nghiệm của một bài toán tối ưu** mã hóa ý tưởng lề cực đại. Có thể nói SVM là Perceptron dạng đối ngẫu nhưng với điều kiện cân bằng lực $\mathbf{y}^T\boldsymbol{\alpha} = 0$, cận trên $C$ và mục tiêu $\frac{1}{2}\boldsymbol{\alpha}^T\mathbf{Q}\boldsymbol{\alpha}$ chọn ra nghiệm duy nhất có lề lớn nhất.

## Cách giải bài toán đối ngẫu

Bài toán đối ngẫu là một quy hoạch toàn phương lồi với $n$ biến, một ràng buộc đẳng thức và ràng buộc hộp. Có ba hướng giải thông dụng:

| Cách giải | Ý tưởng | Đặc điểm |
|---|---|---|
| Bộ giải QP tổng quát | Phương pháp điểm trong trên toàn bộ $\mathbf{Q}$ | Chi phí cỡ $O(n^3)$, cần lưu $\mathbf{Q}$ cỡ $n \times n$, chỉ hợp dữ liệu nhỏ |
| **SMO** | Mỗi bước tối ưu giải tích hai biến $\alpha_i, \alpha_j$ | Không cần lưu cả $\mathbf{Q}$; là lõi của thư viện LIBSVM (*SVC* của scikit-learn) |
| Giải trên bài toán gốc | Gradient/SGD trên hàm hinge (mục 3.6) hoặc tọa độ đối ngẫu | Rất nhanh cho SVM tuyến tính dữ liệu lớn (*LinearSVC*, LIBLINEAR) |

## SMO ở mức trực giác

**SMO (Sequential Minimal Optimization)** dựa trên một quan sát: nếu chỉ cho phép đổi **một** biến $\alpha_i$ thì ràng buộc $\mathbf{y}^T\boldsymbol{\alpha} = 0$ bị phá vỡ. Số biến tối thiểu có thể đổi cùng lúc là **hai**. Với cặp $(i, j)$, ta đổi

$$\alpha_i \leftarrow \alpha_i + y_i t, \qquad \alpha_j \leftarrow \alpha_j - y_j t,$$

thì $\sum_k \alpha_k y_k$ giữ nguyên (vì $y_i^2 = 1$). Khi đó hàm đối ngẫu chỉ còn là một **hàm bậc hai theo** $t$, có nghiệm đóng. Gọi $f(\mathbf{x}_k)$ là hàm quyết định hiện tại và $E_k = f(\mathbf{x}_k) - y_k$ là sai số của mẫu $k$. Đạo hàm của hàm đối ngẫu theo $\alpha_k$ là $1 - (\mathbf{Q}\boldsymbol{\alpha})_k = 1 - y_k(f(\mathbf{x}_k) - b)$. Suy ra độ dốc theo hướng thay đổi $t$ là $E_j - E_i$ và độ cong là $\kappa_{ij} = K_{ii} + K_{jj} - 2K_{ij}$, nên

$$t^* = \frac{E_j - E_i}{\kappa_{ij}}, \qquad \kappa_{ij} = K_{ii} + K_{jj} - 2K_{ij}.$$

Sau đó $t^*$ được **cắt** sao cho cả $\alpha_i$ và $\alpha_j$ vẫn nằm trong $[0, C]$, rồi cập nhật $b$ theo điều kiện KKT. Vòng lặp chọn cặp $(i, j)$ vi phạm KKT nhiều nhất (ví dụ mẫu có $\alpha_i < C$ mà $m_i < 1$, hay $\alpha_i > 0$ mà $m_i > 1$), lặp lại đến khi mọi mẫu thỏa KKT trong một sai số nhỏ. Mỗi bước tăng hàm đối ngẫu và chặn trên bởi $D^*$ nên thuật toán hội tụ. Cái hay của SMO là mỗi bước chỉ cần hai hàng của $\mathbf{K}$, nên bộ nhớ nhỏ.

**Ví dụ một bước SMO.** Lấy bài toán một chiều ở câu hỏi mục 2: $x_1 = 1$ ($y_1 = -1$) và $x_2 = 3$ ($y_2 = +1$), lề cứng (coi $C$ đủ lớn). Khởi tạo $\alpha_1 = \alpha_2 = 0$, $b = 0$. Khi đó $f \equiv 0$ nên $E_1 = 0 - (-1) = 1$ và $E_2 = 0 - 1 = -1$. Các phần tử Gram: $K_{11} = 1$, $K_{22} = 9$, $K_{12} = 3$, nên $\eta_{12} = 1 + 9 - 2 \cdot 3 = 4$. Chọn cặp $(i, j) = (1, 2)$:

$$t^* = \frac{E_2 - E_1}{\eta_{12}} = \frac{-1 - 1}{4} = -\frac{1}{2}, \qquad \alpha_1 = 0 + y_1 t^* = 0 + (-1)\left(-\frac{1}{2}\right) = \frac{1}{2}, \quad \alpha_2 = 0 - y_2 t^* = \frac{1}{2}.$$

Cả hai $\alpha$ nằm trong khoảng cho phép. Từ $\alpha_1 = \alpha_2 = \frac{1}{2}$ ta có $w = \alpha_1 y_1 x_1 + \alpha_2 y_2 x_2 = -\frac{1}{2} + \frac{3}{2} = 1$, và từ vector hỗ trợ $x_2$: $b = y_2 - x_2 w = 1 - 3 = -2$. Đường biên $x = 2$, độ rộng lề $2/|w| = 2$, đúng bằng nghiệm tìm ở mục 2 chỉ sau **một** bước. Với bài toán có nhiều hơn hai biến, SMO lặp bước này trên nhiều cặp khác nhau.

## Tóm tắt mục 4

1. **Cơ chế học.** Điều kiện tối ưu của $J$ cho $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i^T$ và $\mathbf{y}^T\boldsymbol{\alpha} = 0$. Mẫu ngoài lề ($m_i > 1$) có lực $\alpha_i = 0$; mẫu lấn lề hoặc sai ($m_i < 1$) có lực **bị chặn** ở $\alpha_i = C$; mẫu trên biên lề có $0 \le \alpha_i \le C$. Điều kiện $\mathbf{y}^T\boldsymbol{\alpha} = 0$ là cân bằng lực giữa hai lớp và xác định $b$.

2. **Tồn tại và duy nhất.** $J$ luôn có cực tiểu, và $\mathbf{w}^*$ duy nhất nhờ tính lồi chặt của $\frac{1}{2}\|\mathbf{w}\|^2$, kể cả khi dữ liệu phân tách hoàn hảo. Khi dữ liệu phân tách được, lề mềm trùng lề cứng với mọi $C \ge \max_i \alpha_i^{\text{cứng}}$ (ví dụ $\frac{2}{13}$ cho 6 sinh viên).

3. **Dạng đối ngẫu.** $\max \mathbf{1}^T\boldsymbol{\alpha} - \frac{1}{2}\boldsymbol{\alpha}^T\mathbf{Q}\boldsymbol{\alpha}$ với $\mathbf{y}^T\boldsymbol{\alpha} = 0$, $\mathbf{0} \le \boldsymbol{\alpha} \le C\mathbf{1}$, trong đó $\mathbf{Q} = \operatorname{diag}(\mathbf{y})\mathbf{X}\mathbf{X}^T\operatorname{diag}(\mathbf{y})$. Giá trị tối ưu bằng giá trị gốc (đối ngẫu mạnh). Dữ liệu chỉ xuất hiện qua tích vô hướng $\mathbf{x}_i\mathbf{x}_j^T$.

4. **KKT** cho ba loại mẫu: $\alpha_i = 0 \Leftrightarrow m_i > 1$ (hoặc $\geq 1$), $0 < \alpha_i < C \Rightarrow m_i = 1$, $\alpha_i = C \Leftarrow m_i < 1$. Có thể tính $b = y_k - \mathbf{x}_k\mathbf{w}$ từ mẫu $k$ bất kỳ có $0 < \alpha_k < C$.

5. **Tính thưa.** Vector hỗ trợ là các mẫu có $\alpha_i > 0$. Bỏ mẫu có $\alpha_i = 0$ không đổi nghiệm, số lỗi leave-one-out không vượt quá số vector hỗ trợ, và $C$ nhỏ làm số vector hỗ trợ tăng.

6. **Liên hệ Perceptron.** Perceptron dạng đối ngẫu cũng cho $\mathbf{w} = \sum_i \alpha_i \, y_i \mathbf{x}_i^T$ nhưng $\alpha_i$ là số lần sai, không bị chặn và không cân bằng. SVM chọn $\boldsymbol{\alpha}$ bằng bài toán tối ưu. Bài toán đối ngẫu được giải bằng QP hoặc **SMO**, vốn cập nhật hai biến mỗi bước để giữ $\mathbf{y}^T\boldsymbol{\alpha} = 0$.

## Câu hỏi kiểm tra nhanh

1. Với nghiệm ở 4.1 ($C = 1$, 7 sinh viên), sinh viên 4 có $m_4 \approx 0{,}58$ và sinh viên 7 có $m_7 \approx -0{,}92$. Hai sinh viên này thuộc loại nào, và lực của chúng lên $\mathbf{w}$ khác nhau thế nào so với trường hợp sinh viên 7 sai nặng hơn nữa (ví dụ $m_7 = -5$)?

2. Vì sao điều kiện $\mathbf{y}^T\boldsymbol{\alpha} = 0$ gắn với việc $b$ không xuất hiện trong hàm mục tiêu? Nếu tổng $\alpha$ của lớp dương lớn hơn lớp âm thì đường biên có xu hướng dịch về phía nào?

3. Cho hai điểm một chiều $x_1 = 0$ ($y_1 = -1$), $x_2 = 2$ ($y_2 = +1$). Viết ma trận $\mathbf{K}$, $\mathbf{Q}$, bài toán đối ngẫu lề cứng, rồi giải để tìm $\alpha_1, \alpha_2$, $w$ và $b$.

4. Vì sao SVM lề mềm luôn có nghiệm $\mathbf{w}^*$ duy nhất, trong khi Logistic Regression không điều chuẩn trên dữ liệu phân tách hoàn hảo lại không có nghiệm hữu hạn?

5. Nếu thêm vào 6 sinh viên một sinh viên mới nằm xa đường biên, về đúng phía của lớp mình (ví dụ $\mathbf{x} = (10{,}0)$, $y = +1$), nghiệm lề cứng có đổi không? Giải thích bằng $\alpha_i$.

### Gợi ý đáp án

1. Sinh viên 4: $m_4 < 1$ nhưng $> 0$ nên **lấn lề**, thuộc loại 3 với $\alpha_4 = C = 1$. Sinh viên 7: $m_7 < 0$ nên **sai phía**, cũng loại 3 với $\alpha_7 = C = 1$. Dù $m_7 = -0{,}92$ hay $m_7 = -5$, lực vẫn là $\alpha_7 y_7 \mathbf{x}_7^T = -C\,\mathbf{x}_7^T$, **không đổi**. Mức độ sai chỉ ảnh hưởng đến $\xi_7$ (tức giá trị hàm mục tiêu), không ảnh hưởng đến cường độ lực.

2. Biến $b$ không có trong $\frac{1}{2} \| \mathbf{w} \|^2$, nên đạo hàm của Lagrangian theo $b$ chỉ gồm $-\sum_i \alpha_i \, y_i$, buộc nó bằng 0. Nếu $\sum_{y_i=+1} \alpha_i > \sum_{y_i=-1} \alpha_i$, lớp dương "đẩy" mạnh hơn nên đường biên bị đẩy về phía lớp âm, khiến $b$ thay đổi (đường biên dịch về phía lớp âm) cho đến khi hai tổng bằng nhau.

3. $\mathbf{K} = \begin{bmatrix} 0 & 0 \\ 0 & 4 \end{bmatrix}$ (vì $K_{11} = 0,\ K_{12} = 0,\ K_{22} = 4$), $\mathbf{Q} = \begin{bmatrix} 0 & 0 \\ 0 & 4 \end{bmatrix}$ (dấu của $Q_{12}$ không ảnh hưởng vì $K_{12} = 0$). Từ $\mathbf{y}^T\boldsymbol{\alpha} = 0$ ta có $\alpha_1 = \alpha_2 = \alpha$. Hàm đối ngẫu $D(\alpha) = 2\alpha - \frac{1}{2} \cdot 4\alpha^2 = 2\alpha - 2\alpha^2$, nên $\alpha = \frac{1}{2}$. Khi đó $w = \alpha(x_2 - x_1) = \frac{1}{2} \cdot 2 = 1$, và $b = y_2 - x_2 w = 1 - 2 = -1$. Đường biên $x = 1$ (trung điểm của 0 và 2), độ rộng lề $2/|w| = 2$.

4. Với SVM, hinge **dừng thưởng** khi $m \geq 1$ và số hạng $\frac{1}{2} \| \mathbf{w} \|^2$ lồi chặt và phạt $\| \mathbf{w} \|$, nên $J \to \infty$ khi $\| \mathbf{w} \| \to \infty$ và có cực tiểu hữu hạn duy nhất theo $\mathbf{w}$. Logistic không điều chuẩn luôn thưởng thêm khi tăng margin, nên $J$ chỉ giảm dần về 0 khi $\| \mathbf{w} \| \to \infty$ mà không bao giờ đạt.

5. Mẫu $(10,0)$, $y = +1$ có điểm số $\mathbf{x}\mathbf{w} + b = 10 \cdot \frac{6}{13} - \frac{19}{13} = \frac{41}{13} \approx 3{,}15$, nên $m \approx 3{,}15 > 1$: mẫu nằm ngoài dải lề, thuộc loại 1 với $\alpha = 0$. Theo mệnh đề ở 4.4, mẫu mới này không làm đổi nghiệm lề cứng (nghiệm cũ vẫn thỏa mọi điều kiện KKT với $\alpha_{\text{mới}} = 0$).

# 5. SVM phi tuyến: thủ thuật kernel

Mục 2 đến mục 4 xây dựng SVM tuyến tính. Dù ta đã nới lỏng ràng buộc bằng lề mềm, đường biên vẫn là một **siêu phẳng**, nên mô hình vẫn chịu cùng giới hạn như Perceptron và Logistic Regression: không phân loại tốt được dữ liệu như XOR hay hai vòng tròn đồng tâm. Hai bài trước đã chỉ ra cách thoát: **thêm đặc trưng phi tuyến** để dữ liệu trở nên phân tách được trong một không gian mới. Tuy nhiên, cách này có cái giá rất đắt khi số đặc trưng tăng nhanh.

Mục 4 đã để lại một chìa khóa. Ở dạng đối ngẫu (mục 4.3), dữ liệu **chỉ xuất hiện qua các tích vô hướng** $\mathbf{x}_i\mathbf{x}_j^T$, cả khi huấn luyện lẫn khi dự đoán. Điều đó gợi ra một ý tưởng: nếu ta chỉ cần tích vô hướng, liệu ta có thể **tính tích vô hướng trong không gian đặc trưng cao chiều mà không cần dựng tường minh các đặc trưng đó**? Câu trả lời là có, và kỹ thuật này gọi là **thủ thuật kernel (kernel trick)**. Trong mục 5, ta trả lời các câu hỏi:

1. Cách thêm đặc trưng đa thức có cái giá gì, và vì sao cần một cách khác?
2. Thủ thuật kernel thay tích vô hướng $\mathbf{x}_i\mathbf{x}_j^T$ bằng $k(\mathbf{x}_i, \mathbf{x}_j)$ như thế nào mà không dựng đặc trưng?
3. Các kernel phổ biến (tuyến tính, đa thức, RBF) là gì, và ma trận Gram $\mathbf{K}$ có vai trò gì?
4. Dự đoán qua kernel diễn ra ra sao, và $\gamma$ cùng $C$ điều khiển đường biên cong như thế nào?
5. Giải tay XOR bằng kernel đa thức bậc hai, cài đặt SVM kernel từ đầu, và so sánh với mạng nơ-ron.

**Dữ liệu xuyên suốt.** Ta dùng hai bộ dữ liệu quen thuộc từ hai bài trước:

1. **XOR** với $n = 4$ mẫu, nhãn $\{-1, +1\}$ như bài Perceptron:

$$\mathbf{X} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} -1 \\ +1 \\ +1 \\ -1 \end{bmatrix}.$$

2. **Hai vòng tròn đồng tâm có nhiễu**: lớp trong ($y = +1$) và vòng ngoài ($y = -1$), 300 điểm chia thành 210 mẫu huấn luyện và 90 mẫu kiểm tra. Ta sinh lại một bộ tương tự bộ ở bài Logistic Regression (nên con số chính xác không trùng khớp tuyệt đối), và chuẩn hóa đặc trưng theo trung bình và độ lệch chuẩn của tập huấn luyện như ở mục 3.5.

Ký hiệu giữ nguyên như các mục trước. Với hai điểm $\mathbf{x}$, $\mathbf{z}$ (vector hàng), $\mathbf{x}\mathbf{z}^T$ là tích vô hướng thông thường. Số thứ tự hình tiếp tục từ mục 4.

## 5.1. Nhắc lại đặc trưng đa thức (XOR, vòng tròn) và cái giá của việc tự thêm đặc trưng

### Ánh xạ đặc trưng

Khi dữ liệu không phân tách tuyến tính được trong không gian gốc $\mathbb{R}^d$, ta tìm một **ánh xạ đặc trưng (feature map)** $\varphi: \mathbb{R}^d \to \mathbb{R}^D$ sao cho dữ liệu đã ánh xạ $\varphi(\mathbf{x}_i)$ phân tách tuyến tính được. Khi đó ta huấn luyện SVM trong không gian mới, thay mọi $\mathbf{x}_i$ bằng $\varphi(\mathbf{x}_i)$:

$$s = \varphi(\mathbf{x})\,\mathbf{w} + b, \qquad \mathbf{w} \in \mathbb{R}^D,$$

với $\varphi(\mathbf{x})$ là vector hàng $1 \times D$. Mô hình vẫn **tuyến tính theo** $\varphi(\mathbf{x})$ nên mọi thứ ở mục 2 đến mục 4 (lề, vector hỗ trợ, đối ngẫu, KKT) áp dụng nguyên vẹn, chỉ khác là lề được đo trong không gian mới. Nhưng trong không gian gốc, tập $\{\mathbf{x}: s = 0\}$ là một **đường cong**.

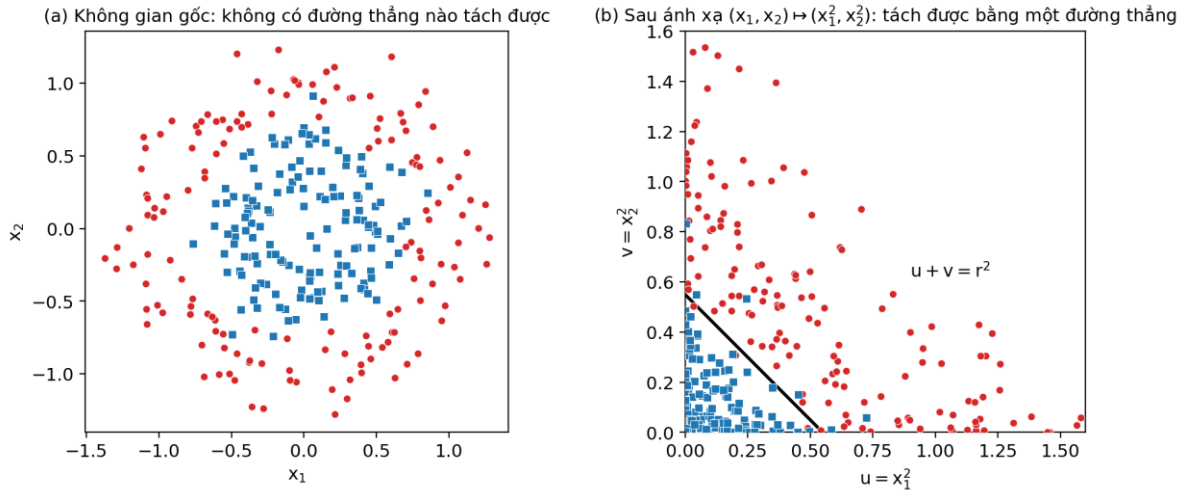
### XOR và vòng tròn

**XOR.** Ở bài Perceptron, ta đã thấy bất kỳ đường thẳng nào cũng sai ít nhất một trong bốn điểm, nên độ chính xác tối đa là $3/4$. Nếu thêm đặc trưng tích $\varphi(\mathbf{x}) = [x_1 \quad x_2 \quad x_1x_2]$, ta chọn được $\mathbf{w} = [1 \quad 1 \quad -2]^T$, $b = -0{,}5$ với điểm số $s = x_1 + x_2 - 2x_1x_2 - 0{,}5$:

$$\mathbf{s} = \begin{bmatrix} -0{,}5 \\ 0{,}5 \\ 0{,}5 \\ -0{,}5 \end{bmatrix}, \qquad \mathbf{m} = \mathbf{y} \odot \mathbf{s} = \begin{bmatrix} 0{,}5 \\ 0{,}5 \\ 0{,}5 \\ 0{,}5 \end{bmatrix} > \mathbf{0}.$$

Mọi margin dương nên XOR đã được phân loại đúng hoàn toàn trong không gian đặc trưng mới.

**Hai vòng tròn đồng tâm.** Đường tròn bán kính $r$ có phương trình $x_1^2 + x_2^2 = r^2$. Nếu đặt $u = x_1^2$ và $v = x_2^2$ thì phương trình này trở thành $u + v = r^2$, tức là **một đường thẳng** trong mặt phẳng $(u, v)$ (Hình 13). Lớp trong ($x_1^2 + x_2^2 < r^2$) và lớp ngoài ($> r^2$) nằm hai bên đường thẳng đó. Với ánh xạ đầy đủ bậc hai $\varphi(\mathbf{x}) = [x_1 \quad x_2 \quad x_1^2 \quad x_1x_2 \quad x_2^2]$, mô hình có thể biểu diễn mọi đường conic (đường tròn, elip, parabol, hyperbol), và ở bài Logistic Regression điều này đã đưa độ chính xác kiểm tra từ 58,9% lên 98,9%.




*Hình 13. (a) Hai vòng tròn đồng tâm: không có đường thẳng nào tách được hai lớp. (b) Sau ánh xạ $(x_1, x_2) \mapsto (x_1^2, x_2^2)$, đường tròn $x_1^2 + x_2^2 = r^2$ trở thành đường thẳng $u + v = r^2$ và hai lớp tách được bằng một siêu phẳng.*

## Cái giá của việc tự thêm đặc trưng

Cách làm này có ba cái giá, đã nhắc ở bài Logistic Regression và trở nên nghiêm trọng hơn với SVM:

3. **Số đặc trưng bùng nổ.** Với $d$ đặc trưng gốc và bậc tối đa $k$, số đặc trưng đa thức (không kể hệ số chệch) là $D = \binom{d+k}{k} - 1$.

| $d$ | Bậc $k$ | Số đặc trưng $D$ |
|---|---|---|
| 2 | 2 | 5 |
| 10 | 3 | 285 |
| 100 | 2 | 5 150 |
| 100 | 3 | 176 850 |
| 100 | 5 | 96 560 645 |
| 784 (ảnh $28 \times 28$) | 3 | 80 931 144 |

Lưu trữ ma trận đặc trưng mới tốn bộ nhớ cỡ $n \times D$ và mỗi phép tính $\mathbf{w}$ hay $\varphi(\mathbf{x})$ tốn cỡ $D$.

1. **Phải tự đoán dạng đặc trưng.** Muốn đường tròn thì thêm $x_1^2, x_2^2$, muốn XOR thì thêm $x_1 x_2$. Với dữ liệu nhiều chiều, ta không biết nên thêm gì.
2. **Nhiều đặc trưng dễ làm mô hình học thuộc** dữ liệu huấn luyện và làm dữ liệu trở nên phân tách hoàn hảo (mục 4.4 của bài Logistic Regression). Với SVM, điều chuẩn $\frac{1}{2} \| \mathbf{w} \|^2$ đã giúp kiểm soát điều này, nhưng chi phí tính toán thì vẫn còn.

Ta muốn **tận dụng sức mạnh của không gian đặc trưng cao chiều mà không trả cái giá** $D$, thậm chí với $D = \infty$. Đối ngẫu cho ta cách làm đó.

## 5.2. Thủ thuật kernel: dạng đối ngẫu chỉ cần tích vô hướng, $\boldsymbol{k(\mathbf{x},\mathbf{z}) = \varphi(\mathbf{x})\varphi(\mathbf{z})^T}$

### Dạng đối ngẫu trong không gian đặc trưng

Ở mục 4.3, bài toán đối ngẫu của SVM lề mềm là

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \frac{1}{2}\sum_i \sum_j \alpha_i\,\alpha_j y_i y_j\,\mathbf{x}_i\mathbf{x}_j^T \quad \text{với} \quad \mathbf{y}^T\boldsymbol{\alpha} = 0,\ \ \mathbf{0} \le \boldsymbol{\alpha} \le C\mathbf{1},$$

và hàm quyết định là $f(\mathbf{x}) = \sum_i \alpha_i\, y_i\, \mathbf{x}\mathbf{x}_i^T + b$. Nếu ta huấn luyện trong không gian đặc trưng, tức thay $\mathbf{x}_i$ bằng $\varphi(\mathbf{x}_i)$ ở mọi nơi, thì dạng đối ngẫu giữ nguyên cấu trúc, chỉ có tích vô hướng đổi thành $\varphi(\mathbf{x}_i)\varphi(\mathbf{x}_j)^T$. **Dữ liệu và ánh xạ $\varphi$ chỉ xuất hiện qua tích vô hướng này**, không bao giờ xuất hiện riêng lẻ. Do đó, nếu ta có một hàm tính nhanh được tích vô hướng đó mà không cần biết $\varphi$, ta sẽ không cần dựng $\varphi(\mathbf{x})$.

### Hàm kernel

**Hàm kernel** là hàm hai biến

$$k(\mathbf{x},\mathbf{z}) = \varphi(\mathbf{x})\,\varphi(\mathbf{z})^T.$$

**Thủ thuật kernel** là thay mọi tích vô hướng $\mathbf{x}_i\mathbf{x}_j^T$ trong dạng đối ngẫu bằng $k(\mathbf{x}_i,\mathbf{x}_j)$. Ma trận $\mathbf{Q}$ trở thành $Q_{ij} = y_i y_j\, k(\mathbf{x}_i,\mathbf{x}_j)$, tức $\mathbf{Q} = \mathrm{diag}(\mathbf{y})\,\mathbf{K}\,\mathrm{diag}(\mathbf{y})$ với **ma trận Gram** $K_{ij} = k(\mathbf{x}_i,\mathbf{x}_j)$ (cỡ $n \times n$). Bài toán đối ngẫu có kernel, và hàm quyết định tương ứng, là

$$\boxed{\max_{\boldsymbol{\alpha}} \mathbf{1}^T\boldsymbol{\alpha} - \frac{1}{2}\boldsymbol{\alpha}^T\mathbf{Q}\,\boldsymbol{\alpha},\ \ \mathbf{y}^T\boldsymbol{\alpha} = 0,\ \ \mathbf{0} \le \boldsymbol{\alpha} \le C\mathbf{1}; \qquad f(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i\, y_i\, k(\mathbf{x}_i,\mathbf{x}) + b.}$$

Mọi thứ khác đều giữ nguyên: điều kiện KKT, ba loại mẫu ($\alpha_i = 0$, $0 < \alpha_i < C$, $\alpha_i = C$), tính thưa, công thức $b = y_k - \sum_i \alpha_i\, y_i k(\mathbf{x}_i,\mathbf{x}_k)$ với $0 < \alpha_k < C$, và các thuật toán giải như QP hay SMO. Chỉ cần đổi cách tính ma trận Gram. Điều quan trọng là bài toán đối ngẫu có $n$ **biến, không phụ thuộc** $D$.

Vector trọng số trong không gian đặc trưng là $\mathbf{w} = \sum_i \alpha_i\, y_i\, \varphi(\mathbf{x}_i)^T \in \mathbb{R}^D$, nhưng ta **không cần tính nó**. Điều này đặc biệt quan trọng khi $D$ rất lớn hoặc vô hạn: ta không bao giờ lưu $\mathbf{w}$, chỉ lưu $\boldsymbol{\alpha}$ và các vector hỗ trợ.

### Ví dụ: kernel đa thức bậc hai

Với $d = 2$, xét hàm $k(\mathbf{x},\mathbf{z}) = (\mathbf{x}\mathbf{z}^T)^2$. Khai triển:

$$(\mathbf{x}\mathbf{z}^T)^2 = (x_1z_1 + x_2z_2)^2 = x_1^2z_1^2 + 2x_1x_2z_1z_2 + x_2^2z_2^2 = \varphi(\mathbf{x})\,\varphi(\mathbf{z})^T,$$

với $\varphi(\mathbf{x}) = \begin{bmatrix} x_1^2 & \sqrt{2}\,x_1x_2 & x_2^2 \end{bmatrix}$. Nghĩa là **hàm $k$ chính là tích vô hướng trong không gian đặc trưng bậc hai**, nhưng ta chỉ cần tính $\mathbf{x}\mathbf{z}^T$ (một tích vô hướng $d$ chiều) rồi bình phương, thay vì dựng $\varphi$ ($D$ chiều). Thêm hằng số 1 để bao gồm cả các bậc thấp:

$$(\mathbf{x}\mathbf{z}^T + 1)^2 = \varphi(\mathbf{x})\,\varphi(\mathbf{z})^T, \qquad \varphi(\mathbf{x}) = \begin{bmatrix} 1 & \sqrt{2}\,x_1 & \sqrt{2}\,x_2 & x_1^2 & \sqrt{2}\,x_1x_2 & x_2^2 \end{bmatrix}.$$

Đây là kernel ta dùng cho XOR ở mục 5.5. Với $d$ và bậc $p$ tổng quát, kernel $(\mathbf{x}\mathbf{z}^T + 1)^p$ ứng với không gian đặc trưng $\binom{d+p}{p}$ chiều, nhưng mỗi lần tính $k$ chỉ tốn cỡ $O(d)$ phép toán. Với $d = 100$ và $p = 3$, đó là $\approx 100$ phép nhân thay vì dựng 176 851 đặc trưng.

| | Dựng đặc trưng tường minh | Thủ thuật kernel |
|---|---|---|
| Một lần tính tích vô hướng | $O(D)$ | $O(d)$ |
| Kích thước bài toán đối ngẫu | $n$ biến | $n$ biến |
| Bộ nhớ chính | $n \times D$ | $n \times n$ (ma trận Gram) |
| Khi $D = \infty$ | Không làm được | Vẫn làm được |

### Điều kiện để một hàm là kernel hợp lệ

Không phải hàm hai biến nào cũng là kernel. Ta cần tồn tại một ánh xạ $\varphi$ (vào một không gian có tích vô hướng) sao cho $k(\mathbf{x}, \mathbf{z}) = \varphi(\mathbf{x})\varphi(\mathbf{z})^T$. **Định lý Mercer** cho biết điều kiện tương đương: $k$ đối xứng và với mọi tập điểm $\mathbf{x}_1, \dots, \mathbf{x}_n$, ma trận Gram **K nửa xác định dương**. Chiều thuận dễ chứng minh: nếu $k = \varphi\varphi^T$ thì với mọi vector $\mathbf{c} \in \mathbb{R}^n$,

$$\mathbf{c}^T\mathbf{K}\,\mathbf{c} = \sum_{i,j} c_i\,c_j\,\varphi(\mathbf{x}_i)\varphi(\mathbf{x}_j)^T = \left\| \sum_i c_i\,\varphi(\mathbf{x}_i) \right\|^2 \geq 0.$$

Điều kiện này không phải thủ tục hình thức: nó bảo đảm $\mathbf{Q} = \mathrm{diag}(\mathbf{y})\mathbf{K}\mathrm{diag}(\mathbf{y})$ cũng nửa xác định dương, nên hàm mục tiêu đối ngẫu vẫn **lõm** và bài toán vẫn có nghiệm tối ưu toàn cục (như ở mục 4). Có thể tạo kernel mới từ kernel cũ: nếu $k_1, k_2$ là kernel hợp lệ và $a > 0$ thì $a\,k_1$, $k_1 + k_2$, $k_1 \cdot k_2$ và $\exp(k_1)$ cũng là kernel hợp lệ.

### Kernel như một thước đo độ tương đồng

Cách nhìn thứ hai là $k(\mathbf{x}, \mathbf{z})$ đo **độ giống nhau** giữa hai mẫu: tích vô hướng lớn nghĩa là hai vector "cùng hướng". Hàm quyết định $f(\mathbf{x}) = \sum_i \alpha_i\,y_i\,k(\mathbf{x}_i, \mathbf{x}) + b$ khi đó là **một cuộc bỏ phiếu có trọng số**: mỗi vector hỗ trợ $\mathbf{x}_i$ bỏ phiếu cho lớp $y_i$ với sức nặng $\alpha_i$, nhân với mức độ tương đồng giữa $\mathbf{x}$ và $\mathbf{x}_i$. Cách nhìn này làm rõ ý nghĩa của kernel RBF ở mục 5.4.

## 5.3. Các kernel phổ biến (tuyến tính, đa thức, RBF) và ma trận Gram K

### Ba kernel phổ biến

| Kernel | Công thức $k(\mathbf{x}, \mathbf{z})$ | Tham số | Số chiều đặc trưng ẩn | Dạng đường biên |
|---|---|---|---|---|
| Tuyến tính | $\mathbf{x}\mathbf{z}^T$ | không | $d$ | Siêu phẳng |

| Kernel | Công thức $k(\mathbf{x}, \mathbf{z})$ | Tham số | Số chiều đặc trưng ẩn | Dạng đường biên |
|---|---|---|---|---|
| Đa thức | $(\gamma\, \mathbf{x}\mathbf{z}^T + r)^p$ | bậc $p$, $\gamma > 0$, $r \geq 0$ | $\binom{d+p}{p}$ (nếu $r > 0$) | Đa thức bậc $p$ |
| RBF (Gauss) | $\exp(-\gamma \parallel \mathbf{x} - \mathbf{z} \parallel^2)$ | $\gamma > 0$ | vô hạn | Đường cong tùy ý, trơn |

**Kernel tuyến tính** chính là SVM đã học, không có ánh xạ nào. **Kernel đa thức** ứng với mọi đơn thức bậc không quá $p$ của các đặc trưng gốc. Hệ số $r$ (gọi là `coef0`) điều khiển trọng số tương đối giữa các bậc thấp và cao: $r = 0$ chỉ cho các đơn thức bậc đúng bằng $p$, $r > 0$ cho cả bậc thấp hơn. Ở mục 5.2, ta dùng $p = 2, \gamma = 1, r = 1$.

**Kernel RBF (Radial Basis Function, còn gọi kernel Gauss)** là kernel được dùng nhiều nhất. Nó phụ thuộc vào khoảng cách Euclid giữa hai mẫu và có thể viết với độ rộng $\sigma$ bằng $\gamma = 1/(2\sigma^2)$. Một số tính chất:

1. $0 < k(\mathbf{x}, \mathbf{z}) \leq 1$, với $k(\mathbf{x}, \mathbf{x}) = 1$. Giá trị gần 1 khi hai điểm gần nhau và tiến về 0 khi hai điểm xa nhau.
2. $\parallel \varphi(\mathbf{x}) - \varphi(\mathbf{z}) \parallel^2 = k(\mathbf{x}, \mathbf{x}) + k(\mathbf{z}, \mathbf{z}) - 2k(\mathbf{x}, \mathbf{z}) = 2 - 2k(\mathbf{x}, \mathbf{z})$. Mọi mẫu đều có $\parallel \varphi(\mathbf{x}) \parallel = 1$, tức nằm trên một mặt cầu đơn vị của không gian đặc trưng, và khoảng cách giữa chúng chỉ phụ thuộc $k$.
3. **Không gian đặc trưng có vô hạn chiều.** Để thấy điều này, khai triển

$$e^{-\gamma\|\mathbf{x}-\mathbf{z}\|^2} = e^{-\gamma\|\mathbf{x}\|^2}\, e^{-\gamma\|\mathbf{z}\|^2}\, e^{2\gamma\, \mathbf{x}\mathbf{z}^T} = e^{-\gamma\|\mathbf{x}\|^2}\, e^{-\gamma\|\mathbf{z}\|^2} \sum_{k=0}^{\infty} \frac{(2\gamma)^k}{k!} (\mathbf{x}\mathbf{z}^T)^k.$$

Mỗi số hạng $(\mathbf{x}\mathbf{z}^T)^k$ là một kernel đa thức bậc $k$ với hệ số dương, nên kernel RBF là tổng (có trọng số) của các kernel đa thức **mọi bậc**, do đó hợp lệ và ứng với không gian đặc trưng vô hạn chiều. Về mặt thực hành, điều này có nghĩa RBF có thể xấp xỉ rất nhiều dạng đường biên, và ta không thể dựng $\varphi$ tường minh mà chỉ dùng được thủ thuật kernel.

**Chọn $\gamma$ mặc định.** Khi dữ liệu đã chuẩn hóa (mỗi đặc trưng có phương sai 1), một điểm xuất phát hợp lý là $\gamma = 1/d$, tương ứng `gamma="scale"` trong scikit-learn. Giá trị tốt nhất sau đó được chọn bằng kiểm định chéo (mục 7.5).

## Ma trận Gram $\mathbf{K}$

Với bộ dữ liệu $\mathbf{x}_1, \ldots, \mathbf{x}_n$ và kernel $k$, ma trận Gram là

$$\mathbf{K} = \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & \cdots & k(\mathbf{x}_1, \mathbf{x}_n) \\ \vdots & \ddots & \vdots \\ k(\mathbf{x}_n, \mathbf{x}_1) & \cdots & k(\mathbf{x}_n, \mathbf{x}_n) \end{bmatrix} \in \mathbb{R}^{n \times n}.$$

Nó đối xứng, nửa xác định dương, và **chứa toàn bộ thông tin về dữ liệu mà SVM cần**: sau khi có $\mathbf{K}$ (cùng $\mathbf{y}$), bài toán đối ngẫu hoàn toàn không cần nhìn lại $\mathbf{X}$. Với kernel tuyến tính, $\mathbf{K} = \mathbf{X}\mathbf{X}^T$ là ma trận ở mục 4.3.

**Ví dụ: ma trận Gram RBF của XOR.** Bốn điểm XOR ở các đỉnh của hình vuông cạnh 1: khoảng cách bình phương giữa hai đỉnh kề nhau là 1 và giữa hai đỉnh đối diện là 2. Với $\gamma = 1$, thứ tự mẫu $(0,0), (0,1), (1,0), (1,1)$:

$$\mathbf{K}_{\text{RBF}} = \begin{bmatrix} 1 & e^{-1} & e^{-1} & e^{-2} \\ e^{-1} & 1 & e^{-2} & e^{-1} \\ e^{-1} & e^{-2} & 1 & e^{-1} \\ e^{-2} & e^{-1} & e^{-1} & 1 \end{bmatrix} \approx \begin{bmatrix} 1 & 0{,}368 & 0{,}368 & 0{,}135 \\ 0{,}368 & 1 & 0{,}135 & 0{,}368 \\ 0{,}368 & 0{,}135 & 1 & 0{,}368 \\ 0{,}135 & 0{,}368 & 0{,}368 & 1 \end{bmatrix}.$$

Đường chéo bằng 1 vì $k(\mathbf{x}, \mathbf{x}) = 1$, và hai điểm kề nhau giống nhau hơn hai điểm đối diện.

**Chi phí.** Tính $\mathbf{K}$ tốn $O(n^2 d)$ thời gian và $O(n^2)$ bộ nhớ. Với $n = 10^5$ mẫu, riêng $\mathbf{K}$ đã cần $10^{10}$ số thực (khoảng 80 GB nếu dùng 64 bit). Đây là nhược điểm chính của SVM kernel trên dữ liệu rất lớn (mục 5.6 và mục 8.3).

## 5.4. Dự đoán qua kernel, ý nghĩa của $\gamma$ và $C$, đường biên cong

### Dự đoán qua kernel

Sau khi huấn luyện, ta chỉ giữ các vector hỗ trợ ($\alpha_i > 0$) cùng hệ số $\alpha_i y_i$. Với mẫu mới $\mathbf{x}$:

$$f(\mathbf{x}) = \sum_{i \in \mathcal{S}} \alpha_i \, y_i \, k(\mathbf{x}_i, \mathbf{x}) + b, \qquad \hat{y} = \operatorname{sign}\big(f(\mathbf{x})\big),$$

với $\mathcal{S}$ là tập vector hỗ trợ và $b = y_k - \sum_{i \in \mathcal{S}} \alpha_i \, y_i \, k(\mathbf{x}_i, \mathbf{x}_k)$ cho một mẫu $k$ có $0 < \alpha_k < C$. Cho một lô $n_{\text{mới}}$ mẫu mới, gọi $\mathbf{K}_{\text{mới}}$ là ma trận $n_{\text{mới}} \times n$ với phần tử $k\big(\mathbf{x}_{\text{mới},a}, \mathbf{x}_i\big)$, thì điểm số của cả lô là

$$\mathbf{f} = \mathbf{K}_{\text{mới}}(\boldsymbol{\alpha} \odot \mathbf{y}) + b\,\mathbf{1}.$$

Chi phí dự đoán một mẫu tỷ lệ với số vector hỗ trợ, đúng như hệ quả của tính thưa ở mục 4.4. Với kernel tuyến tính ta có thể tính trước $\mathbf{w}$, còn với kernel phi tuyến thì phải giữ lại các vector hỗ trợ.

### Đường biên cong và lề trong không gian đặc trưng

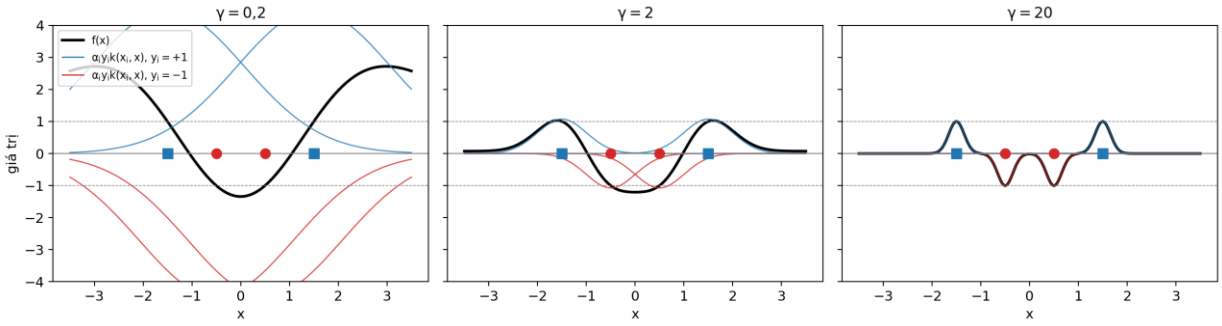
Đường biên quyết định trong không gian gốc là tập $\{\mathbf{x}: f(\mathbf{x}) = 0\}$, thường là một đường cong. Hai "biên lề" là các đường cong $f(\mathbf{x}) = \pm 1$. Về mặt toán học, đó vẫn là siêu phẳng và hai biên lề **trong không gian đặc trưng**, với $\mathbf{w}$ có $\| \mathbf{w} \|^2 = \boldsymbol{\alpha}^T \mathbf{Q} \boldsymbol{\alpha}$ và lề $\gamma = 1/\| \mathbf{w} \|$ đo ở đó. Ta không thể vẽ trực tiếp lề này trong không gian gốc, nhưng vẫn vẽ được các đường mức $f = 0$ và $f = \pm 1$ (Hình 16 và Hình 17).

### Ý nghĩa của $\gamma$ trong kernel RBF

Với kernel RBF, hàm quyết định là tổng của các **"ngọn đồi" Gauss** đặt tại các vector hỗ trợ:

$$f(\mathbf{x}) = \sum_{i \in \mathcal{S}} \alpha_i \, y_i \, e^{-\gamma \|\mathbf{x} - \mathbf{x}_i\|^2} + b.$$

Mỗi vector hỗ trợ dương đóng góp một đồi hướng lên, mỗi vector hỗ trợ âm đóng góp một hố hướng xuống, độ rộng của mỗi đồi hay hố là $\sigma = 1/\sqrt{2\gamma}$. Hình 14 minh họa trên một bài toán một chiều có cấu trúc giống vòng tròn (lớp dương ở hai bên, lớp âm ở giữa), với ba giá trị $\gamma$:



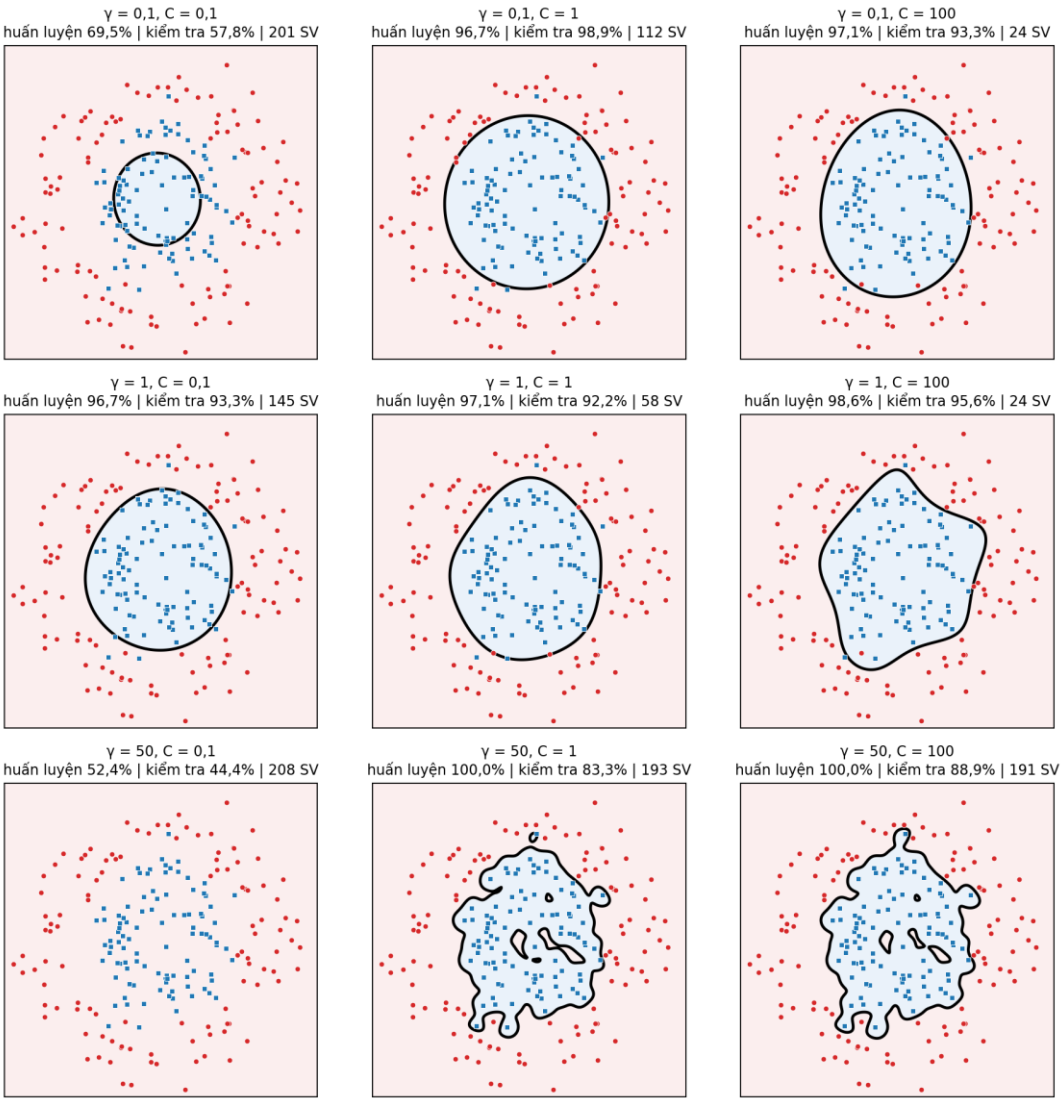

*Hình 14. Hàm quyết định $f(x)$ (nét đen) của SVM RBF trên bốn điểm một chiều, bằng tổng các đồi (xanh, vector hỗ trợ dương) và hố (đỏ, vector hỗ trợ âm) cộng hệ số chệch $b$. $\gamma$ nhỏ cho đồi rộng và $f$ trơn; $\gamma$ lớn cho đồi hẹp, các vector hỗ trợ gần như tách rời nhau, và $f$ chỉ khác $b$ ở quanh các điểm huấn luyện.*

1. $\gamma$ **nhỏ** (đồi rộng): mỗi vector hỗ trợ ảnh hưởng đến vùng rộng, $f$ rất trơn và đường biên đơn giản. Với $\gamma \to 0$ mô hình tiến dần về dạng gần tuyến tính, dễ **dưới khớp**.

2. $\gamma$ **lớn** (đồi hẹp): mỗi vector hỗ trợ chỉ ảnh hưởng quanh nó. Đường biên uốn lượn bám sát từng điểm, mô hình có thể **học thuộc** dữ liệu huấn luyện. Khi $\gamma \to \infty$, ma trận Gram tiến về ma trận đơn vị $\mathbf{K} \to \mathbf{I}$: mọi mẫu hầu như không giống nhau, hầu hết mẫu đều thành vector hỗ trợ, và ở xa các điểm huấn luyện thì $f(\mathbf{x}) \approx b$ (dự đoán không còn phụ thuộc vào dữ liệu).

## Phối hợp $\gamma$ và $C$

$C$ có vai trò như ở mục 3.2: $C$ nhỏ chấp nhận nhiều vi phạm ($\alpha_i \le C$ bị chặn thấp nên lực của từng mẫu yếu, đường biên mượt), $C$ lớn bắt mô hình khớp dữ liệu. Với kernel RBF, hai tham số cùng điều khiển độ phức tạp theo hai cách khác nhau: $\gamma$ quyết định **độ linh hoạt tối đa** của đường biên, $C$ quyết định mô hình **được phép dùng** bao nhiêu phần linh hoạt đó để giảm lỗi huấn luyện. Hình 15 và bảng sau cho thấy chín tổ hợp trên bộ dữ liệu hai vòng tròn có nhiễu (đặc trưng đã chuẩn hóa):




*Hình 15. SVM RBF với chín tổ hợp* $(\gamma, C)$ *trên dữ liệu hai vòng tròn có nhiễu. Hàng trên:* $\gamma = 0{,}1$ *(đồi rộng); hàng giữa:* $\gamma = 1$*; hàng dưới:* $\gamma = 50$ *(đồi rất hẹp). Cột:* $C = 0{,}1;\ 1;\ 100$*. Tiêu đề mỗi ô ghi độ chính xác huấn luyện, kiểm tra và số vector hỗ trợ (SV).*

| $\gamma$ | $C$ | Chính xác huấn luyện | Chính xác kiểm tra | Số vector hỗ trợ | Nhận xét |
|---|---|---|---|---|---|
| 0,1 | 0,1 | 69,5% | 57,8% | 201 | **Dưới khớp**: quá mượt và quá yếu |
| 0,1 | 1 | 96,7% | 98,9% | 112 | Cân bằng tốt |
| 0,1 | 100 | 97,1% | 93,3% | 24 | Khá tốt, ít vector hỗ trợ |
| 1 | 0,1 | 96,7% | 93,3% | 145 | Tốt |
| 1 | 1 | 97,1% | 92,2% | 58 | Tốt |
| 1 | 100 | 98,6% | 95,6% | 24 | Tốt, đường biên bắt đầu uốn nhiều |

| $\gamma$ | $C$ | Chính xác huấn luyện | Chính xác kiểm tra | Số vector hỗ trợ | Nhận xét |
|---|---|---|---|---|---|
| 50 | 0,1 | 52,4% | 44,4% | 208 | **Suy biến**: gần như dự đoán một lớp |
| 50 | 1 | 100% | 83,3% | 193 | **Quá khớp** |
| 50 | 100 | 100% | 88,9% | 191 | **Quá khớp**: gần như mọi mẫu là vector hỗ trợ |

Có bốn điều đáng rút ra:

1. **Vùng "an toàn" là $\gamma$ vừa phải.** Với $\gamma \in \{0{,}1; 1\}$ và $C \geq 1$, mô hình đều đạt trên 92% trên tập kiểm tra.
2. **$\gamma$ quá lớn dẫn đến quá khớp**: độ chính xác huấn luyện 100% nhưng kiểm tra giảm xuống 83% đến 89%, và gần như toàn bộ 210 mẫu thành vector hỗ trợ. Đây là dấu hiệu của $\mathbf{K} \to \mathbf{I}$.
3. **$C$ quá nhỏ kết hợp $\gamma$ quá lớn** làm các hệ số $\alpha_i \leq C$ bé trong khi các đồi lại hẹp, nên $f$ gần như chỉ còn là $b$ và mô hình dự đoán cùng một lớp ở mọi nơi.
4. Số mẫu kiểm tra chỉ là 90, nên chênh lệch vài phần trăm giữa các ô khá nhau có thể là ngẫu nhiên. Việc chọn $(\gamma, C)$ nghiêm túc cần kiểm định chéo (mục 7.5), không nên dựa vào một tập kiểm tra nhỏ.

## 5.5. Ví dụ tính tay: XOR bằng kernel đa thức bậc hai

Ta giải XOR bằng SVM lề cứng (dữ liệu phân tách được trong không gian đặc trưng nên $C$ đủ lớn là tương đương) với kernel $k(\mathbf{x}, \mathbf{z}) = (\mathbf{x}\mathbf{z}^T + 1)^2$. Gọi bốn điểm là $p_1 = (0,0)$, $p_2 = (0,1)$, $p_3 = (1,0)$, $p_4 = (1,1)$ với nhãn $\mathbf{y} = [-1 \quad +1 \quad +1 \quad -1]^T$.

### Bước 1: ma trận Gram và ma trận $\mathbf{Q}$

Các tích vô hướng $\mathbf{x}_i\mathbf{x}_j^T$ giữa các điểm là $p_1 \cdot p_j = 0$ với mọi $j$; $p_2 \cdot p_2 = 1$, $p_2 \cdot p_3 = 0$, $p_2 \cdot p_4 = 1$; $p_3 \cdot p_3 = 1$, $p_3 \cdot p_4 = 1$; $p_4 \cdot p_4 = 2$. Thêm 1 rồi bình phương:

$$\mathbf{K} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 4 & 1 & 4 \\ 1 & 1 & 4 & 4 \\ 1 & 4 & 4 & 9 \end{bmatrix}, \qquad \mathbf{Q} = \operatorname{diag}(\mathbf{y})\,\mathbf{K}\operatorname{diag}(\mathbf{y}) = \begin{bmatrix} 1 & -1 & -1 & 1 \\ -1 & 4 & 1 & -4 \\ -1 & 1 & 4 & -4 \\ 1 & -4 & -4 & 9 \end{bmatrix}.$$

Ma trận $\mathbf{Q}$ có cùng giá trị tuyệt đối với $\mathbf{K}$ và đổi dấu ở các ô nối hai mẫu khác lớp ($y_i y_j = -1$).

### Bước 2: giả thiết cả bốn điểm là vector hỗ trợ và giải hệ KKT

Cả bốn điểm đối xứng nhau về hình học nên ta giả thiết mọi $\alpha_i > 0$, tức mọi $m_i = 1$. Từ $\mathbf{m} = \mathbf{Q}\boldsymbol{\alpha} + b\,\mathbf{y} = \mathbf{1}$ và $\mathbf{y}^T\boldsymbol{\alpha} = 0$, ta có hệ năm phương trình, năm ẩn $(\alpha_1, \ldots, \alpha_4, b)$:

$$\begin{bmatrix} \mathbf{Q} & \mathbf{y} \\ \mathbf{y}^T & 0 \end{bmatrix} \begin{bmatrix} \boldsymbol{\alpha} \\ b \end{bmatrix} = \begin{bmatrix} \mathbf{1} \\ 0 \end{bmatrix}.$$

Viết từng dòng:

$$\begin{aligned} \alpha_1 - \alpha_2 - \alpha_3 + \alpha_4 - b &= 1, \\ -\alpha_1 + 4\alpha_2 + \alpha_3 - 4\alpha_4 + b &= 1, \\ -\alpha_1 + \alpha_2 + 4\alpha_3 - 4\alpha_4 + b &= 1, \\ \alpha_1 - 4\alpha_2 - 4\alpha_3 + 9\alpha_4 - b &= 1, \\ -\alpha_1 + \alpha_2 + \alpha_3 - \alpha_4 &= 0. \end{aligned}$$

Hai điểm $p_2$ và $p_3$ đổi chỗ cho nhau khi hoán đổi $x_1 \leftrightarrow x_2$ (kernel không đổi), nên đặt $\alpha_2 = \alpha_3 = a$, $\alpha_1 = p$, $\alpha_4 = q$. Khi đó:

1. Dòng 5: $p + q = 2a$.
2. Dòng 1: $(p + q - 2a) - b = 1 \Rightarrow -b = 1 \Rightarrow b = -1$.
3. Dòng 4: $p - 8a + 9q - b = 1 \Rightarrow p - 8a + 9q = 0$. Thay $p = 2a - q$: $-6a + 8q = 0$, nên $q = \frac{3}{4}a$.
4. Dòng 2: $-p + 5a - 4q + b = 1 \Rightarrow -p - 4q + 5a = 2$. Thay $p = 2a - q$: $3a - 3q = 2$. Thay $q = \frac{3}{4}a$: $3a - \frac{9}{4}a = \frac{3}{4}a = 2$, nên $a = \frac{8}{3}$.

Từ đó $q = 2$ và $p = 2a - q = \frac{10}{3}$. Vậy

$$\boldsymbol{\alpha} = \begin{bmatrix} \frac{10}{3} & \frac{8}{3} & \frac{8}{3} & 2 \end{bmatrix}^T, \qquad b = -1.$$

Cả bốn $\alpha_i > 0$ nên giả thiết ban đầu đúng (mọi điểm là vector hỗ trợ), và $\mathbf{y}^T\boldsymbol{\alpha} = -\frac{10}{3} + \frac{8}{3} + \frac{8}{3} - 2 = 0$.

## Bước 3: hàm quyết định

Với $k(p_i, \mathbf{x}) = (p_i\mathbf{x}^T + 1)^2$, ta có $k(p_1, \mathbf{x}) = 1$, $k(p_2, \mathbf{x}) = (x_2 + 1)^2$, $k(p_3, \mathbf{x}) = (x_1 + 1)^2$, $k(p_4, \mathbf{x}) = (x_1 + x_2 + 1)^2$. Do đó

$$f(\mathbf{x}) = -\frac{10}{3} \cdot 1 + \frac{8}{3}(x_2 + 1)^2 + \frac{8}{3}(x_1 + 1)^2 - 2\,(x_1 + x_2 + 1)^2 - 1.$$

Khai triển và gom hạng tử:

$$\boxed{f(\mathbf{x}) = \frac{2}{3}x_1^2 + \frac{2}{3}x_2^2 - 4x_1x_2 + \frac{4}{3}x_1 + \frac{4}{3}x_2 - 1.}$$

Kiểm tra bốn điểm huấn luyện: $f(0,0) = -1$, $f(0,1) = \frac{2}{3} + \frac{4}{3} - 1 = 1$, $f(1,0) = 1$, $f(1,1) = \frac{2}{3} + \frac{2}{3} - 4 + \frac{4}{3} + \frac{4}{3} - 1 = -1$. Vậy $m_i = y_i f(p_i) = 1$ với mọi $i$, đúng như giả thiết. Nhãn dự đoán $\operatorname{sign} f = (-1, +1, +1, -1)$ đúng hoàn toàn với XOR.

## Bước 4: ý nghĩa hình học

Đường biên $f(\mathbf{x}) = 0$ là một đường conic có hệ số $B^2 - 4AC = 16 - 4 \cdot \frac{4}{9} > 0$, tức là **hyperbol** với hai nhánh, mỗi nhánh nằm giữa một cặp điểm khác lớp (Hình 16b). Đây là loại đường biên mà không đường thẳng nào làm được.

Trong không gian đặc trưng $\varphi(\mathbf{x}) = \begin{bmatrix} 1 & \sqrt{2}x_1 & \sqrt{2}x_2 & x_1^2 & \sqrt{2}x_1x_2 & x_2^2 \end{bmatrix}$, vector $\mathbf{w} = \sum_i \alpha_i \, y_i \varphi(p_i)^T$ là

$$\mathbf{w} = \begin{bmatrix} 0 & \frac{2\sqrt{2}}{3} & \frac{2\sqrt{2}}{3} & \frac{2}{3} & -2\sqrt{2} & \frac{2}{3} \end{bmatrix}^T, \qquad \| \mathbf{w} \|^2 = \boldsymbol{\alpha}^T \mathbf{Q} \boldsymbol{\alpha} = \frac{32}{3}.$$

Lề trong không gian đặc trưng là $\gamma = 1/\| \mathbf{w} \| = \sqrt{3/32} \approx 0{,}306$. Giá trị đối ngẫu là $\mathbf{1}^T \boldsymbol{\alpha} - \frac{1}{2} \boldsymbol{\alpha}^T \mathbf{Q} \boldsymbol{\alpha} = \frac{32}{3} - \frac{16}{3} = \frac{16}{3}$, **bằng đúng** giá trị gốc $\frac{1}{2} \| \mathbf{w} \|^2 = \frac{16}{3}$ (đối ngẫu mạnh, như ở mục 4.3). Điều đáng chú ý là ta tìm được nghiệm mà **không bao giờ tính** $\varphi$: mọi phép tính đều dùng các số $k(p_i, p_j)$.

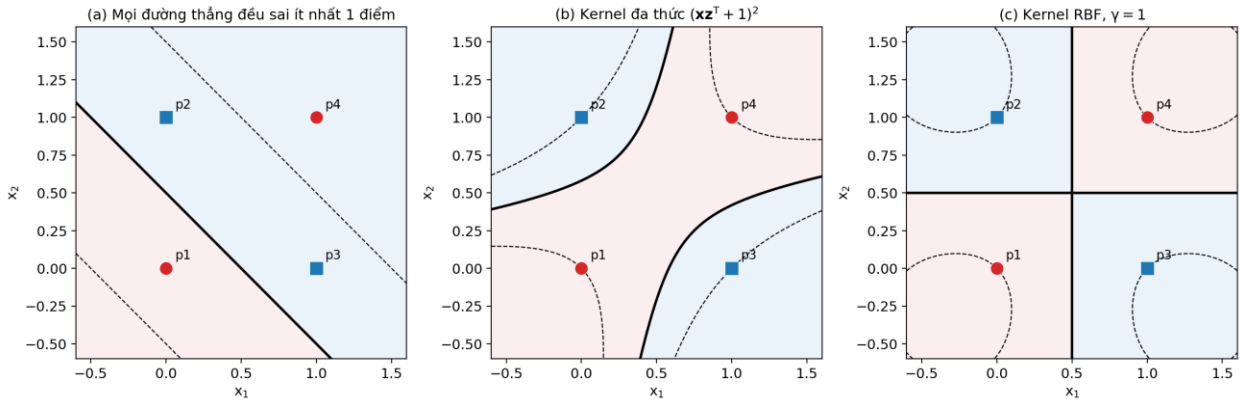
Một vài điểm kiểm tra khác: $f(0{,}5; 0) = -\frac{1}{6} < 0$ nên dự đoán $-1$; $f(0{,}5; 0{,}5) = -\frac{1}{3} < 0$ nên dự đoán $-1$; $f(2; 2) = \frac{8}{3} + \frac{8}{3} - 16 + \frac{16}{3} - 1 = -\frac{19}{3} < 0$ nên dự đoán $-1$.

## Ví dụ phụ: kernel RBF cho XOR

Với $\gamma = 1$ và ma trận Gram ở mục 5.3, cũng nhờ đối xứng ta đặt mọi $\alpha_i = \alpha$ và $b = 0$. Dòng đầu của hệ $\mathbf{Q}\boldsymbol{\alpha} + b\,\mathbf{y} = \mathbf{1}$ cho

$$\alpha(1 - e^{-1} - e^{-1} + e^{-2}) = \alpha(1 - e^{-1})^2 = 1 \;\Rightarrow\; \alpha = \frac{1}{(1 - e^{-1})^2} \approx 2{,}503,$$

và $\mathbf{y}^T \boldsymbol{\alpha} = 0$ thỏa nhờ hai mẫu mỗi lớp. Hàm quyết định là $f(\mathbf{x}) = \alpha\left[e^{-\|\mathbf{x} - p_2\|^2} + e^{-\|\mathbf{x} - p_3\|^2} - e^{-\|\mathbf{x} - p_1\|^2} - e^{-\|\mathbf{x} - p_4\|^2}\right]$. Đường $f = 0$ gồm hai đường thẳng $x_1 = 0{,}5$ và $x_2 = 0{,}5$ (Hình 16c), chia mặt phẳng thành bốn góc phần tư đúng như dữ liệu XOR.




*Hình 16. XOR. (a) Mọi đường thẳng đều sai ít nhất một điểm (ví dụ $x_1 + x_2 = 0{,}5$ sai $p_4$). (b) SVM kernel đa thức bậc hai $(\boldsymbol{x}\boldsymbol{z}^T + 1)^2$ cho đường biên hyperbol, với hai nhánh nằm giữa các cặp điểm khác lớp. (c) SVM kernel RBF với $\gamma = 1$ cho đường biên là hai đường thẳng $x_1 = 0{,}5$ và $x_2 = 0{,}5$. Nét đen liền là $f = 0$, nét đứt là $f = \pm 1$.*

## 5.6. Mã hóa SVM kernel và so với scikit-learn; so sánh với mạng nơ-ron

### Cài đặt SVM kernel từ đầu

Để cài đặt SVM kernel, ta chỉ cần ba thành phần: (i) hàm kernel tính ma trận Gram giữa hai tập điểm, (ii) một bộ giải bài toán đối ngẫu, và (iii) hàm quyết định $f(\mathbf{x}) = \sum_i \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}) + b$. Ta dùng phiên bản **SMO đơn giản hóa** của thuật toán ở mục 4.5: lặp qua các mẫu vi phạm KKT, chọn một mẫu thứ hai, và cập nhật hai hệ số theo công thức nghiệm đóng (với $\kappa = K_{ii} + K_{jj} - 2K_{ij}$). Mỗi bước đều giữ $\mathbf{y}^T\boldsymbol{\alpha} = 0$ và $0 \le \alpha_i \le C$.

```
import numpy as np


def linear_kernel(A, B):
    return A @ B.T

def poly_kernel(A, B, gamma=1.0, coef0=1.0, degree=2):
    return (gamma * (A @ B.T) + coef0) ** degree

def rbf_kernel(A, B, gamma=1.0):
    sq = (A**2).sum(1)[:, None] + (B**2).sum(1)[None, :] - 2 * A @ B.T
    return np.exp(-gamma * np.maximum(sq, 0))


class KernelSVM:
    """SVM lề mềm với kernel, giải bài toán đối ngẫu bằng SMO đơn giản hóa.
    max  1^T a - 1/2 a^T Q a   với  y^T a = 0,  0 <= a <= C"""

    def __init__(self, kernel, C=1.0, tol=1e-3, max_passes=20, seed=0):
        self.kernel, self.C, self.tol = kernel, C, tol
        self.max_passes, self.seed = max_passes, seed
```

```
    def fit(self, X, y):
        n = len(y)
        K = self.kernel(X, X)                          # ma trận Gram n x n
        alpha, b = np.zeros(n), 0.0
        rng = np.random.default_rng(self.seed)
        f = lambda i: (alpha * y) @ K[:, i] + b     # hàm quyết định tại mẫu
i
        passes = 0
        while passes < self.max_passes:
            changed = 0
            for i in range(n):
                E_i = f(i) - y[i]
                # mẫu i vi phạm KKT?
                if (y[i] * E_i < -self.tol and alpha[i] < self.C) or \
                   (y[i] * E_i > self.tol and alpha[i] > 0):
                    j = rng.integers(n - 1)
                    j = j + 1 if j >= i else j      # chọn j khác i
                    E_j = f(j) - y[j]
                    a_i, a_j = alpha[i], alpha[j]
                    if y[i] != y[j]:
                        L, H = max(0, a_j - a_i), min(self.C, self.C + a_j -
a_i)
                    else:
                        L, H = max(0, a_i + a_j - self.C), min(self.C, a_i +
a_j)
                    kappa = K[i, i] + K[j, j] - 2 * K[i, j]
                    if L == H or kappa <= 1e-12:
                        continue
                    a_j_new = np.clip(a_j + y[j] * (E_i - E_j) / kappa, L, H)
                    if abs(a_j_new - a_j) < 1e-8:
                        continue
                    a_i_new = a_i + y[i] * y[j] * (a_j - a_j_new)
                    # cập nhật b theo KKT
                    b1 = b - E_i - y[i] * (a_i_new - a_i) * K[i, i] - y[j] *
(a_j_new - a_j) * K[i, j]
                    b2 = b - E_j - y[i] * (a_i_new - a_i) * K[i, j] - y[j] *
(a_j_new - a_j) * K[j, j]
                    if 0 < a_i_new < self.C:   b = b1
                    elif 0 < a_j_new < self.C: b = b2
                    else:                      b = (b1 + b2) / 2
                    alpha[i], alpha[j] = a_i_new, a_j_new
                    changed += 1
            passes = passes + 1 if changed == 0 else 0
        self.alpha, self.b = alpha, b
        sv = alpha > 1e-8
        self.X_sv, self.coef = X[sv], (alpha * y)[sv]    # chỉ giữ vector hỗ
trợ
        self.dual_value = alpha.sum() - 0.5 * (alpha * y) @ K @ (alpha * y)
        return self
```

```
    def decision_function(self, X):
        return self.kernel(X, self.X_sv) @ self.coef + self.b

    def predict(self, X):
        return np.where(self.decision_function(X) >= 0, 1, -1)
```

Giải thích các phần chính:

1. Ba hàm kernel trả về **ma trận Gram giữa hai tập điểm** $A$ $(n_1 \times d)$ và $B$ $(n_2 \times d)$. *rbf_kernel* dùng khai triển $\| \mathbf{a} - \mathbf{b} \|^2 = \| \mathbf{a} \|^2 + \| \mathbf{b} \|^2 - 2\mathbf{a}\mathbf{b}^T$ để tính cả ma trận bằng phép nhân ma trận mà không dùng vòng lặp.
2. Trong *fit*, *K = self.kernel(X, X)* là ma trận Gram. *f(i)* là hàm quyết định $f(\mathbf{x}_i) = \sum_j \alpha_j y_j K_{ji} + b$, và *E_i = f(i) - y[i]* là sai số $E_i$ ở mục 4.5. Bước cập nhật *a_j + y[j]*(E_i - E_j)/kappa* rồi cắt về $[L, H]$ chính là công thức của SMO.
3. Sau khi huấn luyện, ta chỉ giữ các vector hỗ trợ (*X_sv*) và hệ số $\alpha_i y_i$ (*coef*), nên dự đoán chỉ cần *kernel(X_new, X_sv) @ coef + b*, đúng $\mathbf{f} = \mathbf{K}_{\text{mới}}(\boldsymbol{\alpha} \odot \mathbf{y}) + b\mathbf{1}$ ở mục 5.4.

## Kiểm tra trên XOR và hai vòng tròn

```
from sklearn.svm import SVC

# XOR với kernel đa thức bậc hai (C lớn để xấp xỉ lề cứng)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y = np.array([-1, 1, 1, -1], dtype=float)
poly2 = lambda A, B: poly_kernel(A, B, gamma=1.0, coef0=1.0, degree=2)
m = KernelSVM(poly2, C=1e3, max_passes=50).fit(X, y)
print(m.alpha.round(3), round(m.b, 3), m.predict(X))
# [3.332 2.665 2.665 1.999] -1.0 [-1  1  1 -1]    (tay: 10/3, 8/3, 8/3, 2 ; b
= -1)

# Hai vòng tròn có nhiễu: so sánh với scikit-learn (Ztr, ytr, Zte, yte đã chu
ẩn hóa)
rbf = lambda A, B: rbf_kernel(A, B, gamma=1.0)
mine = KernelSVM(rbf, C=1.0).fit(Ztr, ytr)
ref = SVC(kernel="rbf", gamma=1.0, C=1.0).fit(Ztr, ytr)
print((mine.predict(Zte) == yte).mean(), ref.score(Zte, yte))
```

Kết quả: nghiệm XOR tự cài đặt $\boldsymbol{\alpha} \approx (3{,}33;\ 2{,}67;\ 2{,}67;\ 2{,}00)$, $b = -1{,}00$, khớp phép tính tay $\left(\frac{10}{3}, \frac{8}{3}, \frac{8}{3}, 2\right)$ và $b = -1$ (sai khác nhỏ do dừng ở sai số tol $= 10^{-3}$). Trên hai vòng tròn có nhiễu:

| Kernel | Mô hình | Chính xác huấn luyện | Chính xác kiểm tra | Số vector hỗ trợ | Giá trị đối ngẫu | Thời gian huấn luyện |
|---|---|---|---|---|---|---|
| Đa thức bậc 2 ($C = 1$) | Tự cài đặt | 97,1% | 94,4% | 30 | 21,7135 | 0,9 giây |
| Đa thức bậc 2 ($C = 1$) | *SVC* | 97,1% | 94,4% | 30 | 21,7181 | nhanh hơn |
| RBF ($\gamma = 1,\ C = 1$) | Tự cài đặt | 97,1% | 92,2% | 58 | 35,7681 | 0,7 giây |
| RBF ($\gamma = 1,\ C = 1$) | *SVC* | 97,1% | 92,2% | 58 | 35,7681 | nhanh hơn |

Với RBF, giá trị đối ngẫu và số vector hỗ trợ khớp tuyệt đối với *SVC*, và dự đoán trên 5000 điểm lưới ngẫu nhiên giống nhau 100%. Với kernel đa thức, hai nghiệm khác nhau rất nhỏ (giá trị đối ngẫu 21,7135 so với 21,7181), dự đoán trùng 99,7% trên lưới. Sự khác biệt đó đến từ dung sai dừng của thuật toán đơn giản hóa. *SVC* dùng SMO có chọn cặp thông minh (LIBSVM) nên nhanh và chính xác hơn.

Các tham số tương ứng khi dùng scikit-learn: *kernel="rbf"* với *gamma* là $\gamma$; *kernel="poly"* với *degree* là $p$, *gamma* là $\gamma$, *coef0* là $r$; *kernel="precomputed"* cho phép truyền thẳng ma trận Gram **K** (hữu ích khi ta tự định nghĩa kernel cho chuỗi, đồ thị hay văn bản).

## Chi phí khi $n$ lớn

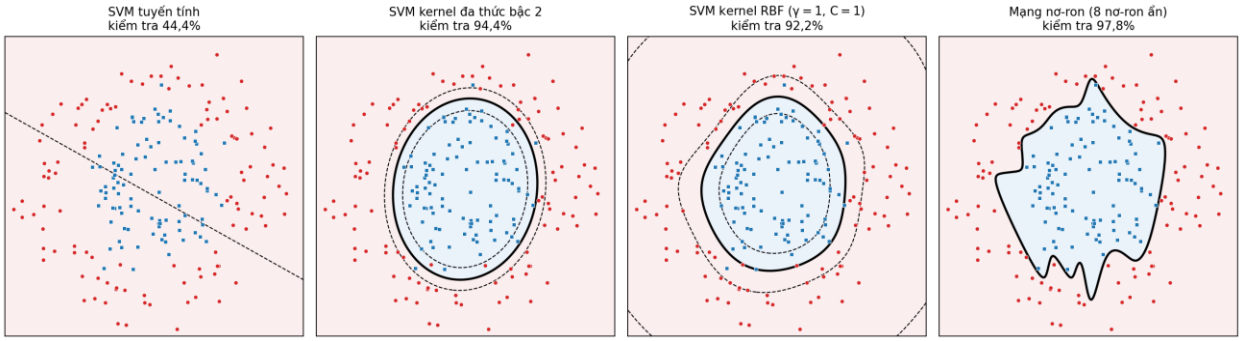
Vì SVM kernel làm việc với ma trận Gram $n \times n$, chi phí huấn luyện tăng nhanh theo $n$. Đo trên cùng bộ dữ liệu hai vòng tròn với *SVC(kernel="rbf")*:

| Số mẫu $n$ | 1 000 | 2 000 | 4 000 | 8 000 |
|---|---|---|---|---|
| Thời gian huấn luyện (giây) | 0,006 | 0,013 | 0,044 | 0,157 |

Thời gian tăng gấp khoảng 2,2, 3,4 và 3,6 lần mỗi khi nhân đôi $n$, tiệm cận mức tăng bậc hai. Bộ nhớ cho ma trận Gram tăng đúng theo $n^2$. Đây là lý do SVM kernel thường dùng cho dữ liệu cỡ vài chục nghìn mẫu trở xuống, còn dữ liệu lớn hơn nên dùng SVM tuyến tính, xấp xỉ kernel hoặc mô hình khác (mục 8.3).

## So sánh với mạng nơ-ron

Cả SVM kernel và mạng nơ-ron đều tạo ra đường biên cong từ cùng một dữ liệu, nhưng bằng hai triết lý khác nhau. Hình 17 so sánh bốn mô hình trên cùng bộ dữ liệu hai vòng tròn có nhiễu.




*Hình 17. Bốn mô hình trên hai vòng tròn có nhiễu (đặc trưng đã chuẩn hóa). SVM tuyến tính không phân loại được. SVM kernel đa thức bậc hai và RBF cho đường biên cong. Mạng nơ-ron 8 nơ-ron ẩn học đặc trưng trực tiếp. Nền xanh/đỏ là hai vùng quyết định, đường đen là biên.*

| Mô hình | Chính xác huấn luyện | Chính xác kiểm tra | Ghi chú |
|---|---|---|---|
| SVM tuyến tính | 52,4% | 44,4% | Không phân loại được |
| SVM kernel đa thức bậc 2 | 97,1% | 94,4% | 30 vector hỗ trợ |
| SVM kernel RBF ($\gamma = 1,\ C = 1$) | 97,1% | 92,2% | 58 vector hỗ trợ |
| Mạng nơ-ron (8 nơ-ron ẩn, tanh) | 100% | 97,8% | Với 5 khởi tạo ngẫu nhiên, kiểm tra từ 90,0% đến 97,8% |

Chênh lệch giữa các hàng nhỏ so với dao động do chỉ có 90 mẫu kiểm tra, nên ta không rút kết luận về mô hình nào "chính xác hơn". Điều đáng so sánh là **bản chất** của hai cách tiếp cận:

| Tiêu chí | SVM kernel | Mạng nơ-ron |
|---|---|---|
| Cách có đường biên cong | Đặc trưng **cố định theo kernel** do ta chọn ($k, \gamma, p$) | Đặc trưng **được học** từ dữ liệu ở các lớp ẩn |
| Hàm mục tiêu | **Lồi**: một nghiệm toàn cục duy nhất, kết quả lặp lại được | **Không lồi**: kết quả phụ thuộc khởi tạo (ví dụ 90% đến 97,8% qua 5 lần) |
| Siêu tham số chính | Kernel, $C$, $\gamma$ (ít, dễ lưới) | Kiến trúc, số nơ-ron, tốc độ học, điều chuẩn,… |
| Chi phí huấn luyện | $O(n^2)$ đến $O(n^3)$, bộ nhớ $O(n^2)$ | Thường $O(n)$ mỗi epoch với SGD, mở rộng được đến dữ liệu rất lớn |
| Dự đoán | Tỷ lệ với số vector hỗ trợ | Cố định theo kích thước mạng |
| Dữ liệu ít | Thường làm tốt, nhờ tối đa hóa lề | Dễ quá khớp, cần điều chuẩn |
| Dữ liệu rất lớn, phi cấu trúc (ảnh, văn bản, âm thanh) | Hạn chế | Thế mạnh, vì học được biểu diễn |

Có thể tóm gọn: **SVM kernel** cho ta bài toán tối ưu lồi, nghiệm duy nhất và ít điều chỉnh, nhưng đặc trưng (qua kernel) do ta chọn và chi phí tăng nhanh theo $n$. **Mạng nơ-ron** tự học đặc trưng và mở rộng tốt đến dữ liệu rất lớn, đổi lại là bài toán không lồi và nhiều siêu tham số. Một liên hệ thú vị: hàm quyết định $f(\mathbf{x}) = \sum_i \alpha_i\, y_i k(\mathbf{x}_i, \mathbf{x}) + b$ của SVM RBF có thể xem là một **mạng một lớp ẩn** với một nơ-ron ẩn $e^{-\gamma\|\mathbf{x}-\mathbf{x}_i\|^2}$ cho mỗi vector hỗ trợ, trong đó vị trí tâm ($\mathbf{x}_i$) và trọng số đầu ra ($\alpha_i y_i$) được chọn tự động nhờ bài toán lồi.

## Tóm tắt mục 5

1. **Đặc trưng phi tuyến** biến đường biên tuyến tính thành đường cong: XOR cần $x_1x_2$, hai vòng tròn cần $x_1^2, x_2^2$. Nhưng số đặc trưng $\binom{d+k}{k} - 1$ tăng bùng nổ và ta phải tự đoán dạng đặc trưng.
2. Dạng đối ngẫu chỉ dùng **tích vô hướng**. **Thủ thuật kernel** thay $\mathbf{x}_i\mathbf{x}_j^T$ bằng $k(\mathbf{x}_i, \mathbf{x}_j) = \varphi(\mathbf{x}_i)\varphi(\mathbf{x}_j)^T$, nên không cần dựng $\varphi$ và vẫn làm việc được với không gian vô hạn chiều. Mọi thứ khác ở mục 4 (KKT, tính thưa, SMO) giữ nguyên.
3. Một hàm là kernel hợp lệ khi ma trận Gram **K** đối xứng và nửa xác định dương. Ba kernel phổ biến: **tuyến tính** $\mathbf{x}\mathbf{z}^T$, **đa thức** $(\gamma\mathbf{x}\mathbf{z}^T + r)^p$ và **RBF** $e^{-\gamma\|\mathbf{x}-\mathbf{z}\|^2}$ (vô hạn chiều).
4. Dự đoán: $f(\mathbf{x}) = \sum_{i\in S} \alpha_i\, y_i\, k(\mathbf{x}_i, \mathbf{x}) + b$, một cuộc bỏ phiếu có trọng số của các vector hỗ trợ. Với RBF, $f$ là tổng các đồi Gauss rộng $\sigma = 1/\sqrt{2\gamma}$: $\gamma$ nhỏ cho đường biên mượt (dưới khớp), $\gamma$ lớn cho đường biên bám từng điểm (quá khớp, $\mathbf{K} \to \mathbf{I}$). $C$ điều khiển mức chấp nhận vi phạm, và cặp $(\gamma, C)$ cần chọn bằng kiểm định chéo.
5. Ví dụ XOR với $(\mathbf{x}\mathbf{z}^T + 1)^2$: $\boldsymbol{\alpha} = \left(\frac{10}{3}, \frac{8}{3}, \frac{8}{3}, 2\right)$, $b = -1$, $f(\mathbf{x}) = \frac{2}{3}x_1^2 + \frac{2}{3}x_2^2 - 4x_1x_2 + \frac{4}{3}x_1 + \frac{4}{3}x_2 - 1$, đường biên là hyperbol, lề $\sqrt{3/32}$ trong không gian đặc trưng.
6. Cài đặt từ đầu chỉ cần hàm kernel và một bộ giải đối ngẫu (SMO), cho kết quả khớp với *svc*. Chi phí là $O(n^2)$ bộ nhớ nên không hợp với dữ liệu rất lớn. So với mạng nơ-ron: SVM kernel có bài toán lồi, ít siêu tham số, nhưng đặc trưng do kernel quyết định và khó mở rộng.

## Câu hỏi kiểm tra nhanh

1. Tính $k(\mathbf{x}, \mathbf{z}) = (\mathbf{x}\mathbf{z}^T + 1)^2$ với $\mathbf{x} = [1 \quad 2]$ và $\mathbf{z} = [2 \quad 0]$, rồi kiểm tra lại bằng $\varphi(\mathbf{x})\varphi(\mathbf{z})^T$ với $\varphi$ ở mục 5.2.
2. Với kernel RBF $\gamma = 0{,}5$, tính $k(\mathbf{x}, \mathbf{z})$ cho $\mathbf{x} = [0 \quad 0]$, $\mathbf{z} = [1 \quad 1]$ và khoảng cách $\| \varphi(\mathbf{x}) - \varphi(\mathbf{z}) \|^2$ trong không gian đặc trưng.

3. Với $d = 100$ đặc trưng và bậc $k = 2$, có bao nhiêu đặc trưng đa thức (không kể hệ số chệch)? So sánh chi phí một lần tính tích vô hướng giữa cách dựng tường minh và kernel $(\mathbf{x}\mathbf{z}^T + 1)^2$.

4. Với SVM RBF, điều gì xảy ra với số vector hỗ trợ và độ chính xác huấn luyện/kiểm tra khi tăng dần $\gamma$ ở một $C$ cố định? Giải thích bằng ma trận Gram.

5. Dùng hàm quyết định ở mục 5.5, tính $f(0{,}5;\ 0)$ và cho biết nhãn dự đoán. Vì sao ta không thể tính được $\mathbf{w}$ tường minh khi dùng kernel RBF, nhưng vẫn dự đoán được?

## Gợi ý đáp án

1. $\mathbf{x}\mathbf{z}^T = 1 \cdot 2 + 2 \cdot 0 = 2$ nên $k = (2 + 1)^2 = 9$. Với $\varphi(\mathbf{x}) = [1 \quad \sqrt{2} \quad 2\sqrt{2} \quad 1 \quad 2\sqrt{2} \quad 4]$ và $\varphi(\mathbf{z}) = [1 \quad 2\sqrt{2} \quad 0 \quad 4 \quad 0 \quad 0]$ ta có $\varphi(\mathbf{x})\varphi(\mathbf{z})^T = 1 + \sqrt{2} \cdot 2\sqrt{2} + 0 + 1 \cdot 4 + 0 + 0 = 1 + 4 + 4 = 9$, khớp.

2. $\| \mathbf{x} - \mathbf{z} \|^2 = 2$ nên $k = e^{-0{,}5 \cdot 2} = e^{-1} \approx 0{,}368$. Khoảng cách trong không gian đặc trưng: $\| \varphi(\mathbf{x}) - \varphi(\mathbf{z}) \|^2 = 2 - 2k \approx 2 - 0{,}736 = 1{,}264$.

3. $\binom{102}{2} - 1 = 5151 - 1 = 5150$ đặc trưng. Dựng tường minh cần tính và nhân hai vector 5150 chiều (cỡ 5150 phép nhân), còn kernel chỉ cần một tích vô hướng 100 chiều rồi cộng 1 và bình phương (cỡ 100 phép nhân).

4. Khi $\gamma$ tăng, các đồi Gauss hẹp lại, $\mathbf{K}$ tiến về $\mathbf{I}$ (mọi mẫu xem như khác hẳn nhau). Số vector hỗ trợ tăng (gần bằng $n$), độ chính xác huấn luyện tăng lên 100% và độ chính xác kiểm tra giảm (quá khớp), như hàng $\gamma = 50$ ở bảng mục 5.4.

5. $f(0{,}5; 0) = \frac{2}{3} \cdot 0{,}25 + 0 - 0 + \frac{4}{3} \cdot 0{,}5 + 0 - 1 = \frac{1}{6} + \frac{2}{3} - 1 = -\frac{1}{6} < 0$, nên dự đoán nhãn $-1$. Với RBF, $\mathbf{w} = \sum_i \alpha_i\, y_i \varphi(\mathbf{x}_i)^T$ nằm trong không gian vô hạn chiều nên không thể lưu như một vector hữu hạn, nhưng dự đoán chỉ cần các giá trị $k(\mathbf{x}_i, \mathbf{x})$ nên vẫn tính được từ $\boldsymbol{\alpha}$ và các vector hỗ trợ.

# 6. Hồi quy vector hỗ trợ (SVR)

Từ mục 2 đến mục 5, ta chỉ làm việc với bài toán **phân loại**: nhãn $y_i \in \{-1, +1\}$ và mục tiêu là tìm đường biên có lề lớn nhất. Nhưng nhiều bài toán thực tế cần dự đoán một **đại lượng liên tục**: điểm thi, giá nhà, nhiệt độ. Mục này cho thấy các ý tưởng đã xây dựng ở mục trước chuyển sang bài toán hồi quy gần như nguyên vẹn, chỉ cần đổi hàm mất mát. Cụ thể, ta giữ lại bốn "viên gạch":

1. điều chuẩn $\frac{1}{2} \| \mathbf{w} \|^2$ để giữ mô hình đơn giản (ở hồi quy, đó là giữ hàm **phẳng**);
2. một hàm mất mát **cắt cụt** (như hinge): bỏ qua sai số nhỏ, và phạt tuyến tính các sai số lớn;
3. biến lỏng và tham số $C$ để cân bằng hai mục tiêu;
4. dạng đối ngẫu chỉ dùng tích vô hướng, nên kernel dùng được ngay.

Kết quả là **hồi quy vector hỗ trợ (Support Vector Regression, SVR)**. Trong mục 6, ta trả lời các câu hỏi:

1. Hồi quy tuyến tính với MSE có hạn chế gì trước mẫu ngoại lai, và ta mong muốn điều gì ở một mô hình hồi quy tốt hơn?
2. Hàm mất mát $\varepsilon$-insensitive và "ống $\varepsilon$" là gì?
3. Bài toán tối ưu của SVR (biến lỏng $\xi_i, \xi_i^*$, dạng không ràng buộc) được viết ra sao?
4. Dạng đối ngẫu, vector hỗ trợ trong hồi quy, và SVR với kernel?
5. Vai trò của $\varepsilon$, $C$, $\gamma$ ra sao, và giải tay một ví dụ một chiều như thế nào?
6. Cài đặt SVR từ đầu và so sánh với hồi quy tuyến tính.

**Dữ liệu xuyên suốt.** Biến phụ thuộc bây giờ là số thực. Ta dùng hai bộ dữ liệu:

1. **Bộ một chiều (để tính tay):** dự đoán điểm thi $y$ từ số giờ tự học $x$ của 5 sinh viên, và một sinh viên thứ 6 là **ngoại lai** ($x_6 = 6, y_6 = 2{,}0$):

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 1{,}0 \\ 2{,}2 \\ 2{,}9 \\ 4{,}0 \\ 5{,}0 \\ 2{,}0 \end{bmatrix}.$$

2. **Bộ phi tuyến:** 60 điểm sinh từ $y = \sin x$ cộng nhiễu Gauss, trong đó có 3 điểm ngoại lai bị lệch mạnh. Chia 42 mẫu huấn luyện và 18 mẫu kiểm tra.

Ký hiệu giữ nguyên như các mục trước, với các thay đổi sau do bài toán hồi quy:

| Đối tượng | Ký hiệu | Kích thước | Ý nghĩa |
|---|---|---|---|
| Vector đích | $\mathbf{y}$ | $n \times 1$ | Giá trị thực $y_i \in \mathbb{R}$ (không còn nhãn $\pm 1$) |
| Dự đoán | $f(\mathbf{x}) = \mathbf{x}\mathbf{w} + b$ | vô hướng | Giá trị dự đoán |
| Phần dư | $r_i = y_i - f(\mathbf{x}_i)$, $\mathbf{r}$ | vô hướng, $n \times 1$ | $\mathbf{r} = \mathbf{y} - \mathbf{X}\mathbf{w} - b\mathbf{1}$ |
| Độ rộng ống | $\varepsilon \geq 0$ | vô hướng | Sai số được bỏ qua |
| Biến lỏng | $\boldsymbol{\xi}, \boldsymbol{\xi}^*$ | $n \times 1$ | Mức vượt ống phía trên, phía dưới |
| Hệ số đối ngẫu | $\boldsymbol{\alpha}, \boldsymbol{\alpha}^*$, $\boldsymbol{\beta} = \boldsymbol{\alpha} - \boldsymbol{\alpha}^*$ | $n \times 1$ | $\beta_i$ là hệ số có dấu của mẫu $i$ |
| Ma trận Gram | $\mathbf{K}$ | $n \times n$ | $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ |

Số thứ tự hình tiếp tục từ mục 5.

## 6.1. Từ hồi quy tuyến tính đến SVR: hạn chế của MSE trước mẫu ngoại lai

### Nhắc lại hồi quy tuyến tính

Hồi quy tuyến tính dự đoán $f(\mathbf{x}) = \mathbf{x}\mathbf{w} + b$ và chọn $(\mathbf{w}, b)$ tối thiểu hóa sai số bình phương trung bình (MSE):

$$J_{\text{MSE}}(\mathbf{w}, b) = \frac{1}{n}\sum_{i=1}^{n} r_i^2 = \frac{1}{n} \parallel \mathbf{y} - \mathbf{X}\mathbf{w} - b\mathbf{1} \parallel^2, \qquad \nabla_{\mathbf{w}} J_{\text{MSE}} = -\frac{2}{n}\mathbf{X}^T\mathbf{r}.$$

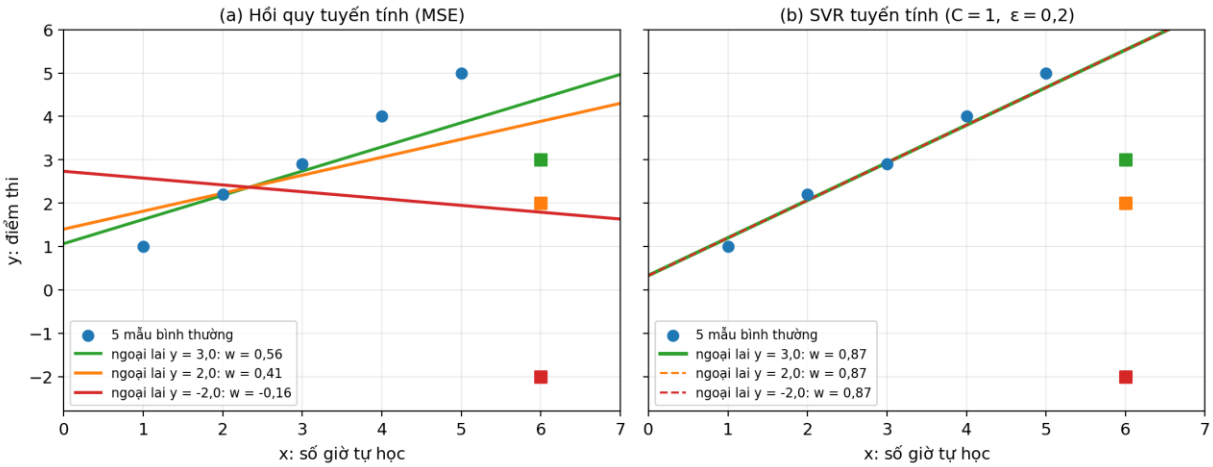
Gradient cho thấy mẫu $i$ kéo $\mathbf{w}$ về phía $\mathbf{x}_i^T$ với cường độ **tỷ lệ thuận với phần dư** $r_i$. Mẫu sai càng nhiều thì càng kéo mạnh, và **không có trần**.

### Một ngoại lai làm lệch cả đường hồi quy

Với 5 sinh viên đầu, đường hồi quy tuyến tính tốt nhất là $y = 0{,}98\,x + 0{,}08$, rất sát dữ liệu (MSE $= 0{,}009$). Thêm sinh viên thứ 6 ($x_6 = 6$) với điểm thi $y_6$ lệch xa xu hướng:

| Điểm của ngoại lai $y_6$ | Hệ số góc $w$ | Hệ số chệch $b$ | MSE |
|---|---|---|---|
| Không có ngoại lai | 0,980 | 0,080 | 0,009 |
| 3,0 | 0,557 | 1,067 | 0,703 |
| 2,0 | 0,414 | 1,400 | 1,252 |
| $-2{,}0$ | $-0{,}157$ | 2,733 | 5,036 |

Chỉ một sinh viên trong sáu người đã đổi hệ số góc từ 0,98 xuống 0,41, thậm chí đổi dấu (xu hướng "học nhiều thì điểm cao" biến thành "học nhiều thì điểm thấp") khi $y_6 = -2$. Nguyên nhân là phạt bình phương: để giảm một phần dư lớn $r$, đường thẳng sẵn sàng hy sinh độ khớp ở năm điểm còn lại, vì chi phí tính theo $r^2$ (Hình 18a).




*Hình 18. (a) Hồi quy tuyến tính (MSE): khi ngoại lai (ô vuông) ở $x = 6$ đổi vị trí, đường hồi quy bị kéo theo, hệ số góc là* 0,56, 0,41 *rồi* $-0{,}16$. *(b) SVR tuyến tính ($C = 1$, $\varepsilon = 0{,}2$): cả ba đường* ***trùng nhau*** *với hệ số góc* 0,87, *vì ảnh hưởng của ngoại lai bị chặn (mục 6.5).*

## Ta mong muốn điều gì?

Từ phân tích trên, một mô hình hồi quy tốt hơn nên có bốn tính chất, trùng khớp với những gì SVM đã làm cho phân loại:

1. **Bỏ qua sai số nhỏ.** Dữ liệu luôn có nhiễu, nên sai số nhỏ hơn một ngưỡng $\varepsilon$ không đáng bị phạt. Tương tự hinge không phạt mẫu có $m \geq 1$.
2. **Ảnh hưởng bị chặn.** Một mẫu sai nặng chỉ nên phạt tuyến tính theo độ lệch (không bình phương), và quan trọng hơn, cường độ kéo của nó nên có trần. Tương tự $\alpha_i \leq C$ ở mục 4.
3. **Hàm đơn giản.** Giữ $\| \mathbf{w} \|$ nhỏ, tức giữ hàm "phẳng".
4. **Phi tuyến qua kernel**, vì dạng đối ngẫu chỉ cần tích vô hướng.

## Đối chiếu SVM phân loại và SVR

| | SVM phân loại | SVR |
|---|---|---|
| Đại lượng đo lỗi | Margin $m = y\, f(\mathbf{x})$ | Phần dư $r = y - f(\mathbf{x})$ |
| Hàm mất mát | Hinge $\max(0, 1 - m)$ | $\varepsilon$-insensitive $\max(0, \lvert r \rvert - \varepsilon)$ |
| Vùng không mất mát | **Ngoài** dải lề ($m \geq 1$) | **Trong** ống ($\lvert r \rvert \leq \varepsilon$) |
| Mẫu có hệ số $\neq 0$ (vector hỗ trợ) | Trên hoặc **trong** dải lề, hoặc sai phía | Trên hoặc **ngoài** ống |
| Hệ số đối ngẫu | $\alpha_i \in [0, C]$ | $\alpha_i, \alpha_i^* \in [0, C]$, $\beta_i = \alpha_i - \alpha_i^*$ |

| | SVM phân loại | SVR |
|---|---|---|
| Cân bằng lực | $\sum_i \alpha_i y_i = 0$ | $\sum_i \beta_i = 0$ |

Điểm cần chú ý là sự **đảo chiều**: ở phân loại, các mẫu "dễ" nằm *ngoài* dải lề và bị bỏ qua; ở hồi quy, các mẫu "dễ" nằm *trong* ống và bị bỏ qua. Vector hỗ trợ luôn là các mẫu "khó": nằm trên biên hoặc ở phía vi phạm.

## 6.2. Hàm mất mát $\varepsilon$-insensitive và "ống $\varepsilon$"

### Định nghĩa

**Hàm mất mát $\varepsilon$-insensitive** (không nhạy với sai số dưới $\varepsilon$) của phần dư $r = y - f(\mathbf{x})$ là

$$\ell_\varepsilon(r) = \max(0,\ |r| - \varepsilon) = \begin{cases} 0 & \text{nếu } |r| \le \varepsilon, \\ |r| - \varepsilon & \text{nếu } |r| > \varepsilon. \end{cases}$$

**Ống** $\varepsilon$ là vùng $\{(\mathbf{x}, y):\ |y - f(\mathbf{x})| \le \varepsilon\}$ nằm giữa hai đường $y = f(\mathbf{x}) \pm \varepsilon$. Mẫu nằm trong ống không bị phạt, mẫu nằm ngoài ống bị phạt bằng khoảng cách **theo chiều dọc** đến biên ống gần nhất. Cần lưu ý $\varepsilon$ có **cùng đơn vị với** $y$, nên khi các giá trị $y$ có thang đo lớn ta nên chuẩn hóa $y$ trước (như ở mục 6.6).

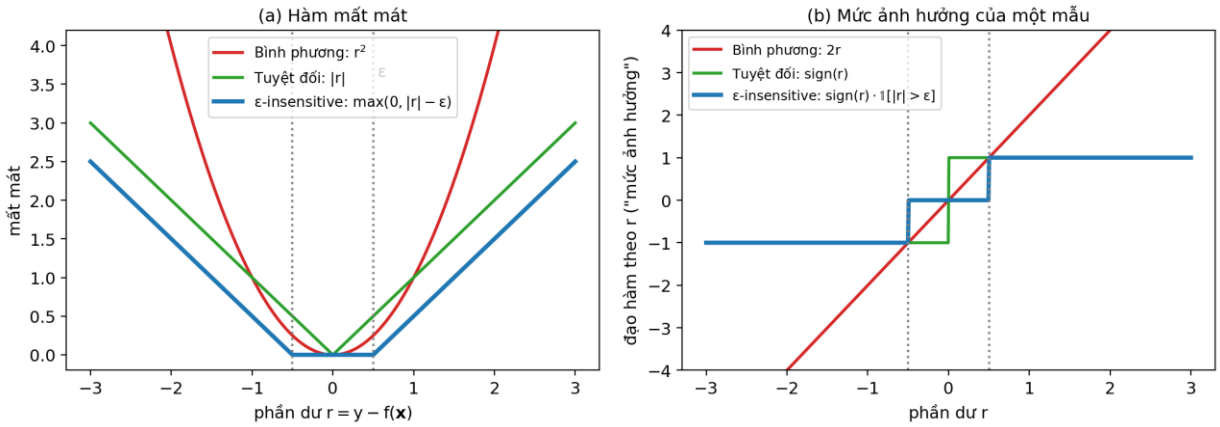
### So sánh với các hàm mất mát khác

Với $\varepsilon = 0{,}5$:

| Phần dư $r$ | Bình phương $r^2$ | Tuyệt đối $\lvert r\rvert$ | $\varepsilon$-insensitive |
|---|---|---|---|
| 0,2 | 0,04 | 0,2 | 0 |
| 0,5 | 0,25 | 0,5 | 0 |
| 1 | 1 | 1 | 0,5 |
| 2 | 4 | 2 | 1,5 |
| 5 | 25 | 5 | 4,5 |

Bình phương phạt mẫu có $r = 5$ gấp 25 lần mẫu có $r = 1$, trong khi $\varepsilon$-insensitive chỉ phạt gấp 9 lần (đây là tỷ lệ $4{,}5/0{,}5$), tăng tuyến tính và không tăng nhanh hơn. Để thấy rõ ảnh hưởng đến $\mathbf{w}$, ta so sánh **đạo hàm theo** $r$ (mức "kéo" của mẫu lên mô hình), như Hình 19b:

$$\frac{d}{dr} r^2 = 2r, \qquad \frac{d}{dr} |r| = \operatorname{sign}(r), \qquad \frac{d}{dr} \ell_\varepsilon(r) = \operatorname{sign}(r)\, \mathbb{1}[|r| > \varepsilon].$$




*Hình 19. (a) Ba hàm mất mát theo phần dư $r$ ($\varepsilon = 0{,}5$). (b) Đạo hàm theo $r$, tức mức ảnh hưởng của một mẫu: bình phương tăng không giới hạn theo $r$, còn $\varepsilon$-insensitive bằng $0$ trong ống và bị chặn ở $\pm 1$ bên ngoài.*

Như vậy $\varepsilon$-insensitive có hai ưu điểm so với MSE: **(i)** bằng 0 trong ống, nên không "chạy theo" nhiễu nhỏ; **(ii)** mức ảnh hưởng bị chặn ở $\pm 1$, nên ngoại lai không thể kéo mạnh tùy ý. Đây chính là điều ta thấy ở Hình 18b. Hai trường hợp đặc biệt: khi $\varepsilon = 0$ thì $\ell_0(r) = |r|$ là sai số tuyệt đối (hồi quy "độ lệch tuyệt đối nhỏ nhất"); khi $\varepsilon$ rất lớn thì mọi mẫu nằm trong ống, mô hình không còn gì để học ngoài điều chuẩn.

### Hàm mục tiêu và ý nghĩa của điều chuẩn

SVR tối thiểu hóa tổng của điều chuẩn và mất mát $\varepsilon$-insensitive, trực tiếp song song với mục 3.3:

$$J(\mathbf{w}, b) = \frac{1}{2} \| \mathbf{w} \|^2 + C \sum_{i=1}^{n} \max(0,\ |y_i - \mathbf{x}_i \mathbf{w} - b| - \varepsilon).$$

Số hạng $\frac{1}{2} \| \mathbf{w} \|^2$ ở hồi quy không còn mang ý nghĩa "lề" như ở phân loại, mà là độ **phẳng** của hàm. Theo bất đẳng thức Cauchy–Schwarz, $|f(\mathbf{x}) - f(\mathbf{z})| = |(\mathbf{x} - \mathbf{z})\mathbf{w}| \leq \| \mathbf{w} \| \, \| \mathbf{x} - \mathbf{z} \|$, nên $\| \mathbf{w} \|$ nhỏ nghĩa là $f$ thay đổi chậm khi $\mathbf{x}$ thay đổi, tức đường dự đoán ít dốc và ít nhạy với nhiễu. SVR vì vậy cân bằng hai thứ: ống phải ôm được dữ liệu (số hạng mất mát) và hàm phải phẳng (số hạng điều chuẩn).

## 6.3. Bài toán tối ưu với biến lỏng $\xi_i$, $\xi_i^*$ và dạng không ràng buộc

### Ràng buộc hai phía và biến lỏng

Ta muốn mọi mẫu nằm trong ống: $-\varepsilon \leq y_i - f(\mathbf{x}_i) \leq \varepsilon$, tức hai bất đẳng thức. Cho phép vi phạm bằng hai biến lỏng không âm: $\xi_i$ cho phía trên ($y_i$ lớn hơn dự đoán quá $\varepsilon$) và $\xi_i^*$ cho phía dưới:

$$\min_{\mathbf{w},b,\boldsymbol{\xi},\boldsymbol{\xi}^*} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}(\xi_i+\xi_i^*) \quad \text{với} \quad \begin{cases} y_i - \mathbf{x}_i\mathbf{w} - b \le \varepsilon + \xi_i, \\ \mathbf{x}_i\mathbf{w} + b - y_i \le \varepsilon + \xi_i^*, \\ \xi_i,\ \xi_i^* \ge 0. \end{cases}$$

Ở dạng ma trận, với $\mathbf{r} = \mathbf{y} - \mathbf{X}\mathbf{w} - b\mathbf{1}$:

$$\min \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\,\mathbf{1}^T(\boldsymbol{\xi}+\boldsymbol{\xi}^*) \qquad \text{với} \qquad \mathbf{r} \le \varepsilon\mathbf{1} + \boldsymbol{\xi}, \quad -\mathbf{r} \le \varepsilon\mathbf{1} + \boldsymbol{\xi}^*, \quad \boldsymbol{\xi}, \boldsymbol{\xi}^* \ge \mathbf{0}.$$

Với mẫu $i$ có phần dư $r_i$, các biến lỏng nhỏ nhất thỏa ràng buộc là

| Vị trí của mẫu | Phần dư | $\xi_i$ | $\xi_i^*$ |
|---|---|---|---|
| Trong ống | $\lvert r_i\rvert \le \varepsilon$ | 0 | 0 |
| Phía trên ống | $r_i > \varepsilon$ | $r_i - \varepsilon$ | 0 |
| Phía dưới ống | $r_i < -\varepsilon$ | 0 | $-r_i - \varepsilon$ |

Tại mỗi mẫu **nhiều nhất một** trong hai biến lỏng khác 0, và $\xi_i + \xi_i^* = \ell_\varepsilon(r_i)$. Như ở mục 3.3, vì hàm mục tiêu tăng theo biến lỏng nên ở nghiệm tối ưu chúng đạt giá trị nhỏ nhất này. Thay vào, ta thu được dạng **không ràng buộc** đã viết ở 6.2:

$$J(\mathbf{w}, b) = \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\,\mathbf{1}^T\max(\mathbf{0},\ |\mathbf{r}| - \varepsilon\mathbf{1}).$$

Ví dụ trên bộ dữ liệu một chiều với ngoại lai: nghiệm tối ưu ở mục 6.5 là $w = \frac{13}{15}$, $b = \frac{1}{3}$, với $\xi_5 = \frac{2}{15}$ và $\xi_6^* = \frac{10}{3}$ (Hình 20), các biến lỏng còn lại bằng 0.

## Dưới đạo hàm và quy tắc cập nhật

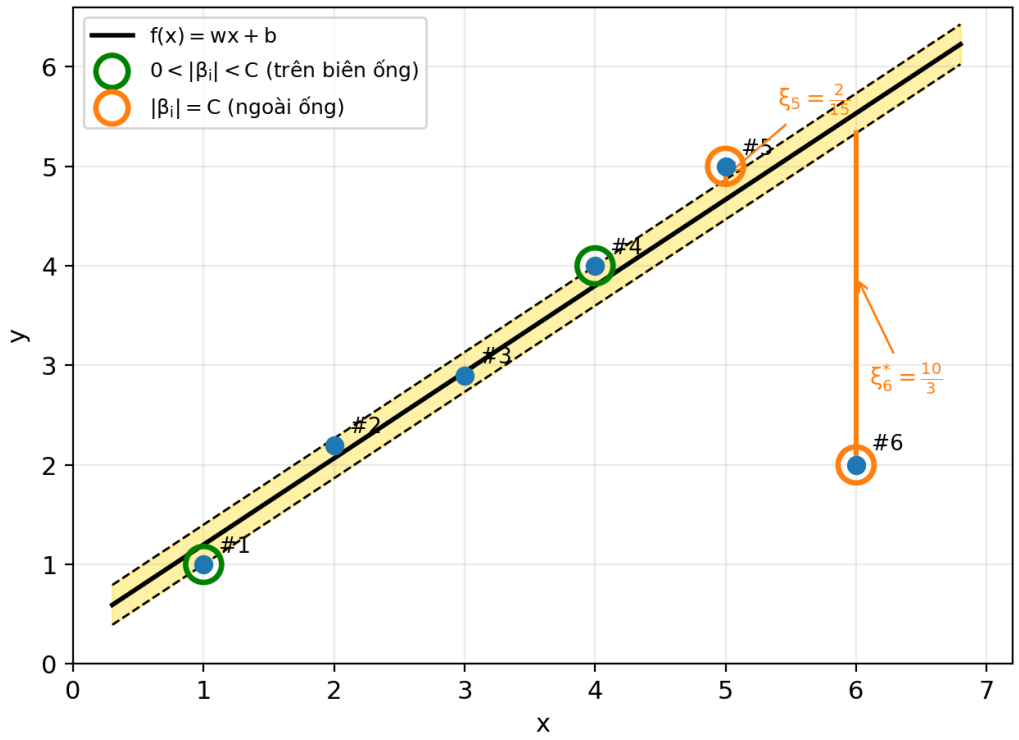
Như ở mục 3.4, hàm mất mát không khả vi tại $|r| = \varepsilon$ nên ta dùng dưới đạo hàm. Đặt $a_i = \mathbb{1}[|r_i| > \varepsilon]$ (mẫu nằm ngoài ống) và $s_i = \operatorname{sign}(r_i)\, a_i$, gom thành vector $\mathbf{s}$ ($n \times 1$, mỗi phần tử thuộc $\{-1, 0, +1\}$). Vì $r_i = y_i - \mathbf{x}_i\mathbf{w} - b$ nên $\partial r_i / \partial \mathbf{w} = -\mathbf{x}_i^T$ và $\partial r_i / \partial b = -1$, do đó

$$\nabla_{\mathbf{w}} J = \mathbf{w} - C\,\mathbf{X}^T\mathbf{s}, \qquad \frac{\partial J}{\partial b} = -C\,\mathbf{1}^T\mathbf{s}.$$

Cập nhật Gradient Descent dưới đạo hàm với tốc độ học $\eta_t = 1/t$ (lý do như ở mục 3.5):

$$\mathbf{w} \leftarrow (1 - \eta_t)\,\mathbf{w} + \eta_t C\,\mathbf{X}^T\mathbf{s}, \qquad b \leftarrow b + \eta_t C\,\mathbf{1}^T\mathbf{s}.$$

So với phân loại (mục 3.4), dạng cập nhật **giống hệt**: một bước "co" $\mathbf{w}$ rồi "đẩy" về phía các mẫu vi phạm. Chỉ khác ở chỗ: ở phân loại, mẫu vi phạm là $m_i < 1$ và hướng đẩy là $y_i\mathbf{x}_i^T$; ở hồi quy, mẫu vi phạm là $|r_i| > \varepsilon$ và hướng đẩy là $\operatorname{sign}(r_i)\,\mathbf{x}_i^T$ (mẫu nằm trên ống thì kéo đường lên, nằm dưới ống thì kéo đường xuống). Mỗi mẫu vẫn chỉ đẩy với cường độ tối đa $C$, và mẫu trong ống không đẩy gì.




*Hình 20. SVR tuyến tính trên 6 sinh viên ($C = 1$, $\varepsilon = 0{,}2$): đường $f(x) = \frac{13}{15}x + \frac{1}{3}$ và ống $\varepsilon$ (vùng vàng). Sinh viên 1 và 4 nằm đúng trên biên ống (vòng xanh lá), sinh viên 5 và 6 nằm ngoài ống (vòng cam) với biến lỏng $\xi_5 = \frac{2}{15}$ và $\xi_6^* = \frac{10}{3}$. Sinh viên 2 và 3 nằm trong ống nên không ảnh hưởng gì đến nghiệm.*

### Tính chất

$J$ là hàm lồi, và lồi chặt theo $\mathbf{w}$ (nhờ $\frac{1}{2} \| \mathbf{w} \|^2$), nên lập luận ở mục 4.2 áp dụng nguyên vẹn: nghiệm tồn tại và $\mathbf{w}^*$ duy nhất. Hệ số chệch $b^*$ duy nhất khi có ít nhất một mẫu nằm đúng trên biên ống.

## 6.4. Đối ngẫu, vector hỗ trợ trong hồi quy và SVR với kernel

### Hàm Lagrange và điều kiện dừng

Gắn nhân tử $\alpha_i \geq 0$ cho ràng buộc phía trên, $\alpha_i^* \geq 0$ cho ràng buộc phía dưới, và $\eta_i, \eta_i^* \geq 0$ cho $\xi_i \geq 0$, $\xi_i^* \geq 0$:

$$L = \frac{1}{2} \| \mathbf{w} \|^2 + C \sum_i (\xi_i + \xi_i^*) - \sum_i \alpha_i \left(\varepsilon + \xi_i - y_i + \mathbf{x}_i \mathbf{w} + b\right)$$
$$- \sum_i \alpha_i^* \left(\varepsilon + \xi_i^* + y_i - \mathbf{x}_i \mathbf{w} - b\right) - \sum_i (\eta_i \xi_i + \eta_i^* \xi_i^*).$$

Đặt $\beta_i = \alpha_i - \alpha_i^*$. Đạo hàm riêng bằng 0 cho

$$\boxed{\mathbf{w} = \sum_{i=1}^{n} \beta_i \, \mathbf{x}_i^T = \mathbf{X}^T\boldsymbol{\beta}}, \qquad \mathbf{1}^T\boldsymbol{\beta} = \sum_i (\alpha_i - \alpha_i^*) = 0, \qquad \alpha_i = C - \eta_i \le C, \quad \alpha_i^* = C - \eta_i^* \le C.$$

Công thức $\mathbf{w} = \sum_i \beta_i \, \mathbf{x}_i^T$ chính là dạng $\mathbf{w} = \sum_i \alpha_i \, y_i \mathbf{x}_i^T$ của mục 4.3, với $\alpha_i y_i$ được thay bằng hệ số có dấu $\beta_i$.

## Bài toán đối ngẫu

Thay lại vào $L$: số hạng chứa $\xi_i$, $\xi_i^*$ và $b$ triệt tiêu. Các số hạng còn lại cho $\sum_i y_i \, \beta_i - \| \mathbf{w} \|^2 + \frac{1}{2} \| \mathbf{w} \|^2 - \varepsilon \sum_i (\alpha_i + \alpha_i^*)$, và vì $\| \mathbf{w} \|^2 = \boldsymbol{\beta}^T \mathbf{K} \boldsymbol{\beta}$ với $\mathbf{K} = \mathbf{X}\mathbf{X}^T$, ta thu được **bài toán đối ngẫu** của SVR:

$$\boxed{\max_{\boldsymbol{\alpha}, \boldsymbol{\alpha}^*} \; -\frac{1}{2}\boldsymbol{\beta}^T \mathbf{K} \boldsymbol{\beta} - \varepsilon \, \mathbf{1}^T(\boldsymbol{\alpha} + \boldsymbol{\alpha}^*) + \mathbf{y}^T\boldsymbol{\beta} \quad \text{với} \quad \mathbf{1}^T\boldsymbol{\beta} = 0, \;\; \mathbf{0} \le \boldsymbol{\alpha}, \boldsymbol{\alpha}^* \le C\mathbf{1}, \;\; \boldsymbol{\beta} = \boldsymbol{\alpha} - \boldsymbol{\alpha}^*.}$$

Đây là một quy hoạch toàn phương lồi trên $2n$ biến. Viết dưới dạng chuẩn với $\boldsymbol{\gamma} = \begin{bmatrix} \boldsymbol{\alpha} \\ \boldsymbol{\alpha}^* \end{bmatrix}$ và ma trận khối $\mathbf{H}$ cỡ $2n \times 2n$:

$$\min_{\boldsymbol{\gamma}} \frac{1}{2}\boldsymbol{\gamma}^T \mathbf{H} \boldsymbol{\gamma} + \mathbf{c}^T\boldsymbol{\gamma}, \qquad \mathbf{H} = \begin{bmatrix} \mathbf{K} & -\mathbf{K} \\ -\mathbf{K} & \mathbf{K} \end{bmatrix}, \quad \mathbf{c} = \begin{bmatrix} \varepsilon\mathbf{1} - \mathbf{y} \\ \varepsilon\mathbf{1} + \mathbf{y} \end{bmatrix}, \quad [\mathbf{1}^T \;\; -\mathbf{1}^T]\boldsymbol{\gamma} = 0, \quad \mathbf{0} \le \boldsymbol{\gamma} \le C\mathbf{1}.$$

Dữ liệu chỉ xuất hiện qua $\mathbf{K}$, nên **thủ thuật kernel của mục 5 dùng được nguyên vẹn**: thay $\mathbf{K} = \mathbf{X}\mathbf{X}^T$ bằng ma trận Gram $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$.

## Điều kiện KKT và ba loại mẫu

Các điều kiện bù trừ là

$$\alpha_i(\varepsilon + \xi_i - r_i) = 0, \quad \alpha_i^*(\varepsilon + \xi_i^* + r_i) = 0, \quad (C - \alpha_i)\xi_i = 0, \quad (C - \alpha_i^*)\xi_i^* = 0.$$

Một hệ quả hữu ích: $\alpha_i$ và $\alpha_i^*$ **không đồng thời dương**. Thật vậy, nếu cả hai dương thì $\varepsilon + \xi_i = r_i$ và $\varepsilon + \xi_i^* = -r_i$; cộng hai vế được $2\varepsilon + \xi_i + \xi_i^* = 0$, vô lý khi $\varepsilon > 0$. Do đó $\beta_i = \alpha_i - \alpha_i^*$ có $|\beta_i| = \max(\alpha_i, \alpha_i^*)$, và xét theo phần dư $r_i$:

| Loại | Vị trí | Phần dư | $\beta_i$ | $\xi_i + \xi_i^*$ | Tên gọi |
|---|---|---|---|---|---|
| 1 | Trong ống | $\lvert r_i \rvert < \varepsilon$ | $0$ | $0$ | Không phải vector hỗ trợ |
| 2 | Đúng trên biên ống | $r_i = \pm\varepsilon$ | $0 < \lvert\beta_i\rvert < C$, cùng dấu $r_i$ | $0$ | Vector hỗ trợ tự do |
| 3 | Ngoài ống | $\lvert r_i \rvert > \varepsilon$ | $\beta_i = C\,\mathrm{sign}(r_i)$ | $\lvert r_i \rvert - \varepsilon$ | Vector hỗ trợ bị chặn |

(Trường hợp $|r_i| = \varepsilon$ cũng có thể ứng với $\beta_i = 0$ hoặc $|\beta_i| = C$ khi suy biến.) Đúng như mục 4.4, ta có **tính thưa**: các mẫu trong ống có $\beta_i = 0$ nên không góp mặt trong $\mathbf{w} = \sum_i \beta_i \mathbf{x}_i^T$, và lập luận "bỏ mẫu loại 1 không đổi nghiệm" ở mục 4.4 áp dụng y nguyên.

Cách đọc theo "lực" ở mục 4.1 cũng giữ nguyên: mẫu nằm **trên** ống kéo đường dự đoán lên với cường độ $\beta_i > 0$, mẫu nằm **dưới** ống kéo xuống với $\beta_i < 0$, cường độ bị chặn ở $C$, và điều kiện $\sum_i \beta_i = 0$ là cân bằng lực giữa hai phía (xác định $b$).

**Hệ số chệch.** Với mẫu $k$ có $0 < \beta_k < C$ (đúng trên biên ống phía trên, $r_k = +\varepsilon$) hoặc $-C < \beta_k < 0$ (biên dưới, $r_k = -\varepsilon$):

$$b = y_k - \varepsilon \text{sign}(\beta_k) - \sum_{i=1}^{n} \beta_i \ \mathbf{x}_i \mathbf{x}_k^T.$$

## SVR với kernel

Thay $\mathbf{x}_i \mathbf{x}_j^T$ bằng $k(\mathbf{x}_i, \mathbf{x}_j)$, hàm dự đoán là

$$f(\mathbf{x}) = \sum_{i=1}^{n} \beta_i \ k(\mathbf{x}_i, \mathbf{x}) + b = \sum_{i \in \mathcal{S}} \beta_i \ k(\mathbf{x}_i, \mathbf{x}) + b,$$

với $\mathcal{S} = \{i : \beta_i \neq 0\}$ là tập vector hỗ trợ. Với kernel RBF, $f$ là tổng các đồi Gauss dương và âm cộng $b$, giống Hình 14 nhưng đóng vai trò **đường cong dự đoán** thay vì hàm quyết định, và các ống $\pm\varepsilon$ bao quanh nó.

## Ví dụ tính tay trong không gian đối ngẫu

Xét 5 sinh viên đầu ($x = 1, \ldots, 5$, $\mathbf{y} = (1{,}0;\ 2{,}2;\ 2{,}9;\ 4{,}0;\ 5{,}0)$) với $\varepsilon = 0{,}2$ và $C$ đủ lớn. Hai mẫu ở hai đầu có xu hướng là vector hỗ trợ: sinh viên 5 nằm trên ống ($\beta_5 = +a$) và sinh viên 1 nằm dưới ống ($\beta_1 = -a$, cân bằng lực $\sum \beta_i = 0$). Với kernel tuyến tính một chiều $K_{ij} = x_i x_j$, ta có $\boldsymbol{\beta}^T \mathbf{K} \boldsymbol{\beta} = (\beta_1 x_1 + \beta_5 x_5)^2 = (-a + 5a)^2 = 16a^2$, $\mathbf{1}^T(\boldsymbol{\alpha} + \boldsymbol{\alpha}^*) = 2a$, $\mathbf{y}^T \boldsymbol{\beta} = -1{,}0a + 5{,}0a = 4a$. Hàm đối ngẫu trở thành

$$D(a) = -\frac{1}{2} \cdot 16a^2 - 0{,}2 \cdot 2a + 4a = 3{,}6\,a - 8a^2, \qquad D'(a) = 3{,}6 - 16a = 0 \ \Rightarrow \ a = 0{,}225.$$

Từ đó $w = \beta_1 x_1 + \beta_5 x_5 = 4a = 0{,}9$ và $b = y_5 - \varepsilon - w\,x_5 = 5{,}0 - 0{,}2 - 4{,}5 = 0{,}3$. Kiểm tra KKT: đường $f(x) = 0{,}9x + 0{,}3$ cho $f = (1{,}2;\ 2{,}1;\ 3{,}0;\ 3{,}9;\ 4{,}8)$ và phần dư $\mathbf{r} = (-0{,}2;\ 0{,}1;\ -0{,}1;\ 0{,}1;\ 0{,}2)$. Mọi $|r_i| \leq \varepsilon = 0{,}2$, trong đó $r_1 = -\varepsilon$ và $r_5 = +\varepsilon$ đúng như giả thiết, ba mẫu còn lại nằm trong ống ($\beta = 0$), và $0 < a = 0{,}225 < C$ nếu $C > 0{,}225$. Giá trị đối ngẫu $D(0{,}225) = 0{,}405$ **bằng** giá trị gốc $\frac{1}{2} w^2 = \frac{1}{2} \cdot 0{,}81 = 0{,}405$ (đối ngẫu mạnh).

## 6.5. Vai trò của $\varepsilon$, $C$, $\gamma$ và ví dụ tính tay một chiều

### Vai trò của $\varepsilon$: độ rộng của ống

Với 5 sinh viên đầu ($C = 1$), thay đổi $\varepsilon$ cho nghiệm sau (hai mẫu ở hai đầu, sinh viên 1 và 5, là vector hỗ trợ khi $\varepsilon \geq 0{,}2$):

| $\varepsilon$ | $w$ | $b$ | Số vector hỗ trợ | Ghi chú |
|---|---|---|---|---|
| 0 | 1,000 | 0,000 | 4 | Hồi quy sai số tuyệt đối, hầu hết mẫu là vector hỗ trợ |
| 0,1 | 0,950 | 0,150 | 4 | |
| 0,2 | 0,900 | 0,300 | 2 | Đúng nghiệm tay ở mục 6.4 |
| 0,5 | 0,750 | 0,750 | 2 | |
| 1,0 | 0,500 | 1,500 | 2 | |
| 2,0 | 0 | 3,000 | 0 | Mô hình hằng số |

Khi hai đầu là vector hỗ trợ và ống ôm sát, đường thẳng phải đi qua hai biên ống ở $x = 1$ và $x = 5$: $f(1) = y_1 + \varepsilon$ và $f(5) = y_5 - \varepsilon$. Từ đó $w = \frac{(y_5 - y_1) - 2\varepsilon}{5-1} = 1 - \frac{\varepsilon}{2}$ và $b = 1{,}5\,\varepsilon$, khớp với bảng ($\varepsilon = 0{,}5$ cho $w = 0{,}75$, $b = 0{,}75$). Có thể rút ra:

1. $\varepsilon$ **lớn thì đường phẳng hơn và ít vector hỗ trợ hơn.** Ống rộng ôm được nhiều mẫu hơn, nên ít mẫu nằm trên biên, và điều chuẩn có "chỗ" để làm $w$ nhỏ đi.
2. $\varepsilon = 2$ **là ngưỡng**: ống rộng $\pm 2$ bao trọn mọi giá trị $y \in [1;\ 5]$ quanh trung điểm 3, nên đường nằm ngang $f = 3$ đã không phạm lỗi nào và $w = 0$.
3. $\varepsilon$ **nhỏ thì bám sát dữ liệu hơn nhưng nhạy với nhiễu.** Khi $\varepsilon = 0$, hầu hết mẫu thành vector hỗ trợ.

Về mặt thực hành, $\varepsilon$ nên cỡ độ lớn của **nhiễu** ($\varepsilon$ bằng khoảng độ lệch chuẩn của nhiễu). Đây là điểm khác với $C$ và $\gamma$ ở chỗ $\varepsilon$ gắn trực tiếp với đơn vị của $y$.

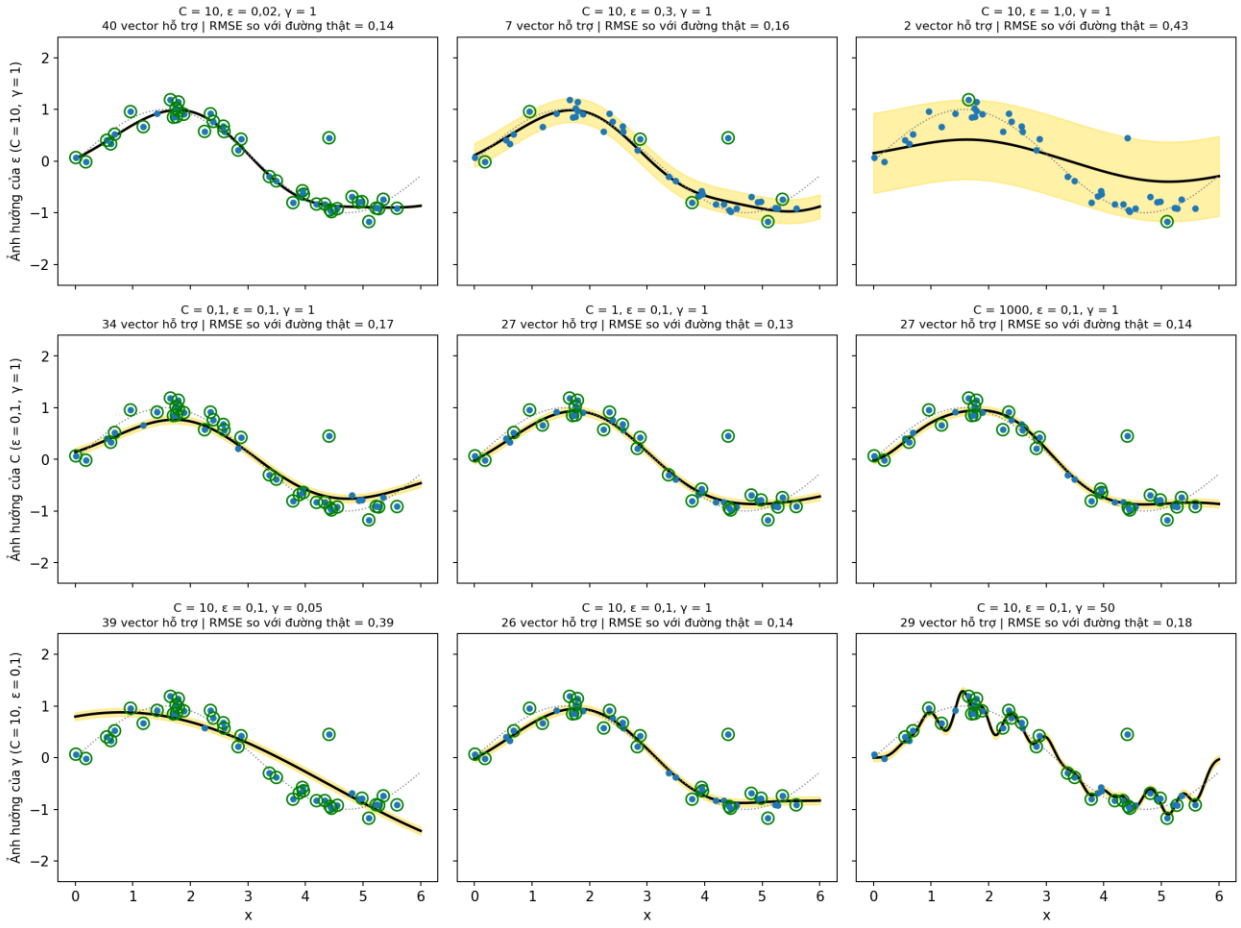
### Vai trò của $C$

$C$ có vai trò như ở mục 3 và 4: giới hạn cường độ lực $|\beta_i| \leq C$. Với cùng dữ liệu và $\varepsilon = 0{,}2$, vì hai vector hỗ trợ có $\beta = \pm 0{,}225$ nên:

1. Nếu $C \geq 0{,}225$: ràng buộc $|\beta_i| \leq C$ không còn tác dụng, nghiệm không đổi ($w = 0{,}9$). Đây là ngưỡng tương tự $C^* = \frac{2}{13}$ ở mục 4.2.
2. Nếu $C < 0{,}225$: các $|\beta_i|$ bị chặn ở $C$, đường nằm "phẳng hơn" vì $w = \sum_i \beta_i\, x_i = 4C$ khi chỉ có hai vector hỗ trợ bị chặn: $C = 0{,}2$ cho $w = 0{,}8$; $C = 0{,}1$ cho $w = 0{,}6$ (lúc này sinh viên 2 và 4 cũng thành vector hỗ trợ).

## Vai trò của $\gamma$ (kernel RBF) và bức tranh chung trên dữ liệu phi tuyến

Với kernel RBF, $\gamma$ điều khiển độ rộng các "đồi" Gauss như ở mục 5.4: $\gamma$ nhỏ cho đường cong rất trơn, $\gamma$ lớn cho đường cong uốn lượn bám từng điểm. Hình 21 minh họa cả ba tham số trên bộ dữ liệu $\sin x$ có nhiễu và ngoại lai (đặc trưng $x$ và nhãn $y$ đã chuẩn hóa; vẽ lại ở thang đo gốc). Nét chấm xám là đường thật $\sin x$, vùng vàng là ống $\pm\varepsilon$ quanh dự đoán.




*Hình 21. SVR RBF trên $\sin x$ có nhiễu và ba ngoại lai (42 điểm huấn luyện). Hàng trên: thay đổi $\varepsilon$ ($0{,}02;\ 0{,}3;\ 1{,}0$). Hàng giữa: thay đổi $C$ ($0{,}1;\ 1;\ 1000$). Hàng dưới: thay đổi $\gamma$ ($0{,}05;\ 1;\ 50$). Vòng xanh lá là vector hỗ trợ. Tiêu đề ghi số vector hỗ trợ và sai số RMSE giữa dự đoán và đường thật $\sin x$.*

| Tham số thay đổi | Giá trị | Số vector hỗ trợ | RMSE so với đường thật | Nhận xét |
|---|---|---|---|---|
| $\varepsilon$ ($C = 10, \gamma = 1$) | 0,02 | 40 | 0,141 | Ống rất hẹp: gần như mọi mẫu là vector hỗ trợ |
| | 0,3 | 7 | 0,155 | Ít vector hỗ trợ mà vẫn khớp tốt |
| | 1,0 | 2 | 0,432 | Ống quá rộng: **dưới khớp** |

| Tham số thay đổi | Giá trị | Số vector hỗ trợ | RMSE so với đường thật | Nhận xét |
|---|---|---|---|---|
| $C$ ($\varepsilon = 0{,}1, \gamma = 1$) | 0,1 | 34 | 0,167 | Lực yếu, đường hơi mượt quá |
| | 1 | 27 | 0,126 | Tốt nhất trong ba giá trị |
| | 1000 | 27 | 0,141 | Ngoại lai bắt đầu kéo đường cong |
| $\gamma$ ($C = 10, \varepsilon = 0{,}1$) | 0,05 | 39 | 0,387 | Quá trơn: **dưới khớp** |
| | 1 | 26 | 0,137 | Cân bằng |
| | 50 | 29 | 0,175 | Quá uốn lượn: **quá khớp** nhiễu |

Cả ba tham số đều tạo ra vùng "dưới khớp" và vùng "quá khớp", và chúng tương tác với nhau, nên cần chọn đồng thời bằng kiểm định chéo và grid search (mục 7.5). Số vector hỗ trợ vẫn là chỉ báo hữu ích: nó tăng khi $\varepsilon$ nhỏ, và đường dự đoán càng phức tạp thì càng nhiều vector hỗ trợ.

## Ví dụ tính tay một chiều có ngoại lai

Ta quay lại bộ dữ liệu 6 sinh viên ở đầu mục ($y_6 = 2{,}0$ là ngoại lai) với $\varepsilon = 0{,}2$ và $C = 1$, và dùng cách "đoán tập vector hỗ trợ, giải, rồi kiểm tra" như ở mục 2.5.

**Bước 1: đoán cấu trúc.** Trực giác từ Hình 18: sinh viên 6 nằm rất xa ống, sinh viên 5 cũng nằm trên ống một chút khi đường bị kéo xuống, còn sinh viên 1 và 4 là hai điểm chạm biên ống. Giả thiết: sinh viên 1 nằm đúng biên dưới ($r_1 = -\varepsilon$, $\beta_1 < 0$), sinh viên 4 đúng biên trên ($r_4 = +\varepsilon$, $\beta_4 > 0$); sinh viên 5 nằm trên ống ($\beta_5 = +C$), sinh viên 6 nằm dưới ống ($\beta_6 = -C$); sinh viên 2 và 3 nằm trong ống ($\beta = 0$).

**Bước 2: giải $w$ và $b$ từ hai biên ống.** Hai điều kiện $r_1 = -0{,}2$ và $r_4 = +0{,}2$:

$$1{,}0 - (w + b) = -0{,}2 \;\Rightarrow\; w + b = 1{,}2, \qquad 4{,}0 - (4w + b) = 0{,}2 \;\Rightarrow\; 4w + b = 3{,}8.$$

Trừ vế theo vế: $3w = 2{,}6$, nên $w = \frac{13}{15} \approx 0{,}8667$ và $b = 1{,}2 - w = \frac{1}{3} \approx 0{,}3333$.

**Bước 3: giải các hệ số đối ngẫu.** Từ $\sum_i \beta_i = 0$ và $\mathbf{w} = \sum_i \beta_i x_i$ với $\beta_5 = 1$, $\beta_6 = -1$:

$$\beta_1 + \beta_4 + 1 - 1 = 0 \;\Rightarrow\; \beta_1 = -\beta_4, \qquad \beta_1 \cdot 1 + \beta_4 \cdot 4 + 1 \cdot 5 - 1 \cdot 6 = 3\beta_4 - 1 = \frac{13}{15} \;\Rightarrow\; \beta_4 = \frac{28}{45} \approx 0{,}622.$$

Vậy $\beta_4 = \frac{28}{45}$ và $\beta_1 = -\frac{28}{45}$, thỏa $0 < |\beta| < C = 1$ đúng như giả thiết (vector hỗ trợ tự do).

**Bước 4: kiểm tra các giả thiết còn lại.** Với $f(x) = \frac{13}{15}x + \frac{1}{3}$:

| Sinh viên | $x_i$ | $y_i$ | $f(x_i)$ | $r_i = y_i - f(x_i)$ | Vị trí | $\beta_i$ |
|---|---|---|---|---|---|---|
| 1 | 1 | 1,0 | 1,200 | $-0{,}200$ | Biên dưới | $-\frac{28}{45}$ |
| 2 | 2 | 2,2 | 2,067 | 0,133 | Trong ống | 0 |
| 3 | 3 | 2,9 | 2,933 | $-0{,}033$ | Trong ống | 0 |
| 4 | 4 | 4,0 | 3,800 | 0,200 | Biên trên | $+\frac{28}{45}$ |
| 5 | 5 | 5,0 | 4,667 | 0,333 | Trên ống | $+1$ |
| 6 | 6 | 2,0 | 5,533 | $-3{,}533$ | Dưới ống | $-1$ |

Mọi giả thiết đều đúng: $|r_2|, |r_3| < \varepsilon,\ r_5 > \varepsilon$ và $r_6 < -\varepsilon$. Các biến lỏng khác 0 là $\xi_5 = 0{,}333 - 0{,}2 = \frac{2}{15}$ và $\xi_6^* = 3{,}533 - 0{,}2 = \frac{10}{3}$. Giá trị gốc:

$$J = \frac{1}{2}w^2 + C(\xi_5 + \xi_6^*) = \frac{1}{2}\cdot\frac{169}{225} + \frac{2}{15} + \frac{10}{3} = \frac{169}{450} + \frac{1560}{450} = \frac{1729}{450} \approx 3{,}842.$$

Hàm đối ngẫu tại nghiệm này là $-\frac{1}{2}w^2 - \varepsilon\sum_i|\beta_i| + \mathbf{y}^T\boldsymbol{\beta} = -0{,}3756 - 0{,}2\cdot 3{,}2444 + 4{,}8667 = 3{,}842$, **bằng** giá trị gốc.

**Bước 5: ảnh hưởng của ngoại lai bị chặn.** Chỉ cần đổi $y_6$ sang giá trị bất kỳ miễn vẫn nằm **dưới ống** (tức $y_6 < f(6) - \varepsilon \approx 5{,}333$), ví dụ $y_6 = 3{,}0$, $2{,}0$ hay $-2{,}0$: mọi giả thiết ở Bước 1 và mọi phương trình ở Bước 2 và 3 **không hề thay đổi**, vì chúng chỉ dùng $y_1$, $y_4$ và việc $\beta_6 = -C$. Do đó nghiệm vẫn là $w = \frac{13}{15}$, $b = \frac{1}{3}$. Đây chính xác là điều Hình 18b cho thấy, và nó đối lập hẳn với hồi quy MSE (hệ số góc 0,56, 0,41, $-0{,}16$). Điều này là hệ quả trực tiếp của việc cường độ lực của mỗi mẫu bị chặn ở $C$: ngoại lai càng xa thì độ lệch của nó càng lớn, nhưng **lực nó tác động vẫn chỉ bằng** $C$.

So với hồi quy trên 5 điểm sạch ($w = 0{,}9$, $b = 0{,}3$), nghiệm có ngoại lai chỉ đổi từ 0,9 sang 0,867, chứ không đổi hoàn toàn như hồi quy tuyến tính (0,98 → 0,41).

## 6.6. Mã hóa SVR từ đầu và so sánh với hồi quy tuyến tính

### Hai cách cài đặt

Ta cài đặt hai phiên bản, tương ứng hai cách giải ở mục 6.3 và 6.4:

1. ***LinearSVR*:** huấn luyện trực tiếp trên bài toán gốc không ràng buộc bằng Gradient Descent dưới đạo hàm, đúng như quy tắc cập nhật ở mục 6.3 và thuật toán batch ở mục 3.6.

2. ***KernelSVR*:** giải bài toán đối ngẫu $\min \frac{1}{2}\boldsymbol{\gamma}^T\mathbf{H}\boldsymbol{\gamma} + \mathbf{c}^T\boldsymbol{\gamma}$ trên $2n$ biến bằng bộ giải toàn phương *SLSQP* của SciPy (đủ cho vài trăm mẫu; các thư viện thật dùng SMO). Sau đó tính $b$ từ các vector hỗ trợ tự do theo công thức ở mục 6.4.

```python
import numpy as np
from scipy.optimize import minimize


def rbf_kernel(A, B, gamma=1.0):
    sq = (A**2).sum(1)[:, None] + (B**2).sum(1)[None, :] - 2 * A @ B.T
    return np.exp(-gamma * np.maximum(sq, 0))


class LinearSVR:
    """SVR tuyến tính huấn luyện trên bài toán gốc không ràng buộc:
    J(w, b) = 1/2 ||w||^2 + C * sum_i max(0, |y_i - x_i w - b| - eps)"""

    def __init__(self, C=1.0, eps=0.1, n_iter=5000):
        self.C, self.eps, self.n_iter = C, eps, n_iter

    def objective(self, X, y):
        r = y - X @ self.w - self.b
        return 0.5 * self.w @ self.w + self.C * np.maximum(0, np.abs(r) - sel
f.eps).sum()

    def fit(self, X, y):
        n, d = X.shape
        self.w, self.b = np.zeros(d), 0.0
        self.history = []
        for t in range(1, self.n_iter + 1):
            r = y - X @ self.w - self.b             # phần dư
            a = (np.abs(r) > self.eps).astype(float)  # mẫu nằm ngoài ống
            s = np.sign(r) * a                      # s_i = sign(r_i) nếu ngo
ài ống, 0 nếu trong
            g_w = self.w - self.C * X.T @ s         # dưới đạo hàm theo w
            g_b = -self.C * s.sum()                 # dưới đạo hàm theo b
            eta = 1.0 / t                           # bước giảm dần
            self.w -= eta * g_w
            self.b -= eta * g_b
            self.history.append(self.objective(X, y))
        return self

    def predict(self, X):
        return X @ self.w + self.b


class KernelSVR:
    """SVR với kernel, giải bài toán đối ngẫu bằng bộ giải toàn phương SLSQP.
    min 1/2 (a-a*)^T K (a-a*) + eps 1^T (a+a*) - y^T (a-a*)
    với 1^T (a-a*) = 0 và 0 <= a, a* <= C"""

    def __init__(self, kernel, C=1.0, eps=0.1):
        self.kernel, self.C, self.eps = kernel, C, eps
```

```
    def fit(self, X, y):
        n = len(y)
        K = self.kernel(X, X)

        def obj(g):
            beta = g[:n] - g[n:]
            return 0.5 * beta @ K @ beta + self.eps * g.sum() - y @ beta

        def grad(g):
            beta = g[:n] - g[n:]
            Kb = K @ beta
            return np.r_[Kb + self.eps - y, -Kb + self.eps + y]

        cons = {"type": "eq", "fun": lambda g: g[:n].sum() - g[n:].sum(),
                "jac": lambda g: np.r_[np.ones(n), -np.ones(n)]}
        res = minimize(obj, np.zeros(2 * n), jac=grad, bounds=[(0, self.C)] *
(2 * n),
                       constraints=[cons], method="SLSQP",
                       options={"maxiter": 500, "ftol": 1e-12})
        beta = res.x[:n] - res.x[n:]
        self.beta = np.where(np.abs(beta) < 1e-7, 0.0, beta)
        sv = self.beta != 0
        self.X_sv, self.coef = X[sv], self.beta[sv]
        # b từ các vector hỗ trợ tự do: 0 < |beta_i| < C
        free = (np.abs(self.beta) > 1e-7) & (np.abs(self.beta) < self.C - 1e-
7)
        f0 = K @ self.beta
        bs = y[free] - f0[free] - self.eps * np.sign(self.beta[free])
        self.b = bs.mean() if free.any() else np.median(y - f0)
        self.dual_value = -res.fun
        return self

    def predict(self, X):
        return self.kernel(X, self.X_sv) @ self.coef + self.b
```

Giải thích các dòng chính:

1. Ở *LinearSVR.fit*, *r = y - X @ w - b* là phần dư $\mathbf{r}$, *a* là vector chỉ báo "ngoài ống" và *s = sign(r) * a* là vector $\mathbf{s}$. Hai dòng *g_w*, *g_b* là $\nabla_{\mathbf{w}} J = \mathbf{w} - C\mathbf{X}^T\mathbf{s}$ và $\partial J / \partial b = -C\mathbf{1}^T\mathbf{s}$. Bước giảm dần $\eta_t = 1/t$ như ở mục 3.

2. Ở *KernelSVR.fit*, biến *g* ghép $\boldsymbol{\gamma} = [\boldsymbol{\alpha}; \boldsymbol{\alpha}^*]$. Hàm *obj* là $\frac{1}{2}\boldsymbol{\beta}^T\mathbf{K}\boldsymbol{\beta} + \varepsilon\mathbf{1}^T\boldsymbol{\gamma} - \mathbf{y}^T\boldsymbol{\beta}$ (dạng cực tiểu của hàm đối ngẫu), ràng buộc đẳng thức là $\mathbf{1}^T\boldsymbol{\beta} = 0$, và *bounds* là $0 \le \gamma_i \le C$.

3. Sau khi giải, chỉ giữ lại các vector hỗ trợ (*X_sv*, *coef*$= \beta_i$) nên dự đoán là *kernel(X_new, X_sv) @ coef + b*, đúng $f(\mathbf{x}) = \sum_{i \in \mathcal{S}} \beta_i \, k(\mathbf{x}_i, \mathbf{x}) + b$.

## Chạy và đối chiếu với scikit-learn

Đoạn mã sau chạy hai ví dụ: bộ một chiều có ngoại lai và bộ $\sin x$ có ngoại lai, so với *LinearRegression* và *SVR* của scikit-learn. Ta chuẩn hóa cả $x$ và $y$ (để $\varepsilon$ có thang đo hợp lý).

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.svm import SVR
from sklearn.model_selection import train_test_split
from svr_scratch import LinearSVR, KernelSVR, rbf_kernel

# --- Ví dụ 1: 5 mẫu bình thường + 1 ngoại lai (x=6, y=2) ---
x = np.array([1, 2, 3, 4, 5, 6], dtype=float)
y = np.array([1.0, 2.2, 2.9, 4.0, 5.0, 2.0])
X = x[:, None]

ols = LinearRegression().fit(X, y)
mine = LinearSVR(C=1.0, eps=0.2, n_iter=20000).fit(X, y)
ref = SVR(kernel="linear", C=1.0, epsilon=0.2, tol=1e-12).fit(X, y)
print("OLS      w, b =", ols.coef_[0].round(4), ols.intercept_.round(4))
print("LinearSVR w, b =", mine.w[0].round(4), round(mine.b, 4), " J =", round
(mine.objective(X, y), 4))
print("sklearn  w, b =", ref.coef_[0, 0].round(4), ref.intercept_[0].round(4)
)

# --- Ví dụ 2: y = sin(x) + nhiễu + 3 ngoại lai ---
rng = np.random.default_rng(3)
xs = np.sort(rng.uniform(0, 6, 60))
ys = np.sin(xs) + rng.normal(0, 0.12, 60)
ys[[10, 30, 45]] += [1.6, -1.8, 1.5]                    # ngoại lai
Xtr, Xte, ytr, yte = train_test_split(xs[:, None], ys, test_size=0.3, random_
state=0)

mu, sd = Xtr.mean(0), Xtr.std(0)                        # chuẩn hóa x
ym, sy = ytr.mean(), ytr.std()                          # chuẩn hóa y (để eps c
ó thang đo hợp lý)
Ztr, Zte, ztr = (Xtr - mu) / sd, (Xte - mu) / sd, (ytr - ym) / sy

kernel = lambda A, B: rbf_kernel(A, B, gamma=1.0)
svr = KernelSVR(kernel, C=10.0, eps=0.1).fit(Ztr, ztr)
ref = SVR(kernel="rbf", gamma=1.0, C=10.0, epsilon=0.1, tol=1e-10).fit(Ztr, z
tr)
grid = (np.linspace(0, 6, 400)[:, None] - mu) / sd
print("số vector hỗ trợ:", len(svr.coef), "| sklearn:", len(ref.support_))
print("chênh lệch dự đoán lớn nhất:", np.abs(svr.predict(grid) - ref.predict(
grid)).max())

def report(name, f):
    p, g = f(Zte), f(grid)
    mae = np.abs(yte - p).mean()
```

```
    rmse = np.sqrt(((yte - p) ** 2).mean())
    r2 = 1 - ((yte - p) ** 2).sum() / ((yte - yte.mean()) ** 2).sum()
    true = np.sin(np.linspace(0, 6, 400))
    print(f"{name:14s} MAE={mae:.3f} RMSE={rmse:.3f} R2={r2:.3f} | RMSE so với sin(x)={np.sqrt(((true-g)**2).mean()):.3f}")

lin = LinearRegression().fit(Ztr, ytr)
report("Hồi quy tuyến tính", lambda Z: lin.predict(Z))
report("SVR tuyến tính", lambda Z: SVR(kernel="linear", C=1, epsilon=0.1).fit(Ztr, ztr).predict(Z) * sy + ym)
report("SVR kernel RBF", lambda Z: svr.predict(Z) * sy + ym)
```

Kết quả:

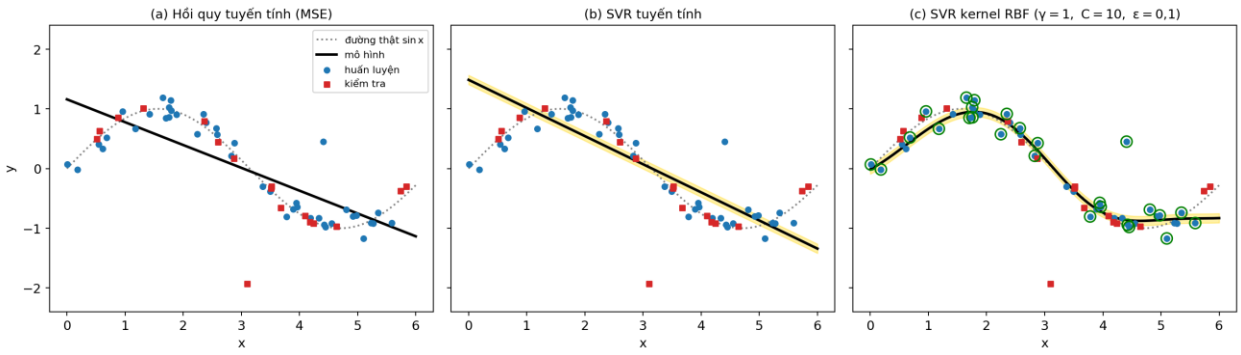
```
OLS      w, b = 0.4143 1.4
LinearSVR w, b = 0.8666 0.3335  J = 3.8423
sklearn  w, b = 0.8667 0.3333
số vector hỗ trợ: 26 | sklearn: 26
chênh lệch dự đoán lớn nhất: 9.975447294863216e-07
Hồi quy tuyến tính MAE=0.531 RMSE=0.732 R2=0.441 | RMSE so với sin(x)=0.422
SVR tuyến tính MAE=0.546 RMSE=0.743 R2=0.424 | RMSE so với sin(x)=0.478
SVR kernel RBF MAE=0.356 RMSE=0.649 R2=0.560 | RMSE so với sin(x)=0.137
```

**Về ví dụ một chiều.** *LinearSVR* tự cài đặt cho $w = 0{,}8666$, $b = 0{,}3335$ và $J = 3{,}8423$, khớp với nghiệm tay $w = \frac{13}{15} \approx 0{,}8667$, $b = \frac{1}{3}$, $J = \frac{1729}{450} \approx 3{,}8422$ và với *SVR* của scikit-learn. Hồi quy tuyến tính cho $w = 0{,}414$, $b = 1{,}4$, lệch hẳn do ngoại lai.

**Về ví dụ phi tuyến.** *KernelSVR* tự cài đặt có 26 vector hỗ trợ giống *SVR* của scikit-learn, và dự đoán khác nhau tối đa cỡ $10^{-6}$ trên 400 điểm lưới. Ba chỉ số đánh giá dùng ở đây (sẽ được định nghĩa chi tiết ở mục 7.4) tính trên $m$ mẫu kiểm tra với dự đoán $\hat{y}_i$ là

$$\text{MAE} = \frac{1}{m}\sum_i |\,y_i - \hat{y}_i|, \quad \text{RMSE} = \sqrt{\frac{1}{m}\sum_i (y_i - \hat{y}_i)^2}\,, \quad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

| Mô hình | MAE | RMSE | $R^2$ | RMSE so với $\sin x$ |
|---|---|---|---|---|
| Hồi quy tuyến tính (MSE) | 0,531 | 0,732 | 0,441 | 0,422 |
| SVR tuyến tính ($C = 1,\ \varepsilon = 0{,}1$) | 0,546 | 0,743 | 0,424 | 0,478 |
| SVR kernel RBF ($C = 10,\ \gamma = 1,\ \varepsilon = 0{,}1$) | **0,356** | **0,649** | **0,560** | **0,137** |




*Hình 22. (a) Hồi quy tuyến tính chỉ có thể là một đường thẳng và không biểu diễn được dạng sóng. (b) SVR tuyến tính vẫn là đường thẳng nên cũng không khớp. (c) SVR kernel RBF bám đường cong thật $\sin x$ và gần như không bị ba ngoại lai kéo lệch (chấm xanh/ô vuông đỏ lệch xa là các ngoại lai). Vùng vàng là ống $\pm\varepsilon$, vòng xanh lá là vector hỗ trợ.*

Cần đọc đúng các con số này:

1. Cột "RMSE so với $\sin x$" cho thấy khác biệt rõ nhất: SVR RBF sai lệch 0,137 so với đường thật, còn hồi quy tuyến tính là 0,422, gấp hơn ba lần. Mô hình tuyến tính không thể biểu diễn sóng sin, bất kể dùng MSE hay $\varepsilon$-insensitive (SVR tuyến tính còn kém nhất, 0,478).
2. Các cột MAE, RMSE, $R^2$ tính trên 18 mẫu kiểm tra **có chứa ngoại lai**, nên đều bị nâng lên (không mô hình nào dự đoán được một ngoại lai, nên sai số lớn là không tránh khỏi). Vì RMSE nhạy với sai số lớn hơn MAE, khác biệt giữa các mô hình ở MAE (0,356 so với 0,531) lớn hơn ở RMSE (0,649 so với 0,732). Điều này cho thấy với dữ liệu có ngoại lai, chỉ số nên dùng là MAE hay sai số so với đường thật (nếu biết), chứ không chỉ RMSE.
3. Chỉ có 18 mẫu kiểm tra nên các con số có độ ngẫu nhiên đáng kể. Nên kiểm chứng bằng kiểm định chéo (mục 7.5).

## Khi nào dùng hồi quy tuyến tính, khi nào dùng SVR?

| Tiêu chí | Hồi quy tuyến tính (MSE) | SVR |
|---|---|---|
| Hàm mất mát | Bình phương, không giới hạn | $\varepsilon$-insensitive, ảnh hưởng bị chặn |
| Ngoại lai | Nhạy | Bền vững hơn (lực chặn ở $C$) |
| Phi tuyến | Phải tự thêm đặc trưng | Kernel, không cần dựng đặc trưng |
| Nghiệm | Công thức đóng $(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$, nhanh, cực nhanh với dữ liệu lớn | Giải bài toán tối ưu; kernel tốn $O(n^2)$ đến $O(n^3)$ |
| Siêu tham số | Gần như không có | $\varepsilon$, $C$, $\gamma$ (cần chọn) |

| Tiêu chí | Hồi quy tuyến tính (MSE) | SVR |
|---|---|---|
| Diễn giải | Hệ số **w** trực tiếp (tuyến tính) | Tuyến tính: giống nhau; kernel: khó diễn giải |
| Dự đoán | $O(d)$ | Tỷ lệ với số vector hỗ trợ (kernel) |

Hồi quy tuyến tính là lựa chọn đơn giản và nhanh khi quan hệ gần tuyến tính và dữ liệu sạch. SVR đáng cân nhắc khi dữ liệu có ngoại lai, quan hệ phi tuyến, hoặc khi ta muốn kiểm soát rõ mức sai số chấp nhận được qua $\varepsilon$.

## Tóm tắt mục 6

1. **MSE nhạy với ngoại lai** vì phạt bình phương làm cường độ kéo $\propto r$ không có trần: một mẫu trong sáu đã đổi hệ số góc từ $0{,}98$ xuống $0{,}41$, thậm chí đổi dấu.
2. **Hàm mất mát $\varepsilon$-insensitive** $\ell_\varepsilon(r) = \max(0, |r| - \varepsilon)$ bằng 0 trong **ống** $|r| \le \varepsilon$ và tăng tuyến tính bên ngoài, nên bỏ qua nhiễu nhỏ và có mức ảnh hưởng bị chặn ở $\pm 1$.
3. **Bài toán gốc:** $\min \frac{1}{2} \| \mathbf{w} \|^2 + C \sum_i (\xi_i + \xi_i^*)$ với $y_i - f(\mathbf{x}_i) \le \varepsilon + \xi_i$, $f(\mathbf{x}_i) - y_i \le \varepsilon + \xi_i^*$, tương đương $J = \frac{1}{2} \| \mathbf{w} \|^2 + C \sum_i \ell_\varepsilon(r_i)$. Cập nhật dưới đạo hàm có cùng dạng "co và đẩy" như phân loại, với hướng đẩy $\text{sign}(r_i)\mathbf{x}_i^T$ cho mẫu ngoài ống.
4. **Đối ngẫu:** $\mathbf{w} = \sum_i \beta_i \mathbf{x}_i^T$ với $\beta_i = \alpha_i - \alpha_i^* \in [-C, C]$, $\sum_i \beta_i = 0$. Mẫu **trong ống** có $\beta_i = 0$; mẫu **trên biên ống** có $0 < |\beta_i| < C$; mẫu **ngoài ống** có $|\beta_i| = C$. Dữ liệu chỉ xuất hiện qua **K**, nên kernel dùng được: $f(\mathbf{x}) = \sum_{i \in \mathcal{S}} \beta_i \, k(\mathbf{x}_i, \mathbf{x}) + b$.
5. **Vai trò tham số:** $\varepsilon$ lớn cho ống rộng, đường phẳng và ít vector hỗ trợ ($\varepsilon = 2$ trên ví dụ cho mô hình hằng số); $C$ chặn lực của mỗi mẫu (với ví dụ, $C \ge 0{,}225$ cho nghiệm không đổi); $\gamma$ điều khiển độ cong của kernel RBF. Ba tham số cùng tạo vùng dưới khớp và quá khớp.
6. **Ví dụ tay:** với ngoại lai, nghiệm $w = \frac{13}{15}$, $b = \frac{1}{3}$, và nghiệm **không đổi** khi ngoại lai di chuyển tùy ý trong vùng dưới ống, trong khi hồi quy MSE đổi từ $0{,}56$ đến $-0{,}16$.
7. **Cài đặt:** `LinearSVR` (gradient dưới đạo hàm) và `KernelSVR` (đối ngẫu) khớp với `SVR` của scikit-learn; trên dữ liệu $\sin x$ có ngoại lai, SVR RBF đạt RMSE so với đường thật là $0{,}137$ so với $0{,}422$ của hồi quy tuyến tính.

## Câu hỏi kiểm tra nhanh

1. Với $\varepsilon = 0{,}5$, tính mất mát $\varepsilon$-insensitive của các phần dư $r = 0{,}3$, $r = -0{,}8$ và $r = 2$. Mẫu nào không bị phạt?
2. Với 5 sinh viên sạch, $\varepsilon = 0{,}5$ và $C$ lớn: dùng công thức $w = 1 - \varepsilon/2$, $b = 1{,}5\varepsilon$ để tìm đường dự đoán, rồi kiểm tra mọi phần dư có thỏa $|r_i| \le \varepsilon$ hay không.
3. Vì sao nghiệm SVR ở ví dụ 6 sinh viên không đổi khi $y_6$ đổi từ $2{,}0$ sang $-10$, trong khi nghiệm hồi quy tuyến tính thay đổi?
4. Với $\varepsilon = 2$ trên 5 sinh viên sạch, vì sao nghiệm là $w = 0$, $b = 3$? Nếu $\varepsilon = 1{,}9$ thì nghiệm có còn bằng 0 không?
5. So sánh vector hỗ trợ của SVM phân loại và của SVR: các mẫu nào là vector hỗ trợ, và ở vị trí nào so với dải lề/ống?

### Gợi ý đáp án

1. $\ell_{0,5}(0{,}3) = \max(0{,}0{,}3 - 0{,}5) = 0$; $\ell_{0,5}(-0{,}8) = 0{,}8 - 0{,}5 = 0{,}3$; $\ell_{0,5}(2) = 2 - 0{,}5 = 1{,}5$. Chỉ phần dư $r = 0{,}3$ (nằm trong ống) không bị phạt.
2. $w = 1 - 0{,}25 = 0{,}75$ và $b = 1{,}5 \cdot 0{,}5 = 0{,}75$, tức $f(x) = 0{,}75x + 0{,}75$, cho $f = (1{,}5;\ 2{,}25;\ 3{,}0;\ 3{,}75;\ 4{,}5)$ và $\mathbf{r} = (-0{,}5;\ -0{,}05;\ -0{,}1;\ 0{,}25;\ 0{,}5)$. Mọi $|r_i| \le 0{,}5$, trong đó sinh viên 1 và 5 nằm đúng trên biên ống (vector hỗ trợ), còn ba sinh viên còn lại nằm trong ống.
3. Sinh viên 6 nằm dưới ống và bị chặn ở $\beta_6 = -C$. Các phương trình xác định $w$, $b$ chỉ dùng $y_1$, $y_4$ (hai điểm trên biên ống) và điều kiện cân bằng $\sum \beta_i = 0$, trong đó $\beta_6$ luôn bằng $-C$ dù $y_6$ là bao nhiêu miễn vẫn nằm dưới ống. Hồi quy tuyến tính có cường độ kéo $\propto r_6$ nên $y_6$ càng thấp thì đường càng bị kéo xuống.
4. Với $\varepsilon = 2$, ống $f \pm 2$ bao trọn mọi $y \in [1{,}5]$ khi $f = 3$ nên mọi phần dư đều $\le 2$ và mất mát bằng 0. Số hạng còn lại là $\frac{1}{2}w^2$ nhỏ nhất ở $w = 0$, nên $w = 0$, $b = 3$. Với $\varepsilon = 1{,}9 < 2$, đường nằm ngang $f = 3$ không còn bao hết dữ liệu ($|y_1 - 3| = |y_5 - 3| = 2 > 1{,}9$), nên nghiệm có $w > 0$ nhỏ (tương ứng $w = 1 - \varepsilon/2 = 0{,}05$, $b = 1{,}5 \cdot 1{,}9 = 2{,}85$ theo công thức của hai điểm hai đầu).
5. SVM phân loại: vector hỗ trợ là mẫu **nằm trên hoặc trong** dải lề (hoặc sai phía), nghĩa là các mẫu "gần ranh giới". SVR: vector hỗ trợ là mẫu **nằm trên hoặc ngoài** ống, nghĩa là các mẫu "khó dự đoán". Phần còn lại (ngoài dải lề, tương ứng trong ống) có hệ số 0 và bị bỏ qua.

# 7. Đánh giá, trực quan hóa và chọn siêu tham số

Sáu mục đầu đã xây dựng mô hình: SVM phân loại (lề cứng, lề mềm, kernel) và SVR. Mục này trả lời câu hỏi mà mọi mô hình học máy đều phải đối mặt sau khi huấn luyện xong: **mô hình này tốt đến đâu, và ta nên chỉnh nó như thế nào?** Cấu trúc của mục này đi song song với mục 5 của bài Logistic Regression: trực quan hóa, các chỉ số đánh giá, rồi chọn siêu tham số bằng kiểm định chéo. Điểm khác là SVM có những đặc thù cần lưu ý: **điểm số của SVM không phải xác suất**, bài toán nhiều lớp phải ghép từ nhiều SVM nhị phân, và có ba siêu tham số $C$, $\gamma$, $\varepsilon$ tương tác với nhau. Các câu hỏi cụ thể:

1. Làm sao nhìn thấy lề, vector hỗ trợ và vùng quyết định của một SVM?
2. Các chỉ số phân loại đã học dùng được cho SVM ra sao, và vì sao điểm số của SVM không phải xác suất? Làm sao chuyển nó thành xác suất (Platt scaling)?
3. SVM nhị phân mở rộng cho bài toán nhiều lớp thế nào (one-vs-rest, one-vs-one)?
4. Hồi quy được đánh giá bằng những chỉ số nào (MAE, RMSE, $R^2$) và chúng khác nhau ở điểm nào?
5. Chọn $C$, $\gamma$, $\varepsilon$ bằng kiểm định chéo và grid search như thế nào cho đúng?
6. SVM tự cài đặt so với `SVC`, `LinearSVC`, `SVR` của scikit-learn ra sao?

**Dữ liệu xuyên suốt.** Để so sánh trực tiếp với hai bài trước, ta dùng **đúng bộ dữ liệu 400 sinh viên mô phỏng** của bài Perceptron và Logistic Regression: hai đặc trưng $x_1$ (số giờ tự học) và $x_2$ (số buổi vắng), hai lớp chồng lấn, chia 280 mẫu huấn luyện (149 đậu) và 120 mẫu kiểm tra (51 đậu). Nhãn dùng quy ước của SVM, $y \in \{-1, +1\}$ với $+1$ là "đậu". Đặc trưng được chuẩn hóa theo trung bình và độ lệch chuẩn của tập huấn luyện (mục 3.5). Mô hình chính là SVM tuyến tính với $C = 1$, cho tham số (trên đặc trưng đã chuẩn hóa)

$$\mathbf{w} = \begin{bmatrix} 2{,}085 \\ -1{,}259 \end{bmatrix}, \qquad b = 0{,}077, \qquad \| \mathbf{w} \| = 2{,}435.$$

Ký hiệu giữ nguyên như các mục trước; ta vẫn dùng ma trận nhầm lẫn $\mathbf{C} = \begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}$ với hàng là nhãn thực, cột là nhãn dự đoán, như ở bài Logistic Regression (ký hiệu $\mathbf{C}$ để chỉ ma trận nhầm lẫn khác với hệ số phạt $C$ vô hướng). Số thứ tự hình tiếp tục từ mục 6.

## 7.1. Trực quan hóa lề, vector hỗ trợ và vùng quyết định

### Vẽ vùng quyết định và hai biên lề

Với bài toán hai chiều, cách trực quan nhất là tính hàm quyết định $f(\mathbf{x}) = \mathbf{x}\mathbf{w} + b$ (hoặc dạng kernel $\sum_i \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}) + b$) trên một **lưới điểm** phủ kín miền dữ liệu, rồi vẽ các đường mức $f = 0$ (đường biên), $f = \pm 1$ (hai biên lề) và tô màu theo dấu của $f$. Đây là phiên bản của "bản đồ xác suất" ở bài Logistic Regression, nhưng với điểm số thay cho xác suất:

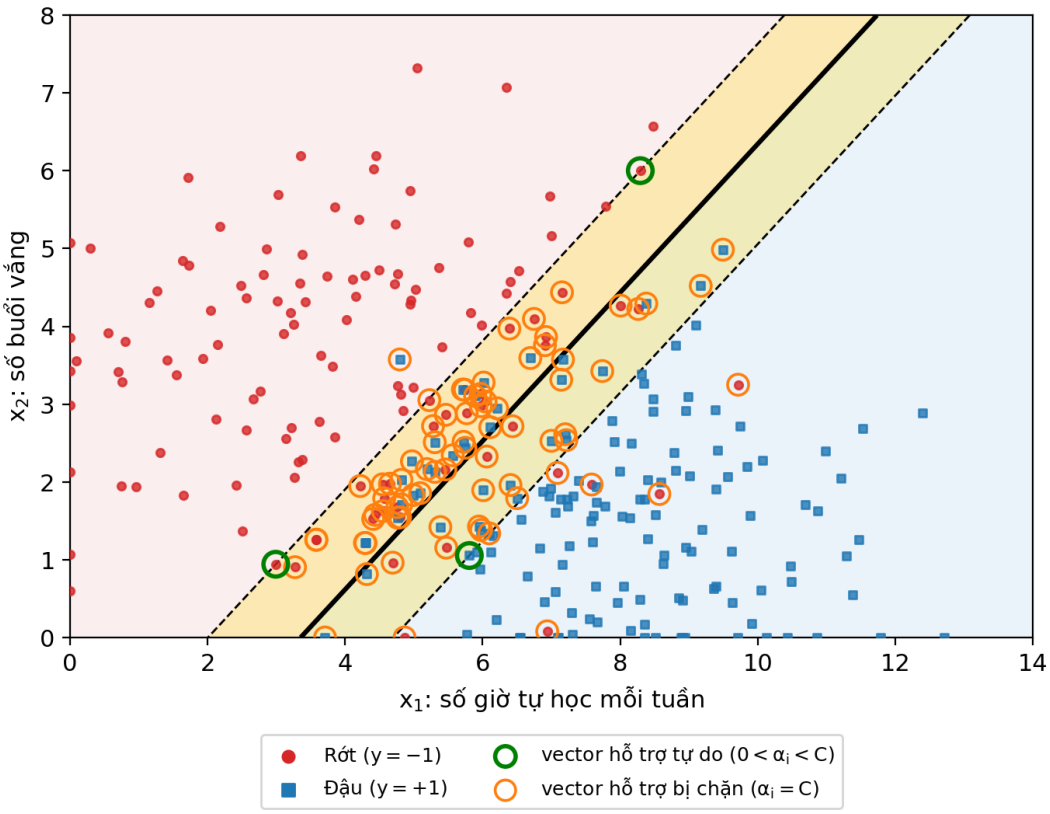
```
import numpy as np
import matplotlib.pyplot as plt

# svm: mô hình đã huấn luyện trên dữ liệu chuẩn hóa; mu, sd: trung bình và độ lệch chuẩn tập huấn luyện
g1, g2 = np.linspace(0, 14, 400), np.linspace(0, 8, 300)
XX, YY = np.meshgrid(g1, g2)
lưới = (np.c_[XX.ravel(), YY.ravel()] - mu) / sd          # đưa lưới về thang đo đã chuẩn hóa
F = svm.decision_function(lưới).reshape(XX.shape)

plt.contourf(XX, YY, F, levels=[-1e9, 0, 1e9], colors=["#fbeeee", "#eaf2fa"])
# hai vùng quyết định
plt.contour(XX, YY, F, levels=[0], colors="k", linewidths=2)
# f = 0
plt.contour(XX, YY, F, levels=[-1, 1], colors="k", linestyles="--")
# f = -1 và f = +1
```

## Đánh dấu vector hỗ trợ theo ba loại

Để thấy cơ chế học ở mục 4, ta đánh dấu các vector hỗ trợ theo loại hệ số: $0 < \alpha_i < C$ (tự do, nằm đúng trên biên lề) và $\alpha_i = C$ (bị chặn, lấn lề hoặc sai). Trong scikit-learn, `svm.support_` là chỉ số của các vector hỗ trợ và `np.abs(svm.dual_coef_)` là $\alpha_i$ của chúng.




*Hình 23. SVM tuyến tính ($C = 1$) trên 280 mẫu huấn luyện. Nét đậm là đường biên $f = 0$, hai nét đứt là $f = \pm 1$ và vùng vàng là dải lề. Có 80 vector hỗ trợ: 3 vector tự do (vòng xanh lá, nằm đúng trên biên lề) và 77 vector bị chặn (vòng cam, nằm trong dải lề hoặc sai phía).*

Một điều đáng chú ý ở Hình 23: vì hai lớp **chồng lấn nhiều**, 77 trong số 80 vector hỗ trợ là loại bị chặn ($\alpha_i = C$), còn chỉ 3 vector nằm đúng trên biên lề. Đây là hình ảnh điển hình của lề mềm trên dữ liệu chồng lấn (xem thêm Hình 12 ở mục 4.4), khác hẳn hình ảnh quen thuộc của lề cứng với vài vector hỗ trợ nằm gọn trên hai biên.

## Ảnh hưởng của $C$ lên lề và số vector hỗ trợ

Chạy lại với các giá trị $C$ khác nhau (đặc trưng đã chuẩn hóa), lề $\gamma = 1/\|\mathbf{w}\|$ tính theo thang đo chuẩn hóa:

| $C$ | $\|\mathbf{w}\|$ | Lề $\gamma$ | Số vector hỗ trợ | Accuracy huấn luyện | Accuracy kiểm tra | AUC kiểm tra |
|---|---|---|---|---|---|---|
| 0,01 | 0,83 | 1,21 | 180 | 89,3% | 86,7% | 0,952 |
| 0,1 | 1,66 | 0,60 | 108 | 90,7% | 89,2% | 0,953 |
| 1 | 2,44 | 0,41 | 80 | 88,2% | 90,0% | 0,953 |
| 10 | 2,74 | 0,36 | 74 | 87,9% | 90,0% | 0,954 |
| 100 | 2,74 | 0,36 | 74 | 87,9% | 90,0% | 0,954 |

Đúng như mục 3.5 và 4.4: $C$ tăng thì $\|\mathbf{w}\|$ tăng, lề thu hẹp, số vector hỗ trợ giảm. Từ $C = 10$ trở lên nghiệm không đổi nữa vì dữ liệu **không phân tách được** (mục 4.2). Accuracy và AUC gần như phẳng theo $C$: trên bộ dữ liệu này, ba chỉ số chỉ khác nhau vài phần trăm, nhỏ hơn dao động ngẫu nhiên của tập kiểm tra 120 mẫu (sai số chuẩn khoảng 2,7 điểm phần trăm như ở bài Logistic Regression).

### Hai hình quan trọng khác

1. **Phân bố điểm số theo lớp thật** (Hình 24a): biểu đồ histogram của $f(\mathbf{x})$ trên tập kiểm tra với ba đường dọc ở $-1$, $0$, $+1$. Hai lớp tách nhau khá rõ quanh $0$, và vùng giữa hai biên lề $-1 < f < 1$ (dải lề) chứa $21{,}7\%$ mẫu kiểm tra, đúng là nơi hầu hết các lỗi xảy ra.
2. **Số vector hỗ trợ và độ lề theo** $C$, như bảng trên. Số vector hỗ trợ gần bằng $n$ là dấu hiệu mô hình quá đơn giản hoặc $C$ quá nhỏ; số vector hỗ trợ rất nhỏ so với $n$ cho dữ liệu chồng lấn đôi khi là dấu hiệu quá khớp (đặc biệt với kernel RBF và $\gamma$ lớn, xem mục 5.4).

## 7.2. Đánh giá phân loại: các chỉ số đã học; SVM không cho xác suất và cách chuẩn hóa điểm số (Platt scaling)

### Các chỉ số đã học áp dụng cho SVM

Mọi chỉ số ở bài Perceptron và Logistic Regression (accuracy, ma trận nhầm lẫn, precision, recall, độ đặc hiệu, F1, ROC, AUC) đều tính được cho SVM, vì chúng chỉ cần **nhãn dự đoán** $\hat{y} = \text{sign}\big(f(\mathbf{x})\big)$ hoặc **điểm số** $f(\mathbf{x})$ để xếp hạng. Với lớp dương là "đậu", trên 120 mẫu kiểm tra, SVM với $C = 1$ cho ma trận nhầm lẫn

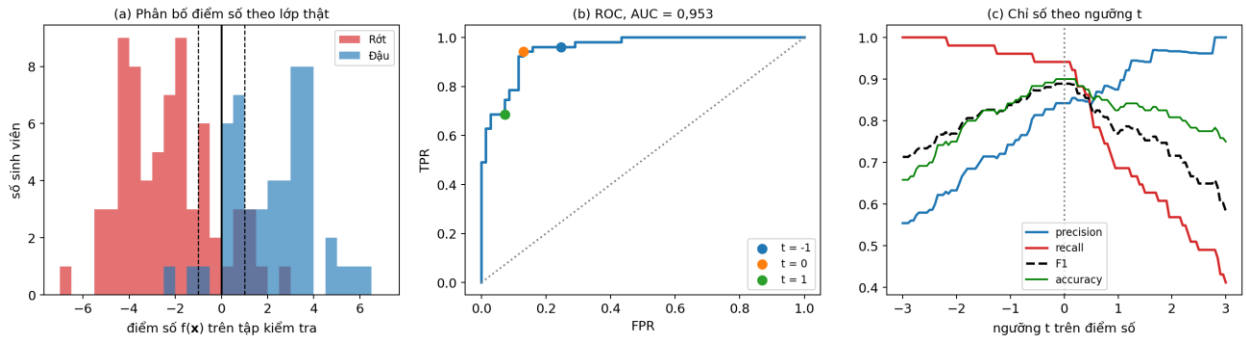
$$\mathbf{C} = \begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix} = \begin{bmatrix} 60 & 9 \\ 3 & 48 \end{bmatrix},$$

**trùng khít** với Logistic Regression ở bài trước. Các chỉ số tính từ $\mathbf{C}$:

$$\text{Accuracy} = \frac{TP + TN}{n} = \frac{108}{120} = 0{,}900, \qquad \text{Precision} = \frac{TP}{TP + FP} = \frac{48}{57} \approx 0{,}842,$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{48}{51} \approx 0{,}941, \qquad \text{Độ đặc hiệu} = \frac{TN}{TN + FP} = \frac{60}{69} \approx 0{,}870, \qquad F_1 = \frac{2\,TP}{2\,TP + FP + FN} = \frac{96}{108} \approx 0{,}889.$$

AUC tính từ điểm số $f$ (không cần xác suất) là $0{,}953$, so với $0{,}953$ của Logistic Regression. Điều này không bất ngờ: với dữ liệu hai lớp chồng lấn và chỉ có hai đặc trưng, mọi mô hình tuyến tính cho đường biên gần như giống nhau, như đã nhận xét ở bài Logistic Regression.




*Hình 24. (a) Phân bố điểm số $f(\mathbf{x})$ trên tập kiểm tra theo lớp thật; các đường dọc ở $f = -1{,}0, +1$. (b) Đường ROC, AUC $= 0{,}953$; ba điểm đánh dấu ứng với ngưỡng $t = -1{,}0, +1$ trên điểm số. (c) Precision, recall, F1 và accuracy khi đổi ngưỡng $t$ trên điểm số: ngưỡng tăng thì precision tăng và recall giảm.*

**Đổi ngưỡng.** Như Logistic Regression đổi ngưỡng xác suất $\tau$, SVM đổi ngưỡng $t$ trên **điểm số**: dự đoán $\hat{y} = +1$ khi $f(\mathbf{x}) \geq t$. Ngưỡng mặc định là $t = 0$. Hình 24c cho thấy $t$ tăng thì precision tăng và recall giảm, cùng quy luật như bài trước. Đổi ngưỡng tương đương với tịnh tiến đường biên song song với chính nó (đổi $b$).

**Báo cáo đầy đủ bằng thư viện:**

```
from sklearn.metrics import classification_report
print(classification_report(y_te, svm.predict(Z_te), target_names=["rot", "dau"], digits=3))
```

```
              precision    recall  f1-score   support

         rot      0.952     0.870     0.909        69
         dau      0.842     0.941     0.889        51

    accuracy                          0.900       120
   macro avg      0.897     0.905     0.899       120
weighted avg      0.906     0.900     0.901       120
```

## Điểm số của SVM không phải xác suất

Logistic Regression cho $p = \sigma(s) \in (0,1)$ và được huấn luyện để $p$ có nghĩa xác suất (log-loss). SVM thì khác: hàm hinge chỉ quan tâm $m = yf \geq 1$ hay không, **không** cố gắng làm $f$ có ý nghĩa xác suất. Cụ thể:

1. $f(\mathbf{x})$ có thể nhận **mọi giá trị thực** (trên tập kiểm tra từ $-6{,}6$ đến $+6{,}1$), và thang đo của nó phụ thuộc vào $C$ (bảng 7.1: $\|\mathbf{w}\|$, cũng như độ lớn trung bình $|f|$, đổi theo $C$).
2. Giá trị $f = 1$ chỉ có nghĩa "đúng trên biên lề", chứ không phải "xác suất 0,73" hay bất kỳ con số nào.

3. Thực tế, mọi mẫu huấn luyện nằm ngoài lề đều có $f \geq 1$ dù chúng "dễ" hay "rất dễ", vì hinge không phân biệt.

Do đó nếu ta cần **xác suất** (để đổi ngưỡng theo chi phí, kết hợp với các mô hình khác, hay báo cáo độ tin cậy) thì phải **hiệu chuẩn** điểm số.

## Platt scaling

Ý tưởng của **Platt scaling** là học một hàm sigmoid đơn giản biến điểm số thành xác suất:

$$P(y = +1 \mid f) = \sigma(Af + B) = \frac{1}{1 + e^{-(Af+B)}},$$

với hai tham số vô hướng $A$ và $B$. Đây chính là **Logistic Regression một chiều trên đặc trưng** $f$, nên các công cụ của bài trước dùng được nguyên vẹn: ta tối thiểu hóa cross-entropy

$$\min_{A,B} -\sum_{i=1}^{n} [t_i \ln p_i + (1 - t_i)\ln(1 - p_i)], \qquad p_i = \sigma(Af_i + B),$$

bằng phương pháp Newton (hay Gradient Descent). Platt đề xuất dùng **nhãn làm mịn** $t_i$ thay cho 0 và 1 để tránh quá khớp trên hai tham số:

$$t_i = \begin{cases} \dfrac{N_+ + 1}{N_+ + 2} & \text{nếu } y_i = +1, \\ \dfrac{1}{N_- + 2} & \text{nếu } y_i = -1, \end{cases}$$

với $N_+, N_-$ là số mẫu dương, âm. Có hai lưu ý quan trọng:

1. **Phải học $A, B$ trên điểm số ngoài mẫu** (out-of-sample). Nếu dùng điểm số của chính các mẫu huấn luyện, hầu hết đều có $f \geq 1$ (ngoài lề) nên sigmoid sẽ quá tự tin. Cách chuẩn là **kiểm định chéo**: với mỗi lần chia, huấn luyện SVM trên phần train và tính $f$ trên phần validation, gộp lại để có điểm số ngoài mẫu cho mọi mẫu huấn luyện, rồi mới khớp $A, B$.
2. Vì $A > 0$ nên sigmoid là hàm đơn điệu tăng: **Platt scaling không đổi thứ tự điểm số**, do đó không đổi nhãn dự đoán ở ngưỡng $p = 0{,}5$ (nếu $B$ nhỏ) và **không đổi AUC**. Nó chỉ làm xác suất "đúng nghĩa".

```
import numpy as np
from sklearn.svm import SVC
from sklearn.model_selection import cross_val_predict, StratifiedKFold

sigma = lambda z: 1 / (1 + np.exp(-z))

def platt_scaling(f, y, n_iter=100):
    """Khớp P(y=+1|f) = sigma(A f + B) bằng phương pháp Newton, với nhãn làm
mịn của Platt."""
```

```
    n_pos, n_neg = (y == 1).sum(), (y == -1).sum()
    t = np.where(y == 1, (n_pos + 1) / (n_pos + 2), 1 / (n_neg + 2))
    A, B = 0.0, np.log((n_pos + 1) / (n_neg + 1))
    for _ in range(n_iter):
        p = sigma(A * f + B)
        g = np.array([((p - t) * f).sum(), (p - t).sum()])           # gradie
nt theo (A, B)
        w = p * (1 - p)
        H = np.array([[(w * f * f).sum(), (w * f).sum()],
                      [(w * f).sum(),     w.sum()]]) + 1e-9 * np.eye(2)  # He
ssian
        step = np.linalg.solve(H, g)
        A, B = A - step[0], B - step[1]
        if np.abs(step).max() < 1e-10:
            break
    return A, B

svm = SVC(kernel="linear", C=1)
cv = StratifiedKFold(5, shuffle=True, random_state=0)
f_cv = cross_val_predict(svm, Z_tr, y_tr, cv=cv, method="decision_function")
# điểm số ngoài mẫu
A, B = platt_scaling(f_cv, y_tr)
svm.fit(Z_tr, y_tr)
p_te = sigma(A * svm.decision_function(Z_te) + B)
# xác suất đậu
print("A =", round(A, 4), "| B =", round(B, 4))
```

```
A = 1.4447 | B = 0.0526
```

## Platt scaling làm gì, khi nào cần?

Hệ số $A$ cho biết sigmoid "ngây thơ" $\sigma(f)$ (tức $A = 1, B = 0$) sai lệch đến đâu. Bảng sau cho $A$, $B$ và log-loss trên tập kiểm tra của sigmoid ngây thơ so với Platt scaling, với nhiều giá trị $C$ và kernel:

| Mô hình | $A$ | $B$ | Log-loss: $\sigma(f)$ ngây thơ | Log-loss: Platt scaling |
|---|---|---|---|---|
| Tuyến tính, $C = 0{,}01$ | 4,31 | $-0{,}50$ | 0,429 | $\mathbf{0,291}$ |
| Tuyến tính, $C = 0{,}1$ | 2,13 | $-0{,}06$ | 0,314 | $\mathbf{0,286}$ |
| Tuyến tính, $C = 1$ | 1,44 | 0,05 | 0,276 | 0,278 |
| Tuyến tính, $C = 100$ | 1,25 | 0,13 | 0,271 | 0,278 |
| RBF, $\gamma = 0{,}1, C = 1$ | 2,19 | $-0{,}04$ | 0,322 | $\mathbf{0,286}$ |
| RBF, $\gamma = 1, C = 100$ | 1,17 | 0,26 | 0,366 | 0,373 |
| RBF, $\gamma = 10, C = 100$ | 1,21 | $-0{,}11$ | 0,512 | 0,515 |

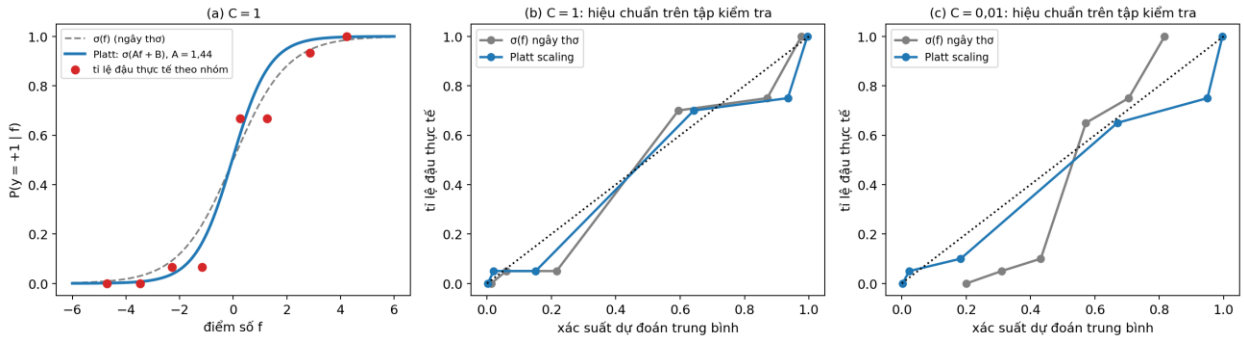
Có ba nhận xét:

1. **Thang đo của $f$ phụ thuộc $C$, nên $A$ cũng đổi theo $C$.** Với $C = 0{,}01$ điểm số rất nhỏ nên $A = 4{,}31$ (cần "khuếch đại" mạnh), với $C = 100$ điểm số lớn hơn và $A = 1{,}25$. Sigmoid ngây thơ $\sigma(f)$ chỉ đúng khi $A \approx 1$. Platt scaling ước lượng $A$ từ dữ liệu thay vì phó mặc cho $C$.

2. **Cải thiện lớn khi $A$ khác xa 1**: với $C = 0{,}01$, log-loss giảm từ $0{,}429$ xuống $0{,}291$ (giảm gần một phần ba). Với $C = 1$ và $C = 100$, vì $A$ gần 1 nên hai cách gần như bằng nhau (và sigmoid ngây thơ thậm chí nhỉnh hơn một chút do dao động ngẫu nhiên của 120 mẫu kiểm tra). Đây là điều nên nói thật: **Platt scaling không phải lúc nào cũng cải thiện rõ rệt**, nó đảm bảo xác suất có thang đo hợp lý mà không phụ thuộc vào một lựa chọn $C$ tùy ý.

3. **Platt scaling không sửa được mô hình tồi.** Với RBF $\gamma = 10$ (quá khớp, accuracy kiểm tra chỉ 80,8%), log-loss $0{,}515$ vẫn rất cao: hiệu chuẩn chỉ sửa thang đo, không sửa thứ tự xếp hạng.

Hình 25 minh họa: (a) sigmoid Platt đi qua các tỉ lệ đậu thực tế theo nhóm điểm số, (b) và (c) biểu đồ hiệu chuẩn (xác suất dự đoán trung bình so với tỉ lệ đậu thực tế) cho $C = 1$ và $C = 0{,}01$. Với $C = 0{,}01$, sigmoid ngây thơ (xám) lệch xa đường chéo lý tưởng, còn Platt scaling (xanh) bám sát hơn.




*Hình 25. (a) Sigmoid Platt $\sigma(Af + B)$ ($A = 1{,}44$, $C = 1$) so với sigmoid ngây thơ và tỉ lệ đậu thực tế theo nhóm điểm số. (b), (c) Biểu đồ hiệu chuẩn trên tập kiểm tra cho $C = 1$ và $C = 0{,}01$. Đường chéo chấm là hiệu chuẩn hoàn hảo. Với $C = 0{,}01$ sigmoid ngây thơ kém tự tin quá mức; Platt scaling sửa lại.*

**Trong scikit-learn**, `SVC(probability=True)` làm đúng quy trình này (kiểm định chéo 5 lần bên trong để học $A, B$), nên huấn luyện chậm hơn khoảng 5 lần. Trên bộ dữ liệu này, `predict_proba` cho log-loss $0{,}2767$ (so với $0{,}2878$ của Logistic Regression ở bài trước) và dự đoán nhãn bằng `predict_proba` $> 0{,}5$ trùng hoàn toàn với `predict`. Thư viện cảnh báo rằng hai cách này **có thể không trùng nhau** ở những mẫu sát biên, vì `predict` dùng dấu của $f$ còn `predict_proba` dùng $\sigma(Af + B)$ với $B \neq 0$ và có sự ngẫu nhiên của kiểm định chéo.

### Chọn ngưỡng theo chi phí

Giống mục 5.3 của bài Logistic Regression, nếu một lần báo nhầm "đậu" (FP) tốn $c_{FP} = 3$ còn bỏ sót một sinh viên đậu (FN) tốn $c_{FN} = 1$, ngưỡng xác suất tối ưu (khi xác suất hiệu chuẩn tốt) là $\tau^* = \frac{c_{FP}}{c_{FP}+c_{FN}} = 0{,}75$. Với Platt scaling ta quy về **ngưỡng trên điểm số**:

$$\sigma(At^* + B) = \tau^* \;\Rightarrow\; t^* = \frac{\ln\frac{\tau^*}{1-\tau^*} - B}{A} = \frac{\ln 3 - 0{,}0526}{1{,}4447} \approx 0{,}724.$$

| Ngưỡng | C | Precision | Recall | Chi phí $3FP + FN$ |
|---|---|---|---|---|
| $t = 0$ | $\begin{bmatrix}60 & 9\\ 3 & 48\end{bmatrix}$ | 0,842 | 0,941 | 30 |
| $t^* = 0{,}724$ | $\begin{bmatrix}63 & 6\\ 12 & 39\end{bmatrix}$ | 0,867 | 0,765 | 30 |

Ngưỡng $t^*$ giảm số lần báo nhầm ($FP$ từ 9 xuống 6) và tăng số lần bỏ sót ($FN$ từ 3 lên 12). Trên 120 mẫu kiểm tra, tổng chi phí **như nhau** (30), nên chưa chứng minh được lợi ích; sự khác biệt giữa hai ngưỡng nằm trong dao động ngẫu nhiên. Ví dụ này nhắc ta rằng ngưỡng theo chi phí là một công cụ hợp lý, nhưng cần đủ dữ liệu kiểm định để thấy được hiệu quả của nó.

## 7.3. Phân loại nhiều lớp bằng one-vs-rest và one-vs-one

SVM về bản chất là bộ phân loại **nhị phân**. Khi có $K > 2$ lớp, ta ghép nhiều SVM nhị phân bằng một trong hai chiến lược. (Khác với Softmax Regression ở bài trước, vốn xử lý trực tiếp nhiều lớp bằng một hàm mục tiêu duy nhất.)

### One-vs-Rest (OvR)

Huấn luyện $K$ **bộ phân loại**: bộ thứ $k$ tách lớp $k$ (nhãn $+1$) khỏi **tất cả các lớp còn lại** (nhãn $-1$), cho hàm quyết định $f_k(\mathbf{x}) = \mathbf{x}\mathbf{w}_k + b_k$. Dự đoán lấy lớp có điểm số **lớn nhất**:

$$\hat{y} = \arg\max_{k\in\{1,\dots,K\}} f_k(\mathbf{x}).$$

Khi có kernel, mỗi $f_k$ có tập vector hỗ trợ riêng.

### One-vs-One (OvO)

Huấn luyện $\binom{K}{2} = \frac{K(K-1)}{2}$ **bộ phân loại**, mỗi bộ cho một cặp lớp $(k, l)$, chỉ dùng các mẫu thuộc hai lớp đó, cho hàm quyết định $f_{kl}(\mathbf{x})$ (dương nghĩa là nghiêng về lớp $l$, âm nghiêng về lớp $k$). Dự đoán theo **bỏ phiếu**: mỗi bộ phân loại bầu cho một lớp, lớp có nhiều phiếu nhất thắng. Khi hòa phiếu, thường chọn lớp có tổng điểm số lớn hơn.

### So sánh hai chiến lược

| Tiêu chí | One-vs-Rest | One-vs-One |
|---|---|---|
| Số bộ phân loại | $K$ | $K(K-1)/2$ |

| Tiêu chí | One-vs-Rest | One-vs-One |
|---|---|---|
| Dữ liệu mỗi bộ | Toàn bộ $n$ mẫu | Chỉ $\approx 2n/K$ mẫu của hai lớp |
| Bài toán nhị phân con | **Mất cân bằng**: 1 lớp so với $K-1$ lớp | Cân bằng hơn, hai lớp |
| Chi phí huấn luyện (nếu $\propto n^2$) | $K \cdot n^2$ | $\frac{K(K-1)}{2}\left(\frac{2n}{K}\right)^2 = \frac{2(K-1)}{K}n^2$ |
| So sánh điểm số | Điểm số $f_k$ của các bộ **không cùng thang đo** nên argmax có thể không tin cậy | Bỏ phiếu, không cần so sánh thang đo |
| Mặc định trong scikit-learn | `LinearSVC` | `SVC` (nội bộ) |

Với phương pháp kernel có chi phí tăng theo $n^2$ như SVM, OvO thường **rẻ hơn** dù có nhiều bộ phân loại hơn. Chẳng hạn với $K = 10$ lớp: OvR tốn $10\,n^2$, còn OvO tốn $\frac{2\cdot 9}{10}n^2 = 1{,}8\,n^2$ (gấp 45 bộ nhưng mỗi bộ chỉ có $n/5$ mẫu). Ngược lại với SVM tuyến tính huấn luyện bằng thuật toán gần tuyến tính theo $n$ (`LinearSVC`), OvR là lựa chọn rẻ hơn và thông dụng hơn.

## Ví dụ ba lớp

Để so sánh, ta xét một bài toán ba lớp mô phỏng: xếp loại sinh viên **yếu**, **khá**, **giỏi** theo $(x_1, x_2) =$ (số giờ tự học, số buổi vắng), mỗi lớp 100 sinh viên từ một phân phối Gauss quanh $(3{,}5;\ 4{,}2)$, $(6{,}0;\ 2{,}6)$ và $(8{,}5;\ 1{,}0)$. Chia 210 huấn luyện và 90 kiểm tra (phân tầng), chuẩn hóa đặc trưng, rồi dùng SVM tuyến tính $C = 1$.

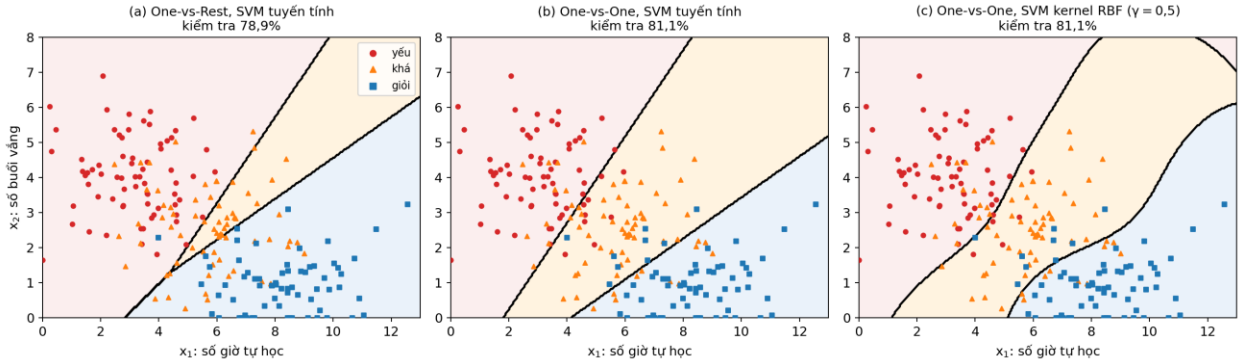
| Chiến lược | Accuracy kiểm tra | Macro-$F_1$ | Ma trận nhầm lẫn (hàng: thật, cột: dự đoán; thứ tự yếu, khá, giỏi) |
|---|---|---|---|
| One-vs-Rest | 78,9% | 0,754 | $\begin{bmatrix} 30 & 0 & 0 \\ 9 & 11 & 10 \\ 0 & 0 & 30 \end{bmatrix}$ |
| One-vs-One | 81,1% | 0,808 | $\begin{bmatrix} 27 & 3 & 0 \\ 5 & 20 & 5 \\ 0 & 4 & 26 \end{bmatrix}$ |

Với bài toán nhiều lớp, ma trận nhầm lẫn là ma trận $K \times K$, và các chỉ số theo lớp (precision, recall, $F_1$) được **gộp** bằng **macro-average** (trung bình đơn giản qua các lớp) hay **weighted-average** (trung bình có trọng số theo số mẫu mỗi lớp):

$$\text{Macro-}F_1 = \frac{1}{K}\sum_{k=1}^{K} F_1^{(k)}, \qquad F_1^{(k)} = \frac{2\,TP_k}{2\,TP_k + FP_k + FN_k}.$$

Hình 26 giải thích kết quả. Lớp "khá" nằm **giữa** hai lớp còn lại. Với OvR, bộ phân loại "khá so với phần còn lại" phải tách một lớp ở giữa khỏi hai lớp hai bên bằng **một đường thẳng**, điều

không thể, nên $f_{\text{khá}}$ gần như luôn âm (11/30 sinh viên khá được nhận ra, 9 bị nhầm thành yếu, 10 thành giỏi). OvO thì mỗi bộ chỉ cần tách **hai lớp**, chuyện luôn dễ hơn.




*Hình 26. Vùng quyết định ba lớp (yếu: đỏ, khá: cam, giỏi: xanh). (a) One-vs-Rest với SVM tuyến tính: lớp "khá" ở giữa bị "lấn át" nên vùng cam rất hẹp. (b) One-vs-One với SVM tuyến tính nhận ra lớp "khá" tốt hơn. (c) One-vs-One với kernel RBF cho các biên cong.*

**Ví dụ tính tay cho một sinh viên** $\mathbf{x} = [6{,}13 \quad 2{,}36]$ (thực tế xếp loại khá). Ba bộ phân loại OvR cho điểm số

$$f_{\text{yếu}} = -1{,}560, \qquad f_{\text{khá}} = -1{,}000, \qquad f_{\text{giỏi}} = -1{,}062.$$

Cả ba đều âm, tức mỗi bộ đều nói "không phải lớp tôi"; argmax chọn lớp "khá" ($-1{,}000$ lớn nhất) nhưng chỉ nhỉnh hơn "giỏi" ($-1{,}062$) rất ít. Với OvO, ba bộ phân loại cho:

| Cặp lớp | Điểm số $f_{kl}$ | Bầu cho |
|---|---|---|
| (yếu, khá) | $+1{,}563$ | khá |
| (yếu, giỏi) | $+0{,}491$ | giỏi |
| (khá, giỏi) | $-1{,}062$ | khá |

Tổng phiếu: yếu 0, khá 2, giỏi 1, nên chọn **khá** (đúng). Hai chiến lược đồng ý trong ví dụ này, nhưng OvO có bằng chứng rõ ràng hơn (hai phiếu trên ba), còn OvR dựa trên so sánh các điểm số khác thang đo.

**Trong scikit-learn:** `SVC` huấn luyện OvO bên trong; tham số `decision_function_shape="ovr"` (mặc định) chỉ **sắp xếp lại** điểm số thành dạng $K$ cột cho tiện dùng, không đổi cách huấn luyện. `LinearSVC` dùng OvR. Các lớp `OneVsRestClassifier` và `OneVsOneClassifier` bọc bất kỳ bộ phân loại nhị phân nào. Trên bộ dữ liệu này, `SVC` tuyến tính cho 81,1%, `LinearSVC` cho 82,2% (khác OvR tự bọc vì dùng hàm mất mát hinge bình phương, mục 7.6), và `SVC` kernel RBF ($\gamma = 0{,}5$) cho 81,1%. Chênh lệch giữa ba kết quả này nhỏ hơn dao động của 90 mẫu kiểm tra.

## 7.4. Đánh giá hồi quy: MAE, RMSE, $R^2$

Với SVR, ta so dự đoán $\hat{y}_i = f(\mathbf{x}_i)$ với giá trị thật $y_i$ trên $m$ mẫu kiểm tra. Gọi $r_i = y_i - \hat{y}_i$ là phần dư và $\bar{y}$ là trung bình của $y_i$ trên tập kiểm tra. Ba chỉ số thông dụng nhất:

$$\text{MAE} = \frac{1}{m}\sum_{i=1}^{m} |\, r_i|, \qquad \text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{m}\sum_{i=1}^{m} r_i^2}\,, \qquad R^2 = 1 - \frac{\sum_i r_i^2}{\sum_i (y_i - \bar{y})^2}$$
$$= 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}.$$

| Chỉ số | Đơn vị | Ý nghĩa | Nhạy với ngoại lai |
|---|---|---|---|
| MAE | Cùng đơn vị với $y$ | Sai số tuyệt đối **trung bình** | Ít (mất mát tuyến tính) |
| RMSE | Cùng đơn vị với $y$ | Căn bậc hai của sai số bình phương trung bình | **Nhiều** (mất mát bình phương) |
| $R^2$ | Không đơn vị | Phần phương sai của $y$ mà mô hình giải thích được; $R^2 = 0$ ứng với dự đoán bằng $\bar{y}$; $R^2 < 0$ nghĩa là tệ hơn dự đoán bằng trung bình | Nhiều (qua $SS_{\text{res}}$) |

Mối quan hệ với hàm mất mát: RMSE tương ứng hàm mất mát bình phương của hồi quy tuyến tính (mục 6.1), MAE tương ứng sai số tuyệt đối, còn SVR được huấn luyện bằng $\varepsilon$-insensitive nằm "giữa" hai thứ đó (mục 6.2). Ta đánh giá mô hình bằng chỉ số **phù hợp với mục tiêu thực tế**, không nhất thiết trùng với hàm mất mát khi huấn luyện.

**Lưu ý về đơn vị.** Nếu ta chuẩn hóa $y$ trước khi huấn luyện (như ở mục 6.6, để $\varepsilon$ có thang đo hợp lý), mọi chỉ số MAE và RMSE phải tính trên $y$ **đã quy về đơn vị gốc**, nếu không con số sẽ không có ý nghĩa thực tế. $R^2$ không phụ thuộc đơn vị nên tính trên thang nào cũng cho cùng giá trị.

## Ví dụ tính tay: tác động của một dự đoán sai nặng

Xét 5 mẫu kiểm tra với $\mathbf{y} = (2;\ 3;\ 5;\ 7;\ 9)$ và dự đoán $\hat{\mathbf{y}} = (2{,}5;\ 2{,}8;\ 5{,}4;\ 6{,}0;\ 9{,}5)$. Phần dư $\mathbf{r} = (-0{,}5;\ 0{,}2;\ -0{,}4;\ 1{,}0;\ -0{,}5)$. Trung bình $\bar{y} = 5{,}2$ và $SS_{\text{tot}} = (-3{,}2)^2 + (-2{,}2)^2 + (-0{,}2)^2 + 1{,}8^2 + 3{,}8^2 = 32{,}8$. Khi đó

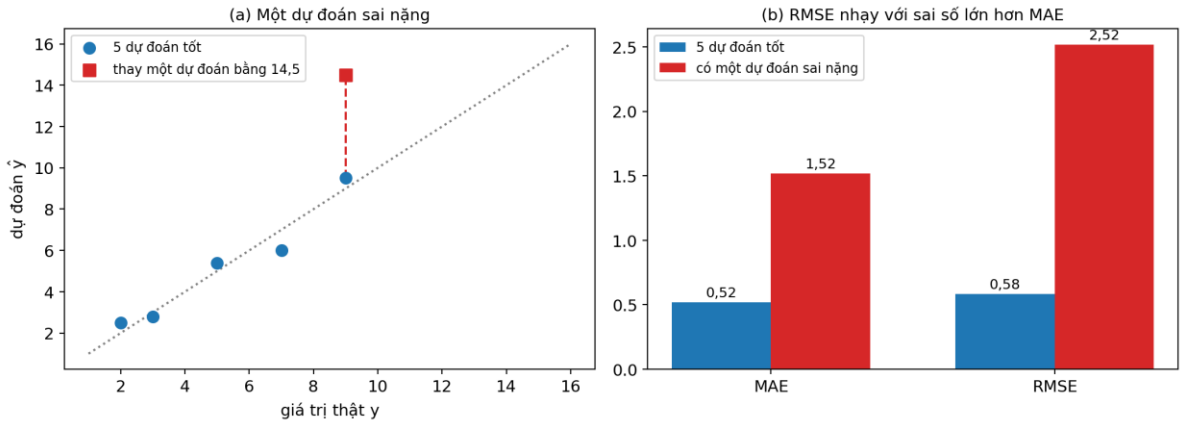
$$\text{MAE} = \frac{0{,}5 + 0{,}2 + 0{,}4 + 1{,}0 + 0{,}5}{5} = 0{,}52, \qquad \text{RMSE} = \sqrt{\frac{0{,}25 + 0{,}04 + 0{,}16 + 1{,}0 + 0{,}25}{5}}$$
$$= \sqrt{0{,}34} \approx 0{,}583,$$
$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{1{,}7}{32{,}8} \approx 0{,}948.$$

Bây giờ thay dự đoán cuối cùng $\hat{y}_5 = 9{,}5$ bằng $\hat{y}_5 = 14{,}5$ (sai 5,5 thay vì 0,5), bốn dự đoán còn lại giữ nguyên:

$$\text{MAE} = \frac{0{,}5 + 0{,}2 + 0{,}4 + 1{,}0 + 5{,}5}{5} = 1{,}52, \qquad \text{RMSE} = \sqrt{\frac{0{,}25 + 0{,}04 + 0{,}16 + 1{,}0 + 30{,}25}{5}} = \sqrt{6{,}34} \approx 2{,}518, \qquad R^2 = 1 - \frac{31{,}7}{32{,}8} \approx 0{,}034.$$

| Chỉ số | 5 dự đoán tốt | Có một dự đoán sai nặng | Tăng |
|---|---|---|---|
| MAE | 0,520 | 1,520 | × 2,9 |
| RMSE | 0,583 | 2,518 | × 4,3 |
| $R^2$ | 0,948 | 0,034 | tụt từ gần 1 về gần 0 |

Một dự đoán sai nặng kéo RMSE tăng 4,3 lần nhưng MAE chỉ tăng 2,9 lần, và $R^2$ tụt từ 0,948 xuống 0,034 (mô hình trông gần như không khá hơn việc luôn dự đoán $\bar{y}$). Tỉ số RMSE/MAE là một **chỉ báo ngoại lai** hữu ích: $0{,}583/0{,}52 \approx 1{,}12$ (sai số khá đều) so với $2{,}518/1{,}52 \approx 1{,}66$ (có sai số đuôi nặng).




*Hình 27. (a) Giá trị thật và dự đoán của* 5 *mẫu:* 4 *dự đoán tốt gần đường chéo, và một dự đoán bị thay bằng* 14,5 *(ô vuông đỏ). (b) MAE và RMSE trước và sau khi có dự đoán sai nặng: RMSE tăng mạnh hơn MAE.*

**Chọn chỉ số nào?** Nếu ngoại lai trong dữ liệu kiểm tra là **nhiễu** ta không muốn nó chi phối, ưu tiên MAE (hoặc sai số tuyệt đối trung vị). Nếu sai số lớn thật sự **tốn kém** hơn nhiều (ví dụ sai 10 tệ gấp mười lần sai 1), ưu tiên RMSE. $R^2$ tiện để so sánh các mô hình trên cùng một tập dữ liệu và với đường cơ sở "luôn dự đoán trung bình". Ở mục 6.6, bộ $\sin x$ có ngoại lai trong tập kiểm tra cho MAE 0,356, RMSE 0,649, $R^2 = 0{,}560$ với SVR RBF, so với 0,531, 0,732, 0,441 của hồi quy tuyến tính; khoảng cách ở MAE rộng hơn ở RMSE đúng như nhận xét trên.

## 7.5. Chọn $C$, $\gamma$, $\varepsilon$ bằng kiểm định chéo và grid search

### Vì sao cần kiểm định chéo?

Mọi đánh giá trên **tập kiểm tra** chỉ được làm **một lần ở cuối**. Nếu ta chọn $C$, $\gamma$ hay $\varepsilon$ bằng cách thử nhiều giá trị rồi giữ giá trị cho accuracy kiểm tra cao nhất, tập kiểm tra đã bị "nhìn thấy" và

ước lượng cuối cùng sẽ lạc quan. Giải pháp là **kiểm định chéo $K$ lần** (như ở bài Logistic Regression): chia tập huấn luyện thành $K$ phần; lần lượt dùng mỗi phần làm tập kiểm định và $K-1$ phần còn lại để huấn luyện, rồi lấy trung bình:

$$\mathrm{CV}(\theta) = \frac{1}{K}\sum_{k=1}^{K} \text{điểm}_k\,(\theta), \qquad \mathrm{SE}(\theta) = \frac{1}{\sqrt{K}}\sqrt{\frac{1}{K-1}\sum_{k=1}^{K}\left(\text{điểm}_k - \mathrm{CV}\right)^2},$$

với $\theta = (C, \gamma, \varepsilon)$ là bộ siêu tham số. Với phân loại dùng **kiểm định chéo phân tầng** để mỗi phần giữ tỉ lệ lớp. Hai quy tắc quan trọng:

1. **Đặt bước chuẩn hóa bên trong pipeline.** Trung bình và độ lệch chuẩn phải được tính chỉ từ phần huấn luyện của từng lần, để thông tin của phần kiểm định không rò rỉ vào quá trình huấn luyện (mục 4.2 bài Logistic Regression).
2. **Thang lưới theo cấp số nhân.** Vì $C$ và $\gamma$ tác động theo cấp số nhân (đổi $C$ từ 1 sang 2 ít ảnh hưởng hơn từ 1 sang 100), ta lưới $C, \gamma$ theo lũy thừa của 10 (hoặc của 2), ví dụ $C \in \{10^{-2}, \ldots, 10^{3}\}$, $\gamma \in \{10^{-3}, \ldots, 10^{1}\}$.

## Grid search cho SVM RBF

Ta thử $6 \times 5 = 30$ tổ hợp $(C, \gamma)$ với kiểm định chéo phân tầng 5 lần, tức 150 lần huấn luyện (mất khoảng 1 giây trên bộ dữ liệu này):

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.svm import SVC

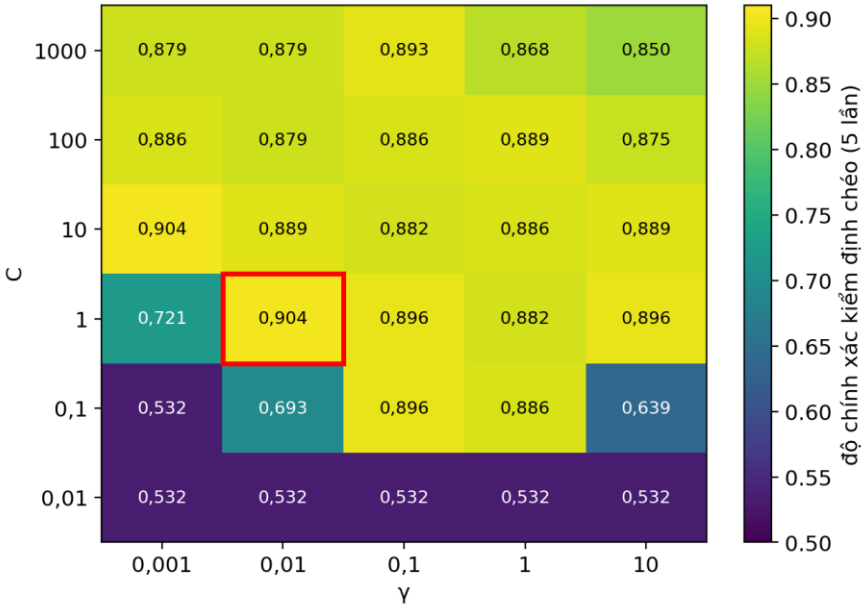
pipe = make_pipeline(StandardScaler(), SVC(kernel="rbf"))        # chuẩn hóa
nằm trong pipeline
luoi = {"svc__C": [0.01, 0.1, 1, 10, 100, 1000],
        "svc__gamma": [0.001, 0.01, 0.1, 1, 10]}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
gs = GridSearchCV(pipe, luoi, cv=cv, scoring="accuracy").fit(X_tr, y_tr)   #
X_tr là đặc trưng gốc
print(gs.best_params_, round(gs.best_score_, 4), "| test:", gs.score(X_te, y_
te))
```

```
{'svc__C': 1, 'svc__gamma': 0.01} 0.9036 | test: 0.8916666666666667
```

Bảng độ chính xác kiểm định chéo (trung bình 5 lần) cho toàn bộ lưới, vẽ ở Hình 28:

| $C \setminus \gamma$ | 0,001 | 0,01 | 0,1 | 1 | 10 |
|---|---|---|---|---|---|
| 0,01 | 0,532 | 0,532 | 0,532 | 0,532 | 0,532 |
| 0,1 | 0,532 | 0,693 | 0,896 | 0,886 | 0,639 |
| 1 | 0,721 | **0,904** | 0,896 | 0,882 | 0,896 |

| $C \setminus \gamma$ | 0,001 | 0,01 | 0,1 | 1 | 10 |
|---|---|---|---|---|---|
| 10 | **0,904** | 0,889 | 0,882 | 0,886 | 0,889 |
| 100 | 0,886 | 0,879 | 0,886 | 0,889 | 0,875 |
| 1000 | 0,879 | 0,879 | 0,893 | 0,868 | 0,850 |




*Hình 28. Độ chính xác kiểm định chéo (5 lần) của SVM RBF theo lưới* $(\gamma, C)$ *trên tập huấn luyện. Ô viền đỏ là tổ hợp tốt nhất. Hàng* $C = 0{,}01$ *cho* $0{,}532$ *(mô hình chỉ dự đoán lớp đa số). Vùng tốt nhất dọc theo một đường chéo* $C \cdot \gamma \approx$ *hằng số.*

Có bốn quan sát cần nhớ:

1. **Có vùng dưới khớp rõ rệt**: với $C = 0{,}01$ cả hàng cho $0{,}532$, đúng tỉ lệ lớp đa số $(149/280)$, nghĩa là mô hình không học được gì. Ở $C = 0{,}1$, $\gamma$ quá nhỏ $(0{,}001)$ cũng như vậy, và $\gamma$ quá lớn $(10)$ khiến mô hình học thuộc cục bộ nên chỉ đạt $0{,}639$.

2. **Hai tổ hợp tốt nhất hòa nhau**: $(C = 1, \gamma = 0{,}01)$ và $(C = 10, \gamma = 0{,}001)$ cùng đạt $0{,}9036 \pm 0{,}0143$ (độ lệch chuẩn qua các lần). Chúng cùng nằm trên một **đường chéo** $C \cdot \gamma = 0{,}01$. Đây không phải trùng hợp: khi $\gamma$ đủ nhỏ, kernel RBF xấp xỉ một kernel tuyến tính, và hai tham số bù trừ nhau (giảm $\gamma$ thì cần tăng $C$ để giữ cùng mức điều chuẩn hiệu dụng).

3. **Nhiều ô nằm trong khoảng một độ lệch chuẩn** của ô tốt nhất $(0{,}9036 - 0{,}0143 \approx 0{,}889)$. Sự khác biệt giữa $0{,}904$ và $0{,}896$ **không có ý nghĩa thống kê**. Một quy tắc thực hành là **quy tắc một độ lệch chuẩn**: trong các cấu hình nằm trong một SE của cấu hình tốt nhất, chọn cấu hình **đơn giản nhất** (ví dụ $\gamma$ nhỏ hơn, tức biên mượt hơn).

4. Tập kiểm tra cho 89,2% với cấu hình đã chọn, **không tốt hơn** SVM tuyến tính ($C = 0{,}1$ cho CV 0,9036 và test 89,2%). Kernel RBF không đem lại lợi thế khi dữ liệu về bản chất là hai đám mây Gauss gần tuyến tính; sức mạnh của nó thể hiện ở dữ liệu như hai vòng tròn ở mục 5.4.

## Grid search cho SVR

Với SVR, lưới có **ba** siêu tham số. Ta chuẩn hóa cả $x$ (bên trong pipeline) và $y$ (bằng *TransformedTargetRegressor*) và chấm điểm bằng sai số tuyệt đối trung bình âm, vì dữ liệu $\sin x$ ở mục 6.6 có ngoại lai:

```
from sklearn.svm import SVR
from sklearn.compose import TransformedTargetRegressor
from sklearn.model_selection import KFold

reg = TransformedTargetRegressor(regressor=make_pipeline(StandardScaler(), SV
R(kernel="rbf")),
                                 transformer=StandardScaler())           # c
huẩn hóa cả y
luoi = {"regressor__svr__C": [0.1, 1, 10, 100],
        "regressor__svr__gamma": [0.1, 0.3, 1, 3],
        "regressor__svr__epsilon": [0.02, 0.1, 0.3]}                      #
48 tổ hợp
gs = GridSearchCV(reg, luoi, cv=KFold(5, shuffle=True, random_state=0),
                  scoring="neg_mean_absolute_error").fit(X_tr, y_tr)
print(gs.best_params_, round(-gs.best_score_, 4))
```

Các cấu hình tốt nhất (MAE kiểm định chéo, tính theo đơn vị gốc của $y$):

| Hạng | $C$ | $\gamma$ | $\varepsilon$ (thang đo chuẩn hóa) | MAE kiểm định chéo |
|---|---|---|---|---|
| 1 | 100 | 0,3 | 0,3 | 0,1477 |
| 2 | 10 | 1 | 0,02 | 0,1507 |
| 3 | 100 | 0,3 | 0,02 | 0,1526 |
| 4 | 1 | 1 | 0,02 | 0,1528 |
| 5 | 1 | 1 | 0,1 | 0,1535 |

Năm cấu hình đầu chỉ khác nhau $\leq 0{,}006$ về MAE, nghĩa là **gần như hòa**, và cấu hình tốt nhất (với $\varepsilon = 0{,}3$) cho đường dự đoán lệch 0,159 so với đường thật $\sin x$, **không tốt hơn** cấu hình chọn tay ở mục 6.6 (0,137 với $C = 10, \gamma = 1, \varepsilon = 0{,}1$). Điều này nhắc ta hai điểm: kiểm định chéo đo trên **dữ liệu có nhiễu và ngoại lai** nên chỉ là một ước lượng, và khi nhiều cấu hình hòa nhau, tiêu chí phụ (đơn giản, ít vector hỗ trợ) quan trọng hơn chênh lệch nhỏ của điểm số.

## Chiến lược tìm kiếm và chi phí

1. **Số lần huấn luyện** là (số tổ hợp) $\times K$; lưới 30 tổ hợp với $K = 5$ là 150 lần, lưới ba tham số $4 \times 4 \times 3 = 48$ tổ hợp là 240 lần. Chi phí tăng nhanh khi thêm siêu tham số.

2. **Tìm thô rồi tinh (coarse-to-fine):** chạy lưới thưa theo lũy thừa 10 để khoanh vùng tốt, rồi lưới mịn hơn quanh vùng đó (ví dụ lũy thừa 2 hay $\sqrt{10}$).
3. **Tìm kiếm ngẫu nhiên** (`RandomizedSearchCV`): lấy mẫu ngẫu nhiên các tổ hợp (từ phân phối log-đều). Thường hiệu quả hơn lưới khi có nhiều siêu tham số mà chỉ một số ít thật sự quan trọng.
4. **Chọn thước đo phù hợp** (`scoring`): `accuracy`, `f1`, `roc_auc` cho phân loại (chọn `f1` hay `roc_auc` khi lớp lệch); `neg_mean_absolute_error`, `neg_root_mean_squared_error`, `r2` cho hồi quy.
5. **Kiểm định chéo lồng nhau (nested CV)**: nếu cần một ước lượng không chệch cho hiệu năng của **cả quy trình** (bao gồm bước chọn siêu tham số), đặt grid search bên trong một vòng kiểm định chéo ngoài.
6. Sau khi chọn xong, **huấn luyện lại trên toàn bộ tập huấn luyện** (`refit=True` mặc định) và đánh giá **một lần** trên tập kiểm tra.

## 7.6. Huấn luyện và so sánh với SVC, LinearSVC, SVR của scikit-learn

### Các lớp SVM trong scikit-learn

| Lớp | Bài toán | Thuật toán bên trong | Kernel | Hàm mất mát | Ghi chú |
|---|---|---|---|---|---|
| `SVC` | Phân loại | LIBSVM (SMO, đối ngẫu) | Mọi kernel | Hinge | Chi phí $O(n^2)$ đến $O(n^3)$; OvO cho nhiều lớp |
| `LinearSVC` | Phân loại | LIBLINEAR (tọa độ) | Chỉ tuyến tính | Squared hinge (mặc định) hoặc hinge | Rất nhanh với $n$ lớn; OvR; hệ số chệch có thể bị điều chuẩn |
| `SGDClassifier(loss="hinge")` | Phân loại | SGD, gần với Pegasos | Tuyến tính | Hinge | Xử lý dữ liệu rất lớn, huấn luyện trực tuyến |
| `SVR` | Hồi quy | LIBSVM (SMO) | Mọi kernel | $\varepsilon$-insensitive | Có `epsilon` |
| `LinearSVR` | Hồi quy | LIBLINEAR | Chỉ tuyến tính | $\varepsilon$-insensitive (mặc định) | Nhanh với $n$ lớn |

Ánh xạ tham số của thư viện sang ký hiệu của bài:

| Tham số scikit-learn | Ký hiệu | Ý nghĩa |
|---|---|---|
| `C` | $C$ | Hệ số phạt vi phạm (như mục 3.2, 6.3) |
| `kernel`, `degree`, `coef0`, `gamma` | $k, p, r, \gamma$ | Kernel và tham số của nó (mục 5.3); `gamma="scale"` ứng với $1/\big(d \cdot \mathrm{Var}(\mathbf{X})\big)$ |
| `epsilon` | $\varepsilon$ | Độ rộng ống của SVR (mục 6.2) |
| `class_weight` | | Nhân $C$ với trọng số theo lớp (xử lý lớp lệch) |
| `probability` | | Bật Platt scaling (mục 7.2) |
| `tol`, `max_iter` | | Dung sai và số bước tối đa của bộ giải |

## So sánh SVM tuyến tính trên bộ 400 sinh viên

Ta huấn luyện cùng một bài toán ($C = 1$, đặc trưng chuẩn hóa, 280 mẫu huấn luyện) bằng các lớp khác nhau và đối chiếu với `LinearSVM` tự cài đặt ở mục 3.7 (batch, 1000 bước). Cột $J$ là giá trị hàm mục tiêu hinge $\frac{1}{2} \| \mathbf{w} \|^2 + C \sum_i \max(0, 1 - m_i)$:

```
from sklearn.svm import SVC, LinearSVC
from sklearn.linear_model import SGDClassifier

svc = SVC(kernel="linear", C=1, tol=1e-6).fit(Z_tr, y_tr)
lin_hinge = LinearSVC(C=1, loss="hinge", dual=True, max_iter=200000, tol=1e-8
).fit(Z_tr, y_tr)
lin_sq = LinearSVC(C=1, max_iter=200000, tol=1e-8).fit(Z_tr, y_tr)
# squared hinge (mặc định)
sgd = SGDClassifier(loss="hinge", alpha=1 / (len(y_tr) * 1.0), max_iter=5000,
tol=None,
                    random_state=0).fit(Z_tr, y_tr)
# alpha = 1/(nC)
```

| Mô hình | $\mathbf{w}^T$ | | $b$ | $J$ | Accuracy kiểm tra | AUC |
|---|---|---|---|---|---|---|
| `LinearSVM` tự cài đặt (batch) | [2,0853 | −1,2582] | 0,0781 | 75,787 | 0,900 | 0,9528 |
| `SVC(kernel="linear")` | [2,0843 | −1,2586] | 0,0765 | 75,786 | 0,900 | 0,9528 |
| `LinearSVC(loss="hinge")` | [2,0843 | −1,2586] | 0,0765 | 75,786 | 0,900 | 0,9528 |
| `SGDClassifier(loss="hinge")` | [2,0848 | −1,2578] | 0,0775 | 75,786 | 0,900 | 0,9528 |
| `LinearSVC()` mặc định (squared hinge) | [1,0161 | −0,6576] | 0,0618 | (khác mục tiêu) | 0,900 | 0,9534 |

Bốn dòng đầu cho **cùng một nghiệm** (khác nhau ở chữ số thập phân thứ ba) và cùng giá trị mục tiêu $J \approx 75{,}79$: điều này khẳng định rằng các cách giải khác nhau, từ đối ngẫu (`SVC`), tọa độ

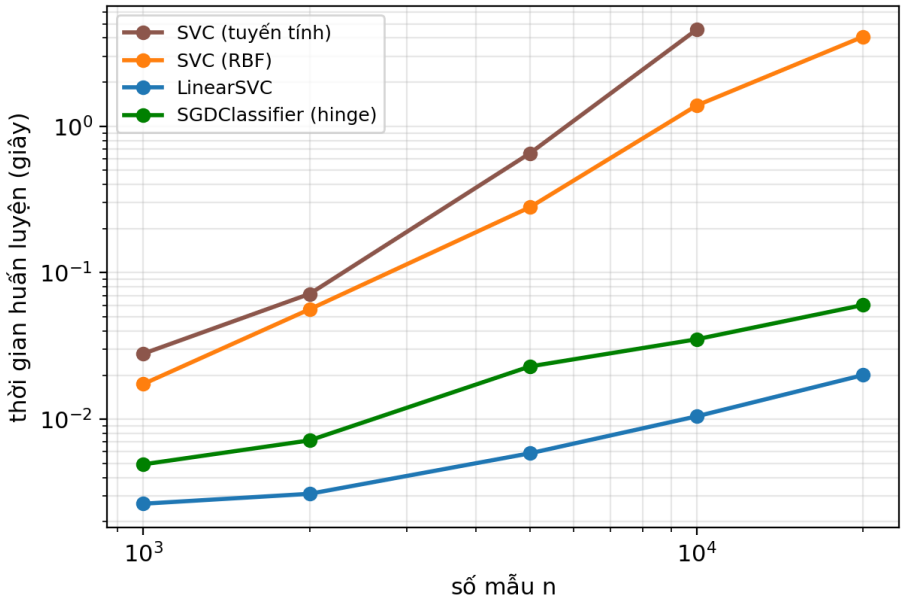
(*LinearSVC*), gradient dưới đạo hàm (mã tự cài đặt) đến SGD (*SGDClassifier*), cùng hội tụ về nghiệm duy nhất của bài toán hinge. Dòng cuối là **một bài toán khác**: *LinearSVC* mặc định dùng hàm mất mát **hinge bình phương** $\max(0,1-m)^2$ (trơn, phạt nặng hơn các vi phạm lớn), cho nghiệm có $\mathbf{w}$ nhỏ hơn khoảng một nửa. Nó cho accuracy bằng nhau trên bộ dữ liệu này nhưng không so sánh được trực tiếp qua $J$.

Một lưu ý về *LinearSVC*: LIBLINEAR coi hệ số chệch như một **đặc trưng thêm vào** (giá trị *intercept_scaling*), nên $b$ có thể bị điều chuẩn cùng với $\mathbf{w}$, khác với định nghĩa ở mục 2.4 (không phạt $b$). Với dữ liệu đã chuẩn hóa như ở đây, ảnh hưởng này không đáng kể (kết quả trùng với *SVC* đến chữ số thập phân thứ tư), nhưng với dữ liệu mà $b$ lớn thì nên tăng *intercept_scaling*.

## Thời gian huấn luyện khi $n$ tăng

Tính chất quan trọng nhất phân biệt các lớp là **cách chi phí tăng theo** $n$. Trên dữ liệu phân loại mô phỏng 10 chiều (đã chuẩn hóa), thời gian huấn luyện (giây) là:

| Số mẫu $n$ | 1 000 | 2 000 | 5 000 | 10 000 | 20 000 |
|---|---|---|---|---|---|
| *SVC(kernel="linear")* | 0,028 | 0,072 | 0,653 | 4,55 | không chạy |
| *SVC(kernel="rbf")* | 0,017 | 0,056 | 0,280 | 1,38 | 4,08 |
| *LinearSVC* | 0,003 | 0,003 | 0,006 | 0,010 | 0,020 |
| *SGDClassifier(loss="hinge")* | 0,005 | 0,007 | 0,023 | 0,035 | 0,060 |




*Hình 29. Thời gian huấn luyện theo số mẫu n (cả hai trục theo thang log).* `SVC` *(giải đối ngẫu với ma trận Gram) tăng gần như theo* $n^2$*;* `LinearSVC` *và* `SGDClassifier` *(giải trực tiếp bài toán gốc) tăng gần tuyến tính theo n và nhanh hơn hàng trăm lần ở* $n = 20\,000$.

Kết quả khớp với phân tích ở mục 3 và 5: thuật toán đối ngẫu với ma trận Gram tốn bộ nhớ và thời gian theo $n^2$, còn giải trực tiếp bài toán hinge không ràng buộc bằng gradient tốn gần tuyến

tính theo $n$. Ở $n = 10\,000$, *SVC* tuyến tính chậm hơn *LinearSVC* khoảng 450 lần dù cho cùng nghiệm. **Quy tắc thực hành:**

| Tình huống | Lựa chọn hợp lý |
|---|---|
| Cần đường biên phi tuyến, $n$ vài nghìn đến vài chục nghìn | *SVC* với kernel RBF |
| SVM tuyến tính, $n$ lớn hoặc nhiều đặc trưng thưa (văn bản) | *LinearSVC* hoặc *SGDClassifier* |
| Dữ liệu rất lớn hoặc đến theo luồng | *SGDClassifier* (cập nhật từng lô) |
| Cần xác suất | *SVC(probability=True)* hoặc tự bọc Platt scaling |

## So sánh SVR

Trên bộ một chiều có ngoại lai của mục 6 ($\varepsilon = 0{,}2$, $C = 1$), cả ba cách cho cùng một nghiệm:

| Mô hình | $w$ | $b$ | $J$ |
|---|---|---|---|
| *SVR(kernel="linear")* | 0,8667 | 0,3333 | 3,8422 |
| *LinearSVR* (LIBLINEAR) | 0,8667 | 0,3333 | 3,8422 |
| *LinearSVR* tự cài đặt (mục 6.6) | 0,8666 | 0,3335 | 3,8423 |
| Nghiệm tay (mục 6.5) | $\frac{13}{15} \approx 0{,}8667$ | $\frac{1}{3} \approx 0{,}3333$ | $\frac{1729}{450} \approx 3{,}8422$ |

Mã *KernelSVR* ở mục 6.6 cũng khớp *SVR* của scikit-learn trên bộ $\sin x$ (cùng 26 vector hỗ trợ, dự đoán lệch tối đa $10^{-6}$). Như vậy các phép tính tay, mã tự cài đặt và thư viện đều **đồng nhất** về kết quả, và sự khác biệt giữa chúng chỉ nằm ở tốc độ và độ tiện dụng.

## Quy trình đề xuất

Gộp các bài học của mục 7 và mục 3 đến 6, một quy trình hợp lý khi dùng SVM trên dữ liệu thật:

1. **Chia** dữ liệu thành tập huấn luyện và tập kiểm tra (phân tầng nếu phân loại), tập kiểm tra để dành đến cuối.
2. **Xây pipeline** gồm chuẩn hóa đặc trưng (và chuẩn hóa nhãn nếu hồi quy) cộng mô hình SVM.
3. **Chọn kernel**: bắt đầu từ tuyến tính (*LinearSVC* nếu $n$ lớn) làm đường cơ sở; nếu thiếu, thử RBF.
4. **Chọn siêu tham số** $C$, $\gamma$, $\varepsilon$ bằng grid search hay random search với kiểm định chéo trên tập huấn luyện, theo thang log; ưu tiên cấu hình đơn giản trong các cấu hình hòa nhau.

5. **Huấn luyện lại** trên toàn bộ tập huấn luyện với siêu tham số đã chọn.
6. **Đánh giá một lần** trên tập kiểm tra bằng đủ chỉ số (ma trận nhầm lẫn, precision, recall, $F_1$, AUC cho phân loại; MAE, RMSE, $R^2$ cho hồi quy), không chỉ accuracy.
7. Nếu cần xác suất hay ngưỡng theo chi phí, **hiệu chuẩn** bằng Platt scaling trên điểm số ngoài mẫu.
8. **Trực quan hóa** lề, vector hỗ trợ và vùng quyết định (nếu có thể) để kiểm tra sự hợp lý, và xem số vector hỗ trợ.

## Tóm tắt mục 7

1. **Trực quan hóa:** vẽ đường mức $f = 0$ và $f = \pm 1$ trên lưới điểm, đánh dấu vector hỗ trợ tự do ($0 < \alpha_i < C$) và bị chặn ($\alpha_i = C$). Trên dữ liệu chồng lấn, hầu hết vector hỗ trợ là loại bị chặn ($77/80$). $C$ tăng thì lề hẹp lại và số vector hỗ trợ giảm.
2. **Phân loại:** mọi chỉ số đã học (ma trận nhầm lẫn $\mathbf{C}$, precision, recall, $F_1$, ROC, AUC) tính được từ nhãn hoặc điểm số. SVM $C = 1$ cho $\mathbf{C} = \begin{bmatrix} 60 & 9 \\ 3 & 48 \end{bmatrix}$, accuracy 90,0%, AUC 0,953, trùng Logistic Regression. Đổi ngưỡng $t$ trên điểm số tương đương dịch $b$.
3. **SVM không cho xác suất.** Điểm số $f$ có thang đo phụ thuộc $C$. **Platt scaling** $P(y = +1 \mid f) = \sigma(Af + B)$ (Logistic Regression một chiều, học trên điểm số **ngoài mẫu**) hiệu chuẩn nó: log-loss giảm từ 0,429 xuống 0,291 với $C = 0{,}01$, nhưng gần như không đổi khi $A \approx 1$, và không thể sửa mô hình tồi.
4. **Nhiều lớp:** OvR ($K$ bộ, chọn $\text{argmax} f_k$) và OvO ($K(K-1)/2$ bộ, bỏ phiếu). OvO thường rẻ hơn với phương pháp kernel và xử lý lớp "ở giữa" tốt hơn; trên ví dụ ba lớp, OvR cho 78,9% và OvO 81,1%. Đánh giá bằng ma trận nhầm lẫn $K \times K$ và macro-$F_1$.
5. **Hồi quy:** MAE, RMSE, $R^2$. RMSE nhạy với ngoại lai hơn MAE: một dự đoán sai nặng đưa MAE từ 0,52 lên 1,52 nhưng RMSE từ 0,58 lên 2,52 và $R^2$ từ 0,95 xuống 0,03. Chỉ số phải tính trên đơn vị gốc của $y$.
6. **Chọn siêu tham số:** kiểm định chéo $K$ lần trong pipeline, lưới theo lũy thừa 10. Trên dữ liệu 400 sinh viên, $(C = 1, \gamma = 0{,}01)$ và $(C = 10, \gamma = 0{,}001)$ hòa nhau ($0{,}904 \pm 0{,}014$), cùng nằm trên đường $C\gamma \approx 0{,}01$. Nhiều cấu hình hòa trong một SE nên chọn cấu hình đơn giản nhất.
7. **Thư viện:** `SVC` (SMO, kernel, $O(n^2)$ đến $O(n^3)$), `LinearSVC` và `SGDClassifier` (gần tuyến tính theo $n$), `SVR`, `LinearSVR`. Bốn cách giải SVM tuyến tính cho cùng

nghiệm; `LinearSVC` mặc định dùng hinge bình phương nên khác. Ở $n = 10\,000$, `LinearSVC` nhanh hơn `SVC` tuyến tính khoảng 450 lần.

## Câu hỏi kiểm tra nhanh

1. Từ $\mathbf{C} = \begin{bmatrix} 63 & 6 \\ 12 & 39 \end{bmatrix}$ (ngưỡng $t^* = 0{,}724$ ở mục 7.2), tính accuracy, precision, recall và $F_1$ của lớp "đậu".
2. Vì sao phải học tham số $A, B$ của Platt scaling trên điểm số **ngoài mẫu** (kiểm định chéo) thay vì trên điểm số của chính tập huấn luyện?
3. Với $K = 4$ lớp và mỗi lớp có $n/4$ mẫu: OvR và OvO cần huấn luyện bao nhiêu bộ phân loại, mỗi bộ dùng bao nhiêu mẫu? Chi phí tương đối nếu thời gian huấn luyện tỉ lệ $n^2$?
4. Cho $\mathbf{y} = (4;\ 6;\ 8)$ và $\hat{\mathbf{y}} = (5;\ 6;\ 6)$. Tính MAE, RMSE và $R^2$.
5. Kết quả grid search cho $(C = 1, \gamma = 0{,}01)$ và $(C = 10, \gamma = 0{,}001)$ cùng đạt $0{,}9036$. Nên chọn cấu hình nào và vì sao? Điều gì cho thấy hai cấu hình này "tương đương"?

### Gợi ý đáp án

1. $n = 120$: accuracy $= \frac{63+39}{120} = \frac{102}{120} = 0{,}850$. Precision $= \frac{39}{39+6} \approx 0{,}867$. Recall $= \frac{39}{39+12} \approx 0{,}765$. $F_1 = \frac{2 \cdot 39}{2 \cdot 39 + 6 + 12} = \frac{78}{96} \approx 0{,}8125$.
2. Với mẫu huấn luyện, hầu hết đều nằm ngoài lề nên điểm số $f \geq 1$ (hinge đã bằng 0), và các điểm số này không phản ánh mức độ không chắc chắn trên dữ liệu mới. Khớp $A, B$ trên chúng sẽ cho sigmoid quá dốc (xác suất quá tự tin). Điểm số ngoài mẫu mô phỏng đúng phân bố điểm số của dữ liệu mới.
3. OvR: 4 bộ, mỗi bộ dùng toàn bộ $n$ mẫu, chi phí $\propto 4n^2$. OvO: $\binom{4}{2} = 6$ bộ, mỗi bộ dùng $2 \cdot \frac{n}{4} = \frac{n}{2}$ mẫu, chi phí $\propto 6\left(\frac{n}{2}\right)^2 = 1{,}5\,n^2$. OvO rẻ hơn $\frac{4}{1{,}5} \approx 2{,}7$ lần.
4. $\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}} = (-1;\ 0;\ 2)$. MAE $= \frac{1+0+2}{3} = 1$. RMSE $= \sqrt{\frac{1+0+4}{3}} = \sqrt{1{,}667} \approx 1{,}291$. $\bar{y} = 6$, $SS_{\text{tot}} = 4 + 0 + 4 = 8$, $SS_{\text{res}} = 5$, nên $R^2 = 1 - \frac{5}{8} = 0{,}375$.
5. Hai cấu hình đều có độ lệch chuẩn qua các lần cỡ $0{,}0143$, lớn hơn nhiều so với mọi khác biệt giữa chúng (chúng cùng đạt $0{,}9036$), nên **không có cơ sở thống kê** để nói cấu hình nào tốt hơn. Cả hai đều dùng khoảng $150$ vector hỗ trợ và cho accuracy kiểm tra $89{,}2\%$ và $88{,}3\%$, chênh nhau đúng một mẫu trên $120$. Khi hòa, ta dùng tiêu chí phụ, chẳng hạn ưu tiên giá trị $C$ nhỏ hơn ($C = 1$) để điều chuẩn

hiệu dụng ổn định hơn và dễ nằm giữa lưới hơn là ở mép. Dấu hiệu cho thấy hai cấu hình "tương đương" là cả hai nằm trên cùng đường $C\gamma = 0{,}01$: khi $\gamma$ đủ nhỏ, kernel RBF xấp xỉ kernel tuyến tính nên $C$ và $\gamma$ bù trừ nhau.

# 8. Tổng kết và mở rộng

Đây là mục cuối của bài giảng. Ta nhìn lại toàn bộ hành trình từ Perceptron và Logistic Regression đến SVM và SVR, đánh giá ưu điểm và hạn chế của SVM/SVR, đặt SVM cạnh các mô hình đã học (Perceptron, Logistic Regression, Softmax Regression, mạng nơ-ron) để thấy chúng thuộc cùng một khung, rồi **mở đường** sang các hướng phát triển tiếp theo: phát hiện bất thường bằng one-class SVM, mở rộng SVM cho dữ liệu lớn, và các mô hình kernel khác. Cuối mục là hệ thống bài tập.

Ta dùng lại quy ước ký hiệu: **chữ thường in đậm** là vector, **chữ hoa in đậm** là ma trận, vector đặc trưng $\mathbf{x}_i$ là **vector hàng**, vector tham số $\mathbf{w}$ là **vector cột**, nhãn phân loại $y_i \in \{-1, +1\}$, điểm số $s_i = \mathbf{x}_i\mathbf{w} + b$, margin $m_i = y_i s_i$. Số thứ tự hình tiếp tục từ mục 7.

## Nhìn lại toàn bộ bài giảng

Bài giảng đi qua bảy bước, mỗi bước trả lời một câu hỏi và để lại một câu hỏi cho bước sau:

| Mục | Câu hỏi | Kết quả chính |
|---|---|---|
| 1 | Perceptron và Logistic Regression còn thiếu gì? | Có vô số đường biên đúng nhưng không có tiêu chí chọn. Khoảng cách có dấu $\gamma_i = m_i/\|\mathbf{w}\|$ không phụ thuộc thang đo; **bài toán lề cực đại** $\max_{\mathbf{w},b}\min_i \gamma_i$. |
| 2 | Biến lề cực đại thành bài toán giải được? | Chốt thang đo $\min_i m_i = 1$: $\min \frac{1}{2}\|\mathbf{w}\|^2$ với $m_i \geq 1$. Lề $= 1/\|\mathbf{w}\|$, $\mathbf{w}$ duy nhất, **vector hỗ trợ** là các mẫu có $m_i = 1$. |
| 3 | Dữ liệu chồng lấn thì sao? | **Lề mềm** với biến lỏng $\xi_i$ và $C$; dạng không ràng buộc $J = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum\max(0, 1 - m_i)$ (**hinge**); gradient dưới đạo hàm, SGD kiểu Pegasos. |
| 4 | Cơ chế học là gì? | $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i^T$, $\mathbf{y}^T\boldsymbol{\alpha} = 0$, $0 \leq \alpha_i \leq C$; KKT cho ba loại mẫu; **tính thưa**; liên hệ Perceptron dạng đối ngẫu; SMO. |
| 5 | Biên cong thì sao? | **Thủ thuật kernel** $k(\mathbf{x}, \mathbf{z}) = \varphi(\mathbf{x})\varphi(\mathbf{z})^T$: dạng đối ngẫu chỉ cần tích vô hướng. Kernel đa thức, RBF; $\gamma$ và $C$ điều khiển độ cong. |
| 6 | Hồi quy thì sao? | **SVR**: mất mát $\varepsilon$-insensitive, ống $\varepsilon$, $\mathbf{w} = \sum_i \beta_i \mathbf{x}_i^T$, ảnh hưởng của ngoại lai bị chặn ở $C$. |
| 7 | Đánh giá và chọn siêu tham số? | Ma trận nhầm lẫn, AUC, **Platt scaling**, one-vs-rest/one-vs-one, MAE/RMSE/$R^2$, kiểm định chéo cho $C, \gamma, \varepsilon$. |

Có bốn ý tưởng cốt lõi nên mang theo:

1. **Chọn nghiệm bằng một tiêu chí hình học.** Khi nhiều mô hình cùng đúng trên dữ liệu huấn luyện, SVM chọn mô hình có lề lớn nhất. Cùng tinh thần đó ở hồi quy là chọn hàm phẳng nhất thỏa ống $\varepsilon$.

2. **Một khung huấn luyện chung: "mất mát cộng điều chuẩn".** $J$ = điều chuẩn + $C\sum_i$ mất mát $(m_i)$. SVM, Logistic Regression có L2 và Perceptron chỉ khác nhau ở hàm mất mát. Với cả ba, mỗi mẫu đẩy $\mathbf{w}$ theo hướng $y_i\mathbf{x}_i^T$ với một trọng số $\varphi(m_i)$ (mục 3.4).
3. **Đối ngẫu cho ta tính thưa và kernel.** Nghiệm là tổ hợp của một số ít mẫu, và dữ liệu chỉ xuất hiện qua tích vô hướng, nên phi tuyến "miễn phí" về mặt tính toán trong không gian đặc trưng (nhưng không miễn phí về $n^2$).
4. **Siêu tham số quyết định chất lượng.** $C, \gamma, \varepsilon$ phải được chọn bằng kiểm định chéo, và các cấu hình hòa nhau nên chọn cấu hình đơn giản nhất (mục 7.5).

## 8.1. Ưu điểm và hạn chế của SVM/SVR

### Ưu điểm

1. **Tiêu chí rõ ràng và nghiệm duy nhất.** Hàm mục tiêu lồi chặt theo $\mathbf{w}$ nên nghiệm tồn tại, duy nhất và không phụ thuộc khởi tạo hay thứ tự duyệt mẫu, kể cả khi dữ liệu phân tách hoàn hảo (mục 4.2). Đây là điểm khác với Perceptron (vô số nghiệm) và Logistic Regression không điều chuẩn (không có cực tiểu hữu hạn).
2. **Nghiệm thưa, chỉ dựa vào vector hỗ trợ.** Bỏ các mẫu có $\alpha_i = 0$ không đổi nghiệm, số lỗi leave-one-out không vượt quá số vector hỗ trợ (mục 4.4), và dự đoán chỉ cần các vector hỗ trợ.
3. **Phi tuyến mà vẫn là bài toán lồi.** Thủ thuật kernel cho đường biên cong, thậm chí trong không gian vô hạn chiều (RBF), mà không phải dựng đặc trưng và không rơi vào cực tiểu địa phương như mạng nơ-ron (mục 5).
4. **Bền vững với ngoại lai.** Lực của mỗi mẫu bị chặn ở $C$ (mục 4.1), nên một mẫu sai nặng không kéo đường biên mạnh tùy ý. SVR giữ nghiệm **không đổi** khi ngoại lai di chuyển tự do vùng dưới ống (mục 6.5), trái với hồi quy MSE.
5. **Hiệu quả khi số chiều lớn so với số mẫu.** Lề lớn kiểm soát độ phức tạp một cách trực tiếp, và dạng đối ngẫu có $n$ biến chứ không phải $d$, nên SVM thường làm tốt với dữ liệu thưa nhiều chiều (văn bản, gen) và dữ liệu ít.
6. **Ít siêu tham số, ý nghĩa rõ.** Chỉ có $C$, kernel và tham số kernel $\gamma$ (cộng $\varepsilon$ với SVR), và mỗi tham số có cách đọc hình học riêng (mục 3.5, 5.4, 6.5).
7. **Một khung cho nhiều bài toán.** Phân loại, hồi quy, phát hiện bất thường (mục 8.3) đều dùng cùng bộ khung điều chuẩn, mất mát cắt cụt và đối ngẫu.

### Hạn chế

Mỗi hạn chế dưới đây đã xuất hiện ở một mục trước và đều có hướng khắc phục:

| Hạn chế | Nguyên nhân | Hướng khắc phục |
|---|---|---|
| **Chi phí cao khi $n$ lớn**: thời gian $O(n^2)$ đến $O(n^3)$, bộ nhớ $O(n^2)$ cho ma trận Gram (mục 5.3, 7.6) | Bài toán đối ngẫu có $n$ biến và ma trận $n \times n$ | *LinearSVC*/SGD cho SVM tuyến tính, xấp xỉ kernel (mục 8.3), lấy mẫu con |
| **Không cho xác suất** (mục 7.2) | Hinge chỉ quan tâm $m \geq 1$, không mô hình hóa xác suất | Platt scaling, hoặc dùng Logistic Regression |
| **Nhạy với thang đo đặc trưng** (mục 3.5) | $\| \mathbf{w} \|$ và khoảng cách Euclid dùng chung cho mọi đặc trưng | Chuẩn hóa đặc trưng trong pipeline |
| **Cần chọn siêu tham số** $C, \gamma, \varepsilon$ bằng kiểm định chéo; nhiều cấu hình hòa nhau (mục 7.5) | Chúng tương tác (ví dụ $C\gamma \approx$ hằng số) | Lưới theo lũy thừa, tìm thô rồi tinh, quy tắc một độ lệch chuẩn |
| **Khó diễn giải với kernel phi tuyến** | $\mathbf{w}$ nằm trong không gian đặc trưng ẩn; chỉ còn các vector hỗ trợ | SVM tuyến tính khi cần diễn giải; vẽ vùng quyết định, xem vector hỗ trợ |
| **Nhiều lớp phải ghép** từ nhị phân (mục 7.3) | Bản chất nhị phân của hinge | One-vs-rest, one-vs-one, hoặc Softmax Regression |
| **Không tự học đặc trưng**: chất lượng phụ thuộc kernel ta chọn | Kernel cố định, không học từ dữ liệu | Thử nhiều kernel, hoặc mạng nơ-ron khi dữ liệu rất lớn và phi cấu trúc |
| **Dữ liệu chồng lấn mạnh làm gần như mọi mẫu là vector hỗ trợ** (mục 7.1) | $\alpha_i = C$ ở các mẫu vi phạm, mất tính thưa | Tăng $C$, thêm đặc trưng, hoặc chấp nhận chi phí cao |
| **Lớp mất cân bằng** làm accuracy gây hiểu lầm | Mô hình thiên về lớp đông | `class_weight`, đổi ngưỡng, dùng recall, precision, AUC |

## Lớp mất cân bằng: `class_weight` là $C$ riêng cho từng lớp

Hạn chế cuối cùng nên xem kỹ vì nó ảnh hưởng trực tiếp đến kết quả. Giả sử lỗi trên lớp thiểu số tốn kém hơn, ta cho mỗi mẫu một hệ số phạt riêng $C_i = C\, w_{y_i}$ với trọng số lớp $w_k$ (thường $w_k = \frac{n}{K\, n_k}$ khi `class_weight="balanced"`):

$$J(\mathbf{w}, b) = \frac{1}{2} \| \mathbf{w} \|^2 + \sum_{i=1}^{n} C_i \max(0,\ 1 - m_i), \qquad \text{đối ngẫu: } 0 \leq \alpha_i \leq C_i.$$

Nghĩa là cận trên của hệ số đối ngẫu ở mục 4.3 giờ phụ thuộc vào lớp của mẫu: mẫu thiểu số được phép "đẩy" mạnh hơn. Ta thử trên 1000 sinh viên mô phỏng với 95% lớp âm và 5% lớp dương (chia 700 huấn luyện với 35 mẫu dương và 300 kiểm tra với 15 mẫu dương). Với *balanced*, $w_+ = \frac{700}{2 \cdot 35} = 10$ và $w_- = \frac{700}{2 \cdot 665} \approx 0{,}526$.

| Mô hình | Accuracy | Recall (lớp dương) | Precision | AUC | Số dự đoán dương |
|---|---|---|---|---|---|
| SVM tuyến tính, không trọng số | 96,0% | 0,333 | 0,714 | 0,972 | 7 |
| SVM tuyến tính, `class_weight="balanced"` | 92,3% | 0,867 | 0,382 | 0,972 | 34 |

Nếu chỉ nhìn accuracy (96% so với 92%), mô hình không trọng số có vẻ tốt hơn, nhưng nó bỏ sót 10 trong 15 ca dương (recall chỉ 0,333). Với trọng số lớp, recall tăng lên 0,867, đổi lại precision giảm. AUC **không đổi** (0,972) vì nó chỉ phụ thuộc thứ tự điểm số; trọng số lớp chủ yếu dịch đường biên, giống việc đổi ngưỡng ở mục 7.2.

### Đa cộng tuyến: điều chuẩn L2 có sẵn giúp ổn định, nhưng chỉ khi $C$ vừa phải

Ở bài Logistic Regression, hai đặc trưng gần như trùng nhau làm trọng số không điều chuẩn trở nên bất ổn (dấu ngược nhau, độ lớn lớn). SVM có sẵn số hạng $\frac{1}{2}\|\mathbf{w}\|^2$ nên được hưởng lợi như điều chuẩn L2. Ta lặp lại thí nghiệm: thêm đặc trưng $x_1' = x_1 + \epsilon$ với nhiễu $\epsilon$ có độ lệch chuẩn 0,05 (hệ số tương quan 0,9998) rồi huấn luyện SVM tuyến tính trên $(x_1, x_1', x_2)$ đã chuẩn hóa:

| Mô hình | $w_1$ | $w_1'$ | $w_2$ | Tổng $w_1 + w_1'$ | Accuracy kiểm tra |
|---|---|---|---|---|---|
| SVM, $C = 1$ | 1,108 | 1,142 | −1,297 | 2,25 | 90,0% |
| SVM, $C = 100$ | −0,957 | 3,387 | −1,432 | 2,43 | 89,2% |

Với $C = 1$, trọng số **chia gần đều** giữa hai bản sao (1,11 và 1,14), vì phạt $\frac{1}{2}(w_1^2 + w_1'^2)$ nhỏ nhất khi tổng cố định mà hai số bằng nhau. Với $C = 100$, điều chuẩn yếu đi nên hiện tượng cũ quay lại: $-0{,}96$ và 3,39 có dấu ngược nhau, dù tổng của chúng (2,43) vẫn gần ổn định. Như vậy đa cộng tuyến vẫn là vấn đề khi $C$ lớn, và ta nên đọc **tổng** trọng số của các đặc trưng tương quan thay vì từng trọng số riêng lẻ.

## 8.2. So sánh Perceptron, Logistic Regression, Softmax Regression, mạng nơ-ron và SVM

### Một khung chung cho các hàm mất mát theo margin

Ba hàm mất mát theo margin $m = y\,s$ mà ta đã gặp thực chất là các trường hợp của **một họ** hàm. Với tham số "độ cứng" $c > 0$ và "ngưỡng" $\theta$, xét

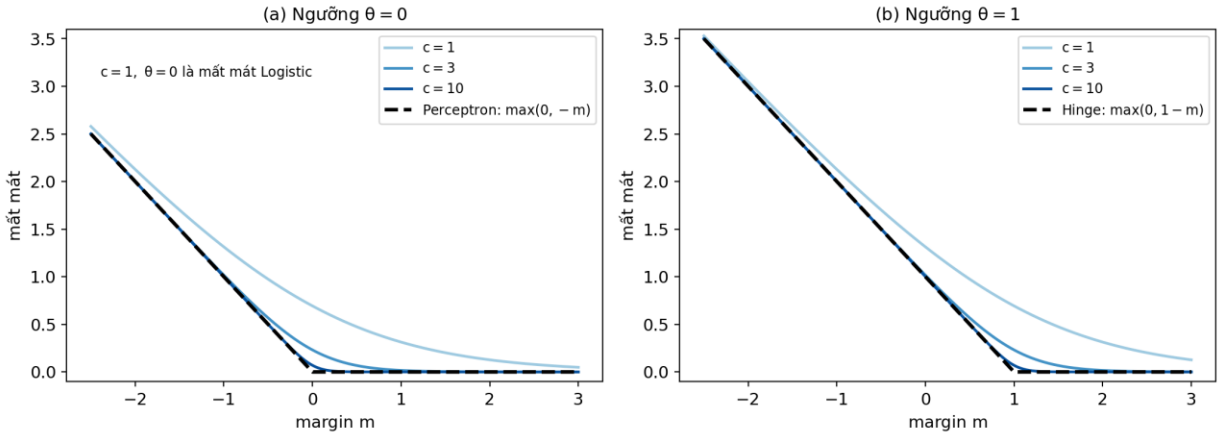
$$\ell_{c,\theta}(m) = \frac{1}{c}\ln\left(1 + e^{\,c(\theta - m)}\right).$$

Đây là hàm softplus dịch chuyển, trơn và lồi. Khi $c \to \infty$, nó tiến tới hàm cắt cụt $\max(0, \theta - m)$ (chứng minh như ở bài Logistic Regression: nếu $m > \theta$ thì $\ell \le e^{-c(m-\theta)}/c \to 0$, nếu $m < \theta$ thì $\ell = \theta - m + \frac{1}{c}\ln\left(1 + e^{-c(\theta - m)}\right) \to \theta - m$). Từ đó:

| Mô hình | Tham số $(c, \theta)$ | Mất mát |
|---|---|---|
| **Logistic Regression** | $c = 1,\ \theta = 0$ | $\ln(1 + e^{-m})$ (trơn) |
| **Perceptron** | $c \to \infty,\ \theta = 0$ | $\max(0, -m)$ |
| **SVM** (hinge) | $c \to \infty,\ \theta = 1$ | $\max(0, 1 - m)$ |

Perceptron và SVM là hai phiên bản "cứng" của cùng một họ, **chỉ khác ngưỡng** $\theta$: Perceptron thỏa mãn khi $m \geq 0$ (chỉ cần đúng), SVM chỉ thỏa mãn khi $m \geq 1$ (đúng và có lề). Bảng số kiểm chứng sự co về hinge ($\theta = 1$):

| Margin $m$ | $c = 1$ | $c = 5$ | $c = 20$ | $c = 100$ | $\max(0, 1 - m)$ |
|---|---|---|---|---|---|
| 0,5 | 0,974 | 0,516 | 0,500 | 0,500 | 0,5 |
| 1,5 | 0,474 | 0,016 | 0,000 | 0,000 | 0 |




*Hình 30. Họ hàm mất mát* $\ell_{c,\theta}(m) = \frac{1}{c} ln\left(1 + e^{c(\theta - m)}\right)$. *(a) Ngưỡng* $\theta = 0$*: khi* $c$ *tăng, đường cong tiến tới mất mát Perceptron* $max(0, -m)$ *(nét đứt); với* $c = 1$ *đó là mất mát Logistic. (b) Ngưỡng* $\theta = 1$*: khi* $c$ *tăng, đường cong tiến tới hinge* $max(0, 1 - m)$.

## Cùng một dạng cập nhật

Ở mục 3.4, ta đã thấy ba mô hình có cùng dạng cập nhật $\mathbf{w} \leftarrow \mathbf{w} + \eta \sum_i \varphi\,(m_i)\, y_i \mathbf{x}_i^T$ với trọng số $\varphi(m_i)$ lần lượt là $\mathbb{1}[m < 0]$, $\sigma(-m)$ và $\mathbb{1}[m < 1]$. Softmax Regression có cùng dạng gradient $\frac{1}{n}\mathbf{X}^T(\mathbf{P} - \mathbf{Y})$ với $\mathbf{P} = \text{softmax}(\mathbf{XW})$ và $\mathbf{Y}$ là ma trận nhãn one-hot; mạng nơ-ron có lớp cuối cùng cũng có dạng này, cộng với lan truyền ngược về các lớp ẩn.

## Bảng so sánh

| | Perceptron | Logistic Regression | Softmax Regression | Mạng nơ-ron | SVM / SVR |
|---|---|---|---|---|---|
| **Bài toán** | Phân loại nhị phân | Phân loại nhị phân | Phân loại $K$ lớp | Phân loại, hồi quy | Phân loại, hồi quy, bất thường |

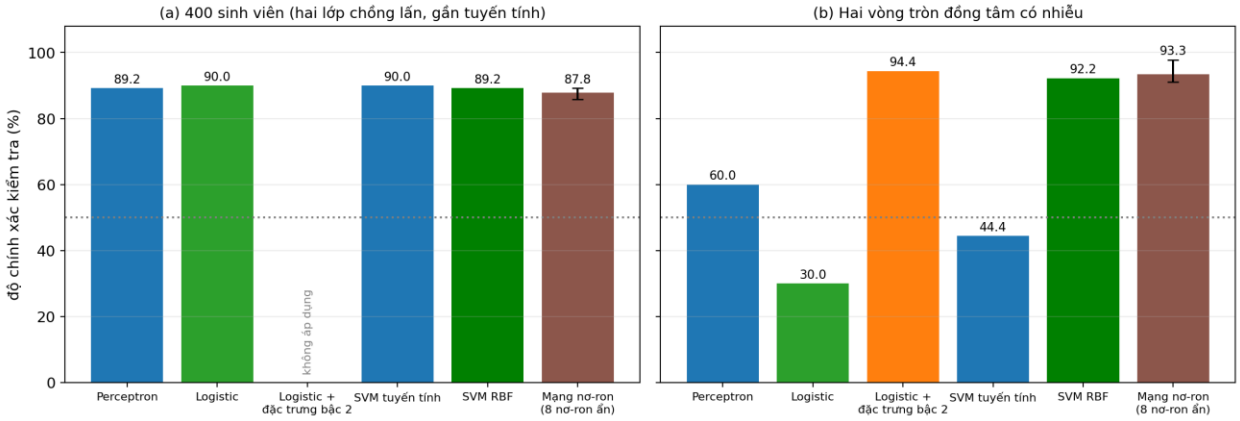
| | Perceptron | Logistic Regression | Softmax Regression | Mạng nơ-ron | SVM / SVR |
|---|---|---|---|---|---|
| **Điểm số** | $s = \mathbf{x}\mathbf{w} + b$ | $s = \mathbf{x}\mathbf{w} + b$ | $\mathbf{S} = \mathbf{X}\mathbf{W}$ | Các lớp ẩn rồi lớp ra | $f = \sum_i \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}) + b$ |
| **Đầu ra** | Nhãn cứng | Xác suất $\sigma(s)$ | Xác suất $K$ lớp | Xác suất hay giá trị | Điểm số $f$ (xác suất qua Platt) |
| **Mất mát theo margin** | $\max(0, -m)$ | $\ln(1 + e^{-m})$ | Cross-entropy $K$ lớp | Cross-entropy hay MSE | $\max(0, 1 - m)$; $\varepsilon$-insensitive |
| **Trọng số mẫu** $\varphi(m)$ | $\mathbb{1}[m < 0]$ | $\sigma(-m)$ | $p_{ik} - y_{ik}$ | Lan truyền ngược | $\mathbb{1}[m < 1]$, bị chặn ở $C$ |
| **Điều chuẩn** | Không | Tùy chọn (L2, L1) | Tùy chọn | Tùy chọn (weight decay) | Có sẵn $\frac{1}{2} \| \mathbf{w} \|^2$ |
| **Hàm mục tiêu** | Không có (cập nhật trực tuyến) | Lồi | Lồi | **Không lồi** | Lồi |
| **Khi dữ liệu phân tách hoàn hảo** | Vô số nghiệm | Không có cực tiểu hữu hạn (cần điều chuẩn) | Tương tự | Nhiều cực tiểu | **Duy nhất**, lề lớn nhất |
| **Mẫu ảnh hưởng** | Mẫu từng sai | Mọi mẫu | Mọi mẫu | Mọi mẫu | **Chỉ vector hỗ trợ** |
| **Xác suất** | Không | Có | Có | Có | Không trực tiếp |
| **Phi tuyến** | Thêm đặc trưng | Thêm đặc trưng | Thêm đặc trưng | **Học đặc trưng** | **Kernel** |
| **Nhiều lớp** | Ghép nhị phân | Ghép nhị phân | **Trực tiếp** | Trực tiếp (softmax ra) | Ghép (OvR, OvO) |
| **Chi phí huấn luyện** | $O(nd)$ mỗi lượt | $O(nd)$ mỗi epoch | $O(ndK)$ mỗi epoch | Tùy kích thước mạng | $O(n^2)$ đến $O(n^3)$ (kernel); gần $O(nd)$ (tuyến tính, primal) |
| **Diễn giải** | Hướng ảnh hưởng | Odds ratio $e^{w_j}$ | Theo từng lớp | Khó | Tuyến tính: dễ; kernel: khó |

| | Perceptron | Logistic Regression | Softmax Regression | Mạng nơ-ron | SVM / SVR |
|---|---|---|---|---|---|
| **Phù hợp** | Dạy khái niệm, học trực tuyến đơn giản | Baseline, cần xác suất, dễ diễn giải | Nhiều lớp, cần xác suất | Dữ liệu lớn, phi cấu trúc | Dữ liệu vừa và nhỏ, nhiều chiều, cần lề, kernel |

## So sánh bằng thực nghiệm

Ta chạy các mô hình trên hai bộ dữ liệu đã dùng xuyên suốt: **400 sinh viên** (hai lớp chồng lấn, đường biên gần tuyến tính) và **hai vòng tròn đồng tâm có nhiễu** (đường biên cong). Tất cả dùng đặc trưng đã chuẩn hóa. Mạng nơ-ron có 8 nơ-ron ẩn, huấn luyện với 3 khởi tạo ngẫu nhiên khác nhau.

| Mô hình | 400 sinh viên: accuracy kiểm tra | AUC | Hai vòng tròn: accuracy kiểm tra | AUC |
|---|---|---|---|---|
| Perceptron | 89,2% | 0,953 | 60,0% | 0,581 |
| Logistic Regression ($C = 1$) | 90,0% | 0,953 | 30,0% | 0,436 |
| Logistic Regression + đặc trưng bậc hai | không cần | | 94,4% | 0,988 |
| SVM tuyến tính ($C = 1$) | 90,0% | 0,953 | 44,4% | 0,518 |
| SVM kernel đa thức bậc hai | | | 94,4% | 0,988 |
| SVM kernel RBF | 89,2% | 0,953 | 92,2% | 0,989 |
| Mạng nơ-ron (3 khởi tạo) | 85,8% đến 89,2% | 0,936 đến 0,950 | 91,1% đến 97,8% | 0,964 đến 0,983 |




*Hình 31. Độ chính xác kiểm tra trên (a) bộ* 400 *sinh viên và (b) bộ hai vòng tròn. Thanh có vạch sai số của mạng nơ-ron là trung bình và khoảng nhỏ nhất đến lớn nhất qua* 3 *khởi tạo ngẫu nhiên. Đường chấm là mức* 50%.

Từ bảng và Hình 31, ta rút ra ba kết luận (cần đọc kỹ vì tập kiểm tra chỉ có 120 và 90 mẫu nên sai số chuẩn cỡ 3 điểm phần trăm):

1. **Trên dữ liệu "dễ", mọi mô hình ngang nhau.** Với 400 sinh viên, Perceptron, Logistic Regression, SVM tuyến tính và SVM RBF đều trong khoảng 89% đến 90% với AUC 0,953, chênh nhau chưa đến hai mẫu kiểm tra. Mạng nơ-ron có phương sai theo khởi tạo cao hơn và không tốt hơn. Đây là điều đã nhận xét ở bài Logistic Regression: lợi thế của một mô hình không nằm ở một con số accuracy.
2. **Khi đường biên cong, sự khác biệt đến từ khả năng phi tuyến.** Mọi mô hình tuyến tính (Perceptron, Logistic, SVM tuyến tính) chỉ ở mức ngẫu nhiên (30% đến 60%; con số dưới 50% là do may rủi trên 90 mẫu, không có ý nghĩa). Các mô hình có phi tuyến (đặc trưng bậc hai, kernel, mạng nơ-ron) đều đạt 91% đến 98%.
3. **Ba cách thêm phi tuyến cho kết quả tương đương trên bài này**, nhưng đánh đổi khác nhau: đặc trưng bậc hai do ta tự đoán, SVM kernel **tự động** chọn qua kernel (thêm hai siêu tham số $C, \gamma$) và cho nghiệm duy nhất lặp lại được, còn mạng nơ-ron **tự học** đặc trưng nhưng kết quả dao động theo khởi tạo (từ 91,1% đến 97,8%).

Với bài toán ba lớp ở mục 7.3, Softmax Regression (`LogisticRegression` đa lớp, huấn luyện trực tiếp) cho 82,2%, còn SVM tuyến tính cho 78,9% (OvR) và 81,1% (OvO), cũng trong phạm vi nhiễu của 90 mẫu kiểm tra.

## Nên chọn mô hình nào?

| Tình huống | Lựa chọn hợp lý |
|---|---|
| Cần xác suất và khả năng diễn giải, dữ liệu gần tuyến tính | **Logistic Regression** (hay Softmax cho nhiều lớp) |
| $n$ vừa phải, nhiều đặc trưng, biên có thể cong, ít dữ liệu | **SVM kernel RBF** (hoặc SVM tuyến tính nếu đủ) |
| $n$ rất lớn hoặc đặc trưng thưa (văn bản) | SVM tuyến tính (`LinearSVC`, SGD) hoặc Logistic Regression |
| Dữ liệu rất lớn, phi cấu trúc (ảnh, âm thanh, văn bản) | **Mạng nơ-ron** |
| Hồi quy có ngoại lai, cần kiểm soát mức sai số chấp nhận được | **SVR** |
| Phát hiện mẫu bất thường khi chỉ có dữ liệu "bình thường" | **One-class SVM** (mục 8.3) |
| Dạy khái niệm, học trực tuyến đơn giản nhất | **Perceptron** |

Trong thực tế, hầu như luôn bắt đầu từ một đường cơ sở tuyến tính (Logistic Regression hay SVM tuyến tính), rồi thêm độ phức tạp (kernel, mạng) chỉ khi đường cơ sở không đủ.

## 8.3. Mở rộng: one-class SVM, SVM cho dữ liệu lớn, hướng sang các mô hình khác

### One-class SVM: phát hiện bất thường

Trong nhiều bài toán ta chỉ có dữ liệu của lớp "bình thường" (giao dịch hợp lệ, máy chạy tốt) và cần phát hiện các điểm "bất thường" chưa từng thấy. Bài toán không có nhãn $\pm 1$, nên ta không thể tìm lề giữa hai lớp. **One-class SVM** (Schölkopf và cộng sự) đổi ý tưởng: tìm siêu phẳng trong không gian đặc trưng **tách dữ liệu khỏi gốc tọa độ với lề lớn nhất**. Gọi $\nu \in (0,1]$ là tham số, bài toán gốc là

$$\min_{\mathbf{w},\rho,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + \frac{1}{\nu n}\sum_{i=1}^{n}\xi_i - \rho \quad \text{với} \quad \varphi(\mathbf{x}_i)\,\mathbf{w} \geq \rho - \xi_i,\ \ \xi_i \geq 0.$$

Siêu phẳng $\varphi(\mathbf{x})\mathbf{w} = \rho$ cách gốc một khoảng $\rho/\|\mathbf{w}\|$, ta muốn khoảng này lớn (số hạng $-\rho$) trong khi vẫn "ôm" phần lớn dữ liệu về một phía (biến lỏng $\xi_i$). Làm tương tự mục 4.3, ta thu được dạng đối ngẫu chỉ cần ma trận Gram $\mathbf{K}$:

$$\min_{\boldsymbol{\alpha}} \frac{1}{2}\boldsymbol{\alpha}^T\mathbf{K}\boldsymbol{\alpha} \quad \text{với} \quad \mathbf{1}^T\boldsymbol{\alpha} = 1,\ \ \mathbf{0} \leq \boldsymbol{\alpha} \leq \frac{1}{\nu n}\mathbf{1},$$

$$f(\mathbf{x}) = \sum_{i=1}^{n}\alpha_i\, k(\mathbf{x}_i,\mathbf{x}) - \rho, \qquad \text{bất thường nếu } f(\mathbf{x}) < 0,$$

với $\rho = \sum_j \alpha_j\, k(\mathbf{x}_j,\mathbf{x}_k)$ lấy từ một mẫu $k$ có $0 < \alpha_k < \frac{1}{\nu n}$. Dạng này giống hệt SVM phân loại đã học: có ràng buộc hộp, ràng buộc tổng, và vector hỗ trợ là các mẫu có $\alpha_i > 0$. Với kernel RBF ($k(\mathbf{x},\mathbf{x}) = 1$ nên mọi $\varphi(\mathbf{x})$ nằm trên mặt cầu đơn vị), bài toán tương đương với việc tìm **hình cầu nhỏ nhất bao dữ liệu** (SVDD).

**Ý nghĩa của $\nu$.** Vì $\sum_i \alpha_i = 1$ và $\alpha_i \leq \frac{1}{\nu n}$, nên số mẫu có $\alpha_i > 0$ phải **ít nhất** $\nu n$. Mặt khác, các mẫu nằm ngoài biên ($f < 0$) phải có $\alpha_i$ đạt cận trên, nên **nhiều nhất** $\nu n$ mẫu như vậy. Tóm lại:

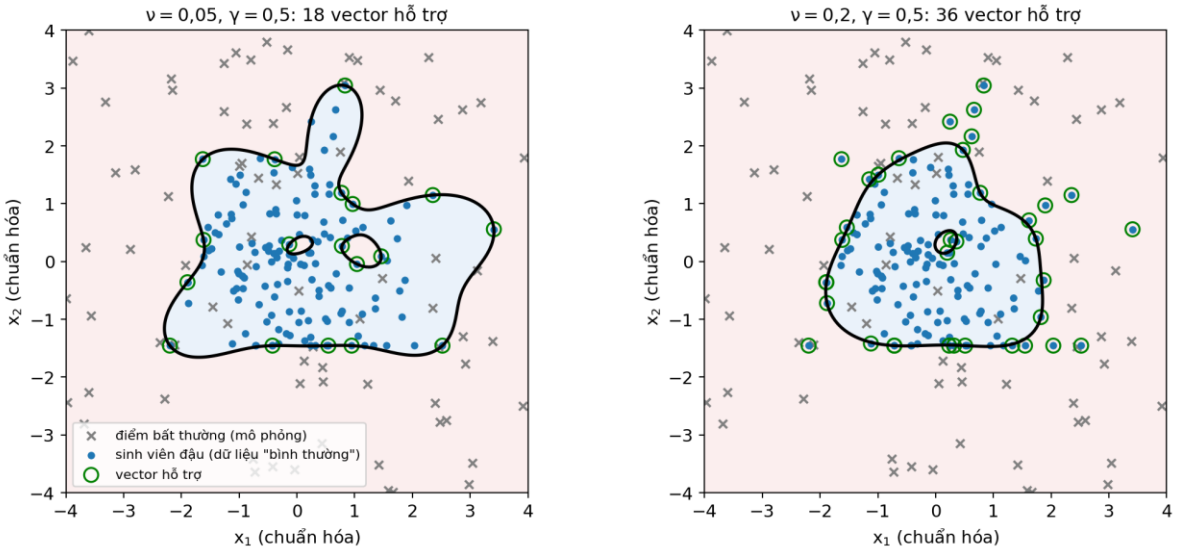
$$\frac{\#\{\text{vector bị chặn } (\alpha_i = 1/\nu n)\}}{n} \leq \nu \leq \frac{\#\{\text{vector hỗ trợ}\}}{n}.$$

Vậy $\nu$ là cận trên của tỉ lệ mẫu huấn luyện bị coi là bất thường và là cận dưới của tỉ lệ vector hỗ trợ. Ta kiểm chứng trên 149 sinh viên đậu của tập huấn luyện (coi là dữ liệu "bình thường", đã chuẩn hóa), kernel RBF với $\gamma = 0{,}5$, rồi thử trên 51 sinh viên đậu của tập kiểm tra và 200 điểm bất thường sinh ngẫu nhiên đều trong $[-4,4]^2$:

| $\nu$ | Tỉ lệ vector bị chặn | Tỉ lệ bị gán bất thường (huấn luyện) | Tỉ lệ vector hỗ trợ | Bình thường bị báo nhầm (kiểm tra) | Bất thường được phát hiện |
|---|---|---|---|---|---|
| 0,01 | 0,7% | 2,7% | 12,1% | 13,7% | 76,5% |
| 0,05 | 0,7% | 5,4% | 12,1% | 13,7% | 76,5% |

| $\nu$ | Tỉ lệ vector bị chặn | Tỉ lệ bị gán bất thường (huấn luyện) | Tỉ lệ vector hỗ trợ | Bình thường bị báo nhầm (kiểm tra) | Bất thường được phát hiện |
|---|---|---|---|---|---|
| 0,1 | 6,0% | 11,4% | 15,4% | 11,8% | 82,0% |
| 0,2 | 16,8% | 21,5% | 24,2% | 27,5% | 83,5% |
| 0,3 | 28,2% | 31,5% | 34,2% | 37,3% | 87,5% |

Cột đầu luôn $\leq \nu$ và cột "vector hỗ trợ" luôn $\geq \nu$, đúng bất đẳng thức trên. Tỉ lệ bị gán bất thường (theo dấu của $f$) bám sát $\nu$ nhưng không bằng chính xác (ví dụ 5,4% với $\nu = 0{,}05$ và 11,4% với $\nu = 0{,}1$) do việc ước lượng $\rho$ có dung sai. Quan trọng hơn, $\nu$ là một **đánh đổi**: $\nu$ lớn bắt được nhiều điểm bất thường hơn (76,5% → 87,5%) nhưng cũng báo nhầm nhiều mẫu bình thường hơn (13,7% → 37,3%). Tham số $\gamma$ cũng ảnh hưởng đến hình dạng vùng "bình thường" (Hình 32).




*Hình 32. One-class SVM với kernel RBF ($\gamma = 0{,}5$) học vùng "bình thường" (nền xanh) từ các sinh viên đậu (chấm xanh). (a) $\nu = 0{,}05$: vùng rộng, ít vector hỗ trợ. (b) $\nu = 0{,}2$: vùng co lại, nhiều vector hỗ trợ và nhiều điểm bị coi là bất thường hơn. Dấu × xám là các điểm bất thường mô phỏng.*

**Một ví dụ thực.** Trên bộ dữ liệu ung thư vú của scikit-learn, ta chỉ dùng các mẫu **lành tính** trong tập huấn luyện để huấn luyện one-class SVM ($\gamma = 0{,}01$, $\nu = 0{,}05$, chuẩn hóa theo lành tính). Khi chấm điểm bất thường $-f(\mathbf{x})$ trên tập kiểm tra có cả hai loại, AUC để phân biệt mẫu ác tính (xem là "bất thường") là 0,959, và 90,6% mẫu ác tính bị gắn nhãn bất thường (với 15,0% mẫu lành tính bị báo nhầm). Mô hình chưa từng thấy một mẫu ác tính nào lúc huấn luyện nhưng vẫn phân biệt khá tốt. Dù vậy, khi **có** nhãn của cả hai lớp, SVM phân loại thông thường luôn tốt hơn (AUC 0,99 ở bài tập B2).

## SVM cho dữ liệu lớn

Ở mục 7.6 ta đã thấy chi phí huấn luyện *SVC* tăng gần bậc hai theo $n$. Có bốn hướng xử lý, từ đơn giản đến phức tạp:

| Hướng | Ý tưởng | Điểm mạnh | Điểm yếu |
|---|---|---|---|
| SVM tuyến tính trên bài toán gốc | *LinearSVC*, SGD kiểu Pegasos (mục 3.6), gradient dưới đạo hàm | Gần tuyến tính theo $n$ | Chỉ biên phẳng |
| **Xấp xỉ kernel** | Ánh xạ về $D$ chiều rồi dùng SVM tuyến tính | Giữ được biên cong, chi phí gần $O(nD)$ | Cần chọn $D$, có sai số xấp xỉ |
| Lấy mẫu con, cắt bớt | Huấn luyện trên tập con, hoặc giới hạn số vector hỗ trợ | Đơn giản | Mất thông tin |
| Cải tiến bộ giải | Shrinking, cache, GPU (SMO song song) | Giữ nghiệm chính xác | Vẫn $O(n^2)$ trong xấu nhất |

**Xấp xỉ kernel bằng đặc trưng Fourier ngẫu nhiên.** Với kernel RBF, định lý Bochner cho biết $k(\mathbf{x},\mathbf{z}) = e^{-\gamma\|\mathbf{x}-\mathbf{z}\|^2}$ là kỳ vọng $\mathbb{E}[\cos(\boldsymbol{\omega}(\mathbf{x}-\mathbf{z})^T)]$ với $\boldsymbol{\omega} \sim \mathcal{N}(\mathbf{0}, 2\gamma\mathbf{I})$. Lấy $D$ mẫu ngẫu nhiên, ta thu được ánh xạ **tường minh** $D$ chiều:

$$\mathbf{z}(\mathbf{x}) = \sqrt{\frac{2}{D}}\cos(\mathbf{x}\boldsymbol{\Omega} + \mathbf{b}), \qquad \boldsymbol{\Omega} \in \mathbb{R}^{d\times D} \text{ (các cột} \sim \mathcal{N}(\mathbf{0}, 2\gamma\mathbf{I})), \quad \mathbf{b} \sim \text{Đều}[0,2\pi]^D,$$

$$k(\mathbf{x},\mathbf{z}) \approx \mathbf{z}(\mathbf{x})\,\mathbf{z}(\mathbf{z})^T.$$

Sau đó, huấn luyện **SVM tuyến tính** trên $\mathbf{z}(\mathbf{x}_i)$ bằng *LinearSVC* hay SGD: chi phí $O(nD)$ thay vì $O(n^2)$. Sai số xấp xỉ giảm như $1/\sqrt{D}$; với hai điểm $\mathbf{x} = (0,0)$, $\mathbf{z} = (1,1)$ và $\gamma = 0{,}5$ (giá trị thật $k = e^{-1} \approx 0{,}3679$), sai số tuyệt đối trung bình qua 20 lần bốc ngẫu nhiên là

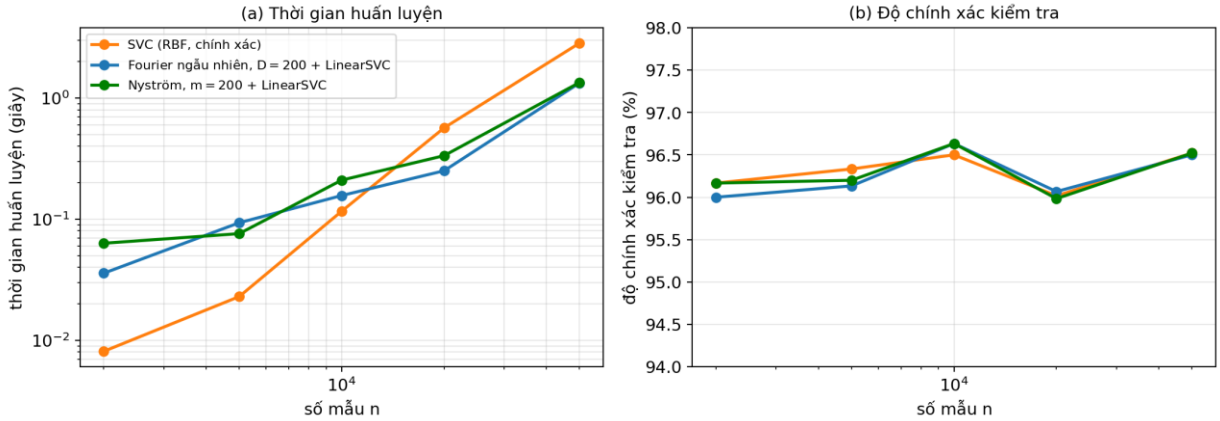
| $D$ | 10 | 100 | 1 000 | 10 000 | 100 000 |
|---|---|---|---|---|---|
| Sai số $\lvert\mathbf{z}(\mathbf{x})\mathbf{z}(\mathbf{z})^T - k\rvert$ | 0,310 | 0,074 | 0,026 | 0,009 | 0,002 |

**Phương pháp Nyström.** Chọn $m \ll n$ mẫu "mốc" và xấp xỉ ma trận Gram bằng hạng thấp: $\mathbf{K} \approx \mathbf{K}_{nm}\mathbf{K}_{mm}^{+}\mathbf{K}_{mn}$ với $\mathbf{K}_{nm}$ là ma trận $n \times m$ các giá trị $k\left(\mathbf{x}_i, \mathbf{x}_j^{\text{mốc}}\right)$. Ánh xạ đặc trưng tương ứng là $\boldsymbol{\Phi} = \mathbf{K}_{nm}\mathbf{K}_{mm}^{-1/2}$ ($n \times m$), rồi dùng SVM tuyến tính. Khác với Fourier ngẫu nhiên, Nyström **phụ thuộc dữ liệu** nên thường chính xác hơn với cùng số chiều.

Ta thử trên bộ hai vòng tròn có nhiễu lớn dần (70% huấn luyện, RBF $\gamma = 1$, $C = 1$; Fourier ngẫu nhiên $D = 200$ và Nyström $m = 200$, cùng với *LinearSVC* phía sau):

| Số mẫu $n$ | *SVC* RBF: thời gian (giây) | Fourier ngẫu nhiên $D = 200$ | Nyström $m = 200$ | *SVC*: accuracy | Fourier: accuracy | Nyström: accuracy |
|---|---|---|---|---|---|---|
| 2 000 | 0,008 | 0,036 | 0,063 | 96,2% | 96,0% | 96,2% |

| Số mẫu $n$ | *SVC* RBF: thời gian (giây) | Fourier ngẫu nhiên $D = 200$ | Nyström $m = 200$ | *SVC*: accuracy | Fourier: accuracy | Nyström: accuracy |
|---|---|---|---|---|---|---|
| 5 000 | 0,023 | 0,093 | 0,076 | 96,3% | 96,1% | 96,2% |
| 10 000 | 0,116 | 0,157 | 0,210 | 96,5% | 96,6% | 96,6% |
| 20 000 | 0,567 | 0,251 | 0,334 | 96,0% | 96,1% | 96,0% |
| 50 000 | 2,84 | 1,33 | 1,34 | 96,5% | 96,5% | 96,5% |
| 100 000 | 9,21 | 3,50 | không chạy | 96,5% | 96,5% | |
| 200 000 | 40,5 | 10,9 | không chạy | 96,3% | 96,3% | |




*Hình 33. So sánh* `SVC` *RBF chính xác với hai phương pháp xấp xỉ kernel cộng* `LinearSVC` *($D = m = 200$). (a) Thời gian huấn luyện theo $n$ (thang log):* `SVC` *tăng nhanh hơn và vượt các phương pháp xấp xỉ khi $n$ khoảng 20 000 trở lên. (b) Độ chính xác kiểm tra: ba cách gần như bằng nhau.*

Kết quả cho ta một bức tranh trung thực: **với $n$ nhỏ, *SVC* chính xác nhanh hơn** (vì phép ánh xạ và `LinearSVC` có chi phí cố định), điểm hòa vốn nằm trong khoảng $n \approx 10^4$ đến $2 \cdot 10^4$, và đến $n = 200\,000$ phương pháp xấp xỉ nhanh hơn khoảng 3,7 lần (10,9 giây so với 40,5 giây) mà độ chính xác **không đổi** (96,3%). Lợi thế sẽ tăng thêm với $n$ lớn hơn hoặc $d$ lớn hơn, vì *SVC* tăng gần bậc hai còn xấp xỉ tăng gần tuyến tính. Lưu ý bộ dữ liệu này khá "dễ" với vài nghìn vector hỗ trợ (ở $n = 200\,000$, *SVC* có khoảng 12 500 vector hỗ trợ); với dữ liệu khó hơn, khoảng cách còn rõ rệt hơn.

## Định lý biểu diễn và họ mô hình kernel

Thủ thuật kernel không chỉ dành cho SVM. **Định lý biểu diễn (representer theorem)** nói rằng với bất kỳ hàm mất mát $\ell$ nào, nghiệm của bài toán "mất mát cộng điều chuẩn" trong không gian hàm $\mathcal{H}$ sinh bởi kernel $k$,

$$\min_{f \in \mathcal{H}} \sum_{i=1}^{n} \ell\left(y_i, f(\mathbf{x}_i)\right) + \lambda \, \| f \|_{\mathcal{H}}^2,$$

luôn có dạng $f(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i \; k(\mathbf{x}_i, \mathbf{x})$ (cộng hệ số chệch). Đây là lý do mọi công thức "$\mathbf{w} = \sum_i \alpha_i \, y_i \mathbf{x}_i^T$" ở mục 4 và 6 xuất hiện. Đổi hàm mất mát $\ell$ ta được các mô hình kernel khác nhau:

| Hàm mất mát $\ell$ | Mô hình | Đặc điểm |
|---|---|---|
| Hinge $\max(0,1-yf)$ | **SVM** | Nghiệm **thưa**, không có xác suất |
| $\varepsilon$-insensitive | **SVR** | Nghiệm thưa, bền với ngoại lai |
| Logistic $\ln(1+e^{-yf})$ | Kernel Logistic Regression | Có xác suất, **không thưa** (mọi $\alpha_i \neq 0$) |
| Bình phương $(y-f)^2$ | **Kernel Ridge Regression** | Nghiệm đóng $\boldsymbol{\alpha} = (\mathbf{K}+\lambda\mathbf{I})^{-1}\mathbf{y}$, không thưa |
| $\max(0,-yf)$ | Kernel Perceptron | Cập nhật trực tuyến (mục 4.5, 5.2) |

Một so sánh minh họa trên bộ $\sin x$ có ngoại lai của mục 6: Kernel Ridge Regression (RBF $\gamma = 1$, $\lambda = 1$) có sai số $0{,}131$ so với đường thật (SVR cho $0{,}137$), tức tương đương, nhưng dùng **cả** $42$ **mẫu** huấn luyện làm hệ số khác không còn SVR chỉ cần $26$ vector hỗ trợ. Đây là cái giá của mất mát bình phương: không có vùng "không mất mát" nên không có tính thưa. Ngoài ra, các mô hình kernel khác gồm Gaussian process (xem $f$ như một phân phối trên hàm với kernel là hiệp phương sai), kernel PCA (giảm chiều phi tuyến) và các kernel cho dữ liệu có cấu trúc (chuỗi, đồ thị, cây).

## Lộ trình học tiếp

Có thể xem các mô hình trong chuỗi bài giảng như các nấc thang, mỗi nấc thay đổi một thành phần của khung "mô hình, hàm mất mát, thuật toán tối ưu":

| Mô hình | Điểm khác so với bước trước |
|---|---|
| Hồi quy tuyến tính | Dự đoán số thực, mất mát MSE |
| **Perceptron** | Thêm hàm dấu, mất mát $\max(0,-m)$ |
| **Logistic Regression** | Làm mềm hàm dấu thành sigmoid, cross-entropy, có xác suất |
| **SVM** | Đổi ngưỡng mất mát từ 0 lên 1 và thêm $\frac{1}{2}\|\mathbf{w}\|^2$: **lề cực đại**, đối ngẫu, tính thưa |
| **SVM kernel, SVR** | Kernel cho biên cong; mất mát $\varepsilon$-insensitive cho hồi quy |
| Softmax Regression | Nhiều lớp: ma trận $\mathbf{W}$, softmax |
| **Mạng nơ-ron** | Xếp nhiều lớp, học đặc trưng, lan truyền ngược |
| Mạng sâu (CNN, RNN, Transformer) | Cùng khung huấn luyện, đổi kiến trúc cho ảnh, chuỗi, văn bản |

Hai hướng đáng tìm hiểu tiếp: **cây quyết định và các mô hình tổ hợp** (rừng ngẫu nhiên, gradient boosting) thường là lựa chọn mạnh nhất cho dữ liệu dạng bảng, và **học sâu** cho dữ liệu phi cấu trúc. Cần nhớ rằng ý tưởng "lề" của SVM vẫn sống trong học sâu, qua các hàm mất mát dựa trên lề (margin loss) ở lớp cuối.

## 8.4. Bài tập thực hành

### Phần A. Tính tay và lý thuyết

**A1.** Dữ liệu một chiều gồm hai mẫu: $x_1 = 1$ với $y_1 = -1$ và $x_2 = 4$ với $y_2 = +1$. Giải bài toán SVM lề cứng để tìm $w$, $b$, đường biên và độ rộng lề. Tính $\alpha_1, \alpha_2$ từ dạng đối ngẫu và kiểm tra giá trị gốc bằng giá trị đối ngẫu.

**A2.** Ba mẫu một chiều $x = (0,1,2)$ với $\mathbf{y} = (-1, -1, +1)$ và tham số $w = 1$, $b = -1{,}5$, $C = 1$. Tính margin, hàm mất mát hinge của từng mẫu, $J$ và dưới đạo hàm theo $w$ và $b$. Kết luận gì về $(w, b)$? Xác định ba loại mẫu theo $\alpha_i$.

**A3.** Cho hai điểm hồi quy $(x_1, y_1) = (0; 0)$ và $(x_2, y_2) = (2; 2{,}4)$, $\varepsilon = 0{,}2$ và $C$ lớn. Tìm $w$, $b$ của SVR tuyến tính, hệ số $\beta_1, \beta_2$ và kiểm tra giá trị gốc bằng giá trị đối ngẫu.

**A4.** Với kernel $k(\mathbf{x}, \mathbf{z}) = (\mathbf{x}\mathbf{z}^T + 1)^2$ và hai điểm $\mathbf{x}_1 = [1 \quad 0]$, $\mathbf{x}_2 = [0 \quad 1]$, tính ma trận Gram $\mathbf{K}$ và các trị riêng của nó. Tính lại với kernel RBF $\gamma = 1$.

**A5.** Một mô hình Platt scaling có $A = 2$, $B = -0{,}5$. Tính xác suất $P(y = +1 \mid f)$ cho $f = 0{,}5$ và $f = 2$. Tìm $f$ để xác suất bằng $0{,}5$.

**A6.** Với $\mathbf{y} = (3; 5; 7; 9)$ và $\hat{\mathbf{y}} = (2; 5; 8; 11)$, tính MAE, RMSE và $R^2$.

**A7.** Bài toán $K = 5$ lớp, mỗi lớp $n/5$ mẫu. Có bao nhiêu bộ phân loại ở one-vs-rest và one-vs-one, mỗi bộ dùng bao nhiêu mẫu? Chi phí tương đối nếu thời gian huấn luyện tỉ lệ $n^2$?

**A8.** Một one-class SVM huấn luyện trên $n = 200$ mẫu với $\nu = 0{,}1$. Cận trên $\alpha_i \le \frac{1}{\nu n}$ bằng bao nhiêu? Có tối đa bao nhiêu mẫu bị coi là bất thường và tối thiểu bao nhiêu vector hỗ trợ?

**A9.** Tính $\ell_{c,1}(m) = \frac{1}{c}\ln\left(1 + e^{c(1-m)}\right)$ với $c = 10$ cho $m = 0{,}5$, $m = 1{,}5$ và $m = -1$, rồi so với $\max(0, 1 - m)$.

**A10.** Với `class_weight="balanced"` trên tập $n = 700$ mẫu gồm 35 mẫu dương và 665 mẫu âm, hãy tính hệ số phạt $C_i$ của mỗi lớp khi $C = 1$ và cận trên của $\alpha_i$.

### Phần B. Lập trình

**B1.** Cài đặt `LinearSVM` từ đầu (mục 3.7) và huấn luyện trên bộ dữ liệu ung thư vú của scikit-learn (nhãn $+1$ là ác tính, chia $70/30$ phân tầng, chuẩn hóa). So accuracy và giá trị $J$ với `SVC(kernel="linear")`.

**B2.** Trên cùng bộ dữ liệu: so sánh `SVC` RBF mặc định, `SVC` tuyến tính, `LinearSVC` và Logistic Regression về accuracy, ma trận nhầm lẫn, AUC. Dùng grid search $C \in \{0{,}1; 1; 10; 100\}$, $\gamma \in \{10^{-3}, 10^{-2}, 10^{-1}, 1\}$ với kiểm định chéo 5 lần và báo cáo kết quả trên tập kiểm tra.

**B3.** Cài đặt Platt scaling (mục 7.2) và áp dụng cho SVM RBF ở B2. So sánh log-loss của $\sigma(f)$, của Platt scaling và của Logistic Regression.

**B4.** Trên bộ Iris, so sánh one-vs-rest, one-vs-one và `LinearSVC` với accuracy kiểm tra, rồi so với Softmax Regression ở bài Logistic Regression.

**B5.** Trên bộ dữ liệu `load_diabetes`, so sánh hồi quy tuyến tính với SVR tuyến tính và SVR RBF (grid search $C, \gamma, \varepsilon$, chuẩn hóa cả $x$ và $y$). Báo cáo MAE, RMSE, $R^2$ trên tập kiểm tra.

**B6.** Cài đặt `KernelSVM` từ đầu bằng SMO (mục 5.6) và kiểm tra trên XOR với kernel đa thức bậc hai: các $\alpha_i$ có khớp với kết quả tay $\left(\frac{10}{3}, \frac{8}{3}, \frac{8}{3}, 2\right)$ không?

**B7.** Cài đặt đặc trưng Fourier ngẫu nhiên từ đầu và vẽ sai số xấp xỉ ma trận Gram theo $D$ trên 200 điểm của bộ hai vòng tròn.

## Phần C. Mở rộng

**C1.** Dùng one-class SVM phát hiện ung thư: huấn luyện chỉ trên mẫu lành tính ở bộ ung thư vú, chấm điểm $-f(\mathbf{x})$ cho tập kiểm tra và tính AUC. Thử các $\gamma$ và $\nu$ khác nhau.

**C2.** Lặp lại thí nghiệm mục 8.3 (thời gian và độ chính xác của `SVC` so với Fourier ngẫu nhiên và Nyström) trên bộ dữ liệu của riêng bạn với $n$ từ $10^3$ đến $10^5$, và tìm điểm hòa vốn.

**C3.** Tạo dữ liệu mất cân bằng (95% lớp âm, 5% lớp dương), huấn luyện SVM có và không có `class_weight="balanced"`, so sánh accuracy, recall, precision, AUC và vẽ đường precision-recall theo ngưỡng $t$.

**C4.** Cài đặt Kernel Ridge Regression $\boldsymbol{\alpha} = (\mathbf{K} + \lambda \mathbf{I})^{-1}\mathbf{y}$ và so sánh với SVR trên dữ liệu $\sin x$ có ngoại lai: sai số so với đường thật, số hệ số khác không, và ảnh hưởng của $\lambda$.

## Gợi ý đáp án

**A1.** Cả hai mẫu nằm trên hai biên lề: $-(w + b) = 1$ và $4w + b = 1$. Trừ vế: $3w = 2$, nên $w = \frac{2}{3}$, $b = -\frac{5}{3}$. Đường biên $\frac{2}{3}x - \frac{5}{3} = 0$ tức $x = 2{,}5$ (trung điểm), độ rộng lề $\frac{2}{w} = 3$. Đối ngẫu: $\alpha_1 = \alpha_2 = \alpha$ và $\boldsymbol{\alpha}^T \mathbf{Q} \boldsymbol{\alpha} = \alpha^2(1 + 16 - 2 \cdot 4) = 9\alpha^2$, nên $D(\alpha) = 2\alpha - \frac{9}{2}\alpha^2$ cực đại tại $\alpha = \frac{2}{9}$. Khi đó $w = \alpha(x_2 - x_1) = \frac{2}{9} \cdot 3 = \frac{2}{3}$ khớp. Giá trị đối ngẫu $D = \frac{4}{9} - \frac{4{,}5 \cdot 4}{81} = \frac{2}{9}$ bằng giá trị gốc $\frac{1}{2}w^2 = \frac{1}{2} \cdot \frac{4}{9} = \frac{2}{9}$.

**A2.** $s = x - 1{,}5 = (-1{,}5;\ -0{,}5;\ 0{,}5)$ nên $m = y\,s = (1{,}5;\ 0{,}5;\ 0{,}5)$ và hinge $= (0;\ 0{,}5;\ 0{,}5)$. $J = \frac{1}{2} \cdot 1^2 + 1 \cdot (0 + 0{,}5 + 0{,}5) = 1{,}5$. Mẫu vi phạm là mẫu 2 và 3 ($m < 1$) nên $\mathbf{a} = (0,1,1)$. Dưới đạo hàm: $\frac{\partial J}{\partial w} = w - C\sum_i a_i\, y_i x_i = 1 - \big((-1) \cdot 1 + (+1) \cdot 2\big) = 0$ và $\frac{\partial J}{\partial b} = -C\sum_i a_i\, y_i = -(-1 + 1) = 0$. Dưới đạo hàm bằng 0 nên $(w, b) = (1; -1{,}5)$ là **nghiệm tối ưu** (hàm lồi). Ba loại mẫu: mẫu 1 có $m = 1{,}5 > 1$ nên $\alpha_1 = 0$; mẫu 2 và 3 có $m < 1$ nên $\alpha_2 = \alpha_3 = C = 1$ (bị chặn). Kiểm tra: $\sum \alpha_i y_i = -1 + 1 = 0$ và $\mathbf{w} = \sum \alpha_i y_i x_i = -1 + 2 = 1$ khớp.

**A3.** Hai điểm ở hai đầu nằm trên hai biên ống: $f(0) = y_1 + \varepsilon$ và $f(2) = y_2 - \varepsilon$ (điểm trái nằm dưới ống, điểm phải nằm trên ống). $b = 0{,}2$, $2w + b = 2{,}2$ nên $w = 1$. Hệ số: $w = \beta_1 x_1 + \beta_2 x_2 = 2\beta_2 = 1$ nên $\beta_2 = 0{,}5$ và $\beta_1 = -0{,}5$ (do $\sum \beta = 0$). Giá trị gốc $\frac{1}{2}w^2 = 0{,}5$. Đối ngẫu:

$D(a) = -\frac{1}{2}(2a)^2 - 0{,}2 \cdot 2a + 2{,}4a = 2a - 2a^2$, cực đại tại $a = 0{,}5$ với $D = 0{,}5$, bằng giá trị gốc.

**A4.** $k(\mathbf{x}_1, \mathbf{x}_1) = (1+1)^2 = 4$, $k(\mathbf{x}_2, \mathbf{x}_2) = 4$, $k(\mathbf{x}_1, \mathbf{x}_2) = (0+1)^2 = 1$: $\mathbf{K} = \begin{bmatrix} 4 & 1 \\ 1 & 4 \end{bmatrix}$ có trị riêng 5 và 3 (đều dương, nửa xác định dương). Với RBF $\gamma = 1$: $\| \mathbf{x}_1 - \mathbf{x}_2 \|^2 = 2$ nên $\mathbf{K} = \begin{bmatrix} 1 & e^{-2} \\ e^{-2} & 1 \end{bmatrix} \approx \begin{bmatrix} 1 & 0{,}135 \\ 0{,}135 & 1 \end{bmatrix}$ có trị riêng $1 \pm e^{-2} \approx 1{,}135$ và $0{,}865$.

**A5.** $P = \sigma(2 \cdot 0{,}5 - 0{,}5) = \sigma(0{,}5) \approx 0{,}622$; $\sigma(2 \cdot 2 - 0{,}5) = \sigma(3{,}5) \approx 0{,}971$. Xác suất bằng 0,5 khi $Af + B = 0$, tức $f = 0{,}25$.

**A6.** $\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}} = (1;\ 0;\ -1;\ -2)$. MAE $= \frac{1+0+1+2}{4} = 1$. RMSE $= \sqrt{\frac{1+0+1+4}{4}} = \sqrt{1{,}5} \approx 1{,}225$. $\bar{y} = 6$, $SS_{\text{tot}} = 9 + 1 + 1 + 9 = 20$, $SS_{\text{res}} = 6$, nên $R^2 = 1 - \frac{6}{20} = 0{,}7$.

**A7.** One-vs-rest: 5 bộ, mỗi bộ dùng cả $n$ mẫu, chi phí $\propto 5n^2$. One-vs-one: $\binom{5}{2} = 10$ bộ, mỗi bộ dùng $2 \cdot \frac{n}{5} = \frac{2n}{5}$ mẫu, chi phí $\propto 10\left(\frac{2n}{5}\right)^2 = 1{,}6\,n^2$. One-vs-one rẻ hơn $\frac{5}{1{,}6} \approx 3{,}1$ lần.

**A8.** $\alpha_i \le \frac{1}{0{,}1 \cdot 200} = 0{,}05$. Vì $\sum \alpha_i = 1$ nên cần ít nhất $\frac{1}{0{,}05} = 20$ vector hỗ trợ (đúng bằng $\nu n$); số mẫu bị coi là bất thường (có $\alpha_i$ đạt cận trên) không quá $\nu n = 20$.

**A9.** $m = 0{,}5$: $\frac{1}{10}\ln(1 + e^5) = \frac{\ln 149{,}41}{10} \approx 0{,}5007$, gần hinge $= 0{,}5$. $m = 1{,}5$: $\frac{1}{10}\ln(1 + e^{-5}) \approx 0{,}00067$, gần 0. $m = -1$: $\frac{1}{10}\ln(1 + e^{20}) \approx 2{,}0000$, bằng hinge $= 2$. Khi $c$ lớn, hàm softplus dịch gần như trùng hinge.

**A10.** $w_+ = \frac{n}{2n_+} = \frac{700}{70} = 10$ và $w_- = \frac{700}{2 \cdot 665} \approx 0{,}526$. Với $C = 1$: $C_i = 10$ cho mẫu dương và $C_i \approx 0{,}526$ cho mẫu âm. Cận trên của $\alpha_i$ cũng là 10 và 0,526: mẫu thiểu số được phép "đẩy" mạnh gấp khoảng 19 lần.

**B1.** Kết quả tham khảo: `LinearSVM` (batch, 2000 bước, $C = 1$) và `SVC` tuyến tính cùng cho accuracy kiểm tra 94,15% trên 171 mẫu kiểm tra. Giá trị $J$: 14,29 (tự cài đặt) so với 13,60 (`SVC`), chưa trùng hẳn vì với 30 đặc trưng, tốc độ học giảm $1/t$ hội tụ chậm hơn ở ví dụ 2 chiều; tăng số bước hoặc dùng SGD kiểu Pegasos sẽ thu hẹp khoảng cách.

**B2.** Kết quả tham khảo (nhãn +1 là ác tính, 398 huấn luyện, 171 kiểm tra, ma trận nhầm lẫn $\begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}$): `SVC` RBF mặc định 95,3%, AUC 0,990, $\begin{bmatrix} 103 & 4 \\ 4 & 60 \end{bmatrix}$; `SVC` tuyến tính 94,2%, AUC 0,986, $\begin{bmatrix} 103 & 4 \\ 6 & 58 \end{bmatrix}$; `LinearSVC` 94,7%, AUC 0,984; Logistic Regression 95,3%, AUC 0,992, $\begin{bmatrix} 103 & 4 \\ 4 & 60 \end{bmatrix}$ (trùng ma trận với bài Logistic Regression). Grid search chọn $(C = 100, \gamma = 0{,}001)$ với CV 98,2% và cho 96,5% trên tập kiểm tra, AUC 0,994, $\begin{bmatrix} 104 & 3 \\ 3 & 61 \end{bmatrix}$.

**B3.** Kết quả tham khảo (SVM RBF $C = 1$, $\gamma$ mặc định): $A \approx 3{,}76$, $B \approx 0{,}33$; log-loss $\sigma(f)$ ngây thơ $\approx 0{,}265$, Platt scaling $\approx 0{,}127$, Logistic Regression $\approx 0{,}107$. Ở đây $A$ lớn hơn nhiều

so với 1 nên Platt scaling cải thiện rõ rệt, và Logistic Regression (được huấn luyện trực tiếp để có xác suất tốt) vẫn nhỉnh hơn một chút.

**B4.** Kết quả tham khảo (Iris, 45 mẫu kiểm tra, phân tầng $70/30$): one-vs-rest tuyến tính 95,6%, one-vs-one tuyến tính 97,8%, `SVC` RBF 97,8%, `LinearSVC` 97,8%; Softmax Regression ở bài Logistic Regression cho 100%. Chênh lệch chỉ một đến hai mẫu kiểm tra.

**B5.** Kết quả tham khảo (`load_diabetes`, 30% kiểm tra): hồi quy tuyến tính MAE 44,6, RMSE 55,7, $R^2 = 0{,}393$; SVR tuyến tính ($C = 1$, $\varepsilon = 0{,}1$) MAE 45,2, RMSE 55,9, $R^2 = 0{,}388$; SVR RBF với siêu tham số tốt nhất ($C = 1$, $\gamma = 0{,}1$, $\varepsilon = 0{,}05$) MAE 46,9, RMSE 59,7, $R^2 = 0{,}301$. Với bộ dữ liệu gần tuyến tính này, **hồi quy tuyến tính không thua** SVR RBF. Kernel chỉ có lợi khi có cấu trúc phi tuyến (như ở bộ $\sin x$ ở mục 6.6).

**B6.** Với $C$ lớn (ví dụ $10^3$), $\alpha \approx (3{,}33;\ 2{,}67;\ 2{,}67;\ 2{,}00)$ và $b \approx -1{,}00$, khớp phép tính tay ở mục 5.5 đến sai số dừng của thuật toán.

**B7.** Sai số trung bình giảm cỡ $1/\sqrt{D}$ (khoảng 0,31, 0,074, 0,026, 0,009 khi $D = 10, 10^2, 10^3, 10^4$ ở ví dụ hai điểm); với ma trận Gram nhiều điểm, sai số tuyệt đối trung bình cũng giảm tương tự khi $D$ tăng.

**C1.** Kết quả tham khảo (chuẩn hóa theo mẫu lành tính, $\gamma = 0{,}01$, $\nu = 0{,}05$): AUC $= 0{,}959$ để phân biệt ác tính, $90{,}6\%$ mẫu ác tính bị gắn nhãn bất thường và $15{,}0\%$ mẫu lành tính bị báo nhầm. Tăng $\nu$ sẽ tăng cả hai tỉ lệ; $\gamma$ quá lớn thì vùng bình thường co quá chặt quanh các điểm huấn luyện.

**C2.** Kết quả mong đợi: `SVC` thắng khi $n \lesssim 10^4$ và thua khi $n$ lớn hơn, với độ chính xác của ba cách gần bằng nhau (xem bảng ở mục 8.3). Điểm hòa vốn phụ thuộc vào $d$, $D$ và độ khó của dữ liệu.

**C3.** Kết quả tham khảo (tập $95/5$, 300 mẫu kiểm tra có 15 mẫu dương): không trọng số cho accuracy 96,0%, recall 0,333, precision 0,714; `balanced` cho accuracy 92,3%, recall 0,867, precision 0,382; AUC bằng nhau (0,972). Trọng số lớp đổi điểm làm việc trên đường precision-recall chứ không đổi chất lượng xếp hạng.

**C4.** Kết quả tham khảo (RBF $\gamma = 1$): Kernel Ridge Regression với $\lambda = 0{,}01;\ 0{,}1;\ 1$ cho sai số so với đường thật $0{,}159;\ 0{,}151;\ 0{,}131$, cả 42 hệ số đều khác 0; SVR ($C = 10$, $\varepsilon = 0{,}1$) cho 0,137 với 26 vector hỗ trợ. Hai mô hình cho sai số tương đương khi $\lambda$ đủ lớn, nhưng chỉ SVR có nghiệm thưa. Với $\lambda$ nhỏ, Kernel Ridge Regression kém hơn vì mất mát bình phương bị ngoại lai kéo.

## Lời kết

Đi từ Perceptron và Logistic Regression đến SVM, ta đã trả lời câu hỏi "trong vô số đường biên đúng, chọn đường nào?" bằng một ý tưởng hình học rất tự nhiên: **chọn đường biên cách xa hai lớp nhất**. Từ ý tưởng đó, toàn bộ bài giảng tự triển khai: bài toán lồi có nghiệm duy nhất, lề mềm cho dữ liệu thực, dạng đối ngẫu cho tính thưa, thủ thuật kernel cho biên cong, SVR cho hồi quy, và một bộ công cụ đánh giá và chọn siêu tham số. Đặt cạnh các mô hình trước đó, SVM không phải một "chiêu thức" riêng lẻ mà là một nấc thang trong cùng một khung: **mô hình, hàm mất mát cộng điều chuẩn, thuật toán tối ưu**, chỉ khác ở chỗ ngưỡng mất mát được dời từ 0 lên

1 và điều chuẩn $\frac{1}{2} \| \mathbf{w} \|^2$ được đặt làm mục tiêu. Hiểu rõ nó, bạn có sẵn nền tảng để đi tiếp sang các mô hình kernel khác, mô hình tổ hợp và học sâu.